

TƯ DUY HỆ THỐNG



TÓM TẮT

Phần B trả lời một câu hỏi duy nhất: làm sao ra quyết định thiết kế đúng khi chưa có mô hình nào chạy. Chín chương là chín dạng ràng buộc — cách đọc một hệ thống lạ, nền lý thuyết của việc học, định nghĩa bài toán, chất lượng nhãn, chia dữ liệu, dịch chuyển phân phối, hàm mất mát, kiến trúc, và phần cứng. Đọc xong bạn loại được phần lớn phương án sai trước khi viết dòng code đầu tiên. Nếu chỉ nhớ một điều: đây là phần có chu kỳ bán rã dài nhất của cây — kiến trúc trong Phần C sẽ cũ, cách suy nghĩ ở đây thì không.

Toàn bộ phần này trả lời một câu hỏi duy nhất: **làm sao ra quyết định thiết kế đúng khi chưa có mô hình nào chạy?** Đó là câu hỏi bị bỏ qua nhiều nhất, vì nó không có code để chạy và không có con số để khoe, nhưng nó quyết định phần lớn kết quả cuối cùng. Một dự án chọn sai đơn vị phân tích ở tuần đầu, hoặc chia dữ liệu để lọt rò rỉ, sẽ cho ra những con số đẹp và vô nghĩa suốt sáu tháng sau đó, và không kiến trúc nào cứu được.

Cách đọc phần này khác với cách đọc một giáo trình. Mỗi chương là *một dạng ràng buộc*, và mọi kỹ thuật xuất hiện trong đó đều là câu trả lời cho một thất bại cụ thể đã xảy ra trước nó. Chương 3 đưa ra quy trình đọc một hệ thống lạ và nguyên lý xuyên suốt cả cây rằng độ khó không biến mất mà chỉ bị đẩy đi chỗ khác. Chương 4 dựng nền lý thuyết tối thiểu về việc học từ dữ liệu, và giới thiệu trục đánh đổi trung tâm giữa giả định và dữ liệu mà mọi phần sau sẽ gặp lại. Chương 5 tới 8 đi theo vòng đời của dữ liệu: định nghĩa bài toán, chất lượng nhãn, chia dữ liệu và giao thức đánh giá, rồi chuyện gì xảy ra khi phân phối lúc triển khai không còn giống lúc huấn luyện. Chương 9 tới 11 chuyển sang ba ràng buộc thiết kế còn lại: hàm mất mát phát biểu điều ta thật sự muốn, kiến trúc phát biểu giả định về lời giải, và phần cứng phát biểu giới hạn cuối cùng mà không lập luận nào vượt qua được.

Đây là phần thuộc tầng [T2], tức tầng có chu kỳ bán rã dài nhất trong cây. Kiến trúc trong phần C sẽ cũ; cách suy nghĩ trong phần B thì không.

Bảng 1. Bản đồ Phần B: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
3	03-giai-phau-he-thong	Quy trình đọc một hệ thống lạ; nguyên lý độ khó bị đẩy đi chỗ khác	cốt lõi
4	04-nen-tang-hoc-tu-du-l	ERM, biểu diễn, manifold, inductive bias, trục đánh đổi giả định và dữ liệu	cốt lõi
5	05-thiet-ke-bai-toan	Đơn vị phân tích, định nghĩa nhãn, taxonomy, dạng đầu ra, population và sampling frame	cốt lõi
6	06-chat-luong-du-lieu	Nhiều nhãn, agreement, tài liệu hoá dữ liệu, phát hiện nhãn sai	cốt lõi
7	07-chia-du-lieu-va-danh	Leakage, chia theo nhóm và thời gian, chọn metric, đánh giá theo slice	cốt lõi
8	08-dich-chuyen-phan-pho	Các loại shift, domain adaptation và generalization, long-tail, OOD	cốt lõi
9	09-thiet-ke-ham-mat-mat	Loss như phát biểu về phân phối; khoảng cách giữa loss và metric	cốt lõi
10	10-thiet-ke-kien-truc	Inductive bias trong kiến trúc, receptive field, đa tỉ lệ, backbone/neck/head	cốt lõi
11	11-gioi-han-tinh-toan	Vì sao FLOPs không dự đoán được latency; roofline, bộ nhớ activation	cốt lõi

Đọc tuần tự. Chương 3 và 4 là tiền đề của bảy chương còn lại; chương 5 tới 8 đi theo vòng đời dữ liệu; chương 9 tới 11 là ba ràng buộc thiết kế còn lại.

Giải phẫu một hệ thống thị giác (Anatomy of a Vision System)

PHỤ THUỘC không bắt buộc; đây là chương mở đầu của Phần B. Đọc trước A/02-vat-ly-tao-anh nếu muốn hiểu vì sao “luồng dữ liệu” phải bắt đầu từ cảm biến chứ không từ tensor đầu vào.

TÓM TẮT

Chương này đưa ra ba thứ. Một quy trình bảy câu hỏi để mổ xẻ một repo hoặc paper lạ. Một bảng nói rõ paper, code và **ablation** mỗi nguồn che giấu điều gì. Và nguyên lý *độ khó không biến mất, chỉ bị đẩy đi chỗ khác*, công cụ dùng được suốt phần còn lại của cây. Đọc xong, chỉ sau một buổi bạn nói ra được các **chế độ hỏng** (*failure mode*) của một hệ thống mới gặp, thay vì chỉ liệt kê module của nó. Bạn cũng biết lúc nào nên đóng một paper lại thay vì đọc tiếp. Nếu chỉ nhớ một điều: đọc một hệ thống là đi tìm chỗ nó phải trả giá. Ràng buộc cứng loại bỏ phần lớn phương án trước khi độ chính xác kịp lên tiếng, nên hãy tìm ràng buộc trước và tìm ý tưởng sau.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, nhận ảnh làm đầu vào và trả về dự đoán (nhãn, hộp, mask). “Huấn luyện” nghĩa là đi tìm bộ tham số làm dự đoán sát nhãn nhất; “suy luận” (*inference*) là chạy hàm đó trên dữ liệu mới.
- **hệ thống thị giác và pipeline** — một hệ thống thật không chỉ có mô hình. Nó là một chuỗi khối nối tiếp — cảm biến, tiền xử lý, mô hình, hậu xử lý, quyết định — và chuỗi đó gọi là *pipeline*. Mỗi khối trong chuỗi gọi là một *module*. Phân biệt hai thứ này là điều kiện để đọc chương.
- **tín hiệu giám sát (supervision signal)** — thứ nói cho thuật toán biết dự đoán đúng là gì. Thường là nhãn do người gán nhãn tạo ra, nhưng cũng có thể là một ràng buộc vật lý, một cảm biến khác, hoặc chính dữ liệu (tự giám sát). Biết tín hiệu giám sát đến từ đâu là biết hệ thống học được điều gì và không học được điều gì.
- **giả định (assumption)** — một mệnh đề về thế giới mà hệ thống coi là đúng nhưng không kiểm tra — ví dụ “vật thể luôn nằm trên mặt đất” hay “ảnh đủ sáng”. Giả định không được nói ra vẫn có hiệu lực; hệ thống hỏng đúng lúc giả định bị vi phạm.
- **ablation** — thí nghiệm bỏ đi từng thành phần của hệ thống rồi đo lại kết quả, để biết thành phần nào thực sự đóng góp. Từ này không có bản dịch tiếng Việt thông dụng nên tài liệu giữ nguyên.
- **baseline** — phương pháp đối chứng đơn giản hơn mà phương pháp mới phải vượt qua thì mới có ý nghĩa. Nếu baseline không được tinh chỉnh cẩn thận thì mọi so sánh với nó đều vô giá trị.
- **độ trễ (latency) và FPS** — độ trễ là thời gian xử lý một khung hình, tính bằng mili-giây; FPS (*frames per second*) là số khung hình xử lý được mỗi giây. Hai đại lượng nghịch đảo nhau và là ràng buộc cứng phổ biến nhất trong hệ thống thời gian thực.
- **mAP** — thước đo phổ biến cho bài toán phát hiện vật thể, gộp độ chính xác và độ bao phủ trên nhiều ngưỡng thành một con số duy nhất. Ở chương này bạn chỉ cần biết nó là “điểm số trong bảng kết quả”; định nghĩa đầy đủ nằm ở Phần E.

CÂU HỎI MỞ ĐẦU

- Q1. Hai kỹ sư đọc cùng một repo trong cùng một buổi: vì sao một người ra về với danh sách module, còn người kia dự đoán được hệ thống sẽ sập trong một hành lang tường trắng?
- Q2. Nếu phần method là phần dài nhất của mọi paper, vì sao nó lại là phần đáng đọc *sau cùng*?
- Q3. Một paper báo cáo 45 ms còn bạn có 33 ms mỗi khung hình — vì sao phép tính hai dòng đó loại bỏ nhiều phương án hơn cả bảng kết quả?
- Q4. Nếu bảng ablation nói bỏ khối A không mất gì và bỏ khối B cũng không mất gì, bạn có được kết luận cả hai đều vô dụng không?

1.1 Đặt vấn đề

Đứng trước một repo hoặc một paper lạ, làm sao đi từ “không hiểu gì” tới “biết nó giả định gì và hỏng ở đâu” trong một buổi, thay vì mò mẫm hàng tuần? Câu hỏi này nghe như một câu hỏi về kỹ năng đọc, nhưng thực ra nó là câu hỏi về **cách phân loại**. Người mới đọc hệ thống theo đặc điểm bề mặt: tên module, tên tầng, con số trong bảng kết quả. Người có kinh nghiệm đọc theo cấu trúc sâu: bài toán gì, giả định gì, đánh đổi gì. Hai cách đọc cho ra hai bản đồ hoàn toàn khác nhau về cùng một hệ thống, và chỉ bản đồ thứ hai dùng được khi bạn phải sửa hoặc thay một khối.

Sự khác biệt này không phải chuyện học thuật. Bạn đã gặp nó ở phía SLAM. Hai người cùng đọc ORB-SLAM3 [1] trong một buổi. Người thứ nhất ra về với một danh sách bộ phận: có ORB extractor, có DBoW2, có **hiệu chỉnh chùm tia** (*bundle adjustment*) cục bộ, có **đóng vòng** (*loop closing*). Người thứ hai ra về với một câu đánh cược: hệ này tin rằng góc ảnh còn lặp lại được giữa hai khung hình liên tiếp. Từ câu đó suy ra ngay hai chỗ sập. Ảnh mờ làm góc nhoe đi nên không còn khớp được; bề mặt trơn thì ngay từ đầu đã không có góc nào để tìm. Người thứ hai cũng biết hệ bù lại bằng một tầng **tái định vị** (*relocalization*) đắt tiền. Chỉ anh ta dự đoán được chuyện gì xảy ra trong một hành lang tường trắng. Danh sách bộ phận không sinh ra dự đoán; bản đồ giả định thì có.

Cái làm cho việc đọc trở nên khó là không gian thiết kế quá rộng. Một hệ thống thị giác hiện đại có hàng chục lựa chọn gắn kết nhau, và paper hầu như luôn kể câu chuyện theo trục “chúng tôi nghĩ ra ý tưởng đẹp này”. Trục đó gần như không bao giờ là trục thật đã sinh ra thiết kế. Trục thật thường là một ràng buộc: chỉ có bằng ấy nhân, chỉ có bằng ấy mili-giây, chỉ có bằng ấy bộ nhớ. Vì vậy chương này không dạy bạn đọc nhanh hơn; nó dạy bạn đọc theo đúng trục, để hầu hết nội dung tự rơi vào chỗ của nó.

TRỰC GIÁC

Đọc một hệ thống thị giác giống khám một bệnh nhân chứ không giống đọc một cuốn catalogue. Catalogue liệt kê bộ phận; khám thì hỏi *bộ phận này gánh chức năng gì, và nếu nó hỏng thì triệu chứng gì hiện ra ở đâu*. Một người thuộc lòng tên hai trăm chi tiết máy vẫn không sửa được xe; người biết “nếu bơm xăng yếu thì máy chết ở vòng tua cao” thì sửa được. Thước đo của cả chương là bạn có phát biểu được các triệu chứng hay không.

1.2 Bảy câu hỏi để mổ xẻ (Seven Questions for Dissection)

A. Vì sao thứ tự của các câu hỏi là thông tin

Thứ tự dưới đây không tùy tiện: mỗi câu hỏi chỉ trả lời được khi câu trước đã có đáp án, và mỗi câu sinh ra vì câu trước để lại một chỗ hở. Nói gọn thì mạch đi từ *bài toán* (câu 1–2) sang *không gian thiết kế bị ràng buộc* (câu 3–4) rồi sang *đóng góp thật và chỗ hỏng* (câu 5–7). Đọc lệch thứ tự thì bạn dành công sức cho những chi tiết mà ràng buộc cứng đàng nào cũng xoá bỏ.

Mạch câu hỏi khi mổ xẻ một hệ thống lạ

- 1 Vấn đề** không biết bắt đầu từ đâu, nên bắt đầu từ chỗ dễ nhìn nhất là code.

Giải pháp tách ba tầng **bài toán** → **nguyên lý** → **cài đặt**, và trước khi đọc một dòng code, ép mình viết ra đúng một câu cho mỗi tầng.

Hệ quả nó ngăn bạn nhầm chi tiết framework với ý tưởng cốt lõi. Cách họ viết **data-loader** hay đặt tên **checkpoint** là chi tiết. Vì sao **tín hiệu giám sát** (*supervision signal*) đến từ nguồn này chứ không nguồn khác mới là ý tưởng.

Giá phải trả mất nửa giờ viết ba câu trông tầm thường; đổi lại không mất ba ngày đọc nhầm tầng.
- 2 Vấn đề** “bài toán” vẫn là một khái niệm mơ hồ, ai cũng gật đầu nhưng mỗi người hiểu một kiểu.

Giải pháp định nghĩa nó bằng đúng ba thứ — đầu vào, đầu ra, và tín hiệu giám sát. Mọi thứ còn lại là lựa chọn triển khai.

Giả định ba thứ này xác định bài toán đủ chặt để hai người khác nhau viết ra cùng một bản mô tả.

Hệ quả nếu chưa trả lời được cả ba thì đọc tiếp là vô ích; hãy quay về phần dataset, vì câu trả lời nằm ở đó chứ không ở phần method.
- 3 Vấn đề** không gian thiết kế quá rộng để suy luận bằng trực giác.

Giải pháp truy tìm **ràng buộc cứng** (*hard constraint*) trước tiên: **độ trễ** (*latency*), bộ nhớ, lượng nhân có được, quyền riêng tư, năng lượng.

Giả định tác giả không tự do như phần method làm ra vẻ.

Hệ quả ràng buộc cứng loại bỏ phần lớn phương án trước khi độ chính xác kịp lên tiếng, nên nó giải thích nhiều lựa chọn thiết kế hơn toàn bộ phần method cộng lại. Con số “khoảng 90%” hay được nhắc là kinh nghiệm thực hành chứ không phải kết quả đo được; nhưng chiều của nhận định thì rất vững.
- 4 Vấn đề** thiết kế nào cũng trông đẹp khi đọc trong paper.

Giải pháp áp nguyên lý **độ khó không biến mất, chỉ bị đẩy đi chỗ khác** và hỏi thẳng: họ đã đẩy nó đi đâu — sang dữ liệu, sang hậu xử lý, sang thời gian huấn luyện, hay sang một giả định về đầu vào?

Giả định mọi hệ thống đều trả giá ở đâu đó cho cái nó làm dễ đi ở chỗ khác.

Hệ quả câu trả lời gần như luôn tồn tại; nếu bạn không tìm ra thì kết luận đúng là bạn chưa hiểu hệ thống.
- 5 Vấn đề** không biết phần nào trong bài là đóng góp thật.

Giải pháp đọc bảng **ablation** theo chiều ngược — bỏ thành phần nào thì hệ thống sập, chứ không phải thêm gì thì con số nhích lên.

Giả định ablation được chạy đàng hoàng và **baseline** được tinh chỉnh ngang với phương pháp đề xuất.

Vấn đề mới giả định đó thường sai; có cả một dòng công trình cho thấy phần lớn “tiến bộ” trong một số lĩnh vực biến mất khi baseline được tinh chỉnh tử tế [2], [3]. Chương 24 dạy cách kiểm tra chính giả định này.

↪ **tiền đề** E/24 — độ tin cậy của một bảng ablation là điều kiện để quy trình đọc này còn giá trị

6 Vấn đề

Giải pháp

Hệ quả

paper hầu như không nói về chỗ nó hỏng.

đọc phần *limitations* và *failure case* trước phần kết quả, vì đó là chỗ tác giả không có động cơ phóng đại.

bạn bước vào phần kết quả với một danh sách nghi vấn sẵn có, nên các con số đẹp không còn quyền định khung suy nghĩ của bạn. Nguyên tắc phía sau: hiểu một hệ thống nghĩa là biết nó hỏng ở đâu, không phải biết nó chạy được gì.

7 Vấn đề

Giải pháp

Hệ quả

lỗi thật khi triển khai thường không nằm trong model.

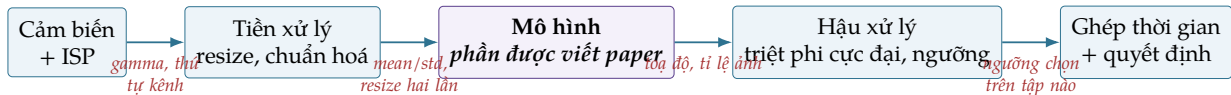
vẽ **sơ đồ luồng dữ liệu** (*data flow diagram*) đầy đủ, từ cảm biến tới quyết định cuối, và đánh dấu mọi đoạn nối giữa hai khối.

phần lớn lỗi hệ thống nằm ở các đoạn nối — sai thứ tự kênh màu, resize hai lần, đơn vị góc lệch nhau, *timestamp* lệch pha — chứ không nằm ở khối được viết paper [4]. Bất kỳ ai từng debug một pipeline SLAM đều đã sống qua điều này.

↪ **tiền đề** A/02 — hai khối đầu của luồng — cảm biến và ISP — là vật lý tạo ảnh, và chúng quyết định phần lớn khoảng cách train/production

B. Luồng dữ liệu: chỗ paper im lặng

Câu hỏi thứ bảy đáng được vẽ ra chứ không chỉ nói suông, vì bản chất của nó là không gian: cái ta muốn thấy là tỉ lệ giữa phần được viết paper và phần còn lại của đường đi. Hình 1.1 cho thấy một luồng điển hình; khối tô đậm là khối duy nhất paper mô tả, và bốn mũi tên nối là nơi các lỗi tồn thời gian nhất sinh ra.



Hình 1.1. Luồng dữ liệu đầy đủ của một hệ thống thị giác. Paper mô tả một khối; các nhãn đỏ dưới mũi tên là những đoạn nối nơi lỗi triển khai thực tế hay sinh ra và không bao giờ xuất hiện trong bảng kết quả.

1.3 Ba nguồn thông tin: paper, code, ablation

Có đúng ba nguồn thông tin về một hệ thống lạ. Người mới thường chọn một nguồn theo thói quen — kỹ sư đọc code, sinh viên đọc paper — rồi tin rằng mình đã hiểu. Vấn đề là mỗi nguồn không chỉ *thiếu* thông tin, nó còn *che giấu* một loại thông tin cụ thể, và loại bị che luôn là loại bạn cần khi phải ra quyết định.

Bảng 1.1. Ba cách tiếp cận một hệ thống lạ, và cái mà mỗi cách che giấu. Cột “che giấu” là cột có giá trị nhất: nó nói cho bạn biết mình đang mù ở đâu.

Cách đọc	Cho bạn biết	Che giấu	Hỏng khi nào
Đọc paper	Ý định của tác giả, vị trí lịch sử của ý tưởng	Chi tiết thật sự quyết định kết quả: gán mẫu (<i>label assignment</i>), augmentation, lịch học	Khi câu chuyện được viết lại sau khi kết quả đã có
Đọc code	Sự thật về cái đang chạy, kể cả chỗ paper không kể	Vì sao chọn như vậy; đâu là thiết yếu, đâu là tàn dư lịch sử	Khi repo lớn tới mức bạn hiểu chi tiết mà mất tăng bài toán
Đọc ablation	Trọng số đóng góp thật của từng khối theo chính tác giả	Tương tác giữa các khối, chất lượng tinh chỉnh của baseline	Khi hai khối làm cùng một việc và che lẫn nhau

Bảng 1.1 cho thấy không nguồn nào đủ một mình, và quan trọng hơn, cho thấy cách phối hợp hiệu quả nhất: đọc ablation trước để biết phần nào đáng quan tâm, rồi đọc code của đúng phần đó, rồi mới quay lại paper để hiểu tác giả nghĩ gì — ngược hoàn toàn với thứ tự trang giấy. Lý do là ablation có tỉ lệ tín hiệu trên độ dài cao nhất, còn phần method có tỉ lệ thấp nhất vì nó được viết để thuyết phục người phản biện. Đây là dạng đọc chọn lọc mà Keshav mô tả dưới tên “đọc ba lượt” [5], chỉ khác là ta chọn điểm vào theo ablation thay vì theo phần mở đầu.

Cột cuối của bảng đáng được nói rõ hơn ở một điểm: ablation cho biết *đóng góp biên* của một khối khi các khối còn lại đã có mặt, chứ không cho biết giá trị tuyệt đối của nó. Hai khối làm cùng một việc sẽ che nhau — bỏ khối nào đi một mình cũng không mất gì, nên bảng kết luận cả hai đều vô dụng, trong khi bỏ cả hai thì hệ thống sập. Chuyện này xảy ra thường xuyên và giải thích vì sao ablation không thay được việc đọc code.

1.4 Độ khó không biến mất (Conservation of Difficulty)

A. Nguyên lý và ba ví dụ

Không có thiết kế nào làm cho một bài toán dễ đi; chỉ có thiết kế di chuyển phần khó sang một trục mà bạn sẵn lòng trả giá hơn. Nói gọn thì mỗi “cải tiến” là một phép đổi tiền tệ, và việc của bạn khi đọc là tìm ra tỉ giá.

• Ba chỗ độ khó hay bị đẩy tới, kèm ví dụ cụ thể:

- **Sang thời gian huấn luyện.** Mô hình *one-stage* bỏ bước sinh đề xuất vùng nên chạy nhanh hơn. Nhưng vẫn phải quyết định vị trí nào trên ảnh chịu trách nhiệm cho vật nào. Câu hỏi đó không biến mất, nó chỉ đổi tên thành bài toán gán mẫu và kéo theo cả một dòng heuristic [6], [7]. DETR bỏ được **triệt phi cực đại** (*non-maximum suppression*) và **anchor**, đổi lại số epoch cần để hội tụ tăng vọt [8].
- **Sang giả định về đầu vào.** SLAM trực tiếp bỏ được bước trích đặc trưng, đổi lại phải giả định **độ sáng không đổi** (*brightness constancy*) giữa các khung hình và chấp nhận một miền hội tụ hẹp hơn nhiều. Đây chính là trục *giả định mạnh với ít dữ liệu* đổi lại *giả định yếu với nhiều dữ liệu* mà chương 4 sẽ đặt tên.

→ cùng nguyên lý với B/04 — “độ khó không biến mất là phát biểu định tính của cùng một trục mà chương 4 định lượng

- **Sang dữ liệu và hậu xử lý.** Một hệ thống tự nhận là “đơn giản, không cần module X” thường đã chuyển gánh nặng sang chỗ khác. Chỗ đó có thể là một tập dữ liệu lớn hơn

nhiều lần. Cũng có thể là một chuỗi luật lọc đầu ra chiếm nửa số dòng trong repo mà paper không nhắc tới.

B. Ngân sách mili-giây: một phép tính hai dòng

Cách kiểm chứng nguyên lý này bằng số học rất đơn giản: lập một ngân sách và xem con số nào phải nhường chỗ. Giả sử camera chạy 30 Hz, tức mỗi khung hình có $1000/30 \approx 33$ ms. Nếu bạn dành 4 ms cho tiền xử lý, 3 ms cho hậu xử lý và 6 ms cho phần ghép thời gian thì mô hình chỉ còn

$$33 - (4 + 3 + 6) = 20 \text{ ms},$$

và sau khi chừa biên an toàn thì thực tế là khoảng 16 ms. Công thức chỉ nói một điều: *ngân sách của mô hình là phần dư sau khi mọi khối khác đã lấy phần của chúng*, nên nó luôn nhỏ hơn con số bạn tưởng. Bây giờ đọc một paper báo cáo 45 ms trên GPU máy chủ với **batch** bằng 8: con số đó vừa vượt ngân sách hơn hai lần, vừa được đo ở chế độ batch mà bạn không bao giờ có vì bạn xử lý từng khung hình đến. Phép tính hai dòng này loại bỏ cả một họ phương pháp nhanh hơn bất kỳ bảng mAP nào. Các con số trong ví dụ là do tôi đặt ra để minh họa cách tính, không trích từ một hệ thống cụ thể.

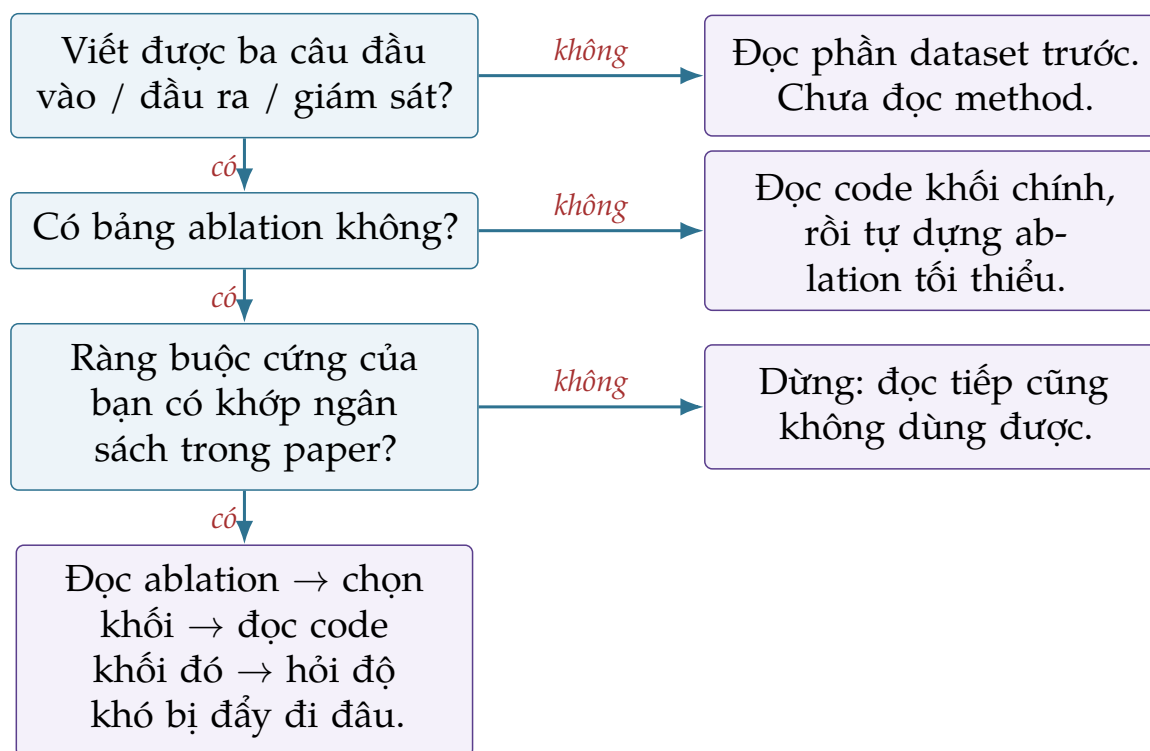
↔ **ràng buộc bởi B/11** — ngân sách mili-giây ở đây là dạng thô sơ của mô hình roofline, nơi ràng buộc thật là băng thông bộ nhớ chứ không phải số phép tính

1.5 Ba quy tắc quyết định

Bảy câu hỏi ở trên là quy trình; ba quy tắc dưới đây là các điểm dừng — những chỗ bạn được phép kết luận mà không cần đọc thêm. Giá trị của chúng nằm ở chỗ mỗi quy tắc cho phép bạn đóng một cuốn paper sớm.

1. **Chưa viết được ba câu đầu vào / đầu ra / tín hiệu giám sát?** Dừng đọc phần method, quay lại phần dataset. Đọc tiếp trong trạng thái đó chỉ nạp vào đầu các chi tiết không có chỗ để gắn.
2. **Nếu ràng buộc độ trễ siết đi năm lần thì khối nào bị thay đầu tiên?** Trả lời được câu này nghĩa là bạn đã hiểu kiến trúc theo nghĩa dùng được. Nó đòi hỏi hai thứ: biết chi phí tính toán phân bố ra sao, và biết khối nào có phương án thay thế rẻ hơn.
3. **Paper không có phần limitations?** Coi đó là tín hiệu cảnh báo về mức độ trung thực của phần còn lại, không phải dấu hiệu hệ thống hoàn hảo. Mọi hệ thống thật đều có chế độ hồng; im lặng về chúng là một lựa chọn biên tập.

Hình 1.2 gói ba quy tắc và mạch bảy câu hỏi thành một cây quyết định dùng được ngay khi mở một repo.



Hình 1.2. Cây quyết định khi mở một hệ thống lạ. Ba nút hỏi tương ứng ba quy tắc dừng; chỉ khi cả ba đều thoả thì việc đọc sâu mới có lãi.

1.6 Thực hành

Quy trình bảy câu hỏi chỉ trở thành phản xạ khi bạn chạy nó trên mã nguồn thật vài lần.

Repo và tài nguyên. Ba loại repo đáng mổ xẻ, theo thứ tự tăng dần độ phức tạp:

- **Tối giản, đọc hết trong một buổi** — [karpathy/minGPT](#), [lucidrains/vit-pytorch](#). Dùng để luyện tách ba tầng khi không có nhiều.
- **Production đầy đủ tầng** — [ultralytics/ultralytics](#), [open-mmlab/mmdetection](#), [facebookresearch/detectron2](#). Dùng để thấy một hệ thống thật được tổ chức ra sao.
- **Hệ thống nhiều khối ghép** — [nerfstudio-project/nerfstudio](#), [colmap/colmap](#). Dùng để thấy các đoạn nối trong Hình 1.1 mới là phần khó.

Danh sách tên mô hình dẫn đầu bảng chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; repo hạ tầng như COLMAP hay [timm](#) bền hơn nhiều, nên hãy chia thời gian theo tỉ lệ đó.

Câu hỏi kiểm tra hiểu. Trong repo bạn chọn, ranh giới giữa “bài toán”, “nguyên lý” và “chi tiết cài đặt” nằm ở đâu — chỉ đúng ba file hoặc ba hàm đánh dấu ranh giới đó. Độ khó của bài toán đã bị đẩy đi đâu, và bạn chứng minh bằng dòng code nào?

Mini project. Chọn một repo và viết *system datasheet* dài một trang: đầu vào, đầu ra, tín hiệu giám sát, ràng buộc cứng, ba giả định ngầm, ba chế độ hỏng. Ràng buộc của bài tập là không đọc paper, chỉ đọc code — đây chính là chỗ bạn phát hiện mình vẫn đang suy đoán từ tên biến thay vì từ hành vi. Vài giờ đến hai ngày.

1.7 Giới hạn và trường hợp hỏng

Quy trình này có ba chỗ dễ gây, và biết trước chúng thì bạn không tin nó quá mức.

- **Nó giả định ablation chạy đẳng hoàng** — một giả định thường sai. Ablation có thể chạy trên một cấu hình, một *seed*; trong học tăng cường người ta đã cho thấy phương sai giữa các seed đôi khi lớn hơn khoảng cách giữa các phương pháp [9]. Kinh nghiệm benchmark SLAM của bạn nói cùng một điều: một hệ đa luồng cho hai kết quả khác nhau trên cùng một bag, nên một dòng trong bảng ablation không phải một phép đo.
- **Nó tối ưu cho việc hiểu, không cho việc tái hiện.** Rất nhiều hệ thống chỉ đạt con số đã công bố nhờ những chi tiết không được coi là ý tưởng: độ dài lịch học, cách khởi tạo, một phép chuẩn hoá đặt đúng chỗ. Bạn có thể hiểu hoàn hảo một hệ thống mà vẫn không tái hiện được nó.
- **Nó không áp dụng tốt cho công trình mà đóng góp chính là dữ liệu hoặc phép đo.** Với một paper giới thiệu benchmark, bảy câu hỏi phần lớn trả về câu trả lời rỗng; khung đọc đúng là khung của chương 7 và chương 24 — tập này lấy mẫu từ đâu, chia thế nào, và metric thường cho hành vi gì.

CẠM BÃY

Cạm bẫy phổ biến nhất là nhầm “tôi đọc hiểu từng dòng code” với “tôi hiểu hệ thống”. Đọc hết code cho bạn tăng cài đặt ở độ phân giải rất cao và tăng bài toán ở độ phân giải bằng không. Dấu hiệu nhận biết: bạn mô tả được hệ thống làm gì theo trình tự thời gian, nhưng không nói được điều gì sẽ hỏng nếu ảnh đầu vào tối đi hai *stop*. Nếu rơi vào trạng thái đó, đừng đọc thêm code — hãy quay lại câu hỏi số hai.

CẦN NHỚ

Hiểu một hệ thống nghĩa là biết nó *hỏng ở đâu*, không phải biết nó chạy được gì. Quy trình gọn nhất: viết ba câu đầu vào / đầu ra / giám sát → tìm ràng buộc cứng → đọc ablation ngược → hỏi độ khó đã bị đẩy đi đâu. Nếu chưa tìm ra chỗ độ khó bị đẩy tới, bạn chưa đọc xong. [T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Bài toán, nguyên lý, cài đặt	Ba tầng để tách một hệ thống ra trước khi đọc dòng code đầu tiên. Viết đúng một câu cho mỗi tầng thì bạn không còn nhầm một chi tiết framework với một ý tưởng cốt lõi.
••• Tín hiệu giám sát	Thứ nói cho thuật toán biết dự đoán đúng là gì: nhãn người gán, một ràng buộc vật lý, một cảm biến khác, hoặc chính dữ liệu. Biết nó đến từ đâu là biết hệ thống học được gì và không học được gì.
••• Ràng buộc cứng	Giới hạn mà tác giả không thể vượt qua: độ trễ, bộ nhớ, lượng nhân, quyền riêng tư, năng lượng. Nó loại bỏ phần lớn phương án trước khi độ chính xác kịp lên tiếng.
•• Vì sao ràng buộc cứng giải thích thiết kế nhiều hơn cả phần method?	Vì nó là ràng buộc loại trừ: nó xoá cả một họ phương án bất kể chúng chính xác tới đâu. Phần method chỉ xếp hạng những gì đã lọt qua.
••• Độ khó không biến mất	Nguyên lý trung tâm: không thiết kế nào làm bài toán dễ đi, nó chỉ đẩy phần khó sang dữ liệu, hậu xử lý, thời gian huấn luyện, hoặc một giả định về đầu vào. Không tìm ra chỗ bị đẩy tới nghĩa là chưa hiểu hệ thống.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• Gán mẫu	Bước quyết định vị trí nào trên ảnh chịu trách nhiệm cho vật nào. Nó sinh ra đúng lúc mô hình một giai đoạn bỏ bước sinh đề xuất vùng, và là ví dụ điển hình của độ khó bị đẩy đi chỗ khác.
•• Ablation	Thí nghiệm bỏ đi từng thành phần rồi đo lại, để biết thành phần nào thực sự đóng góp. Cách đọc đúng là đọc ngược: bỏ cái gì thì hệ thống sập.
•• Đóng góp biên	Cái mà ablation thực sự đo: mức mất mát khi bỏ một khối trong khi các khối còn lại vẫn có mặt. Nó không phải giá trị tuyệt đối của khối đó.
••• Bỏ khối A không mất gì, bỏ khối B cũng không mất gì. Kết luận?	Không được kết luận cả hai vô dụng. Rất có thể chúng làm cùng một việc và che nhau; phép thử phân biệt là bỏ đồng thời cả hai, nếu hệ thống sập thì chúng dư thừa lẫn nhau chứ không thừa.
• Baseline	Phương pháp đối chứng đơn giản hơn mà phương pháp mới phải vượt qua. Nếu nó không được tinh chỉnh cẩn thận thì mọi so sánh với nó đều vô giá trị.
•• Vì sao đọc ablation trước rồi mới đọc code?	Ablation có tỉ lệ tín hiệu trên độ dài cao nhất, còn phần method thấp nhất vì nó được viết để thuyết phục người phản biện. Đọc ablation để chọn khối đáng quan tâm, rồi chỉ đọc code của khối đó.
•• Vì sao đọc limitations trước phần kết quả?	Vì đó là chỗ tác giả không có động cơ phóng đại. Bạn bước vào bảng số với một danh sách nghi vấn sẵn có, nên con số đẹp không định khung suy nghĩ của bạn.
• Paper không có mục limitations	Đọc như một tín hiệu cảnh báo về mức trung thực của phần còn lại, không phải dấu hiệu hệ thống hoàn hảo. Mọi hệ thống thật đều có chế độ hỏng; im lặng về chúng là một lựa chọn biến tập.
•• Sơ đồ luồng dữ liệu	Đường đi đầy đủ từ cảm biến tới quyết định cuối, có đánh dấu mọi đoạn nối. Vẽ nó ra là cách nhanh nhất thấy phần được viết paper chỉ chiếm một khối trong năm.
•• Đoạn nối	Chỗ ghép giữa hai khối: thứ tự kênh màu, chuẩn hoá, tỉ lệ toạ độ, timestamp. Phần lớn lỗi triển khai nằm ở đây chứ không nằm trong mô hình.
•• Ngân sách độ trễ	Thời gian cho mỗi khung hình, chia cho từng khối. Ngân sách của mô hình luôn là phần dư sau khi các khối khác đã lấy phần của chúng.
•• Vì sao camera 30 Hz chỉ chừa cho mô hình khoảng 16 ms?	Mỗi khung hình có 33 ms. Trừ 4 ms tiền xử lý, 3 ms hậu xử lý và 6 ms ghép thời gian còn 20 ms, rồi phải chừa biên an toàn.
•• System datasheet	Bản một trang mô tả một hệ thống: đầu vào, đầu ra, tín hiệu giám sát, ràng buộc cứng, ba giả định ngầm, ba chế độ hỏng. Viết được nó nghĩa là bạn đã đọc xong hệ thống.
•• Bạn kể được hệ thống làm gì theo trình tự nhưng không nói được nó hỏng khi nào	Bạn có tăng cài đặt ở độ phân giải rất cao và tăng bài toán bằng không. Phép thử: hỏi điều gì xảy ra nếu ảnh tối đi hai stop; trả lời không được thì quay lại ba câu đầu vào, đầu ra, giám sát.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Datasheets for datasets, model cards	Chính là “system datasheet” của chương này được chuẩn hoá thành tài liệu bắt buộc kèm mô hình và tập dữ liệu	Thông lệ ở nhiều nhóm lớn từ 2019, ít hết hạn

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
mmdetection , detectron2	Tách ba tầng bài toán / nguyên lý / cài đặt thành cấu hình, registry và module — đọc chúng là bài tập tự nhiên cho quy trình bày câu hỏi	Hạ tầng, bền
colmap/colmap	Hệ thống nhiều khối ghép, nơi các đoạn nối — hiệu chuẩn, thứ tự ảnh, ngưỡng khớp — quyết định kết quả hơn bất kỳ khối đơn lẻ nào	Hạ tầng, bền
Dòng DETR thời gian thực	Ví dụ sống của “độ khó bị đẩy đi chỗ khác”: bỏ triết phi cực đại và anchor, đổi lại chi phí hội tụ và cả một dòng nghiên cứu về thiết kế query	Tính tới 9/2026

MINI PROJECT SLAM · System datasheet cho ORB-SLAM3, chỉ đọc code · 1–2 ngày

Mục tiêu	biến quy trình bày câu hỏi thành một sản phẩm kiểm tra được, trên chính hệ thống bạn chạy hằng ngày. Bài tập sẽ lộ ra những chỗ bạn tưởng mình hiểu mà thực ra chỉ đang suy đoán từ tên biến.
Dữ liệu	mã nguồn ORB-SLAM3 [1] và một cấu hình EuRoC đang chạy được. Ràng buộc của bài tập: <i>không mở paper</i> , chỉ đọc code và file cấu hình.
Các bước	viết một trang gồm đúng sáu phần — (1) đầu vào, đầu ra, tín hiệu giám sát; (2) ràng buộc cứng đọc được từ code, ví dụ số đặc trưng tối đa, số mức kim tự tháp, ngưỡng số điểm khớp để coi là bám được; (3) sơ đồ luồng dữ liệu từ ảnh tới pose, đánh dấu mọi đoạn nối; (4) ba giả định ngầm, mỗi giả định kèm dòng code chứng minh; (5) ba chế độ hỏng suy ra từ ba giả định đó; (6) độ khó đã bị đẩy đi đâu.
Xong khi	bạn trả lời được bằng văn bản câu “nếu ràng buộc độ trễ siết đi năm lần thì khối nào bị thay đầu tiên”. Câu trả lời phải kèm phần trăm thời gian của từng khối, đo bằng bộ đếm bạn tự chèn chứ không bằng cảm giác. Kiểm tra chéo: ba chế độ hỏng bạn nêu phải tái hiện được ít nhất một cái trên dữ liệu thật (ví dụ làm mờ ảnh đầu vào hoặc giảm số đặc trưng) và quan sát đúng triệu chứng bạn đã dự đoán.
Nối với	B/05 dùng lại datasheet này khi thiết kế bài toán học máy đầu tiên gắn vào hệ; B/11 thay ước lượng phần trăm thời gian bằng mô hình roofline; E/24 cho cách đọc con số đo được khi harness có thành phần ngẫu nhiên.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai người đọc theo hai cách phân loại khác nhau: người thứ nhất theo đặc điểm bề mặt (tên module, tên tầng), người thứ hai theo cấu trúc sâu (bài toán, giả định, đánh đổi). Chỉ bản đồ giả định mới sinh ra dự đoán về triệu chứng.
- A2. Vì phần method có tỉ lệ tín hiệu trên độ dài thấp nhất — nó được viết để thuyết phục người phân biện — trong khi ablation và limitations nói thẳng phần nào đóng góp và phần nào hỏng. Đọc ablation trước để chọn điểm vào, rồi đọc code của đúng phần đó.
- A3. Vì ràng buộc cứng là ràng buộc loại trừ: nó xoá cả một họ phương án bất kể chúng chính xác tới đâu, còn bảng kết quả chỉ xếp hạng trong số những phương án đã lọt qua. Thêm nữa, 45 ms đó thường được đo ở chế độ batch mà hệ thống thời gian thực của bạn không bao giờ có.
- A4. Không. Ablation đo đóng góp biên khi các khối còn lại đã có mặt, nên hai khối dư thừa lẫn nhau sẽ cùng cho ra “bỏ đi không mất gì” trong khi bỏ cả hai thì hệ thống sập. Đây là lý do ablation không thay được việc đọc code.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn ngồi ăn ở một quán cơm nhỏ và muốn hiểu vì sao thực đơn của họ đúng như vậy. Bạn có thể hỏi chủ quán, và được nghe một câu chuyện đẹp về công thức gia truyền. Bạn có

thể xin vào bếp xem họ nấu, và thấy chính xác những gì đang xảy ra nhưng vẫn không hiểu vì sao. Nhưng nếu bạn đo cái bếp — chỉ có hai bếp lửa, một tủ lạnh nhỏ, và khách phải có đồ ăn trong tám phút — thì gần như cả thực đơn tự giải thích. Món nào cần hầm ba tiếng thì phải hầm sẵn từ sáng: công sức không mất đi, nó chỉ chuyển sang lúc khác.

Đọc một hệ thống máy tính lạ cũng vậy. Bài toán gốc là bạn có vài trăm nghìn dòng mã cùng một bài báo dài, còn thời gian thì chỉ một buổi. Mẹo là đừng bắt đầu từ ý tưởng, hãy bắt đầu từ những giới hạn mà người làm ra nó không thể vượt qua: máy phải trả lời trong bao nhiêu phần nghìn giây, có sẵn bao nhiêu dữ liệu đã được đánh dấu, bộ nhớ bao nhiêu. Những giới hạn ấy gạt bỏ phần lớn cách làm khác trước khi chuyện hay dở kịp được bàn tới.

Cái giá phải trả là bạn buộc phải đọc kỹ đúng phần trông nhàm chán nhất, và để dành phần hấp dẫn nhất tới cuối.

Chỗ không rút gọn được: chuyện “bỏ bộ phận nào ra thì máy vẫn chạy, nên bộ phận đó vô dụng” thì không rút gọn được mà vẫn đúng. Nếu hai bộ phận cùng làm một việc, bỏ riêng từng cái đều không thấy mất gì, mà bỏ cả hai thì máy đứng. Mọi cách nói ngắn hơn điều này đều dẫn tới kết luận sai.

Một câu: Muốn hiểu một cái máy lạ, hãy đo cái hộp mà người ta buộc phải nhét nó vào, đừng nghe câu chuyện về nó.

1.8 Kết nối

Chương này là tiền đề của cả Phần B; mỗi chương sau là một lát cắt chi tiết của một trong bảy câu hỏi.

- **Câu 2 (đầu vào / đầu ra / giám sát)** → [B/05-thiet-ke-bai-toan](#): ở đó việc chọn đơn vị phân tích và dạng đầu ra khoá chặt mọi thứ phía sau.
- **Câu 3 (ràng buộc cứng)** → [B/11-gioi-han-tinh-toan](#): ngân sách mili-giây trong ví dụ trên được thay bằng mô hình *roofline* nghiêm túc [10].
- **Câu 4 (độ khó bị đẩy đi đâu)** → [B/04-nen-tang-hoc-tu-du-lieu](#): ở đó nó mang tên chính thức là trục đánh đổi giả định $\leftrightarrow$ dữ liệu.
- **Câu 5 (ablation đáng tin không)** → chương 24 ở Phần E.
- **Hai khối đầu của Hình 1.1** → [A/02-vat-ly-tao-anh](#), tiền đề vật lý của chúng.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao ràng buộc cứng loại bỏ nhiều phương án thiết kế hơn cả phần method cộng lại, và điều đó gợi ý gì về thứ tự đọc một paper?
2. Trong một repo bạn đã đọc, độ khó đã bị đẩy đi đâu — và bạn kiểm chứng bằng bằng chứng nào trong code?
3. Bảng ablation che giấu điều gì mà việc đọc code lộ ra, và trong tình huống cụ thể nào thì nó kết luận sai?
4. Vì sao nên đọc phần limitations trước phần kết quả, chứ không phải sau?
5. Nếu ngân sách độ trễ bị siết từ 33 ms xuống 8 ms mỗi khung hình, khối nào trong Hình 1.1 bị thay đầu tiên, và vì sao không phải khối mô hình?
6. Quy trình bảy câu hỏi hổng ở loại công trình nào, và khung đọc nào thay thế nó ở đó?

Nguồn

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *Rec-Sys*, 2019.
- [3] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [4] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [5] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [6] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [7] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [8] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [9] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [10] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [11] V. N. Vapnik, “An overview of statistical learning theory,” *IEEE Trans. Neural Networks*, vol. 10, no. 5, pp. 988–999, 1999.
- [12] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [13] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [15] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [16] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [17] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [18] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE Trans. Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.
- [19] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.

- [20] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [21] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [22] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [23] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [24] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [25] R. Geirhos, J.-H. Jacobsen, C. Michaelis, et al., “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.
- [26] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [27] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [28] T. Gebru, J. Morgenstern, et al., “Datasheets for Datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [29] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [30] M. Mitchell, S. Wu, et al., “Model Cards for Model Reporting,” in *FAT**, 2019.
- [31] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [32] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [33] C. G. Northcutt, L. Jiang, and I. L. Chuang, “Confident Learning: Estimating Uncertainty in Dataset Labels,” *JAIR*, vol. 70, pp. 1373–1411, 2021.
- [34] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [35] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [36] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [37] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [38] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [39] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [40] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [41] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London: Butterworths, 1979.
- [42] B. W. Matthews, “Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme,” *Biochimica et Biophysica Acta (BBA) — Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
- [43] D. Chicco and G. Jurman, “The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation,” *BMC Genomics*, vol. 21, p. 6, 2020.

- [44] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in *International Conference on Machine Learning (ICML)*, 2006, pp. 233–240.
- [45] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.
- [46] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [47] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [48] M. P. Naeni, G. F. Cooper, and M. Hauskrecht, "Obtaining Well Calibrated Probabilities Using Bayesian Binning," in *AAAI Conference on Artificial Intelligence*, 2015.
- [49] G. W. Brier, "Verification of Forecasts Expressed in Terms of Probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
- [50] A. H. Murphy, "A New Vector Partition of the Probability Score," *Journal of Applied Meteorology*, vol. 12, no. 4, pp. 595–600, 1973.
- [51] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [52] T.-Y. Lin, M. Maire, S. Belongie, et al., "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.
- [53] J. Hosang, R. Benenson, P. Dollár, et al., "What Makes for Effective Detection Proposals?" *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 4, pp. 814–830, 2016.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] F. Perazzi, J. Pont-Tuset, B. McWilliams, et al., "A Benchmark Dataset and Evaluation Methodology for Video Object Segmentation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [56] G. Csurka, D. Larlus, and F. Perronnin, "What Is a Good Evaluation Measure for Semantic Segmentation?" In *British Machine Vision Conference (BMVC)*, 2013.
- [57] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [58] K. Mikolajczyk and C. Schmid, "A Performance Evaluation of Local Descriptors," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 10, pp. 1615–1630, 2005.
- [59] M. Dusmanu, I. Rocco, T. Pajdla, et al., "D2-Net: A Trainable CNN for Joint Description and Detection of Local Features," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [60] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [61] S. Sagawa, P. W. Koh, T. B. Hashimoto, et al., "Distributionally Robust Neural Networks for Group Shifts," in *ICLR*, 2020.
- [62] M. Arjovsky, L. Bottou, I. Gulrajani, et al., "Invariant Risk Minimization," *arXiv:1907.02893*, 2019.
- [63] I. Gulrajani and D. Lopez-Paz, "In Search of Lost Domain Generalization," in *ICLR*, 2021.
- [64] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," in *ICLR*, 2019.

- [65] R. Taori, A. Dave, V. Shankar, et al., “Measuring Robustness to Natural Distribution Shifts in Image Classification,” in *NeurIPS*, 2020.
- [66] D. Hendrycks and K. Gimpel, “A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks,” in *ICLR*, 2017.
- [67] Y. Ovadia, E. Fertig, et al., “Can You Trust Your Model’s Uncertainty?” In *NeurIPS*, 2019.
- [68] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [69] A. Kendall, Y. Gal, and R. Cipolla, “Multi-Task Learning Using Uncertainty to Weigh Losses,” in *CVPR*, 2018.
- [70] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [71] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [72] F. Milletari, N. Navab, and S.-A. Ahmadi, “V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation,” in *3DV*, 2016.
- [73] H. Rezatofighi, N. Tsoi, J. Gwak, et al., “Generalized Intersection over Union,” in *CVPR*, 2019.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [76] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [77] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [78] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [79] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [80] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [81] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [82] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [83] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [84] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [85] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [86] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [87] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [88] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [89] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [90] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.

- [91] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [92] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [93] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [94] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [95] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [96] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [97] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [98] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [99] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [100] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [101] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.

Nền tảng học từ dữ liệu (Foundations of Learning from Data)

PHỤ THUỘC B/03-giai-phau-he-thong cho khung “bài toán → nguyên lý → cài đặt”. Nguyên lý “độ khó không biến mất” ở chương đó, tại đây có tên chính thức là trục đánh đổi giả định ↔ dữ liệu. Nền toán: xác suất cơ bản và chuẩn vector.

TÓM TẮT

Chương này giải thích khe hở duy nhất mà từ đó mọi vấn đề của học máy sinh ra: ta tối ưu một trung bình trên mẫu hữu hạn nhưng muốn tốt trên cả phân phối thật. Nó đi từ *ERM* qua thất bại của *KNN* trên pixel, qua **lời nguyền số chiều** (*curse of dimensionality*) và **giả thuyết đa tạp** (*manifold hypothesis*), tới bốn nguồn của *inductive bias*, tới *double descent*, và kết ở *identifiability*. Đọc xong bạn trả lời được câu hỏi đầu tiên nhất trong mọi dự án, bằng phép đo chứ không bằng cảm tính: mô hình chưa đủ tốt thì nên thêm dữ liệu, thêm giả định, hay thêm **sức chứa** (*capacity*)? Bạn cũng nhận ra được những bài toán mà thêm dữ liệu chỉ là ném tiền qua cửa sổ. Nếu chỉ nhớ một điều: bạn không chọn được “không có giả định”, bạn chỉ chọn giả định nào, và trục duy nhất là *giả định mạnh với ít dữ liệu* đối lại *giả định yếu với nhiều dữ liệu*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình, huấn luyện, suy luận** — mô hình là một hàm f có tham số, nhận đầu vào x và trả về dự đoán $f(x)$. Huấn luyện là đi tìm bộ tham số làm dự đoán sát nhất trên dữ liệu đã có; suy luận là chạy hàm đó trên dữ liệu mới. Cả chương này nói về khoảng cách giữa hai việc đó.
- **phân phối dữ liệu và giả định i.i.d.** — “phân phối” là quy luật xác suất sinh ra dữ liệu bạn sẽ gặp — ảnh nào hay xuất hiện, nhãn nào đi kèm. Giả định i.i.d. (*independent and identically distributed*) nói rằng mỗi mẫu được rút độc lập từ cùng một phân phối; đây là giả định ngầm của gần như mọi kết quả trong chương.
- **hàm mất mát và rủi ro (loss, risk)** — hàm mất mát ℓ đo một dự đoán sai bao nhiêu so với nhãn của một mẫu. Rủi ro là kỳ vọng của hàm mất mát trên toàn phân phối — tức mức sai trung bình trên thế giới thật. Ta muốn tối thiểu rủi ro nhưng chỉ đo được trung bình trên mẫu đã có.
- **tham số và sức chứa (capacity)** — tham số là các con số bên trong mô hình mà huấn luyện điều chỉnh. Sức chứa là mức độ phong phú của tập hàm mà mô hình biểu diễn được — càng nhiều tham số và càng ít ràng buộc thì sức chứa càng lớn, và mô hình càng dễ khớp mọi thứ, kể cả nhiễu.
- **overfitting và underfitting** — overfitting là khi mô hình khớp rất tốt dữ liệu huấn luyện nhưng kém trên dữ liệu mới — nó học thuộc chi tiết riêng của mẫu. Underfitting là khi nó kém ở cả hai, vì sức chứa hoặc thời gian huấn luyện chưa đủ. Phân biệt hai trạng thái này quyết định bạn nên thêm dữ liệu hay thêm mô hình.
- **bias** — ba nghĩa khác nhau — (a) trong thống kê, độ lệch hệ thống giữa kỳ vọng của bộ ước lượng và giá trị thật — đây là nghĩa dùng trong cụm “bias-variance”; (b) *inductive bias*, tập giả định mà mô hình mang sẵn về dạng lời giải, nghĩa dùng nhiều nhất trong chương này; (c) tham số cộng thêm b trong $Wx + b$ của một tầng tuyến tính — nghĩa này *không* xuất hiện ở đây. Khi đọc, luôn hỏi đang là nghĩa nào.
- **vector, chuẩn L_2 , số chiều** — một ảnh xám 32×32 được xem như một vector 1024 chiều bằng cách xếp các pixel thành một dãy số. Chuẩn L_2 của hiệu hai vector, $\|A - B\|_2 = \sqrt{\sum_i (A_i - B_i)^2}$, là

khoảng cách Euclid quen thuộc mở rộng lên nhiều chiều — “số chiều” ở đây chỉ đơn giản là số pixel.

- **SGD và batch** — thuật toán huấn luyện phổ biến nhất là *stochastic gradient descent*: lặp đi lặp lại việc lấy một nhóm nhỏ mẫu (một *batch*), tính hướng làm giảm hàm mất mát trên nhóm đó, rồi dịch tham số một bước theo hướng ngược lại. Chương này chỉ cần bạn biết rằng lựa chọn thuật toán tối ưu cũng là một giả định, không cần biết chi tiết cơ chế.

CÂU HỎI MỞ ĐẦU

- Q1. Nếu KNN trên pixel thất bại thảm hại còn KNN trên đặc trưng DINOv3 lại rất mạnh, mà thuật toán phân loại không đổi một dòng, thì “trí tuệ” của hệ thống thực ra nằm ở đâu?
- Q2. Một ảnh xám trông rõ ràng lại gần một cạnh dọc hơn chính cạnh đó dịch một pixel — vậy thứ gì đã hỏng trong khái niệm “khoảng cách”?
- Q3. Giáo trình nói tăng sức chứa thì sai số kiểm thử phải xấu đi. Vì sao những mạng có nhiều tham số hơn cả số mẫu vẫn tổng quát hoá tốt?
- Q4. Có bài toán nào mà gán nhãn thêm một triệu ảnh cũng không cải thiện được gì không, và làm sao biết trước điều đó thay vì biết sau khi đã tiêu tiền?

2.1 Đặt vấn đề

Bạn tối ưu trên một tập hữu hạn nhưng muốn hệ thống hoạt động tốt trên thế giới. Khoảng cách đó lấp bằng gì? Câu hỏi nghe hiển nhiên tới mức dễ bị bỏ qua, nhưng nó là khe hở duy nhất mà từ đó *mọi* vấn đề của ngành sinh ra: *overfitting*, *domain gap*, *shortcut learning*, mô hình tốt trên *benchmark* mà vô dụng khi triển khai. Hãy hiểu chính xác khe hở này rộng bao nhiêu và cái gì thu hẹp nó lại. Khi đó phần lớn kỹ thuật ở Phần C và Phần D không còn là một danh sách mẹo rời rạc. Chúng trở thành các câu trả lời khác nhau cho cùng một câu hỏi.

Chương này khác chương 3 ở chỗ nó không dạy cách đọc mà dạy cách *dự đoán*: vì sao một giả định lại giúp ích, và trong hoàn cảnh nào chính nó quay ra làm hại. Đó là điều kiện cần để trả lời câu hỏi thực dụng nhất trong mọi dự án — mô hình chưa đủ tốt, nên thêm dữ liệu, thêm giả định, hay thêm sức chứa?

Có một lý do riêng để một kỹ sư SLAM đọc kỹ chương này. Trong hình học đa tầm nhìn, bạn biết chính xác khi nào bài toán suy biến vì **ma trận thông tin** (*information matrix*) mất hạng. Học từ dữ liệu không cho sự rõ ràng đó miễn phí. Nhưng khái niệm identifiability ở cuối chương chính là bản dịch của trực giác “suy biến” sang ngôn ngữ thống kê. Nó cho bạn quyền nói “thêm dữ liệu sẽ không cứu được” bằng một lập luận, thay vì bằng một linh cảm.

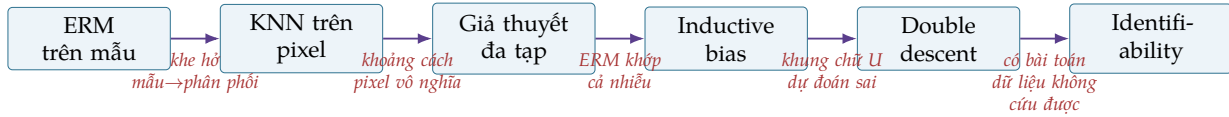
TRỰC GIÁC

Huấn luyện là chụp ảnh một địa hình từ vài trăm điểm quan sát rồi dựng lại toàn bộ bản đồ. Dữ liệu cho bạn các điểm; inductive bias là giả thiết về địa hình *giữa* các điểm — nó phẳng, nó liên tục, nó không đổi khi dịch ngang. Không có giả thiết nào thì có vô hạn địa hình đi qua đúng những điểm ấy và bạn không chọn được cái nào; giả thiết sai thì bạn dựng ra một bản đồ trơn tru, tự tin, và sai ở đúng những chỗ bạn chưa nhìn tới.

2.2 Từ ERM tới no free lunch (From ERM to No Free Lunch)

A. Bản đồ của mạch trước khi đi vào từng bước

Mạch dưới đây có tám bước, và mỗi bước tồn tại vì bước trước để lại đúng một nút thắt. Hình 2.1 cho thấy chuỗi nút thắt đó trước, để bạn đọc từng bước với bối cảnh sẵn có thay vì phải chờ tới cuối mới thấy hình dạng tổng thể.



Hình 2.1. Chuỗi nút thắt của chương. Mỗi khái niệm ra đời để gỡ nút thắt ghi dưới mũi tên đi vào nó; đọc theo chiều này thì không khái niệm nào là tùy tiện.

B. Tám bước

I Mạch từ ERM tới no free lunch

1 Vấn đề

cái ta muốn tối thiểu thì không đo được. Ta muốn tối thiểu **rủi ro kỳ vọng** (expected risk) $R(f) = \mathbb{E}_{(x,y) \sim P} [\ell(f(x), y)]$ trên phân phối thật P , nhưng chỉ có n mẫu rút từ nó.

Giải pháp

tối thiểu hoá rủi ro thực nghiệm (empirical risk minimization): tối thiểu $\hat{R}(f) = \frac{1}{n} \sum_i \ell(f(x_i), y_i)$ và hy vọng nó xấp xỉ $R(f)$ [11]. Công thức chỉ nói một điều: ta thay một tích phân không tính được bằng một trung bình cộng đếm được.

Giả định

mẫu đại diện cho phân phối, và phân phối lúc triển khai vẫn là P .

Giá phải trả

mọi vấn đề tổng quát hoá của ngành sinh ra từ khe hở giữa $\hat{R}$ và R .

↪ **đối lập với B/08** — chương dịch chuyển phân phối bỏ đúng giả định i.i.d. mà ERM dựa vào, nên mọi cận ở đây mất hiệu lực ở đó

2 Vấn đề

liệu bộ phân loại có phải chỗ đáng đầu tư không?

Giải pháp

chạy thí nghiệm mở màn rẻ nhất: **k láng giềng gần nhất** (k -nearest neighbours) trên pixel thô, với khoảng cách L_1 hoặc L_2 . Nó không huấn luyện gì và đồn toàn bộ chỉ phí sang lúc suy luận. Trên ảnh, nó thất bại thảm hại.

Hệ quả

bài học không phải “KNN dở”, vì cùng thuật toán ấy chạy trên đặc trưng của một mô hình **tự giám sát** (self-supervised) lại rất mạnh [12], [13]. Thất bại nằm ở **biểu diễn** (representation): khoảng cách pixel không tương ứng với khoảng cách ngữ nghĩa, và toàn bộ deep learning cho thị giác là nỗ lực học một không gian mà ở đó khoảng cách mới có nghĩa.

3 Vấn đề

vì sao khoảng cách pixel lại vô nghĩa đến vậy?

Giải pháp

lời nguyên số chiều: trong không gian nhiều nghìn chiều, thể tích tập trung ở vỏ ngoài và mọi cặp điểm gần như cách đều nhau, nên “láng giềng gần nhất” mất phần lớn nội dung thông tin.

Vấn đề mới

nếu vậy thì học trong không gian đó lẽ ra phải bất khả thi, mà thực tế thì không.

Giải pháp

giả thuyết đa tạp: dữ liệu thật không rải đều khắp không gian mà nằm gần một đa tạp số chiều thấp hơn nhiều nhúng trong đó [14].

4 Vấn đề

ERM một mình sẽ khớp cả nhiều. Một mạng đủ lớn học thuộc được cả nhãn ngẫu nhiên, nên đặt $\hat{R} = 0$ không nói gì về R [15].

Giải pháp

thêm **inductive bias** — giả định về lời giải, thứ thu hẹp không gian hàm *trước khi* dữ liệu lên tiếng.

Giả định

giả định đó đúng với bài toán của bạn.

- Hệ quả nhận thức then chốt: bạn không thể “không có giả định”; bạn chỉ chọn giả định nào, và việc không nói ra không làm nó biến mất.
- 5 Vấn đề** nên đặt bao nhiêu giả định?
- Giải pháp** nhìn nó như một trục liên tục: **giả định mạnh + ít dữ liệu** ở một đầu, **giả định yếu + nhiều dữ liệu** ở đầu kia.
- Giả định** bạn ước lượng đúng vị trí mình đang đứng trên trục đó, mà vị trí ấy do lượng dữ liệu quyết định chứ không do sở thích.
- Hệ quả** đây chính là nguyên lý “độ khó bị đẩy đi chỗ khác” của chương 3, lần này có tên và có trục. Bạn sẽ gặp lại nó nguyên vẹn hai lần nữa: ở CNN so với **ViT** tại chương 13, và ở hình học cổ điển so với mô hình **tiền thẳng** (*feed-forward*) tại chương 16.
- ↔ **tiền đề** C/13 — CNN đứng ở đầu giả định mạnh, ViT ở đầu giả định yếu. Đó là toàn bộ lý do ViT cần nhiều dữ liệu hơn
- 6 Vấn đề** khung lý thuyết cổ điển dự đoán sai với mạng lớn.
- Giả định** khung **bias–variance**: tăng sức chứa thì độ lệch hệ thống giảm nhưng kết quả dao động mạnh hơn giữa các lần huấn luyện, và sai số kiểm thử có dạng chữ U.
- Vấn đề mới** quan sát mâu thuẫn — mạng có nhiều tham số hơn cả số mẫu huấn luyện vẫn tổng quát hoá tốt.
- Giải pháp** **double descent**: vượt qua **ngưỡng nội suy** (*interpolation threshold*), sai số kiểm thử tăng vọt rồi *giảm trở lại* [16], [17].
- Hệ quả** đừng suy luận về overfitting bằng trực giác lấy từ giáo trình cũ; hãy đo.
- 7 Vấn đề** có những bài toán mà thêm dữ liệu không cứu được.
- Giải pháp** khái niệm identifiability: nếu nhiều nghiệm khác nhau sinh ra cùng một quan sát thì mọi lượng dữ liệu đều vô ích.
- Hệ quả** hành động đúng không phải thêm mẫu mà là đổi bài toán, thêm ràng buộc, hoặc thêm cảm biến.
- 8 Vấn đề** vậy cuối cùng nên chọn kiến trúc nào?
- Hệ quả** kết luận của cả chương là **no free lunch**: lấy trung bình trên mọi bài toán có thể, không thuật toán nào tốt hơn thuật toán nào [18]. Ở dạng nghiêm ngặt, định lý nói về một không gian bài toán mà thế giới thật không lấy mẫu từ đó, nên đừng dùng nó để phủ nhận việc so sánh. Ở dạng thực dụng: không có kiến trúc tốt nhất, chỉ có kiến trúc mà giả định của nó khớp với dữ liệu của bạn.

2.3 Vì sao khoảng cách pixel vô nghĩa (Why Pixel Distance Fails)

A. Một phép tính 4×4 làm được trên giấy

Bước hai của mạch trên dễ bị chấp nhận như một khẩu hiệu, nên hãy tính tay một ví dụ đủ nhỏ. Xét “ảnh” 4×4 một kênh.

- **Ba ảnh dùng trong ví dụ:**

- *A* — hai cột trái bằng 1, hai cột phải bằng 0: một cạnh dọc.
- *B* — chính *A* dịch sang phải một pixel (cột 2, 3 bằng 1). Về ngữ nghĩa, *A* và *B* là cùng một vật.

- G — ảnh xám đồng nhất, mọi pixel bằng 0,5: không cạnh, không cấu trúc, không vật thể.

Hiệu $A - B$ khác không ở cột 1 và cột 3, mỗi cột bốn pixel với độ lệch 1; còn $A - G$ lệch 0,5 ở cả mười sáu pixel. Do đó

$$\|A - B\|_2 = \sqrt{4 \cdot 1^2 + 4 \cdot 1^2} = \sqrt{8} \approx 2,83, \quad \|A - G\|_2 = \sqrt{16 \cdot 0,5^2} = 2,00.$$

Hai con số này nói một điều rất khó chịu: *ảnh xám trông rõ ràng gần A hơn chính A dịch một pixel*. Đây không phải tình huống bệnh lý được dựng lên. Dịch chuyển, đổi độ sáng và đổi nền là ba biến đổi phổ biến nhất trong ảnh thật. Cả ba đều giữ nguyên nội dung nhưng làm khoảng cách pixel nhảy vọt.

B. Lờn nguyên số chiều bằng con số

Với ảnh CIFAR-10, không gian có $32 \times 32 \times 3 = 3072$ chiều. Muốn phủ khối lập phương đơn vị trong không gian đó bằng một lưới rất thô, mười mức mỗi trục, bạn cần 10^{3072} điểm — một con số không có nghĩa vật lý nào. Ý nghĩa của phép tính: không tập dữ liệu nào “phủ” được không gian pixel, nên láng giềng gần nhất trong đó không bao giờ thực sự gần, và mọi phương pháp dựa vào khoảng cách thô đều đang đo một đại lượng gần như ngẫu nhiên. Đây cũng chính là lý do các công cụ chiếu xuống chiều thấp như **UMAP** [19] chỉ hữu ích khi áp lên *đặc trưng đã học*, không phải lên pixel.

2.4 Bốn nguồn của inductive bias (Four Sources of Inductive Bias)

Nói “thêm giả định” thì dễ, nhưng giả định được cài vào hệ thống ở đúng bốn chỗ, và biết nó nằm ở chỗ nào quyết định bạn sửa cái gì khi nó sai.

• Bốn nguồn, kèm giả định cụ thể mà mỗi nguồn phát biểu:

- **Kiến trúc** — *convolution* phát biểu hai điều: đặc trưng hữu ích thì cục bộ, và ý nghĩa của một mẫu hình không đổi khi nó dịch vị trí. Đó là lý do CNN học được từ vài chục nghìn ảnh trong khi mạng toàn kết nối cần nhiều hơn hẳn.
- **Hàm mất mát** — *sai số bình phương* (*squared error*) ngầm giả định nhiễu Gauss đồng nhất; *cross-entropy* ngầm giả định đầu ra là một phân phối phạm trù. Chọn loss là chọn mô hình nhiễu, dù bạn có nghĩ tới điều đó hay không.

↪ **trường hợp riêng của B/09** — cả chương thiết kế hàm mất mát là việc chọn một nguồn inductive bias duy nhất trong bốn nguồn ở đây

- **Dữ liệu** — mỗi phép **tăng cường dữ liệu** (*data augmentation*) là một phát biểu “nhân không đổi dưới phép này”. Lật ngang là phát biểu về đối xứng trái phải của thế giới, và nó sai ngay khi bài toán cần đọc chữ hoặc phân biệt tay trái với tay phải.
- **Thuật toán tối ưu** — *SGD* không dừng ở một nghiệm bất kỳ trong tập nghiệm; có bằng chứng thực nghiệm rằng *batch* lớn dẫn tới nghiệm “nhon” hơn kèm tổng quát hoá kém hơn [20]. Cơ chế vẫn còn tranh cãi, nên hãy coi đây là quan sát thực nghiệm, không phải định luật.

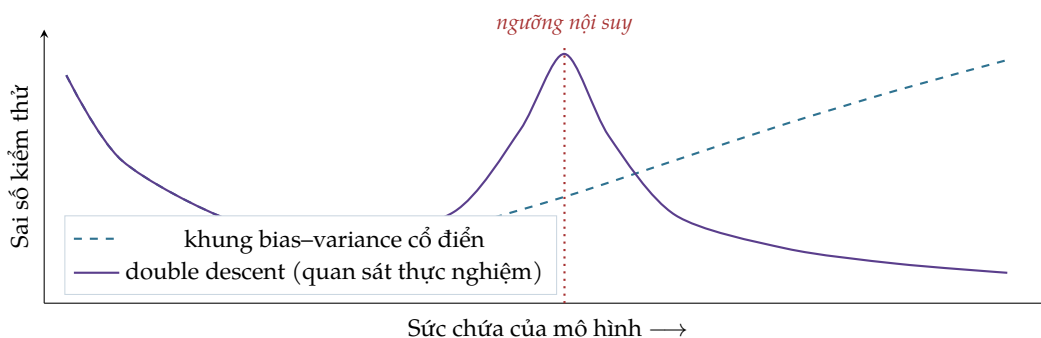
Bốn nguồn này không tương đương về chi phí sửa chữa, và đó mới là thông tin dùng được khi chẩn đoán (Bảng 2.1).

Bảng 2.1. Bốn nguồn inductive bias xếp theo chi phí sửa khi bạn phát hiện đã đặt sai giả định. Chẩn đoán nên đi từ trên xuống.

Nguồn	Giả định nó phát biểu	Chi phí sửa	Hòng khi nào
Augmentation	Nhân bất biến dưới một nhóm phép biến đổi	Một buổi, huấn luyện lại	Khi phép biến đổi thật sự đối nhân (chữ, trái/phải, màu chẩn đoán)
Thuật toán tối ưu	Nghiệm “phẳng” tổng quát hoá tốt hơn	Vài lần chạy để dò lại	Khi thay đổi batch size hoặc lịch học làm dịch nghiệm mà không ai để ý
Hàm mất mát	Dạng phân phối của nhiễu và của đầu ra	Huấn luyện lại, thường phải sửa cả metric	Khi nhiễu lệch nặng, hoặc khi lớp mất cân bằng cực đoan
Kiến trúc	Cấu trúc không gian của tín hiệu (cục bộ, đẳng biến tịnh tiến)	Thiết kế lại và huấn luyện lại từ đầu	Khi quan hệ tầm xa mới là thứ quyết định, hoặc dữ liệu không có cấu trúc lưới

2.5 Chữ U cũ và hai lần đi xuống (Bias–Variance vs Double Descent)

Hình 2.2 đặt cạnh nhau điều giáo trình cổ điển dự đoán và điều thường quan sát được với mạng lớn. Cần đọc hình này cẩn thận, vì nó rất dễ bị hiểu thành “cứ tăng mô hình là tốt”.



Hình 2.2. Hai dự đoán về sai số kiểm thử khi sức chứa tăng. Trước ngưỡng nội suy — điểm mà mô hình vừa đủ sức khớp toàn bộ tập huấn luyện — hai khung trùng nhau; sau ngưỡng đó chúng đi ngược chiều. Trục không có đơn vị: đây là hình minh hoạ dạng hàm, không phải một phép đo.

Điều đáng rút ra không phải là “mô hình càng lớn càng tốt” — vùng nguy hiểm nhất trong hình là quanh ngưỡng nội suy, và một dự án ngân sách hẹp rất dễ dừng lại đúng ở đó. Điều đáng rút ra là lời khuyên cổ điển “train loss thấp hơn test loss nhiều thì phải giảm mô hình” không còn là suy luận hợp lệ; nó là một giả thuyết phải kiểm chứng bằng cách quét kích thước mô hình. Cũng cần nói thẳng rằng double descent hiện rõ trong một số cấu hình và mờ nhạt hoặc vắng mặt trong nhiều cấu hình khác, nên đừng trông đợi thấy nó trên mọi bài toán.

2.6 Khi thêm dữ liệu là vô ích (Identifiability)

Ví dụ sạch nhất nằm ngay trong lĩnh vực bạn đã làm. Ước lượng độ sâu từ một ảnh đơn: nhân toàn bộ cảnh và toàn bộ khoảng cách camera với cùng một hệ số $s > 0$ thì ảnh thu được giống hệt. Nói gọn thì phép chiếu đã xoá thông tin về tỉ lệ tuyệt đối, nên không hàm nào nhận ảnh làm

đầu vào phục hồi được tỉ lệ đó từ hình học thuần túy — thêm một triệu ảnh nữa cũng không đổi. Đó là lý do các công trình độ sâu đơn ảnh dùng hàm mất mát bất biến theo tỉ lệ, hoặc theo cả tỉ lệ lẫn dịch chuyển, thay vì hồi quy độ sâu tuyệt đối [21], [22].

Ở đây cần *phức tạp hoá* thay vì cắt gọn, vì bức tranh thật sự không đơn giản. Các mô hình đơn ảnh vẫn đoán được tỉ lệ ở mức nào đó — nhưng nhờ tiên nghiệm về kích thước quen thuộc của vật thể (người cao chừng ấy, xe rộng chừng ấy) chứ không nhờ hình học. Nói cách khác, phần không định danh được về mặt hình học được lấp bằng một inductive bias học từ dữ liệu, và bias ấy sập ngay khi cảnh chứa vật thể lạ kích thước. Vì vậy câu “mô hình này ước lượng được độ sâu tuyệt đối” vừa đúng vừa sai, và biết nó đúng theo nghĩa nào là điều kiện để dùng nó an toàn. Hệ SLAM đơn camera mang đúng tính chất này: bản đồ và quỹ đạo chỉ xác định tới một hệ số tỉ lệ. Cách xử lý đúng không phải thu thập thêm ảnh mà là thêm một phép đo khác: IMU, *stereo baseline*, hoặc một vật mốc đã biết kích thước [23].

↔ **tiền đề B/05** — learning curve phẳng có đúng hai nguyên nhân: hết tín hiệu trong dữ liệu, hoặc bài toán vốn không identifiable. Phân biệt được chúng thì mới biết nên tiêu tiền vào đâu

2.7 Ba đòn bẩy khi mô hình chưa đủ tốt (Three Levers)

Toàn bộ chương quy về một quyết định lặp đi lặp lại trong mọi dự án. Bảng 2.2 là dạng vận hành được của nó; cột cuối đáng đọc nhất vì nó cho bạn dấu hiệu nhận ra mình đang kéo nhầm đòn bẩy trước khi tiêu hết ngân sách.

Bảng 2.2. Ba nguồn cải thiện khi mô hình chưa đủ tốt, và dấu hiệu bạn đang chọn sai.

Đòn bẩy	Giả định ngầm	Hiệu quả nhất khi	Hỏng khi nào
Thêm dữ liệu	Bài toán identifiable và nhân nhất quán	<i>learning curve</i> theo kích thước dữ liệu chưa bão hoà	Curve đã phẳng từ lâu mà vẫn tiếp tục thu thập
Thêm giả định (kiến trúc, loss, augmentation)	Bạn thật sự biết một tính chất đúng của bài toán	Dữ liệu ít, tri thức miền rõ ràng	Thêm bias sai làm mô hình tệ hơn cả baseline
Thêm sức chứa	Đang <i>underfit</i> chứ không phải thiếu tín hiệu	Train loss còn cao và còn giảm đều	Train loss đã gần 0 mà vẫn phóng to mô hình

Ba đòn bẩy cạnh tranh cùng một ngân sách, nên có một thứ tự chẩn đoán rõ nhất:

1. Nhìn train loss để loại hoặc xác nhận underfit. Đây là bước rõ nhất vì bạn đã có sẵn số.
2. Vẽ learning curve theo kích thước dữ liệu để biết mình còn ở đoạn dốc hay đã bão hoà — quy trình cụ thể nằm ở [B/05-thiet-ke-bai-toan](#).
3. Chỉ khi cả hai bước trên đều nói “không” mới đặt câu hỏi về giả định, vì đó là đòn bẩy đòi hỏi hiểu biết miền và cũng là đòn bẩy dễ dùng sai nhất.

2.8 Thực hành

Chương này trừu tượng hơn các chương khác, nên bài tập được thiết kế để biến một mệnh đề trừu tượng — “vấn đề nằm ở biểu diễn” — thành một con số bạn tự đo được.

Repo và tài nguyên. [scikit-learn](#) cho KNN và baseline cổ điển; [facebookresearch/dinov3](#) để lấy đặc trưng tự giám sát mạnh làm đối chứng; [faiss](#) khi vét cạn không còn khả thi; [umap-learn](#) để nhìn cấu trúc đa tạp bằng mắt [19]. Sách nền: Goodfellow và cộng sự [14] cho manifold hypothesis, Murphy [24] cho phần thống kê. Tên mô hình nền tảng chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; [scikit-learn](#) và [faiss](#) bền hơn nhiều, nên hãy học giao diện chứ đừng học tên *checkpoint*.

Câu hỏi kiểm tra hiểu. Vì sao KNN trên pixel thất bại còn KNN trên đặc trưng DINOv3 lại rất mạnh, dù thuật toán phân loại không đổi một dòng — điều đó nói gì về vị trí thật sự của “trí tuệ” trong hệ thống? Double descent phá vỡ lời khuyên cổ điển nào mà có thể bạn vẫn đang áp dụng theo phản xạ? Nếu ai đó đề nghị gán nhãn thêm mười nghìn ảnh để cải thiện một mô hình ước lượng độ sâu tuyệt đối từ ảnh đơn, bạn phản biện thế nào trong ba câu?

Bài tập phụ. Nếu muốn một phiên bản không dính tới SLAM để khởi động, chạy KNN trên CIFAR-10 ở ba biểu diễn: pixel thô, *HOG*, và đặc trưng DINOv3 đóng băng. Sau đó vẽ độ chính xác theo k cho cả ba trên cùng một trục. Mini project SLAM ở cuối chương là phiên bản nghiêm túc của chính thí nghiệm này, trên dữ liệu của bạn.

2.9 Giới hạn và trường hợp hỏng

- **Giả định i.i.d. gần như luôn sai ở mức độ nào đó.** Mọi lập luận về tổng quát hoá trong chương này sụp đổ *hoàn toàn* khi phân phối dịch chuyển, chứ không suy giảm dần dần. Cả chương [B/08-dịch-chuyen-phan-phoi](#) tồn tại để xử lý phần sai đó.
- **Khung này im lặng về shortcut learning**, một chế độ hỏng nguy hiểm hơn overfitting. Mô hình có thể đạt $\hat{R}$ rất thấp và tổng quát hoá tốt trên tập kiểm thử cùng phân phối. Nó vẫn có thể đang dựa vào một tương quan giả: nền ảnh, *watermark*, hay kết cấu thay vì hình dạng. Tương quan đó biến mất thì mô hình sập [25]. ERM không phân biệt được đặc trưng nhân quả với đặc trưng chỉ tương quan; nó chỉ tối thiểu một trung bình.
- **Các khái niệm ở đây mô tả giới hạn chứ không phải quy trình.** Chúng nói khi nào một hướng đi là vô vọng, nhưng không nói kiến trúc nào nên thử trước. Hãy dùng chúng để loại bỏ những thí nghiệm không cần chạy.

CẠM BẦY

Cạm bẫy hay gặp nhất là dùng “no free lunch” như một cái cớ để không so sánh. Định lý phát biểu về trung bình trên *mọi* bài toán khả dĩ, kể cả những bài toán mà nhãn hoàn toàn ngẫu nhiên — một không gian mà thế giới thật không lấy mẫu từ đó. Trên lớp bài toán thực tế, có những kiến trúc tốt hơn hẳn những kiến trúc khác một cách ổn định, và câu “không có kiến trúc nào tốt nhất” không cho phép bạn bỏ qua việc đo. Cách dùng đúng: để đặt câu hỏi “giả định của mô hình này khớp với dữ liệu của tôi ở điểm nào”, chứ không phải để kết thúc tranh luận.

CẦN NHỚ

Bạn không chọn được “không có giả định” — bạn chỉ chọn giả định nào, và trực đánh đổi duy nhất là *giả định mạnh với ít dữ liệu* đổi lại *giả định yếu với nhiều dữ liệu*. Trước khi kéo bất kỳ đòn bẩy nào, kiểm tra hai điều theo thứ tự: bài toán có identifiable không (nếu không, mọi dữ liệu thêm vào đều lãng phí), và train loss đã thấp chưa (nếu chưa, bạn đang underfit chứ không thiếu dữ liệu). [T1]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
ERM	Tối thiểu sai số trung bình trên mẫu đã có, rồi hy vọng nó xấp xỉ sai số trên phân phối thật. Đây là thứ mọi thuật toán học có giám sát thực sự làm.
Rủi ro kỳ vọng	Sai số trung bình trên toàn phân phối thật, tức đại lượng ta muốn tối thiểu nhưng không đo được. Khe hở giữa nó và trung bình trên mẫu là nơi mọi vấn đề tổng quát hoá sinh ra.
Giả định i.i.d.	Mỗi mẫu được rút độc lập từ cùng một phân phối, và phân phối lúc triển khai vẫn là phân phối đó. Nó sập thì mọi lập luận về tổng quát hoá mất hiệu lực hoàn toàn chứ không suy giảm dần.
KNN trên pixel	Phép thử mở màn rẻ nhất: phân loại bằng láng giềng gần nhất, không huấn luyện gì. Nó thất bại thảm hại trên ảnh, và cùng thuật toán đó chạy trên đặc trưng tự giám sát lại rất mạnh.
Vì sao khoảng cách pixel không đo được ngữ nghĩa?	Vì nó đo chênh lệch độ sáng ở từng vị trí. Một ảnh xám trông rõ ràng cách một cạnh dọc 2,00 đơn vị, còn chính cạnh đó dịch một pixel thì cách 2,83 đơn vị.
Lời nguyên số chiều	Trong không gian nhiều nghìn chiều, mọi cặp điểm gần như cách đều nhau, nên “láng giềng gần nhất” mất phần lớn nội dung thông tin. Không tập dữ liệu nào phủ nổi không gian pixel.
Giả thuyết đa tạp	Dữ liệu thật không rải đều khắp không gian mà nằm gần một mặt có số chiều thấp hơn nhiều. Nó giải thích vì sao học vẫn khả thi dù lời nguyên số chiều có thật.
Inductive bias	Giả định về dạng lời giải, cài vào trước khi dữ liệu lên tiếng. Nó đến từ đúng bốn nguồn: kiến trúc, hàm mất mát, augmentation, và thuật toán tối ưu.
Augmentation	Một phép biến đổi áp lên ảnh huấn luyện là một phát biểu “nhân không đổi dưới phép này”. Lật ngang sai ngay khi bài toán cần đọc chữ hoặc phân biệt tay trái với tay phải.
Vì sao thuật toán tối ưu cũng là một nguồn giả định?	Vì SGD không dừng ở một nghiệm bất kỳ trong tập nghiệm. Có bằng chứng thực nghiệm rằng batch lớn dẫn tới nghiệm “nhộn” hơn kèm tổng quát hoá kém hơn, dù cơ chế vẫn còn tranh cãi.
Trực giả định — dữ liệu	Một đầu là giả định mạnh với ít dữ liệu, đầu kia là giả định yếu với nhiều dữ liệu. Bạn không chọn được “không có giả định”, chỉ chọn giả định nào và đứng ở đâu trên trục.
Vì sao train loss bằng không không nói gì về tổng quát hoá?	Vì một mạng đủ lớn học thuộc được cả nhãn hoàn toàn ngẫu nhiên. Khả năng khớp dữ liệu huấn luyện không phải bằng chứng về khả năng đoán dữ liệu mới.
Double descent	Vượt qua ngưỡng nội suy, tức điểm mà mô hình vừa đủ sức khớp toàn bộ tập huấn luyện, sai số kiểm thử tăng vọt rồi giảm trở lại. Hệ quả: trực giác chữ U từ giáo trình cũ không còn là suy luận hợp lệ.
Identifiability	Bài toán identifiable là bài toán mà quan sát xác định nghiệm duy nhất. Nếu nhiều nghiệm khác nhau sinh ra cùng một quan sát thì mọi lượng dữ liệu thêm vào đều vô ích.
Vì sao thêm dữ liệu vô ích với độ sâu tuyệt đối từ ảnh đơn?	Nhân cả cảnh và khoảng cách camera với cùng một hệ số thì ảnh thu được giống hệt. Phép chiếu đã xoá tỉ lệ tuyệt đối, nên không hàm nào nhận ảnh làm đầu vào phục hồi được nó.
Ba đòn bẩy	Thêm dữ liệu, thêm giả định, thêm sức chứa. Chúng cạnh tranh cùng một ngân sách, nên phải chần đoán trước khi kéo.
Train loss còn cao và còn giảm đều. Kéo đòn bẩy nào?	Bạn đang underfit, nên thêm sức chứa hoặc huấn luyện lâu hơn. Phép phân biệt: nếu train loss đã gần không mà test vẫn kém thì vấn đề nằm ở dữ liệu hoặc ở giả định, không ở kích thước mô hình.
Tốt trên tập kiểm thử cùng phân phối rồi sập ngoài đời. Nghi ngờ gì?	Shortcut learning: mô hình dựa vào một tương quan giả như nền ảnh, watermark hay kết cấu. Phép phân biệt là dựng một tập kiểm thử phá vỡ đúng tương quan đó.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• No free lunch	Lấy trung bình trên mọi bài toán khả dĩ, không thuật toán nào tốt hơn thuật toán nào. Dùng nó để hỏi giả định của mô hình khớp dữ liệu của bạn ở đâu, không phải để từ chối đo.

• • • phải thuộc • • nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
DINOv2, DINOv3	Bằng chứng trực tiếp cho luận điểm “vấn đề nằm ở biểu diễn”: KNN và probe tuyến tính trên đặc trưng đóng băng đạt mức mà pixel thô không bao giờ với tới	Mô hình dẫn đầu tính tới 9/2026
Scaling law (Kaplan, Chinchilla)	Dạng đo được của trực giả định $\leftrightarrow$ dữ liệu: khi giả định yếu đi thì dữ liệu và tính toán phải tăng theo quy luật lũy thừa để bù	Kết quả đã công bố, dùng để lập ngân sách huấn luyện
MiDaS và dòng ước lượng độ sâu đơn ảnh	Xử lý trực tiếp một bài toán không identifiable bằng cách bỏ tỉ lệ tuyệt đối ra khỏi hàm mất mát thay vì cố học nó	Hạ tầng ước lượng độ sâu, bền
faiss	Tìm láng giềng gần nhất ở quy mô lớn chỉ có nghĩa sau khi không gian đã được học — đây là hiện thực công nghiệp của bước hai trong mạch	Hạ tầng, bền

MINI PROJECT SLAM • Khoảng cách nào mới có nghĩa: ghép cặp mảnh ảnh quanh đặc trưng ORB • 1–2 ngày

Mục tiêu	kiểm chứng luận điểm trung tâm của chương ngay trên dữ liệu của chính bạn: thất bại nằm ở biểu diễn chứ không ở thuật toán. Cách kiểm chứng là giữ nguyên bộ phân loại và chỉ đổi không gian đo khoảng cách.
Dữ liệu	một chuỗi EuRoC bất kỳ. Cho ORB-SLAM3 chạy và xuất ra, với mỗi đặc trưng ORB, một mảnh ảnh 32×32 quanh nó cùng với id của điểm 3D mà nó được gán. Nhân của một <i>cặp</i> mảnh là nhị phân: cùng điểm 3D hay không. Lấy vài chục nghìn cặp, cân bằng dương/âm.
Các bước	chạy KNN trên tập cặp và đo độ chính xác ghép cặp theo k , lần lượt với ba biểu diễn: pixel thô; mô tả từ ORB 256 bit sẵn có, dùng khoảng cách Hamming; và đặc trưng từ một mô hình tự giám sát đóng băng. Chú ý dùng đúng khoảng cách của từng không gian, và chia tập theo <i>chuỗi con</i> chứ không theo cặp, nếu không hai mảnh của cùng một điểm 3D sẽ nằm ở cả hai phía.
Xong khi	bạn có một hình duy nhất với ba đường độ chính xác ghép cặp theo k , và ba câu giải thích chênh lệch trong đó ít nhất một câu nói về <i>khoảng cách</i> chứ không về “mô hình mạnh hơn”. Con số phụ bắt buộc: khoảng cách L_2 trung bình giữa hai mảnh của cùng một điểm 3D, đặt cạnh khoảng cách L_2 trung bình giữa hai mảnh ngẫu nhiên. Hai con số gần nhau nghĩa là bạn vừa tái hiện ví dụ 4×4 ở mục trên, lần này trên dữ liệu thật.
Nối với	B/03 cung cấp datasheet nói cho bạn biết đặc trưng và mô tả từ nằm ở khối nào; B/05 dùng chính tập cặp này làm ví dụ về đơn vị phân tích và rò rỉ theo chuỗi; B/07 cho giao thức chia tập chống rò rỉ.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

A1. Nằm ở biểu diễn, không ở bộ phân loại. Cùng một KNN chạy trên hai không gian khác nhau cho hai kết quả khác hẳn, nên phần tạo ra chênh lệch là ánh xạ từ pixel sang đặc trưng — và đó là thứ deep learning thực sự học.

- A2. Hồng ở chỗ khoảng cách Euclid trên pixel đo sự khác nhau về *giá trị sáng từng vị trí*, không đo sự khác nhau về *nội dung*. Dịch chuyển, đổi độ sáng và đổi nền đều làm nó nhảy vọt trong khi giữ nguyên ngữ nghĩa.
- A3. Vì chữ U cổ điển chỉ mô tả vùng trước ngưỡng nội suy. Sau ngưỡng đó, trong nhiều cấu hình sai số kiểm thử giảm trở lại — hiện tượng double descent — nên khoảng cách train/test lớn không còn tự động là bằng chứng phải thu nhỏ mô hình.
- A4. Có: các bài toán không identifiable, mà ví dụ sạch nhất là tỉ lệ tuyệt đối trong ước lượng độ sâu đơn ảnh. Biết trước bằng cách tìm một phép biến đổi tham số giữ nguyên quan sát; tìm được thì mọi dữ liệu thêm vào đều vô ích, và việc đúng là thêm ràng buộc hoặc thêm cảm biến.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn nấu một nồi canh to và muốn biết đã vừa miệng chưa. Bạn không thể uống hết nồi, nên bạn nếm một thìa. Toàn bộ chuyện máy học từ dữ liệu nằm gọn trong hành động đó: đo trên một thìa rồi kết luận về cả nồi.

Mẹo này chạy được nhờ hai điều. Thứ nhất, bạn khuấy đều trước khi nếm; nếu mức đúng chỗ vừa đổ muối vào thì thìa ấy nói dối, và mọi kết luận sau đó đều hồng chứ không phải chỉ lệch đi một chút. Thứ hai, khi thìa quá nhỏ so với nồi, bạn phải bù bằng hiểu biết có sẵn: bạn biết canh thường mặn ở đáy, biết loại rau này hút muối. Chính phần hiểu biết thêm vào ấy giúp bạn đoán đúng phần nồi mình chưa nếm. Điều quan trọng là bạn không thể chọn “không thêm hiểu biết nào” — bạn chỉ chọn thêm cái gì. Thêm đúng thì một thìa là đủ; thêm sai thì nếm mười thìa vẫn kết luận sai, và sai một cách rất tự tin.

Cái giá phải trả là mọi kết luận đều là một vụ đặt cược rằng thìa của bạn giống phần còn lại. Và có những câu hỏi mà nếm bao nhiêu cũng vô ích: nếm không cho biết nồi này được bắc lên bếp lúc mấy giờ. Gặp loại câu hỏi đó, việc đúng là đi tìm cái đồng hồ, chứ không phải nếm thêm.

Chỗ không rút gọn được: vì sao một cái máy khổng lồ, đủ sức học thuộc cả những chỗ dữ liệu bị ghi sai, mà vẫn đoán tốt trên dữ liệu mới, thì tới nay chưa ai nói gọn được mà vẫn đúng. Mọi lời giải thích ngắn hiện có đều bỏ sót một trường hợp ngược lại.

Một câu: Bạn luôn kết luận về cả nồi từ một thìa, nên việc quan trọng nhất không phải là nếm nhiều hơn mà là biết thìa của mình đại diện cho cái gì.

2.10 Kết nối

- **Bốn nguồn inductive bias** tách thành các chương riêng: hàm mất mát ở [B/09-thiet-ke-ham-mat-mat](#), kiến trúc ở [B/10-thiet-ke-kien-truc](#) rồi cả Phần C, dữ liệu và thuật toán tối ưu ở Phần D.
- **Chương đối lập trực tiếp** là [B/08-dich-chuyen-phan-phoi](#): ở đây ta giả định phân phối đứng yên, ở đó ta bỏ giả định ấy.
- **Learning curve** — công cụ quyết định trong Bảng 2.2 — được xây dựng cụ thể ở [B/05-thiet-ke-bai-toan](#), chương kế tiếp, nơi “đầu tư dữ liệu hay đầu tư mô hình” trở thành quyết định kinh tế có con số.
- **Ngược lên**, [B/03-giai-phau-he-thong](#) dạy cách phát hiện giả định của người khác; chương này dạy cách chọn giả định của mình.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao ERM là một xấp xỉ chứ không phải một phép giải, và chính xác đại lượng nào trong định nghĩa của nó là thứ ta không đo được?
2. KNN trên pixel thất bại còn KNN trên đặc trưng tự giám sát thành công. Kết luận đúng rút ra là gì, và kết luận sai mà nhiều người rút ra là gì?

3. Bốn nguồn inductive bias là gì, và xếp theo chi phí sửa chữa khi phát hiện đặt sai giả định thì thứ tự của chúng ra sao?
4. Vì sao “train loss thấp hơn test loss nhiều nên phải giảm kích thước mô hình” không còn là suy luận hợp lệ?
5. Trong lĩnh vực của bạn, bài toán nào mà thêm dữ liệu chắc chắn vô ích, và phép biến đổi cụ thể nào giữ nguyên quan sát chứng minh nó không identifiable?
6. Shortcut learning khác overfitting ở chỗ nào, và vì sao tập kiểm thử rút cùng phân phối không phát hiện được nó?

Nguồn

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *Rec-Sys*, 2019.
- [3] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [4] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [5] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [6] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [7] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [8] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [9] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [10] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [11] V. N. Vapnik, “An overview of statistical learning theory,” *IEEE Trans. Neural Networks*, vol. 10, no. 5, pp. 988–999, 1999.
- [12] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [13] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [15] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [16] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [17] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [18] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE Trans. Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.
- [19] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.

- [20] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [21] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [22] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [23] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [24] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [25] R. Geirhos, J.-H. Jacobsen, C. Michaelis, et al., “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.
- [26] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [27] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [28] T. Gebru, J. Morgenstern, et al., “Datasheets for Datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [29] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [30] M. Mitchell, S. Wu, et al., “Model Cards for Model Reporting,” in *FAT**, 2019.
- [31] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [32] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [33] C. G. Northcutt, L. Jiang, and I. L. Chuang, “Confident Learning: Estimating Uncertainty in Dataset Labels,” *JAIR*, vol. 70, pp. 1373–1411, 2021.
- [34] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [35] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [36] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [37] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [38] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [39] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [40] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [41] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London: Butterworths, 1979.
- [42] B. W. Matthews, “Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme,” *Biochimica et Biophysica Acta (BBA) — Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
- [43] D. Chicco and G. Jurman, “The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation,” *BMC Genomics*, vol. 21, p. 6, 2020.

- [44] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in *International Conference on Machine Learning (ICML)*, 2006, pp. 233–240.
- [45] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.
- [46] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [47] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [48] M. P. Naeni, G. F. Cooper, and M. Hauskrecht, "Obtaining Well Calibrated Probabilities Using Bayesian Binning," in *AAAI Conference on Artificial Intelligence*, 2015.
- [49] G. W. Brier, "Verification of Forecasts Expressed in Terms of Probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
- [50] A. H. Murphy, "A New Vector Partition of the Probability Score," *Journal of Applied Meteorology*, vol. 12, no. 4, pp. 595–600, 1973.
- [51] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [52] T.-Y. Lin, M. Maire, S. Belongie, et al., "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.
- [53] J. Hosang, R. Benenson, P. Dollár, et al., "What Makes for Effective Detection Proposals?" *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 4, pp. 814–830, 2016.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] F. Perazzi, J. Pont-Tuset, B. McWilliams, et al., "A Benchmark Dataset and Evaluation Methodology for Video Object Segmentation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [56] G. Csurka, D. Larlus, and F. Perronnin, "What Is a Good Evaluation Measure for Semantic Segmentation?" In *British Machine Vision Conference (BMVC)*, 2013.
- [57] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [58] K. Mikolajczyk and C. Schmid, "A Performance Evaluation of Local Descriptors," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 10, pp. 1615–1630, 2005.
- [59] M. Dusmanu, I. Rocco, T. Pajdla, et al., "D2-Net: A Trainable CNN for Joint Description and Detection of Local Features," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [60] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [61] S. Sagawa, P. W. Koh, T. B. Hashimoto, et al., "Distributionally Robust Neural Networks for Group Shifts," in *ICLR*, 2020.
- [62] M. Arjovsky, L. Bottou, I. Gulrajani, et al., "Invariant Risk Minimization," *arXiv:1907.02893*, 2019.
- [63] I. Gulrajani and D. Lopez-Paz, "In Search of Lost Domain Generalization," in *ICLR*, 2021.
- [64] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," in *ICLR*, 2019.

- [65] R. Taori, A. Dave, V. Shankar, et al., “Measuring Robustness to Natural Distribution Shifts in Image Classification,” in *NeurIPS*, 2020.
- [66] D. Hendrycks and K. Gimpel, “A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks,” in *ICLR*, 2017.
- [67] Y. Ovadia, E. Fertig, et al., “Can You Trust Your Model’s Uncertainty?” In *NeurIPS*, 2019.
- [68] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [69] A. Kendall, Y. Gal, and R. Cipolla, “Multi-Task Learning Using Uncertainty to Weigh Losses,” in *CVPR*, 2018.
- [70] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [71] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [72] F. Milletari, N. Navab, and S.-A. Ahmadi, “V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation,” in *3DV*, 2016.
- [73] H. Rezatofighi, N. Tsoi, J. Gwak, et al., “Generalized Intersection over Union,” in *CVPR*, 2019.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [76] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [77] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [78] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [79] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [80] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [81] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [82] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [83] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [84] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [85] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [86] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [87] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [88] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [89] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [90] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.

- [91] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [92] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [93] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [94] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [95] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [96] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [97] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [98] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [99] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [100] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [101] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.

Thiết kế bài toán và định nghĩa dữ liệu (Problem Framing and Data Definition)

PHỤ THUỘC B/03-giai-phau-he-thong (bộ ba đầu vào / đầu ra / tín hiệu giám sát); B/04-nen-tang-hoc-tu-du-lieu (identifiability và ba đòn bẩy — chương này biến đòn bẩy “thêm dữ liệu” thành một quyết định có con số).

TÓM TẮT

Chương này nói về những quyết định bạn phải ra *trước khi* có mô hình nào chạy: một mẫu là gì, nhãn được định nghĩa thế nào ở các ca biên, gộp hay tách lớp, dùng *bbox* hay *mask*, lấy mẫu từ đâu, và nên đầu tư tiếp vào dữ liệu hay vào mô hình. Đọc xong bạn tự chẩn đoán được nguyên nhân của phần lớn thất bại triển khai, kể cả những nguyên nhân không hiện lên trên bất kỳ metric nội bộ nào. Bạn cũng trả lời được câu “gán nhãn thêm bao nhiêu thì đáng” bằng một phép tính hai dòng trên *learning curve*, thay vì bằng cảm tính. Nếu chỉ nhớ một điều: **đơn vị phân tích** (*unit of analysis*) và dạng đầu ra khoá chặt mọi thứ phía sau, còn thiên lệch sinh ra ở khâu thu thập thì không hàm mất mát nào sửa được — chỉ có thu thập lại.

KIẾN THỨC CẦN CÓ TRƯỚC

- **nhãn (label) và tập dữ liệu** — nhãn là câu trả lời đúng gắn với một ảnh — tên lớp, tọa độ hộp, hay mask. Tập dữ liệu là tập các cặp (ảnh, nhãn) dùng để dạy và để đánh giá mô hình. Cả chương này nói về việc *định nghĩa* nhãn chứ không về việc học từ nó.
- **mẫu (sample) — hai nghĩa** — (a) một điểm dữ liệu đơn lẻ, “một mẫu ảnh”; (b) trong thống kê, cả tập được rút ra từ một quần thể, “mẫu gồm 500 ảnh”. Chương này dùng nghĩa (a) khi nói “số mẫu” và nghĩa (b) khi nói “lấy mẫu”; ngữ cảnh phân biệt được.
- **tập huấn luyện và tập kiểm thử** — tập dữ liệu được chia làm ít nhất hai phần: phần dùng để điều chỉnh tham số (huấn luyện) và phần giữ riêng, không cho mô hình nhìn thấy, dùng để ước lượng chất lượng trên dữ liệu mới (kiểm thử). Nếu một mẫu lọt vào cả hai phần thì con số kiểm thử lạc quan giả — hiện tượng gọi là rò rỉ dữ liệu.
- **các dạng nhãn không gian** — *bounding box* (bbox) là hình chữ nhật bao quanh vật, ghi bằng bốn số; *mask* là nhãn cho từng pixel, nói pixel đó thuộc vật nào; *polygon* là đường bao đa giác, xấp xỉ rẻ hơn của mask; *keypoint* là một tập điểm có ý nghĩa cố định (khớp vai, khuỷu tay); *hộp 3D* là hộp trong không gian ba chiều, cần thêm thông tin hình học.
- **lớp (class) và taxonomy** — lớp là một phạm trù mà nhãn có thể nhận (“xe”, “người”). Taxonomy là toàn bộ danh sách lớp cùng cách chúng gộp hay tách khỏi nhau — “xe” hay là “ô tô / xe tải / xe buýt”. Đây là một quyết định thiết kế, không phải một sự thật có sẵn trong thế giới.
- **quần thể và khung lấy mẫu** — quần thể (*population*) là toàn bộ tình huống hệ thống sẽ gặp khi chạy thật. Khung lấy mẫu (*sampling frame*) là phần của quần thể đó mà bạn thực sự chụp được. Hai thứ này gần như không bao giờ trùng nhau, và chênh lệch giữa chúng là chủ đề trung tâm của chương.
- **learning curve — hai nghĩa** — (a) đường cong sai số theo số vòng huấn luyện (*epoch*), dùng để xem mô hình đã hội tụ chưa; (b) đường cong sai số theo *kích thước tập dữ liệu*, dùng để quyết định có nên thu thập thêm dữ liệu không. Chương này chỉ dùng nghĩa (b), và đó là nghĩa ít người vẽ hơn hẳn.
- **IoU và mAP** — IoU (*intersection over union*) đo mức chồng lấn giữa dự đoán và nhãn: diện tích phần giao chia cho diện tích phần hợp, bằng 1 khi trùng khít. mAP là điểm tổng hợp cho bài

toán phát hiện, dựng trên IoU. Ở chương này chỉ cần biết chúng là các thước đo phụ thuộc vào dạng đầu ra bạn đã chọn.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao hai người gán nhãn giỏi, cùng đọc một guideline, vẫn cho ra hai tập nhãn khác nhau — và vì sao con số bất đồng đó lại là trần hiệu năng của mô hình?
- Q2. Một mô hình ổn định qua năm lần chia tập rồi sập ngay tuần đầu triển khai. Nếu không phải overfitting thì là gì?
- Q3. Đổi từ bbox sang mask nghe như một thay đổi kiến trúc; vì sao nó thực ra là quyết định đắt nhất của cả dự án?
- Q4. Câu hỏi “nên gán nhãn thêm hay nên đổi mô hình” có trả lời được bằng một con số thay vì bằng thâm niên không?

3.1 Đặt vấn đề

Trước khi có mô hình, bạn phải biến một nhu cầu mơ hồ thành một bài toán học máy. Mọi sai lầm ở bước này không sửa được ở bước sau — đó là điều làm cho chương này khác hẳn mọi chương kỹ thuật khác trong cây. Một **hàm mất mát** (*loss function*) chọn sai thì đổi loss; một kiến trúc chọn sai thì huấn luyện lại; nhưng một đơn vị phân tích chọn sai, hay một dạng đầu ra chọn sai, thì phải gán nhãn lại từ đầu, và trong hầu hết dự án điều đó đồng nghĩa với việc dự án dừng.

Người mới thường coi phần này là “việc chuẩn bị” và muốn đi nhanh qua để tới phần thú vị. Quan sát lặp lại — và cũng là quan sát chung trong kỹ thuật hệ thống học máy [4] — là điều ngược lại: khi một dự án thị giác thất bại, nguyên nhân hiếm khi nằm ở kiến trúc. Nó thường nằm ở một trong hai chỗ: nhãn được định nghĩa không nhất quán, hoặc dữ liệu được thu thập từ một quần thể khác với quần thể sẽ dùng hệ thống. Cả hai nguyên nhân này đều *vô hình* đối với mọi metric nội bộ: bạn có thể đạt **mAP** rất cao trên tập kiểm thử của chính mình và vẫn thất bại hoàn toàn khi triển khai, mà không con số nào trong bảng cảnh báo bạn.

Nếu bạn đã làm **benchmark** SLAM thì bạn có sẵn trực giác đúng cho chương này, chỉ là dưới tên khác. Một quỹ đạo **chân trị** (*ground truth*) trên EuRoC được định nghĩa trong một hệ quy chiếu cụ thể; nếu bạn đánh giá quỹ đạo camera bằng chân trị của thân máy bay, mọi con số đều ra, đều đẹp, và đều sai. Sai lầm không nằm ở thuật toán mà nằm ở định nghĩa của đại lượng được đo. Chương này là phiên bản tổng quát của bài học đó, áp cho toàn bộ khâu thiết kế bài toán.

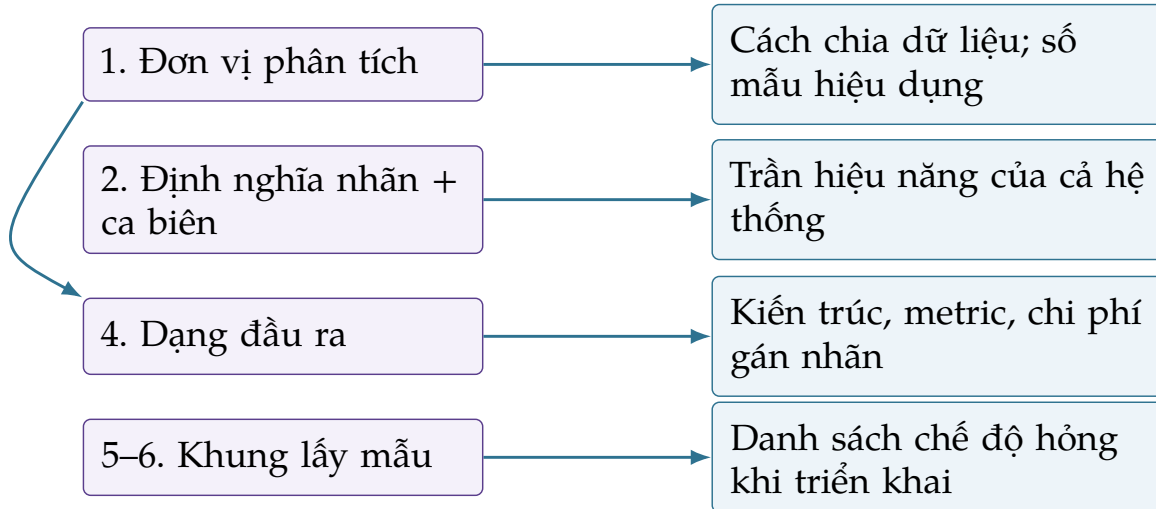
TRỰC GIÁC

Định nghĩa bài toán giống việc đúc một cái khuôn: mọi thứ đổ vào sau đó đều mang hình dạng của nó. Chọn đơn vị phân tích là chọn kích thước khuôn; chọn dạng đầu ra là chọn hình dạng khoang; định nghĩa nhãn kèm ca biên là mài các cạnh cho hai người thợ khác nhau đúc ra cùng một chi tiết. Bạn có thể đổi kim loại đổ vào — đổi kiến trúc, đổi loss, đổi **bộ tối ưu** (*optimizer*) — bao nhiêu lần tùy thích, nhưng đổi khuôn thì phải phá cả mẻ đã đúc.

3.2 Tám quyết định theo thứ tự ràng buộc (Eight Decisions, Ordered by Reach)

A. Quyết định nào khoá cái gì

Thứ tự của mạch dưới đây đi từ quyết định ràng buộc nhiều thứ nhất tới quyết định ràng buộc ít nhất. Đảo ngược thứ tự là sai lầm phổ biến: chọn dạng đầu ra trước khi biết đơn vị phân tích là gì thì bạn sẽ phát hiện mâu thuẫn sau khi đã gán vài nghìn ảnh. Hình 3.1 cho thấy tầm với của từng quyết định — đọc nó trước sẽ tiết kiệm cho bạn cả mạch phía sau.



Hình 3.1. Bốn quyết định có tầm với xa nhất và thứ mà mỗi cái khoá lại. Mũi tên cong bên trái nhắc rằng đơn vị phân tích còn ràng buộc cả dạng đầu ra: chọn video làm đơn vị thì nhãn theo khung hình mất phần lớn ý nghĩa.

B. Tám bước

I Mạch thiết kế một bài toán thị giác

1 Vấn đề

“một mẫu” là gì, câu hỏi tưởng như không cần hỏi.

Giải pháp

chọn đơn vị phân tích một cách có ý thức — một khung hình, một video, một bệnh nhân, một ca chụp, một chuyến đi.

Hệ quả

nó quyết định luôn cách chia dữ liệu ở chương 7: chọn khung hình làm đơn vị rồi chia ngẫu nhiên là công thức tạo rò rỉ dữ liệu (*data leakage*). Hai khung hình cách nhau 33 ms gần như là cùng một ảnh, nên chúng sẽ nằm ở cả tập huấn luyện lẫn tập kiểm thử.

Giá phải trả

chọn đơn vị lớn hơn làm số mẫu hiệu dụng giảm mạnh, và bạn phải chấp nhận con số đó thay vì con số đẹp hơn nhưng vô nghĩa.

↪ **tiền đề B/07** — quy tắc chia dữ liệu ở chương đánh giá chỉ đúng khi đơn vị phân tích đã được chọn đúng ở đây

2 Vấn đề

nhãn nghe có vẻ hiển nhiên nhưng không phải.

Giả định

phần lớn cái ta gọi là “mô hình kém” thực ra là nhãn được định nghĩa không nhất quán: biên của mask đặt ở đâu khi vật nhoè, che khuất bao nhiêu phần trăm thì vẫn gán, vật nhỏ tới mức nào thì bỏ qua.

Giải pháp

viết định nghĩa nhãn kèm các **ca biên** trước khi gán mẫu đầu tiên, rồi thử nghiệm nó trên vài chục ảnh với hai người gán nhãn độc lập.

Hệ quả

mức đồng thuận giữa người gán nhãn (*inter-annotator agreement*) là trần hiệu năng thực tế của hệ thống; không mô hình nào vượt qua độ nhất quán của dữ liệu dạy nó một cách đáng tin.

- 3 Vấn đề** chia lớp thế nào cho đúng.
Giải pháp thiết kế *taxonomy*, tức quyết định gộp hay tách.
Giá phải trả gộp làm bài toán dễ và metric đẹp nhưng đầu ra kém hữu ích; tách làm đầu ra hữu ích hơn nhưng đẩy chi phí gán nhãn lên và làm các lớp hiếm quá thừa để học.
Hệ quả quy tắc thực dụng: tách khi và chỉ khi hai lớp dẫn tới hai *hành động* khác nhau ở phía sau hệ thống; nếu hệ thống phản ứng như nhau thì việc tách chỉ tạo công gán nhãn và bất đồng giữa người gán nhãn.
- 4 Vấn đề** chọn dạng đầu ra nào.
Giải pháp cân nhắc giữa nhãn ảnh, bbox, mask, *polygon*, *keypoint*, **hộp 3D** (*3D box*) — xem Bảng 3.1.
Giá phải trả đây là quyết định đắt nhất trong cả dự án, vì nó ràng buộc *đồng thời* kiến trúc, metric và chi phí gán nhãn. Đổi từ bbox sang mask giữa chừng nghĩa là gán lại từ đầu.
Hệ quả nên trả giá suy nghĩ ở đây bằng một thử nghiệm nhỏ, đừng trả giá bằng nửa ngân sách gán nhãn.
- 5 Vấn đề** bạn huấn luyện trên tập nào và triển khai cho ai?
Giải pháp phân biệt rành mạch **quần thể** (*population*) — toàn bộ tình huống hệ thống sẽ gặp khi chạy thật — với **khung lấy mẫu** (*sampling frame*), phần của quần thể đó mà bạn thực sự lấy mẫu được.
Giả định hai thứ trùng nhau, một giả định gần như luôn sai.
Hệ quả khoảng cách giữa chúng là nguồn thất bại số một khi triển khai, và nó không hiện lên trên bất kỳ metric nội bộ nào vì tập kiểm thử của bạn cũng rút từ chính khung lấy mẫu.
- 6 Vấn đề** thiên lệch được cài vào ngay từ khâu đầu.
Giả định *selection bias* khi thu thập: bạn chụp ban ngày vì tiện, dùng ba thiết bị vì chỉ có ba, lấy ảnh ở những địa điểm cho phép quay.
Giải pháp nguyên tắc cứng — thiên lệch sinh ra ở khâu thu thập thì không mô hình, loss, hay *reweighting* nào sửa được ở khâu cuối. Reweighting chỉ sửa được lệch *tỉ lệ* giữa các nhóm đã có mặt; nó không tạo ra được nhóm chưa từng xuất hiện.
Hệ quả chỉ có thu thập lại, nên chi phí của một quyết định lấy mẫu cầu thả là toàn bộ ngân sách dữ liệu.
- 7 Vấn đề** nên đầu tư vào dữ liệu hay vào mô hình?
Giải pháp một phép đo thay cho tranh luận — vẽ learning curve theo kích thước dữ liệu: huấn luyện cùng một cấu hình ở 10%, 25%, 50%, 100% dữ liệu và nhìn hình dạng đường cong.
Hệ quả curve còn dốc thì thu thập tiếp; curve đã phẳng thì tiền gán nhãn tiếp theo gần như bị đốt, và bạn phải đổi mô hình, đổi dạng đầu ra, hoặc đổi bài toán. Đây chính là đòn bẩy “thêm dữ liệu” của chương 4, lần này có quy trình đo cụ thể.
 ↪ **hệ quả của B/04** — bảng ba đòn bẩy nói có ba lựa chọn; learning curve nói *chọn cái nào*
- 8 Vấn đề** dữ liệu không miễn phí, mà thảo luận về nó thường không có đơn vị.
Giải pháp so *chi phí thu thập* với *giá trị biên mỗi mẫu* đọc từ chính đường cong ở bước trên.
Hệ quả nó biến một quyết định kỹ thuật thành quyết định kinh tế phát biểu được bằng con số — cách nghĩ dùng được ở mọi dự án thật, kể cả khi con số chỉ chính xác tới một bậc độ lớn.

3.3 Chọn dạng đầu ra (Choosing the Output Representation)

Trong tám bước trên, bước bốn đáng được mở rộng vì nó là bước duy nhất mà sai lầm không có đường lùi rẽ. Cột chi phí trong Bảng 3.1 là *tương đối* và chỉ nên đọc ở mức bậc độ lớn: các con số này phản ánh kinh nghiệm thực hành lưu truyền trong cộng đồng gắn nhãn, không phải một phép đo có kiểm soát. Chúng còn thay đổi mạnh khi có trợ lý gắn nhãn bằng mô hình nền tảng.

Bảng 3.1. Chọn dạng đầu ra. Cột “hóng khi nào” là cột quyết định: nó cho biết loại lỗi bạn tự nguyện chấp nhận vĩnh viễn khi chọn dạng đó.

Dạng	Giả định về bài toán	Chi phí nhãn (tương đối)	Hóng khi nào
Nhãn ảnh	Chỉ cần biết trong ảnh có gì	1×	Khi hệ thống cần định vị hoặc đếm
Bbox	Vật tách rời, hình chữ nhật đủ mô tả nó	5–10×	Vật dài, cong, hoặc chồng lấn: hộp chứa chủ yếu là nền
Mask	Cần biên chính xác tới mức pixel	20–50×	Biên mơ hồ (tóc, khối, kính) gây bất đồng lớn giữa người gắn nhãn
Polygon	Cần biên nhưng chấp nhận xấp xỉ đa giác	10–30×	Biên phức tạp bị làm trơn quá mức
Keypoint	Vật có cấu trúc cố định đã biết trước	10–20×	Vật không có cấu trúc chung, hoặc bị che một phần
Hộp 3D	Có hình học hoặc cảm biến phụ đồng bộ	50× trở lên	Lệch hiệu chuẩn (<i>calibration</i>) hoặc lệch thời gian giữa các cảm biến

Cách chọn đúng không phải là “chọn dạng giàu thông tin nhất mà ngân sách cho phép”, mà là đi ngược từ hành động phía sau: hệ thống của bạn *làm gì* với đầu ra? Nếu robot chỉ cần biết “có chướng ngại trong hành lang bên trái hay không” thì mask là lãng phí nhiều chục lần và còn làm chậm cả pipeline. Nếu nó phải bám theo mép một vật dài và cong thì bbox *sai về nguyên tắc*, không chỉ kém chính xác. Hình chữ nhật bao quanh một vật nằm chéo chứa phần lớn diện tích là nền. Hậu quả kép: bộ điều khiển phía sau không dùng được dự đoán đó, còn mọi metric dựa trên *IoU* thì vẫn khen nó.

↔ **tiền đề B/03** — đây chính là câu hỏi “đầu ra là gì của quy trình bảy câu hỏi, lần này phải trả lời cho hệ thống của chính bạn

Một dấu hiệu nhận biết mình chọn sai dạng đầu ra: bạn phải viết ngày càng nhiều luật hậu xử lý để dịch đầu ra của mô hình thành thứ hệ thống thật cần.

3.4 Quần thể và khung lấy mẫu (Population vs Sampling Frame)

Bước năm và sáu nói cùng một chuyện từ hai phía, và đáng được tách ra vì chúng là nguồn thất bại triển khai hay gặp nhất. Nói gọn thì có ba tập lồng nhau, không phải hai: quần thể là tập tất cả tình huống hệ thống sẽ gặp; khung lấy mẫu là tập tình huống bạn có khả năng chụp được; tập dữ liệu là một mẫu rút từ khung lấy mẫu, *không phải* từ quần thể. Tập kiểm thử của bạn cũng rút từ khung lấy mẫu, nên nó đo được sai số trong khung và mù hoàn toàn với phần quần thể nằm ngoài khung.

Điều này giải thích một hiện tượng gây bối rối: mô hình đạt kết quả tốt và ổn định qua nhiều lần chia tập, rồi sập khi triển khai. Không có gì mâu thuẫn — mọi phép chia đều nằm trong cùng một khung lấy mẫu, nên chúng đo lại cùng một thứ nhiều lần thay vì đo thêm thứ mới. Các

benchmark có kiểm soát về dịch chuyển phân phối được xây dựng chính vì lý do này [26], [27]. Bài học chung từ chúng: độ chính xác trong phân phối không dự đoán tốt độ chính xác ngoài phân phối.

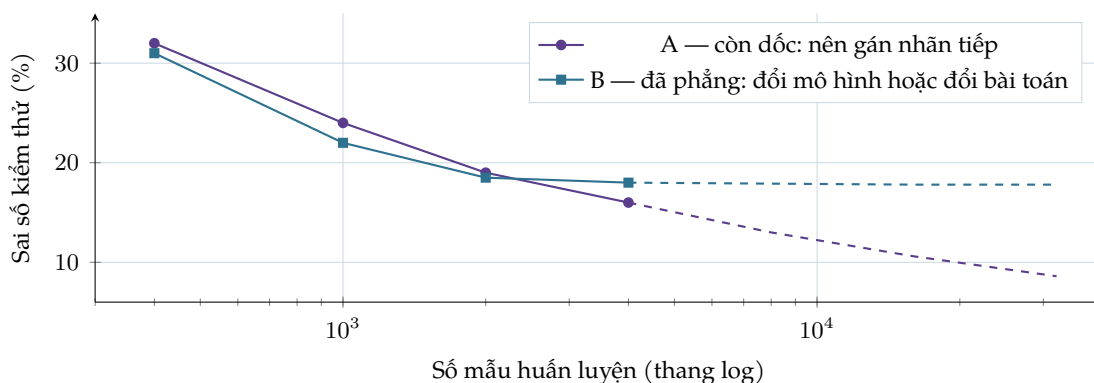
↔ **tiền đề B/08** — chênh lệch quần thể ↔ khung lấy mẫu là dạng tĩnh của dịch chuyển phân phối, chỉ khác ở chỗ nó có mặt ngay từ ngày đầu

Hệ quả hành động thì đơn giản và không dễ chịu. Trước khi thu thập, hãy viết ra bằng lời quần thể mục tiêu và liệt kê các trục biến thiên của nó: thiết bị, ánh sáng, mùa, địa điểm, người dùng. Sau đó đánh dấu trục nào khung lấy mẫu của bạn không phủ. Danh sách đó chính là danh sách chế độ hỏng đã biết trước của hệ thống, và nó nên nằm trong *datasheet* của tập dữ liệu [28] chứ không nằm trong đầu một người. Nếu một trục quan trọng không phủ được, quyết định đúng là thu hẹp phạm vi cam kết của sản phẩm, chứ không phải hy vọng mô hình tự tổng quát hoá.

3.5 Đường cong học theo kích thước dữ liệu (Data-Size Learning Curve)

A. Hai hình dạng, hai quyết định trái ngược

Câu hỏi “nên gán nhãn thêm hay nên đổi mô hình” thường được tranh luận bằng trực giác và thâm niên. Nó không cần phải như vậy: một buổi huấn luyện ở bốn tỉ lệ dữ liệu cho bạn câu trả lời kiểm chứng được. Điều quan trọng là bạn phân biệt hai trường hợp bằng *hình dạng* đường cong chứ không bằng một con số đơn lẻ (Hình 3.2).



Hình 3.2. Hai hình dạng learning curve theo kích thước dữ liệu, vẽ trên trục log. Đường liền là bốn điểm đo được ở 10%, 25%, 50%, 100% dữ liệu; đường đứt là ngoại suy. Đường A gần thẳng trên trục log — dấu hiệu quy luật lũy thừa còn hiệu lực; đường B đã chạm trần, và trần đó thường là mức nhất quán của nhãn chứ không phải giới hạn của mô hình. Số liệu là minh họa, không phải phép đo trên một tập cụ thể.

B. Phép tính hai dòng biến nó thành quyết định kinh tế

Với đường A, sai số giảm từ 24% ở 1000 mẫu xuống 16% ở 4000 mẫu. Giả sử dạng lũy thừa $e(n) \approx a n^{-b}$, lấy tỉ số hai điểm ta có

$$b = \frac{\log(24/16)}{\log(4000/1000)} = \frac{\log 1,5}{\log 4} \approx \frac{0,405}{1,386} \approx 0,29.$$

Số mũ b nói một điều duy nhất: *mỗi lần nhân đôi dữ liệu thì sai số nhân với 2^{-b}* . Với $b \approx 0,29$, nhân tư số mẫu lên 16,000 cho $e \approx 16 \cdot 4^{-0,29} \approx 10,6\%$, nghĩa là 12,000 ảnh gán nhãn thêm mua được khoảng 5,4 điểm. Nếu chi phí gán một ảnh là c thì bạn đang cân nhắc bỏ 12,000 c để đổi lấy 5,4 điểm, và bạn so nó với các phương án khác — thuê thêm một kỹ sư trong một tháng để thử kiến trúc mới, hay giảm c bằng trợ lý gán nhãn tự động. Với đường B thì phép tính không cần làm: b gần 0, nên mọi khoản đầu tư dữ liệu đều trả về gần 0.

Hai cảnh báo về công cụ này. Thứ nhất, ngoại suy quy luật lũy thừa ra ngoài dải đã đo là suy đoán và thường lạc quan quá mức, vì đường cong thật cuối cùng cũng phải chạm trần nhãn; hãy coi 10,6% là cận trên của kỳ vọng chứ không phải một dự báo. Thứ hai, mỗi điểm phải được huấn luyện với cùng một quy trình và có phương sai được ước lượng, nếu không bạn đang đọc nhiều như đọc xu hướng — đây chính là bài học từ benchmark có thành phần ngẫu nhiên mà bạn đã gặp ở phía SLAM.

3.6 Thực hành

Chương này chỉ chuyển thành kỹ năng khi bạn tự chịu hậu quả của một định nghĩa nhãn lỏng lẻo. Vì vậy cả hai bài tập đều xoay quanh một phép đo: mức bất đồng giữa hai người gán nhãn. Đó là con số hầu như không dự án nào đo, mà hầu như dự án nào cũng bị nó chặn.

Repo và tài nguyên. Ba nhóm công cụ:

- **Gán nhãn** — [HumanSignal/label-studio](#), [cvat-ai/cvat](#), Roboflow Annotate.
- **Trợ lý gán nhãn bằng mô hình nền tảng** — dòng SAM cho mask theo *prompt* [29] và dòng Grounding DINO cho box theo mô tả văn bản. Chúng làm dịch chuyển đáng kể cột chi phí trong Bảng 3.1, nhưng *không* làm dịch chuyển vấn đề nhất quán của định nghĩa nhãn.
- **Soát và tài liệu hoá dữ liệu** — [voxel51/fiftyone](#) để khám phá; datasheets [28] và *model cards* [30] để viết tài liệu kèm.

Danh sách tên mô hình chốt tại tháng 9/2026 và sẽ cũ trong sáu đến mười hai tháng; CVAT và FiftyOne bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Trong bài toán của bạn, hai người gán nhãn giỏi sẽ bất đồng ở đâu — và bạn xử lý bằng cách siết guideline hay bằng cách đổi định nghĩa nhãn? Nếu chuyển đơn vị phân tích từ khung hình sang video thì tập kiểm thử phải chia lại thế nào, và số mẫu hiệu dụng thay đổi bao nhiêu lần? Quần thể mục tiêu của bạn có trực biến thiên nào mà khung lấy mẫu hiện tại không phủ?

Bài tập phụ. Nếu bài toán của bạn không phải SLAM, phiên bản tổng quát của bài tập này là: viết *annotation guideline* hai trang, tự gán 200 ảnh, rồi nhờ một người khác gán 50 ảnh trong số đó. Đo mức bất đồng bằng **hệ số kappa của Cohen** (*Cohen's kappa*) [31] nếu là bài toán phân loại, hoặc bằng tỉ lệ khớp ở một ngưỡng IoU cố định nếu là phát hiện. Mini project SLAM ở cuối chương là phiên bản gắn với hệ thống của bạn.

3.7 Giới hạn và trường hợp hồng

- **Khung này giả định bạn có quyền quyết định về dữ liệu.** Rất nhiều dự án bắt đầu từ một tập đã tồn tại với nhãn đã gán; khi đó phần lớn tám bước trở thành công việc khảo cổ — đọc ngược từ dữ liệu ra định nghĩa nhãn ngầm mà người trước đã dùng. Việc này vẫn đáng làm và thường phát hiện những mâu thuẫn giải thích được vì sao mô hình kém, nhưng nó không thay thế được quyền thiết kế.
- **Nó giả định quần thể mục tiêu đứng yên đủ lâu.** Có những hệ thống mà môi trường đổi theo mùa, theo phiên bản phần cứng, hay theo hành vi người dùng. Ở đó “quần thể” là một mục tiêu di động. Bài toán đúng không phải thiết kế một lần cho tốt, mà là dựng một vòng lặp thu thập lại; đó là nội dung của [B/08-dich-chuyen-phan-phoi](#).
- **Mức đồng thuận là trần thực dụng, không phải trần lý thuyết.** Có trường hợp mô hình vượt từng người gán nhãn đơn lẻ vì nó học phân tín hiệu chung và trung bình hoá phần nhiễu

riêng của mỗi người; ngược lại, có những tập dữ liệu nổi tiếng mà chính nhãn kiểm thử chứa lượng lỗi đủ lớn để làm sai lệch bảng xếp hạng [32]. Hãy dùng nó như tín hiệu cảnh báo, đừng dùng như hằng số vật lý.

CẠM BẮY

Cạm bẫy tốn kém nhất trong chương này là tin rằng một thiên lệch thu thập có thể sửa được ở khâu huấn luyện. Reweighting, *resampling* hay các hàm mất mát cân bằng nhóm chỉ điều chỉnh được *tỉ lệ* giữa những nhóm đã có mặt trong dữ liệu. Giả sử hệ thống sẽ gặp điều kiện ban đêm mà tập dữ liệu không có một ảnh ban đêm nào. Không trọng số nào tạo ra được thông tin đó. Mô hình sẽ đưa ra dự đoán tự tin và sai, còn tập kiểm thử của bạn thì không hề một lời. Dấu hiệu nhận biết: cuộc thảo luận về “xử lý mất cân bằng dữ liệu” diễn ra hoàn toàn bằng ngôn ngữ hàm mất mát, không ai nhắc tới việc đi chụp thêm.

CẦN NHỚ

Đơn vị phân tích và dạng đầu ra là hai quyết định khoá chặt mọi thứ phía sau: sai ở đó thì phải gán nhãn lại từ đầu. Thiên lệch sinh ra ở khâu thu thập chỉ sửa được bằng cách thu thập lại — không loss nào, không trọng số nào chạm tới nó. Và trước khi tranh luận “thêm dữ liệu hay đổi mô hình”, hãy vẽ learning curve: số mẫu b trả lời câu hỏi đó bằng một con số. [T1]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Đơn vị phân tích	Thứ được tính là “một mẫu”: một khung hình, một video, một bệnh nhân, một chuyến đi. Nó quyết định luôn cách chia dữ liệu và số mẫu hiệu dụng, nên chọn sai là phải làm lại từ đầu.
••• Rò rỉ dữ liệu	Cùng một thông tin có mặt ở cả tập huấn luyện lẫn tập kiểm thử, làm con số kiểm thử lạc quan giả. Chọn khung hình làm đơn vị rồi chia ngẫu nhiên là công thức tạo ra nó.
••• Ca biên	Các trường hợp lấp lửng trong định nghĩa nhãn: biên vật nhoè, mức che khuất, kích thước tối thiểu. Phải viết ra giấy trước khi gán mẫu đầu tiên, vì sau đó sửa nghĩa là gán lại.
••• Mức đồng thuận giữa người gán nhãn	Tỉ lệ hai người gán độc lập cho ra cùng một nhãn. Đây là trần hiệu năng thực tế của hệ thống, vì mô hình chỉ học được từ những nhãn họ tạo ra.
•• Khi nào mô hình vượt được trần đó?	Khi mỗi người nhầm theo một kiểu riêng. Mô hình học phần tín hiệu chung và trung bình hoá phần nhiễu riêng của từng người, nên nhỉnh hơn từng người đơn lẻ.
•• Taxonomy	Danh sách lớp cùng cách chúng gộp hay tách. Quy tắc thực dụng: chỉ tách khi hai lớp dẫn tới hai hành động khác nhau ở phía sau hệ thống.
••• Dạng đầu ra	Nhãn ảnh, bbox, mask, polygon, keypoint, hay hộp 3D. Đây là quyết định đắt nhất dự án vì nó ràng buộc đồng thời kiến trúc, metric và chi phí gán nhãn.
••• Vì sao đổi dạng đầu ra giữa chừng lại đắt tới vậy?	Kiến trúc thì huấn luyện lại là xong, nhưng nhãn phải gán lại toàn bộ từ đầu. Đổi từ bbox sang mask là nhân chi phí gán nhãn lên vài lần.
•• Bbox	Hình chữ nhật bao quanh vật, ghi bằng bốn số. Nó sai về nguyên tắc với vật dài và cong, vì hộp bao một vật nằm chéo chứa phần lớn diện tích là nền.
•• Bạn phải viết ngày càng nhiều luật hậu xử lý. Triệu chứng của gì?	Dạng đầu ra chọn sai: bạn đang dịch đầu ra của mô hình thành thứ hệ thống thật cần. Phép phân biệt là hỏi hệ thống làm gì với đầu ra, rồi chọn dạng đi ngược từ đó.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Quần thể	Toàn bộ tình huống hệ thống sẽ gặp khi chạy thật. Đây là thứ bạn cam kết phục vụ, không phải thứ bạn đo được.
●●● Khung lấy mẫu	Phần của quần thể mà bạn thực sự chụp được. Tập kiểm thử cũng rút từ đây, nên nó mù hoàn toàn với phần quần thể nằm ngoài khung.
●●● Ổn định qua năm lần chia tập rồi sập khi triển khai. Nguyên nhân?	Chênh lệch giữa quần thể và khung lấy mẫu, không phải overfitting. Cả năm phép chia đều nằm trong cùng một khung nên chúng đo lại cùng một thứ; phép phân biệt là dựng một tập kiểm thử từ ngoài khung.
●●● Selection bias	Thiên lệch cài vào ngay khâu thu thập: chụp ban ngày vì tiện, dùng ba thiết bị vì chỉ có ba. Nó không sửa được ở khâu cuối, chỉ có thu thập lại.
●●● Vì sao reweighting không sửa được selection bias?	Reweighting chỉ chỉnh tỉ lệ giữa những nhóm đã có mặt trong dữ liệu. Nó không tạo ra được nhóm chưa từng xuất hiện, ví dụ ảnh ban đêm khi tập dữ liệu không có tấm nào.
●● Cả nhóm bàn “mất cân bằng dữ liệu” hoàn toàn bằng ngôn ngữ hàm mất mát	Dấu hiệu đang rơi vào bẫy reweighting. Phép kiểm tra: hỏi trong dữ liệu có bao nhiêu mẫu của nhóm thiểu; nếu là không thì việc phải làm là đi chụp thêm.
●●● Learning curve theo kích thước dữ liệu	Sai số kiểm thử vẽ theo số mẫu huấn luyện, đo ở 10, 25, 50 và 100 phần trăm dữ liệu. Nó biến câu “dữ liệu hay mô hình” thành một con số thay vì một cuộc tranh luận theo thâm niên.
●● Số mũ b của learning curve	Độ dốc trên trục log: mỗi lần nhân đôi dữ liệu thì sai số nhân với hai mũ trừ b. Với b khoảng 0,29 thì nhân tư dữ liệu mua được vài điểm; với b gần không thì mọi khoản đầu tư dữ liệu trả về gần không.
●● Datasheet của tập dữ liệu	Tài liệu ghi nguồn, quần thể mục tiêu, khung lấy mẫu, quy trình gán nhãn, mức đồng thuận và các lớp bị thiếu. Danh sách trục không phủ được chính là danh sách chế độ hỏng đã biết trước.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Datasheets for datasets, model cards	Chuẩn hoá đúng việc chương này yêu cầu: ghi lại quần thể mục tiêu, khung lấy mẫu, quy trình gán nhãn và các lớp bị thiếu	Thông lệ ở nhiều nhóm lớn từ 2019, ít hết hạn
Dòng SAM và Grounding DINO trong quy trình gán nhãn voxel51/fiftyone	Kéo chi phí của mask và bbox xuống một bậc, tức dịch chuyển trực tiếp cột chi phí trong bảng chọn dạng đầu ra	Đã thành mặc định tính tới 9/2026
	Công cụ soi chênh lệch giữa khung lấy mẫu và quần thể bằng cách lát cắt dữ liệu theo thiết bị, ánh sáng, địa điểm	Hạ tầng, bền
ImageNet-v2 và WILDS	Bằng chứng đo được rằng độ chính xác trong khung lấy mẫu không dự đoán tốt độ chính xác ngoài nó	Kết quả đã công bố

MINI PROJECT SLAM • Biến “có nên làm keyframe không” thành một bài toán học máy • 1–2 ngày

Mục tiêu đi hết tám quyết định của chương trên một bài toán bạn đã có sẵn trực giác kỹ thuật.
Mục đích là thấy rõ chỗ nào trực giác đó *không* đủ để thay một định nghĩa viết ra giấy.

Dữ liệu	hai chuỗi EuRoC, một dễ và một khó (ví dụ một chuỗi MH và một chuỗi V2), cùng nhật ký của ORB-SLAM3 ghi lại từng khung hình và quyết định keyframe hiện tại của heuristic có sẵn.
Các bước	viết một tài liệu định nghĩa bài toán gồm sáu phần — (1) đơn vị phân tích: một khung hình, một đoạn, hay một cặp khung hình so với keyframe gần nhất, và hệ quả của lựa chọn đó lên cách chia dữ liệu; (2) định nghĩa nhãn dương/âm kèm ít nhất năm ca biên viết rõ (khung hình mờ nhưng nhiều đặc trưng, camera đứng yên, quay nhanh tại chỗ, vừa mất bám xong, vùng đã lập bản đồ dày); (3) taxonomy: nhị phân hay ba mức, và mỗi mức dẫn tới hành động khác nhau nào trong hệ; (4) dạng đầu ra và metric đi kèm; (5) khung lấy mẫu so với quần thể thật của bạn, ghi rõ trục nào EuRoC không phủ; (6) giao thức chia dữ liệu chống rò rỉ theo chuỗi. Sau đó tự gán 300 khung hình và nhờ một người khác gán 80 khung hình trong số đó theo đúng tài liệu, không được hỏi lại.
Xong khi	hai người gán độc lập theo tài liệu của bạn mà không hỏi thêm một câu nào, và bạn báo cáo được mức đồng thuận trên 80 khung hình chung bằng một con số (κ hoặc tỉ lệ khớp). Ngưỡng để coi là đạt: sau một vòng sửa tài liệu, mức đồng thuận phải tăng, và bạn liệt kê được ít nhất ba ca biên mà vòng đầu không nghĩ ra.
Nối với	B/03 cho biết keyframe nằm ở khối nào trong luồng dữ liệu và độ khó bị đẩy đi đâu khi bạn thay heuristic bằng mô hình; B/06 biến quy trình gán nhãn này thành một quy trình có ngân sách; B/07 kiểm tra giao thức chia dữ liệu ở phần (6); B/04 dùng chính tập nhãn này để vẽ learning curve và quyết định có đáng gán tiếp không.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì nhãn không phải một sự thật có sẵn mà là kết quả của một định nghĩa, và mọi định nghĩa đều dễ hờ các ca biên: biên vật nhòe, mức che khuất, kích thước tối thiểu. Mô hình học từ tập nhãn đó nên không thể nhất quán hơn chính dữ liệu dạy nó một cách đáng tin.
- A2. Là chênh lệch giữa quần thể và khung lấy mẫu. Cả năm lần chia tập đều nằm trong cùng một khung lấy mẫu, nên chúng đo lại cùng một thứ năm lần thay vì đo thêm thứ mới; phần quần thể nằm ngoài khung không xuất hiện trong bất kỳ phép chia nào.
- A3. Vì nó đổi *đồng thời* kiến trúc, metric và toàn bộ nhãn đã gán. Kiến trúc thì huấn luyện lại là xong; nhãn thì phải làm lại từ đầu với chi phí lớn hơn nhiều lần, và đó là thứ thường giết dự án.
- A4. Có: huấn luyện cùng một cấu hình ở bốn tỉ lệ dữ liệu, khớp $e(n) \approx a n^{-b}$ và đọc số mũ b . Với $b \approx 0,29$, nhân tư dữ liệu mua được vài điểm; với b gần 0, mọi khoản đầu tư dữ liệu tiếp theo đều trả về gần không.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn nhờ người khác đi chợ hộ. Nếu mẫu giấy chỉ ghi “mua ít rau”, bạn sẽ nhận về thứ khác với thứ mình hình dung; nhờ người khác thì lại nhận về thứ thứ ba. Không ai sai cả — mẫu giấy sai. Bài toán gốc của chương này đúng như vậy, chỉ khác là người đi chợ là một cái máy. Trước khi có cái máy nào chạy, bạn phải viết mẫu giấy đó: mỗi lần nó nhìn vào cái gì, cái gì được coi là đúng, và những trường hợp lấp lửng thì tính sao. Rau hơi héo có mua không? Bó to gấp đôi thì tính là một hay hai?

Mẹo gồm ba việc. Một, viết các trường hợp lấp lửng ra giấy trước khi bắt đầu chứ không phải sau. Hai, nhờ hai người cùng đi chợ với cùng mẫu giấy rồi xem họ mua khác nhau bao nhiêu; chênh lệch đó chính là mức tốt nhất cái máy có thể đạt, vì nó chỉ học được từ những gì hai người kia mang về. Ba, nhớ rằng bạn chỉ nhờ mua ở chợ đầu ngõ nên sẽ không bao giờ biết chợ khác bán gì; nếu nhà sắp chuyển đi nơi khác thì việc phải làm là đi chợ mới, chứ không phải viết lại mẫu giấy cho hay hơn.

Cái giá phải trả là viết mẫu giấy vừa tốn thời gian vừa nhầm chán, và phải làm xong trước khi được chạm vào phần thú vị.

Chỗ không rút gọn được: câu “cái máy không bao giờ giỏi hơn hai người đi chợ” gần đúng nhưng không đúng hẳn. Nếu mỗi người nhầm theo một kiểu riêng, cái máy học từ cả hai có thể trung bình hoá được phần nhầm và nhỉnh hơn từng người. Nói gọn hơn nữa là sai.

Một câu: *Cái máy chỉ làm được đúng cái mẫu giấy bạn viết, nên phần khó nhất của dự án nằm ở tờ giấy chứ không nằm ở cái máy.*

3.8 Kết nối

- **Ngược lên** — ba câu hỏi đầu của [B/03-giai-phau-he-thong](#) (đầu vào, đầu ra, giám sát) ở đây được trả lời cho *bài toán của bạn* thay vì cho bài toán của người khác; và đôn bẩy “thêm dữ liệu” trong bảng ba đôn bẩy của [B/04-nen-tang-hoc-tu-du-lieu](#) ở đây có được quy trình đo cụ thể.
- **Đi tiếp** — [B/06-chat-luong-du-lieu](#) nhận vấn đề nhất quán nhãn ở bước hai và biến nó thành quy trình có ngân sách; [B/07-chia-du-lieu-va-danh-gia](#) nhận đơn vị phân tích ở bước một và cho thấy chọn sai nó sinh ra rò rỉ thế nào; [B/08-dich-chuyen-phan-phoi](#) xử lý phần quần thể nằm ngoài khung lấy mẫu.
- **Sang ngang** — việc chọn dạng đầu ra ràng buộc luôn thước đo, nên Phần E về đánh giá đọc dễ hơn nhiều nếu bạn quay lại Bảng 3.1 khi đọc nó.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao chọn khung hình làm đơn vị phân tích rồi chia dữ liệu ngẫu nhiên lại tạo ra rò rỉ, và triệu chứng của rò rỉ đó hiện ra ở con số nào?
2. Vì sao mức bất đồng giữa hai người gán nhãn lại là trần hiệu năng thực tế, và trong hoàn cảnh nào mô hình vượt được trần đó?
3. Đưa ra một bài toán mà bbox không chỉ kém chính xác hơn mask mà còn sai về nguyên tắc. Sai ở đâu, và metric nào che giấu cái sai đó?
4. Một mô hình đạt kết quả ổn định qua năm lần chia tập rồi sập khi triển khai — cặp khái niệm quần thể / khung lấy mẫu giải thích hiện tượng đó thế nào, mà không cần dùng tới từ “overfitting”?
5. Vì sao reweighting không sửa được selection bias, và chính xác thì nó sửa được loại lệch nào, không sửa được loại nào?
6. Learning curve của bạn có số mũ $b \approx 0,05$. Kết luận gì, và bước tiếp theo là gì?
7. Trong bốn quyết định — đơn vị phân tích, taxonomy, dạng đầu ra, ngưỡng của metric — cái nào rẻ nhất để đổi sau khi đã gán nhãn xong, và vì sao?

Nguồn

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *Rec-Sys*, 2019.
- [3] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [4] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [5] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [6] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [7] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [8] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [9] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [10] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [11] V. N. Vapnik, “An overview of statistical learning theory,” *IEEE Trans. Neural Networks*, vol. 10, no. 5, pp. 988–999, 1999.
- [12] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [13] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [15] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [16] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [17] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [18] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE Trans. Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.
- [19] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.

- [20] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [21] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [22] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [23] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [24] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [25] R. Geirhos, J.-H. Jacobsen, C. Michaelis, et al., “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.
- [26] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [27] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [28] T. Gebru, J. Morgenstern, et al., “Datasheets for Datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [29] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [30] M. Mitchell, S. Wu, et al., “Model Cards for Model Reporting,” in *FAT**, 2019.
- [31] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [32] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [33] C. G. Northcutt, L. Jiang, and I. L. Chuang, “Confident Learning: Estimating Uncertainty in Dataset Labels,” *JAIR*, vol. 70, pp. 1373–1411, 2021.
- [34] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [35] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [36] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [37] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [38] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [39] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [40] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [41] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London: Butterworths, 1979.
- [42] B. W. Matthews, “Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme,” *Biochimica et Biophysica Acta (BBA) — Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
- [43] D. Chicco and G. Jurman, “The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation,” *BMC Genomics*, vol. 21, p. 6, 2020.

- [44] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in *International Conference on Machine Learning (ICML)*, 2006, pp. 233–240.
- [45] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.
- [46] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [47] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [48] M. P. Naeni, G. F. Cooper, and M. Hauskrecht, "Obtaining Well Calibrated Probabilities Using Bayesian Binning," in *AAAI Conference on Artificial Intelligence*, 2015.
- [49] G. W. Brier, "Verification of Forecasts Expressed in Terms of Probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
- [50] A. H. Murphy, "A New Vector Partition of the Probability Score," *Journal of Applied Meteorology*, vol. 12, no. 4, pp. 595–600, 1973.
- [51] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [52] T.-Y. Lin, M. Maire, S. Belongie, et al., "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.
- [53] J. Hosang, R. Benenson, P. Dollár, et al., "What Makes for Effective Detection Proposals?" *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 4, pp. 814–830, 2016.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] F. Perazzi, J. Pont-Tuset, B. McWilliams, et al., "A Benchmark Dataset and Evaluation Methodology for Video Object Segmentation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [56] G. Csurka, D. Larlus, and F. Perronnin, "What Is a Good Evaluation Measure for Semantic Segmentation?" In *British Machine Vision Conference (BMVC)*, 2013.
- [57] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [58] K. Mikolajczyk and C. Schmid, "A Performance Evaluation of Local Descriptors," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 10, pp. 1615–1630, 2005.
- [59] M. Dusmanu, I. Rocco, T. Pajdla, et al., "D2-Net: A Trainable CNN for Joint Description and Detection of Local Features," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [60] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [61] S. Sagawa, P. W. Koh, T. B. Hashimoto, et al., "Distributionally Robust Neural Networks for Group Shifts," in *ICLR*, 2020.
- [62] M. Arjovsky, L. Bottou, I. Gulrajani, et al., "Invariant Risk Minimization," *arXiv:1907.02893*, 2019.
- [63] I. Gulrajani and D. Lopez-Paz, "In Search of Lost Domain Generalization," in *ICLR*, 2021.
- [64] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," in *ICLR*, 2019.

- [65] R. Taori, A. Dave, V. Shankar, et al., “Measuring Robustness to Natural Distribution Shifts in Image Classification,” in *NeurIPS*, 2020.
- [66] D. Hendrycks and K. Gimpel, “A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks,” in *ICLR*, 2017.
- [67] Y. Ovadia, E. Fertig, et al., “Can You Trust Your Model’s Uncertainty?” In *NeurIPS*, 2019.
- [68] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [69] A. Kendall, Y. Gal, and R. Cipolla, “Multi-Task Learning Using Uncertainty to Weigh Losses,” in *CVPR*, 2018.
- [70] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [71] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [72] F. Milletari, N. Navab, and S.-A. Ahmadi, “V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation,” in *3DV*, 2016.
- [73] H. Rezatofighi, N. Tsoi, J. Gwak, et al., “Generalized Intersection over Union,” in *CVPR*, 2019.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [76] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [77] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [78] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [79] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [80] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [81] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [82] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [83] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [84] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [85] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [86] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [87] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [88] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [89] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [90] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.

- [91] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [92] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [93] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [94] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [95] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [96] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [97] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [98] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [99] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [100] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [101] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.

Chất lượng dữ liệu và nhãn (Data and Label Quality)

PHỤ THUỘC B/05-thiet-ke-bai-toan (đơn vị phân tích (*unit of analysis*), thiên lệch chọn mẫu (*selection bias*)); B/04-nen-tang-hoc-tu-du-lieu (vì sao thêm dữ liệu không phải lúc nào cũng giúp). Chương này là tiền đề trực tiếp của B/07-chia-du-lieu-va-danh-gia.

TÓM TẮT

Chương này nói về chỗ mà trần hiệu năng của một hệ thống thị giác thật sự nằm: không ở kiến trúc, mà ở độ nhất quán của người gán nhãn. Đọc xong bạn đặt được một con số lên trần đó, trước khi huấn luyện mô hình đầu tiên. Bạn phân biệt được hai loại nhiễu nhãn có cơ chế gây hại khác hẳn nhau. Bạn tìm ra vài nghìn nhãn sai trong một trăm nghìn ảnh mà không phải xem hết bằng mắt. Nếu chỉ nhớ một điều: nhãn là một phép đo có sai số riêng, nên mức đồng thuận giữa người gán nhãn là trần trên của mọi con số bạn báo cáo. Và 3% nhiễu hệ thống hại hơn 10% nhiễu ngẫu nhiên, vì mô hình học được cái thứ nhất.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model), huấn luyện (training) và hàm mất mát (loss function)** — mô hình là một hàm có tham số, nhận một ảnh và trả về dự đoán. Hàm mất mát là con số đo mức lệch giữa dự đoán và nhãn của *một* mẫu, và huấn luyện chính là đi tìm bộ tham số làm trung bình con số đó nhỏ nhất. Cần biết vì cả chương bàn về chuyện nhãn sai; khi nhãn sai thì “làm dự đoán sát nhãn” không còn là mục tiêu đúng. Cần biết thêm vì cách tìm nhãn sai rẻ nhất là xếp hạng mẫu theo giá trị mất mát.
- **nhãn (label) và chân trị (ground truth)** — nhãn là đáp án gán cho mỗi mẫu: với phân loại là tên lớp, với phát hiện là khung bao kèm lớp, với phân vùng là mặt nạ từng pixel. *Chân trị* là toàn bộ tập nhãn được coi là đúng khi chấm điểm. Luận điểm trung tâm của chương là chân trị cũng chỉ là một phép đo có sai số riêng, không phải chân lý.
- **accuracy, precision, recall và IoU** — accuracy là tỉ lệ dự đoán đúng trên toàn bộ mẫu; precision là trong số lần mô hình báo “có” thì bao nhiêu phần đúng; recall là trong số ca “có” thật thì mô hình bắt được bao nhiêu phần. IoU là tỉ số giữa diện tích phần giao và diện tích phần hợp của hai vùng, bằng 1 khi trùng khít và 0 khi rời nhau. Ba con số đầu khác nhau rất xa khi một lớp hiếm; IoU là cách đo “hai người vẽ mask có giống nhau không”.
- **mức đồng thuận giữa người gán nhãn (inter-annotator agreement) và Cohen kappa** — cho hai người cùng gán một tập ảnh rồi đếm xem họ trùng nhau ở bao nhiêu phần trăm — đó là mức đồng thuận trần. Cohen kappa sửa con số đó bằng cách trừ đi phần trùng nhau chỉ nhờ may rủi, rồi chia cho phần dư địa còn lại: $\text{kappa} = 0$ nghĩa là chỉ ngang mức đoán bừa, $\text{kappa} = 1$ nghĩa là đồng ý hoàn toàn. Cần nó vì với lớp hiếm, hai người gán bừa vẫn đồng ý tới hơn 90%.
- **trùng lặp gần đúng (near-duplicate)** — hai ảnh khác nhau từng byte nhưng giống hệt nhau với mắt người — cùng một tấm bị nén lại, cắt cúp, đổi kích thước, thêm watermark, hoặc hai khung hình liền nhau trong video. Cần biết vì nếu một ảnh nằm ở tập huấn luyện còn bản sao gần đúng của nó nằm ở tập kiểm tra thì kết quả đo được là kết quả giả.
- **bias — phân biệt ba nghĩa** — (a) *độ lệch* thống kê: sai luôn về một hướng của một phép đo hay một tập dữ liệu so với thực tế; (b) *thiên lệch quy nạp (inductive bias)*: các giả định được cài sẵn vào kiến trúc hay hàm mất mát; (c) tham số cộng thêm b trong tầng tuyến tính $Wx + b$. Chương này luôn dùng nghĩa (a).

- **embedding** — vector vài trăm chiều mà một mạng đã huấn luyện sinh ra để biểu diễn một ảnh; hai ảnh giống nhau về nội dung cho hai vector gần nhau, đo bằng khoảng cách thông thường. Cần biết vì đây là cách duy nhất phát hiện được trùng lặp gần đúng — so từng byte thì không thấy gì.
- **dự đoán ngoài mẫu (out-of-sample) và kiểm định chéo (cross-validation)** — dự đoán ngoài mẫu là dự đoán cho một mẫu mà mô hình *không* được nhìn thấy lúc huấn luyện. Kiểm định chéo là cách lấy được dự đoán như vậy cho *mọi* mẫu: chia dữ liệu thành k phần, lần lượt giữ một phần ra ngoài, huấn luyện trên $k - 1$ phần còn lại rồi đoán cho phần bị giữ. Không có nó thì mọi phép “dùng mô hình soi nhãn” đều vô nghĩa.

CÂU HỎI MỞ ĐẦU

- Q1. Hai người gán nhãn giỏi chỉ đồng ý với nhau ở 85% số ảnh. Vậy con số accuracy 92% mà bạn vừa báo cáo đang đo cái gì — sự thật, hay sự bất đồng của con người?
- Q2. Vì sao 3% nhãn sai lại có thể tàn phá một mô hình nặng hơn 10% nhãn sai, dù ít hơn ba lần?
- Q3. Tập validation của ImageNet — thứ cả ngành dùng để xếp hạng suốt hơn một thập kỷ — chứa khoảng 6% nhãn sai. Bao nhiêu phần của bảng xếp hạng đó là cuộc đua khớp lỗi nhãn?
- Q4. Bạn có 100 nghìn ảnh, nghi vài nghìn nhãn sai, và không có ngân sách xem lại bằng mắt. Làm sao bắt chính mô hình chỉ ra chúng mà không để nó bao che cho lỗi nó đã học thuộc?

4.1 Đặt vấn đề

Ở chương trước, bài toán đã được định nghĩa: đơn vị phân tích là gì, đầu ra dạng nào, mẫu lấy từ đâu. Bây giờ có người ngồi gán nhãn, và câu hỏi trở nên khó chịu hơn nhiều: *nhãn của bạn đến từ con người, và con người thì không nhất quán — vậy trần hiệu năng của hệ thống nằm ở đâu, và làm sao nâng nó lên với một ngân sách hữu hạn?*

Câu hỏi này thường bị bỏ qua vì nó không có vẻ là một câu hỏi kỹ thuật: tập dữ liệu bị coi như hằng số của bài toán, còn công sức thì dồn hết vào phần *mô hình*. Cái giá không hiện ra ngay. Nó hiện ra ba tháng sau, dưới dạng một câu hỏi không trả lời được: đã thử mười kiến trúc, con số kẹt ở 91% và không nhúc nhích, tại sao? Nếu hai người gán nhãn giỏi chỉ đồng ý với nhau ở 90% số ảnh thì 91% chính là chỗ mọi thứ phải dừng, và mười kiến trúc kia chưa bao giờ là biến số quyết định.

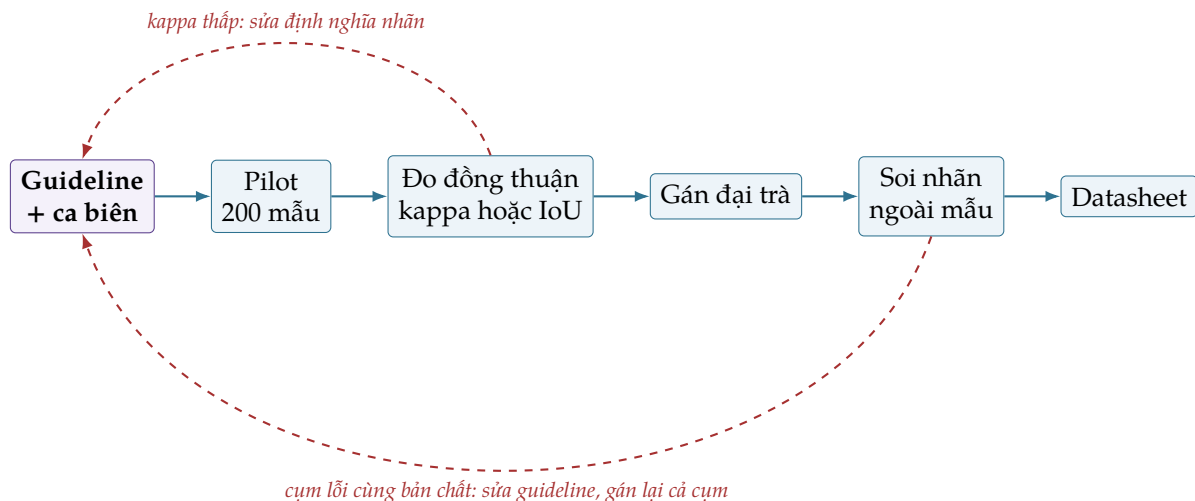
Người đọc đã làm SLAM thấy ngay chuyện này quen. Trong benchmark visual-inertial, **chân trị** (*ground truth*) là đầu ra của một hệ đo khác: Vicon, Leica hay RTK-GNSS. Hệ đó có sai số riêng và độ trễ riêng. Phép **hiệu chỉnh ngoại** (*extrinsic calibration*) nối nó với camera cũng chỉ là một ước lượng, và sai số của phép hiệu chỉnh cộng thẳng vào chân trị. Khi sai số quỹ đạo của thuật toán tụt xuống cùng bậc độ lớn với sai số của hệ tham chiếu, bảng so sánh giữa các thuật toán thôi không còn đo cái nó tự nhận là đo. Chất lượng nhãn trong thị giác máy tính là đúng hiện tượng đó, chỉ khác là ít ai nói ra.

TRỰC GIÁC

Coi tập nhãn như một hệ đo tham chiếu chứ không như chân lý. Mọi hệ đo có ba đặc trưng: một độ phân giải (chi tiết nhỏ hơn mức này thì nó không thấy), một sai số ngẫu nhiên (đo lại thì lệch), và một sai số hệ thống (lệch cùng một hướng mọi lần). Ba thứ này quyết định bạn được phép kết luận gì. Một thước kẹp chia vạch 0,1 mm không cho phép bạn tranh luận về chênh lệch 0,02 mm, dù có đo bao nhiêu lần đi nữa; và nếu thước bị cong, đo nhiều lần chỉ làm bạn tự tin hơn vào một con số sai.

4.2 Gán nhãn là một quy trình đo (Annotation as Measurement)

Nói gọn thì: chất lượng nhãn không phải thứ đo được ở khâu nghiệm thu, mà là thứ *được tạo ra* ở khâu quy trình. Kiểm tra cuối chỉ bắt được nhiều ngẫu nhiên; nó gần như mù với nhiều hệ thống, vì nhiều hệ thống trông hoàn toàn nhất quán nên trông giống như đúng. Hình 4.1 cho thấy điều làm nên khác biệt không phải các khối, mà là hai vòng phản hồi quay ngược về guideline.



Hình 4.1. Quy trình gán nhãn. Các khối là phần ai cũng làm; hai mũi tên đứt đỏ là phần quyết định chất lượng. Vòng trên bắt lỗi *định nghĩa nhãn* khi còn rẻ (200 mẫu); vòng dưới bắt lỗi *hệ thống* sau khi đã có mô hình. Bỏ hai vòng này thì quy trình biến thành dây chuyền một chiều, và mọi hiểu nhầm ban đầu được nhân lên đúng bằng kích thước tập dữ liệu.

A. Từ “có nhãn” tới “biết nhãn tốt tới đâu”

Mạch 1 — bốn bước dựng trần hiệu năng

1 Vấn đề

chất lượng nhãn không tự có.

Giải pháp

viết guideline kèm ca biên trước khi gán mẫu đầu tiên, chạy pilot vài trăm mẫu, đọc lại chỗ bất đồng, rồi *sửa guideline* và gán lại. Sản phẩm chính của pilot là vòng sửa guideline chứ không phải mấy trăm nhãn kia.

Hệ quả

chất lượng được quyết định ở khâu hiệu chuẩn, không ở khâu đọc số.

2 Vấn đề

không biết nhãn tốt tới đâu.

Giải pháp

đo **mức đồng thuận giữa người gán nhãn** (*inter-annotator agreement*) bằng cách cho hai người gán chồng lên nhau một tập con.

Hệ quả

nếu hai người chỉ đồng ý ở 85% số mẫu thì accuracy 92% trên test không nói về sự thật, nó nói về mức mô hình khớp với *nhiều của người gán*. Đồng thuận là trần trên, và mọi phần trăm giành được phía trên trần đó đáng nghi hơn đáng mừng.

3 Vấn đề

không phải nhiễu nào cũng như nhau.

Giải pháp

tách **nhiều ngẫu nhiên** (*random label noise*) (người gán mệt, bấm nhầm, kéo lệch một biên — phân bố đều, chủ yếu làm chậm hội tụ) khỏi **nhiều hệ thống** (*systematic label noise*) (guideline mơ hồ ở một chỗ cụ thể: mọi người gán đều bỏ vật nhỏ hơn 20 pixel, đều coi xe đạp có người ngồi là “người”).

Hệ quả mạng học được cái thứ hai và tái tạo nó rất trung thành, nên 3% nhiễu hệ thống nguy hiểm hơn 10% nhiễu ngẫu nhiên; việc mạng sâu thừa sức thuộc lòng cả một tập nhãn ngẫu nhiên hoàn toàn [15] cho thấy **sức chứa** (*capacity*) không phải hàng rào bảo vệ nào cả.

4 Vấn đề

Giả định

Hệ quả

ta coi chân trị là chân lý.

nó cũng chỉ là một ước lượng, có sai số của thiết bị đo và của người gán.

khi sai số của mô hình tụt xuống dưới sai số của chân trị, mọi bảng so sánh mất ý nghĩa và bạn đang tối ưu nhiễu. Đây là lúc hành động đúng là đi cải thiện phép đo, không phải thử kiến trúc thứ mười một.

B. Cohen kappa, và vì sao “đồng ý 95%” có thể là con số rỗng

Đo đồng thuận bằng tỉ lệ đồng ý trần (*raw agreement*) có một lỗ hổng chết người: khi một lớp hiếm, hai người gán bữa cũng đồng ý gần như tuyệt đối. *Cohen's kappa* [31] sửa chuyện đó bằng cách trừ đi phần đồng thuận giải thích được bởi ngẫu nhiên:

$$\kappa = \frac{p_o - p_e}{1 - p_e}.$$

Công thức này nói: *trong khoảng cải thiện còn lại phía trên mức ngẫu nhiên, hai người gán đã đi được bao nhiêu phần*. Mẫu số $1 - p_e$ là toàn bộ dư địa; tử số là phần thực sự đạt được. Ở đây p_o là tỉ lệ đồng ý quan sát được, p_e là tỉ lệ đồng ý kỳ vọng nếu hai người gán độc lập nhưng giữ nguyên tần suất từng lớp của mình.

Hai trường hợp tính tay được, cùng cỡ mẫu 100 ảnh nhị phân, cho thấy toàn bộ vấn đề:

- **Lớp dương phổ biến.** Cùng dương 45, cùng âm 40, bất đồng 15 (A dương B âm 10, A âm B dương 5). Vậy $p_o = 0,85$; A nói dương 55% số lần, B nói dương 50%, nên $p_e = 0,55 \cdot 0,50 + 0,45 \cdot 0,50 = 0,50$ và $\kappa = 0,35/0,50 = 0,70$.
- **Lớp dương hiếm.** Cùng dương 3, cùng âm 92, bất đồng 5. Bây giờ $p_o = 0,95$ — nghe tuyệt vời — nhưng $p_e = 0,06 \cdot 0,05 + 0,94 \cdot 0,95 \approx 0,896$, nên $\kappa \approx 0,054/0,104 \approx 0,52$.

Bảng đồng thuận trông đẹp hơn hẳn lại cho kappa thấp hơn hẳn. Bài học không phải “kappa tốt hơn accuracy”. Với lớp hiếm, cả đồng thuận trần lẫn accuracy đều bị nền âm tính khổng lồ kéo lên, vì mẫu số của chúng là toàn bộ tập chứ không phải riêng phần đáng quan tâm. Hai con số vì thế nói dối theo cùng một cách. Đây chính là lý do B/07 phải dành hẳn một mục cho việc chọn metric.

→ **tiền đề B/07** — nền âm tính khổng lồ kéo cả accuracy lẫn đồng thuận trần lên; ở đó cùng cơ chế này làm AUROC nói dối khi lớp dương chỉ chiếm một phần trăm.

Kappa chỉ định nghĩa cho nhãn phân loại rời rạc. Với mask, bbox hay keypoint, hãy đo đồng thuận bằng chính metric hình học của bài toán: IoU giữa hai mask của hai người gán, hoặc sai số pixel giữa hai bộ keypoint. Và đọc *phân bố* của con số đó chứ đừng đọc trung bình, vì bất đồng giữa người gán dồn hết vào phần đuôi — chỗ vật nhỏ và biên mờ. Cách này không có nền lý thuyết gọn như kappa nhưng trả lời đúng câu hỏi cần: mô hình còn bao nhiêu chỗ trống trước khi chạm trần người.

4.3 Lỗi nhãn trong chính tập test chuẩn (Label Errors in Benchmarks)

Có một phản bác hợp lý: nhiễu nhãn nghe như vấn đề của dữ liệu tự thu thập, còn ImageNet hay COCO đã qua bao năm soi xét. Phản bác này sai theo một cách đã được đo: khảo sát mười tập

test chuẩn của học máy tìm thấy tỉ lệ lỗi nhãn trung bình vài phần trăm, riêng tập validation của ImageNet khoảng 6% [32]. Hệ quả sắc hơn con số: khi sửa lại các nhãn sai đó, *thứ tự xếp hạng giữa các mô hình có thể đảo*, và mô hình lớn hơn — thứ khớp tập test gốc tốt hơn — hoá ra cũng là thứ khớp nhiều nhãn tốt hơn.

Từ đó ra câu hỏi thực dụng: tìm nhãn sai trong 100 nghìn ảnh bằng cách nào? Câu trả lời là dùng chính mô hình làm cảm biến, ở ba mức từ rẻ tới đắt:

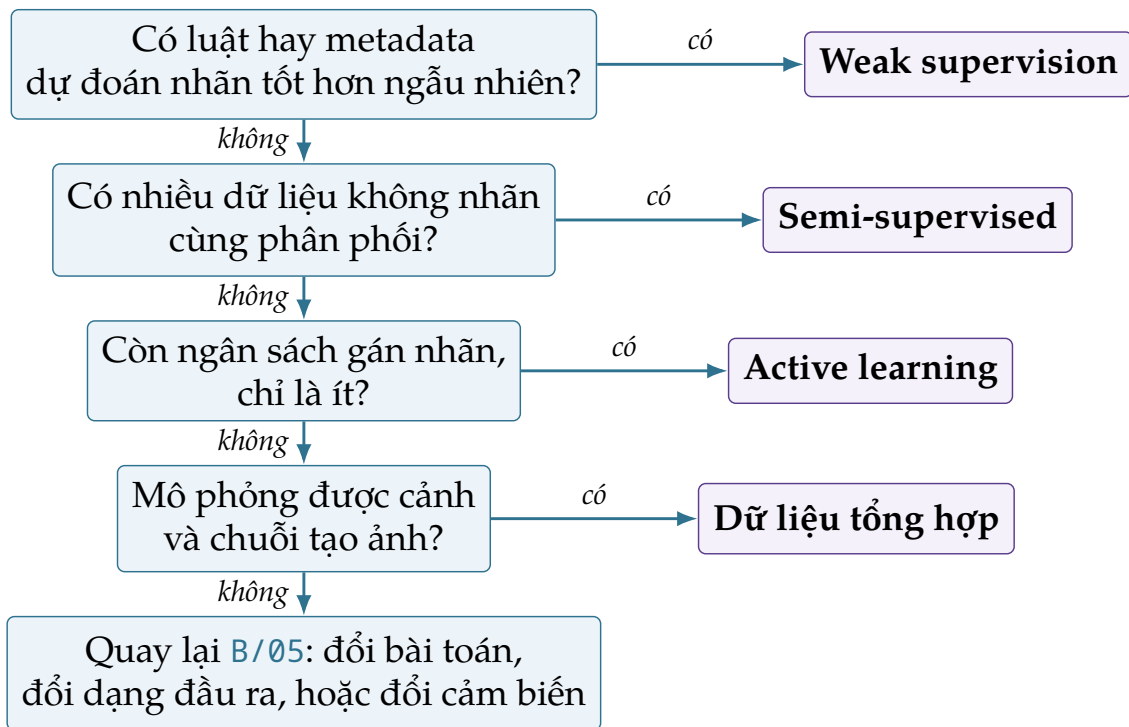
- **Lọc theo loss.** Xếp hạng mẫu theo giá trị mất mát sau khi huấn luyện; đuôi cao chứa cả nhãn sai lẫn mẫu khó. Rẻ nhất, nhiều nhất, nhưng thường đủ để bắt đầu.
- **Confident learning** [33] — ước lượng ma trận chung giữa nhãn quan sát được và nhãn tiềm ẩn từ xác suất dự đoán *ngoài mẫu*, rồi suy ra ngưỡng cắt từ chính ma trận đó thay vì đặt ngưỡng bằng tay. Đây là mức nên dùng mặc định.
- **influence function** — hỏi mẫu huấn luyện nào đang kéo dự đoán trên tập val đi sai. Đắt nhất, nhưng là công cụ duy nhất trả lời được câu hỏi “mẫu nào gây ra lỗi này”.

Cả ba đều giả định mô hình đủ tốt để sự bất đồng của nó mang tín hiệu; với một mô hình chưa học được gì, danh sách nghi vấn chỉ là danh sách ngẫu nhiên. Và cả ba đều đánh dấu nhầm các mẫu *khó nhưng đúng*, nên đây không phải bộ lọc tự động mà là bộ sắp thứ tự cho mắt người: nó biến “xem 100k ảnh” thành “xem 2k ảnh đáng ngờ nhất”. Ai để nó tự xoá mẫu sẽ vô tình xoá đúng phần đuôi khó — chính phần mô hình cần nhất — và tạo ra một tập dữ liệu sạch một cách giả tạo. Khi phát hiện một cụm lỗi cùng bản chất (mọi vật bị che quá nửa đều bị bỏ), đó là lỗi guideline chứ không phải lỗi người gán: sửa guideline rồi gán lại cả cụm, đúng vòng phản hồi dưới của Hình 4.1.

↔ **tiền đề** E/24 — cùng một kỹ thuật “dùng mô hình soi dữ liệu”, ở đó dùng để phân rã lỗi theo loại thay vì để tìm nhãn sai.

4.4 Ngân sách hữu hạn: bốn đường thoát (Label-Efficient Learning)

Giả sử quy trình đã tốt và ta biết nhãn đáng tin tới đâu. Vẫn còn ràng buộc cuối: không đủ tiền gán nhãn. Cơ chế của bốn hướng thoát đều đơn giản, nên đó không phải chỗ đáng học. Chỗ đáng học là *mỗi hướng mua sự rẻ bằng một giả định khác nhau*. Hình 4.2 xếp bốn hướng theo đúng thứ tự để kiểm chứng của giả định đó.



Hình 4.2. Cây quyết định chọn hướng giảm chi phí nhãn. Thứ tự các câu hỏi không tùy tiện: nó đi từ hướng có giả định dễ kiểm chứng nhất (bạn *biết* mình có luật hay không) tới hướng có giả định khó kiểm chứng nhất (mô phỏng “đủ gần” là một phán đoán). Nhánh cuối cùng — không hướng nào áp dụng được — là một kết quả hợp lệ, và nó nói rằng vấn đề nằm ở khâu thiết kế bài toán chứ không ở khâu dữ liệu.

Bảng 4.1. Bốn cách giảm chi phí nhãn. Cột cuối đáng đọc nhất: mỗi hướng hòng theo một kiểu riêng, và cả bốn kiểu hòng đều tạo ra nhiều *hệ thống*.

Hướng	Cơ chế	Giả định	Hòng khi nào
giám sát yếu (<i>weak supervision</i>)	Sinh nhãn từ heuristic, luật, metadata	Heuristic tốt hơn ngẫu nhiên và sai một cách đo được	Nhiều bám đúng thiên lệch của heuristic → mô hình học lại chính heuristic đó
học chủ động (<i>active learning</i>)	Chọn mẫu đáng gán nhất theo độ bất định	Ước lượng bất định đáng tin	Bất định kém hiệu chuẩn; tập thu được lệch phân phối nên khó tái dùng cho mô hình khác
học bán giám sát (<i>semi-supervised learning</i>), <i>pseudo-labeling</i>	Khai thác dữ liệu không nhãn quanh cụm	Biên quyết định nằm ở vùng mật độ thấp	Mô hình khởi đầu lệch → vòng lặp tự củng cố lỗi, càng train càng tự tin sai
dữ liệu tổng hợp (<i>synthetic data</i>), sim2real	Render cảnh có nhãn miễn phí	Mô phỏng đủ gần ở những đặc trưng mô hình thật sự dùng	Khoảng cách miền nằm ở khâu tạo ảnh (nhiều cảm biến, ISP, quang sai) chứ không ở khâu hình học

Bảng 4.1 có một quy luật ngầm đáng nhớ: cả bốn hướng đều *đổi nhiều ngẫu nhiên đắt tiền lấy nhiều hệ thống rẻ tiền* — hợp lý khi bạn biết mình đang đổi gì, là bấy khi không biết. Đây chính là trục đánh đổi “thêm dữ liệu hay thêm giả định” ở B/04, lần này xuất hiện dưới dạng một quyết định về ngân sách. Riêng sim2real, chỗ hờ hiếm khi nằm ở hình học (render chính xác tuyệt đối) mà nằm ở chuỗi tạo ảnh: nhiễu đọc của cảm biến, đường cong đáp ứng, khử nhiễu và làm nét của ISP, ảnh mờ do chuyển động — đó là lý do A/02-vat-1y-tao-anh có giá trị rất thực tế ở đây.

↔ **ràng buộc bởi A/02** — khoảng cách giữa ảnh render và ảnh thật nằm ở chuỗi tạo ảnh, nên nó bị chặn bởi mức bạn mô phỏng được cảm biến và ISP.

↔ **cùng nguyên lý với B/04** — cả bốn hướng đều là “thêm giả định thay cho thêm dữ liệu”, đúng trực đánh đổi đã dựng ở chương nền tảng.

4.5 Ba vấn đề âm thầm (Duplicates, Missingness, Documentation)

Ba thứ còn lại ít hào nhoáng nhưng gây thiệt hại lớn, và chúng có chung một đặc điểm: không thứ nào báo lỗi.

- **trùng lặp gần đúng (*near-duplicate*)**. Khi kết quả đẹp một cách khó hiểu, đây nên là nghi ngờ đầu tiên. Ảnh crawl từ web có vô số bản resize, crop, nén lại, thêm watermark; video thì mọi frame liền kề gần như trùng nhau.
 - **Vì sao hash thất bại** — chỉ cần nén lại một lần là mã băm đổi hoàn toàn, trong khi ảnh với mắt người vẫn y hệt.
 - **Công cụ đúng** — khử trùng lặp trên không gian *embedding* với một ngưỡng khoảng cách, hoặc trên *perceptual hash*.
 - **Nổi sang nền có sẵn** — hai frame cách nhau 33 ms không phải hai quan sát độc lập, đúng như hai keyframe liền kề trong bundle adjustment không cho hai ràng buộc độc lập.
- **dữ liệu thiếu (*missing data*): ba loại, không phải một**. Cách dữ liệu bị thiếu tự nó mang thông tin.
 - **MCAR** (thiếu hoàn toàn ngẫu nhiên) — bỏ mẫu là an toàn, chỉ tốn dữ liệu.
 - **MAR** (thiếu phụ thuộc các biến *quan sát được*) — bù được bằng mô hình.
 - **MNAR** (thiếu phụ thuộc chính giá trị bị thiếu) — trường hợp độc: ảnh quá tối nên người gán bỏ qua, cảm biến depth trả về lỗi đúng ở bề mặt bóng và vật trong suốt. Áp cách xử lý của MCAR lên đây không loại được thiên lệch mà tạo ra một thiên lệch mới, và thiên lệch mới thì vô hình vì nó vừa được “dọn sạch”.
- **Trí nhớ của tổ chức**. Sáu tháng sau không ai còn nhớ tập dữ liệu này là gì: thư mục còn đó nhưng lý do chọn nguồn, tiêu chí loại ảnh, phiên bản guideline, mức đồng thuận đo được đều bốc hơi. *Datasheets for Datasets* [28] và *Model Cards* [30] là câu trả lời rẻ đến bất ngờ: vài trang về động cơ thu thập, quy trình, thành phần, lớp bị thiếu, và những cách dùng *không* nên dùng. Đây là một dạng cụ thể của khoản nợ kỹ thuật ẩn trong hệ thống học máy [4] — chi phí không nằm ở lúc viết mà ở lúc không viết.

Và cuối cùng, thứ không công cụ nào thay thế được: *ngồi xem 200 ảnh bằng mắt*. Đây là kinh nghiệm thực hành, không phải kết quả đo. Nhưng nó nhất quán tới mức đáng coi là quy tắc. Lần nào cũng lộ ra thứ mà không thống kê nào cho thấy: một lớp được gán bằng hai định nghĩa ở hai đợt, hay một watermark xuất hiện đúng ở các mẫu dương. Lý do đơn giản là thống kê chỉ đếm được thứ bạn đã nghĩ ra để đếm.

↔ **hệ quả của B/05** — mọi thứ chương này phải dọn dẹp đều bắt nguồn từ định nghĩa nhãn và khung lấy mẫu đã chốt ở khâu thiết kế bài toán.

4.6 Giới hạn và trường hợp hỏng

Toàn bộ chương này đứng trên vài giả định nên nói rõ. Khái niệm “trần đồng thuận” giả định có một sự thật khách quan mà người gán đang cố xấp xỉ; với các nhãn mang tính đánh giá — mức độ hư hỏng, độ thẩm mỹ, mức nguy hiểm — giả định đó sai, và bất đồng giữa người gán là tín hiệu chứ không phải nhiễu. Khi đó thứ cần làm không phải ép về một nhãn mà là giữ lại phân bố nhãn và huấn luyện với nhãn mềm.

Confident learning và mọi biến thể “dùng mô hình soi nhãn” giả định nhiều nhãn độc lập với đặc trưng đầu vào khi đã biết lớp thật; với nhiều hệ thống gắn với ngoại hình, giả định này vỡ đúng ở chỗ ta quan tâm nhất và phương pháp sẽ báo sạch một tập vẫn còn lệch. Đo đồng thuận cũng hỏng lạng lẽ khi hai người gán do cùng một người huấn luyện: họ chia sẻ cùng một hiểu nhầm nên kappa cao mà cả hai cùng sai — thuốc giải là để một người thứ ba, không đọc guideline của bạn, gán mù một tập nhỏ.

CẠM BÃY

Cạm bẫy phổ biến nhất là kiểm tra chất lượng bằng chính mô hình đã được train trên tập cần kiểm tra, rồi kết luận “dữ liệu sạch” vì loss thấp đều. Mô hình đã hấp thụ nhiều hệ thống và bây giờ đồng ý với nó — bạn đang hỏi bị cáo xem mình có tội không. Muốn tín hiệu có nghĩa, dự đoán dùng để soi nhãn phải là dự đoán *ngoài mẫu* (**kiểm định chéo** (*cross-validation*)), đúng như confident learning yêu cầu.

CẦN NHỚ

Mức đồng thuận giữa người gán nhãn là trần trên của mọi con số bạn báo cáo, nên hãy đo nó *trước* khi huấn luyện mô hình đầu tiên. Và 3% nhiễu hệ thống hại hơn 10% nhiễu ngẫu nhiên, vì mô hình học được cái thứ nhất và chỉ chậm hội tụ vì cái thứ hai. [T2]

4.7 Thực hành

Repo và tài nguyên. [cleanlab/cleanlab](#) (confident learning); [visual-layer/fastdup](#) và [idealo/imagededup](#) (trùng lặp gần đúng); [voxel51/fiftyone](#) (soát dữ liệu có hệ thống); [snorkel-team/snorkel](#) (weak supervision); [baal-org/baal](#) và [modAL](#) (active learning); [lightly-ai/lightly](#) (chọn mẫu theo embedding). Dữ liệu tổng hợp: Blender kèm [BlenderProc](#), hoặc NVIDIA Omniverse Replicator. Danh sách này chốt tại tháng 9/2026 và tên công cụ sẽ cũ trong 6–12 tháng; hạ tầng (FiftyOne, Blender) bền hơn nhiều so với tên phương pháp đang dẫn đầu.

Câu hỏi kiểm tra hiểu. Nhãn nhiễu ngẫu nhiên 10% và nhãn nhiễu hệ thống 3% — cái nào hại hơn và cơ chế gây hại khác nhau ở đâu? Nếu mức đồng thuận giữa người gán nhãn là 85% thì con số accuracy 92% trên test đang nói điều gì? Vì sao khử trùng lặp bằng mã băm byte gần như vô dụng với ảnh web?

Bài tập phụ. Làm hỏng 10% nhãn CIFAR-10 ngẫu nhiên, train một mạng nhỏ, dùng cleanlab tìm lại, báo cáo precision/recall của việc phát hiện. Lặp lại với nhiễu hệ thống (đổi lớp A thành B ở 3% số mẫu): trường hợp sau khó phát hiện hơn hẳn dù tỉ lệ nhiễu thấp hơn ba lần.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Chân trị	Tập nhãn được coi là đúng khi chấm điểm. Nó do thiết bị và con người tạo ra nên có sai số riêng, và khi sai số của mô hình tụt xuống dưới sai số đó thì mọi bảng so sánh mất ý nghĩa.
••• <i>Inter-annotator agreement</i>	Mức đồng thuận giữa người gán nhãn: tỉ lệ hai người gán độc lập trùng nhau trên cùng một tập con. Nó là trần trên của mọi con số bạn báo cáo, nên phải đo trước khi huấn luyện mô hình đầu tiên.
••• Nhiễu hệ thống	Một quy tắc sai được lặp lại nhất quán, ví dụ mọi vật nhỏ hơn 20 pixel đều bị bỏ. Mô hình học được nó và tái tạo nó trên dữ liệu mới, nên nó không tự lộ ra ở khâu nghiệm thu.
•• Nhiễu ngẫu nhiên	Sai sót rải đều: người gán mệt, bấm nhầm, kéo lệch một biên. Nó chủ yếu làm chậm hội tụ chứ không cài một thiên lệch vào mô hình.
•• Vì sao 3% nhiễu hệ thống hại hơn 10% nhiễu ngẫu nhiên?	Vì mạng học được quy tắc sai và áp lại nó cho dữ liệu mới. Nhiễu ngẫu nhiên thì trung bình về không, còn dung lượng lớn không cứu được: mạng sâu thừa sức thuộc lòng cả một tập nhãn ngẫu nhiên.
••• Cohen kappa	Mức đồng thuận sau khi trừ đi phần trùng nhau do may rủi, chia cho phần dư địa còn lại. Nó chỉ định nghĩa cho nhãn rời rạc; với mask hay keypoint phải đo bằng IoU hoặc sai số pixel.
•• Hai người gán đồng ý 95% mà kappa chỉ 0,52 — nguyên nhân?	Lớp dương quá hiếm, nên nền âm tính khổng lồ kéo tỉ lệ đồng ý trần lên. Phân biệt bằng cách nhìn tỉ lệ lớp: nếu lớp dương dưới vài phần trăm thì luôn đọc kappa, đừng đọc tỉ lệ đồng ý.
••• Confident learning	Ước lượng ma trận chung giữa nhãn quan sát được và nhãn tiềm ẩn, rồi suy ngưỡng cắt từ chính ma trận đó. Nó giả định nhiều độc lập với ngoại hình khi đã biết lớp thật, nên vỡ với nhiễu hệ thống gắn ngoại hình.
•• Vì sao phải soi nhãn bằng dự đoán ngoài mẫu?	Vì mô hình đã hấp thụ nhiễu hệ thống và bây giờ đồng ý với nó. Dự đoán dùng để soi nhãn phải là dự đoán ngoài mẫu, lấy bằng kiểm định chéo.
•• Weak supervision	Sinh nhãn từ heuristic, luật hoặc metadata thay vì từ người. Giá phải trả là nhiễu bám đúng thiên lệch của heuristic, nên mô hình học lại chính heuristic đó.
•• Active learning	Chỉ gán nhãn những mẫu mô hình bất định nhất. Giá phải trả là tập thu được lệch phân phối, nên khó tái dùng cho một mô hình khác.
• Pseudo-labeling	Dùng dự đoán của chính mô hình làm nhãn cho dữ liệu không nhãn. Nó giả định biên quyết định nằm ở vùng mật độ thấp, và hóng thành vòng lặp tự củng cố lỗi nếu mô hình khởi đầu lệch.
•• Sim2real	Huấn luyện trên ảnh render rồi triển khai trên ảnh thật. Khoảng cách hiếm khi nằm ở hình học mà nằm ở chuỗi tạo ảnh: nhiễu cảm biến, đường cong đáp ứng, khử nhiễu của ISP, ảnh mờ do chuyển động.
••• Trùng lặp gần đúng	Hai ảnh khác nhau từng byte nhưng giống hệt với mắt người: bản nén lại, cắt cúp, đổi kích thước, hay hai khung hình liên nhau. Phải khử trước khi chia dữ liệu.
•• Vì sao mã băm byte vô dụng với trùng lặp gần đúng?	Vì chỉ cần nén lại một lần là mã băm đổi hoàn toàn, trong khi nội dung ảnh không đổi. Công cụ đúng là khoảng cách trên embedding hoặc <i>perceptual hash</i> .
•• Mô hình mới vọt 8 điểm trên test mà không đổi gì lớn — nghi ngờ đầu tiên?	Trùng lặp gần đúng giữa train và test, thường do ảnh crawl web hoặc khung hình liên nhau. Phân biệt với “mô hình thật sự tốt” bằng cách khử trùng lặp theo embedding rồi đo lại: nếu con số tụt hẳn về mức cũ thì thủ phạm là bản sao.
• MNAR	Dữ liệu thiếu mà lý do thiếu phụ thuộc chính giá trị bị thiếu, ví dụ ảnh quá tối nên người gán bỏ qua. Xử lý nó như thiếu ngẫu nhiên sẽ tạo ra một thiên lệch mới đã được “dọn sạch” nên vô hình.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• Datasheet for datasets	Vài trang ghi nguồn, quy trình gán nhãn, mức đồng thuận đo được, các lớp bị thiếu, và những cách dùng không nên dùng. Không có nó thì sáu tháng sau tập dữ liệu thành một thư mục bí ẩn.
•• Khi nào bắt đầu của người gán là tín hiệu?	Với nhãn mang tính đánh giá: mức hư hỏng, độ thẩm mỹ, mức nguy hiểm. Khi đó hãy giữ lại phân bố nhãn và huấn luyện với nhãn mềm thay vì ép về một nhãn.
•• Thứ mười kiến trúc mà con số kết ở 91% — đo gì tiếp theo?	Đo mức đồng thuận giữa hai người gán trên hai trăm mẫu. Nếu nó cũng quanh 90% thì bạn đã chạm trần và phải đầu tư vào nhãn; nếu nó 98% thì vấn đề đúng là ở mô hình.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
cleanlab	Cài đặt trực tiếp confident learning: ước lượng ma trận chung giữa nhãn quan sát và nhãn tiềm ẩn từ dự đoán ngoài mẫu	Chính nhóm này công bố khảo sát lỗi nhãn trên mười tập test chuẩn
FiftyOne (voxel51)	Biến “xem 200 ảnh bằng mắt” thành thao tác có hệ thống: lọc theo lát, sắp theo độ nghi ngờ, so hai phiên bản nhãn	Hạ tầng soát dữ liệu, bền hơn tên mô hình
Trợ lý gán nhãn bằng mô hình nền tảng (SAM 3, Grounding DINO)	Chính là giám sát yếu, chỉ khác là nguồn nhãn thô đến từ một mô hình lớn thay vì từ luật viết tay	Tên mô hình sẽ đổi nhanh; cơ chế và cái giá (nhiều hệ thống theo thiên lệch của mô hình nguồn) thì không
snorkel	Gộp nhiều luật nhiều thành nhãn xác suất, mô hình hoá luôn độ tin cậy của từng luật	Ý tưởng đã lan sang các pipeline lọc dữ liệu huấn luyện mô hình ngôn ngữ
Dataset card và model card trên Hugging Face Hub	Đúng nội dung của Datasheets và Model Cards, được đưa thành mục mặc định khi công bố tập dữ liệu	Bảng chứng rõ nhất rằng ý tưởng tài liệu hoá đã thành chuẩn ngành

MINI PROJECT SLAM • Đo mức đồng thuận của chính bạn trên bài toán đóng vòng • 1–2 ngày

Mục tiêu	biến “trần hiệu năng” từ một khái niệm thành một con số của riêng bạn, trên đúng quyết định mà mọi hệ đóng vòng phải đưa ra: hai khung hình này có phải cùng một chỗ hay không.
Dữ liệu	200 cặp khung hình rút từ hai chuỗi EuRoC (ví dụ MH_03 và V1_02), trộn ba nhóm với tỉ lệ ngang nhau: cặp thật sự cùng chỗ, cặp gần nhau về thời gian nhưng khác chỗ, và cặp ngẫu nhiên.
Các bước	viết guideline một trang <i>trước</i> khi gán mẫu đầu tiên, chốt rõ ca biên (bao nhiêu phần cảnh chồng lấn thì tính là cùng chỗ, lệch góc nhìn tới đâu thì thôi, ảnh mờ tới mức nào thì bỏ). Gán lần một. Chờ ba tới năm ngày. Gán lại đúng 200 cặp đó với thứ tự đã xáo trộn. Lập bảng 2×2 giữa hai lần gán, tính p_o , p_e và kappa bằng tay.
Xong khi	bạn có con số kappa của chính mình kèm bảng 2×2 , và một danh sách ít nhất mười cặp bạn tự mâu thuẫn với bản thân, mỗi cặp kèm đúng một dòng sửa guideline.
Nối với	B/07-chia-du-lieu-va-danh-gia dùng lại chính 200 cặp này làm tập test có nhãn cho mini project chương sau; E/24-thi-nghiem-va-ablation đọc kappa như trần trên khi so hai bộ phát hiện đóng vòng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Nó đo mức mô hình khớp với *nhiều của người gán*, không đo sự thật. 85% là trần: mọi phần trăm giành được phía trên nó chỉ nói rằng mô hình đã học luôn cả các hiểu nhầm nhất quán trong guideline.
- A2. Vì hai loại nhiễu có cơ chế gây hại khác hẳn. Nhiễu ngẫu nhiên phân bố đều nên chủ yếu làm chậm hội tụ; nhiễu hệ thống là một quy tắc sai được lặp lại, và mô hình học được quy tắc đó rồi tái tạo nó trên dữ liệu mới.
- A3. Để đảo thứ hạng: khi sửa lại nhãn, mô hình lớn hơn — thứ khớp tập test gốc tốt hơn — hoá ra cũng là thứ khớp nhiều nhãn tốt hơn. Một phần cuộc đua nhiều năm là cuộc đua khớp lỗi nhãn, không phải khớp thực tế.
- A4. Bằng dự đoán *ngoài mẫu*: chia k phần, mỗi mẫu chỉ được chấm bởi mô hình chưa nhìn thấy nó, rồi dùng confident learning để suy ngưỡng từ ma trận nhầm lẫn. Dùng mô hình đã train trên chính tập đó thì nó chỉ xác nhận nhiễu nó đã hấp thụ.

NÓI LẠI THẬT ĐƠN GIẢN

Tưởng tượng bạn thuê hai người chấm cùng một tập hai trăm bài văn, thang mười điểm. Chấm xong, bạn so hai bảng điểm và thấy trung bình họ lệch nhau một điểm rưỡi. Bây giờ bạn xây một cái máy chấm tự động và khoe rằng nó chỉ lệch nửa điểm so với bảng điểm của người thứ nhất. Nửa điểm đó không có nghĩa là máy giỏi hơn người. Nó chỉ có nghĩa là máy đã học được thói quen chấm của đúng một người, kể cả những chỗ người đó chấm sai.

Bài toán gốc là như vậy: thứ bạn dùng làm đáp án cũng do người làm ra, nên nó có sai số riêng. Mẹo thoát ra gồm hai bước rẻ tiền. Một, cho hai người chấm chồng lên nhau một phần nhỏ rồi đo xem họ lệch bao nhiêu; con số đó là mức tốt nhất mà máy có quyền đạt tới, và mọi thứ vượt qua nó nên bị nghi ngờ. Hai, viết trước cách xử lý các bài khó — bài lạc đề nhưng viết hay thì mấy điểm — rồi sửa bản hướng dẫn chấm, chứ không sửa từng bài một.

Cái giá là vài ngày làm việc nhằm chán trước khi có mô hình đầu tiên, cộng với việc phải thừa nhận rằng một phần công sức trước đây của bạn đã đổ vào đuổi theo nhiễu.

Chỗ không rút gọn được: ranh giới giữa “bài bị chấm sai” và “bài khó nhưng chấm đúng” thì không nói gọn được. Mọi cách diễn đạt đơn giản đều biến nó thành một quy tắc máy móc, trong khi chính ranh giới đó là thứ mà cả người lẫn máy đang phải học.

Một câu: Trước khi hỏi mô hình giỏi tới đâu, hãy hỏi đáp án của bạn đúng tới đâu — vì con số thứ hai chặn trên con số thứ nhất.

4.8 Kết nối

Chương này là phần thực thi của [B/05-thiet-ke-bai-toan](#): ở đó ta quyết định nhãn là gì, ở đây ta đối mặt với việc nhãn *trở thành* gì sau khi qua tay người. Nó là tiền đề bắt buộc của [B/07-chia-du-lieu-va-danh-gia](#), vì trùng lặp gần đúng chính là một dạng rò rỉ và trần đồng thuận quyết định chênh lệch nào giữa hai mô hình mới đáng bàn. Nó đối lập hữu ích với [B/08-dich-chuyen-phan-phoi](#): ở đây nhãn sai vì con người không nhất quán, ở đó nhãn đúng nhưng thể giới đã đổi. Phần “dùng mô hình soi nhãn” quay lại đầy đủ ở [E/24-thi-nghiem-va-ablation](#).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao mức đồng thuận giữa người gán nhãn là trần trên của accuracy, chứ không chỉ là một con số tham khảo?
2. Cùng một bảng đồng thuận cho $p_o = 0,95$ lại có thể cho kappa thấp hơn một bảng có $p_o = 0,85$. Cơ chế nào gây ra chuyện đó, và nó cảnh báo gì về accuracy?

3. Confident learning giả định gì về cấu trúc của nhiễu, và loại nhiễu nào làm giả định đó vỡ đúng ở chỗ ta cần nhất?
4. Vì sao dùng chính mô hình đã train trên tập đó để soi nhãn lại cho kết luận vô nghĩa, và sửa bằng cách nào?
5. Bạn có 500 giờ công gán nhãn: khi nào tiêu vào việc *gán lại* dữ liệu cũ có giá trị hơn gán thêm dữ liệu mới?
6. Trong cây quyết định ở Hình 4.2, vì sao câu hỏi về heuristic lại đứng trước câu hỏi về mô phỏng?

Nguồn

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *Rec-Sys*, 2019.
- [3] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [4] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [5] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [6] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [7] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [8] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [9] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [10] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [11] V. N. Vapnik, “An overview of statistical learning theory,” *IEEE Trans. Neural Networks*, vol. 10, no. 5, pp. 988–999, 1999.
- [12] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [13] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [15] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [16] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [17] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [18] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE Trans. Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.
- [19] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.

- [20] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [21] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [22] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [23] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [24] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [25] R. Geirhos, J.-H. Jacobsen, C. Michaelis, et al., “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.
- [26] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [27] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [28] T. Gebru, J. Morgenstern, et al., “Datasheets for Datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [29] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [30] M. Mitchell, S. Wu, et al., “Model Cards for Model Reporting,” in *FAT**, 2019.
- [31] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [32] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [33] C. G. Northcutt, L. Jiang, and I. L. Chuang, “Confident Learning: Estimating Uncertainty in Dataset Labels,” *JAIR*, vol. 70, pp. 1373–1411, 2021.
- [34] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [35] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [36] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [37] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [38] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [39] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [40] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [41] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London: Butterworths, 1979.
- [42] B. W. Matthews, “Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme,” *Biochimica et Biophysica Acta (BBA) — Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
- [43] D. Chicco and G. Jurman, “The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation,” *BMC Genomics*, vol. 21, p. 6, 2020.

- [44] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in *International Conference on Machine Learning (ICML)*, 2006, pp. 233–240.
- [45] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.
- [46] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [47] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [48] M. P. Naeni, G. F. Cooper, and M. Hauskrecht, "Obtaining Well Calibrated Probabilities Using Bayesian Binning," in *AAAI Conference on Artificial Intelligence*, 2015.
- [49] G. W. Brier, "Verification of Forecasts Expressed in Terms of Probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
- [50] A. H. Murphy, "A New Vector Partition of the Probability Score," *Journal of Applied Meteorology*, vol. 12, no. 4, pp. 595–600, 1973.
- [51] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [52] T.-Y. Lin, M. Maire, S. Belongie, et al., "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.
- [53] J. Hosang, R. Benenson, P. Dollár, et al., "What Makes for Effective Detection Proposals?" *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 4, pp. 814–830, 2016.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] F. Perazzi, J. Pont-Tuset, B. McWilliams, et al., "A Benchmark Dataset and Evaluation Methodology for Video Object Segmentation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [56] G. Csurka, D. Larlus, and F. Perronnin, "What Is a Good Evaluation Measure for Semantic Segmentation?" In *British Machine Vision Conference (BMVC)*, 2013.
- [57] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [58] K. Mikolajczyk and C. Schmid, "A Performance Evaluation of Local Descriptors," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 10, pp. 1615–1630, 2005.
- [59] M. Dusmanu, I. Rocco, T. Pajdla, et al., "D2-Net: A Trainable CNN for Joint Description and Detection of Local Features," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [60] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [61] S. Sagawa, P. W. Koh, T. B. Hashimoto, et al., "Distributionally Robust Neural Networks for Group Shifts," in *ICLR*, 2020.
- [62] M. Arjovsky, L. Bottou, I. Gulrajani, et al., "Invariant Risk Minimization," *arXiv:1907.02893*, 2019.
- [63] I. Gulrajani and D. Lopez-Paz, "In Search of Lost Domain Generalization," in *ICLR*, 2021.
- [64] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," in *ICLR*, 2019.

- [65] R. Taori, A. Dave, V. Shankar, et al., “Measuring Robustness to Natural Distribution Shifts in Image Classification,” in *NeurIPS*, 2020.
- [66] D. Hendrycks and K. Gimpel, “A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks,” in *ICLR*, 2017.
- [67] Y. Ovadia, E. Fertig, et al., “Can You Trust Your Model’s Uncertainty?” In *NeurIPS*, 2019.
- [68] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [69] A. Kendall, Y. Gal, and R. Cipolla, “Multi-Task Learning Using Uncertainty to Weigh Losses,” in *CVPR*, 2018.
- [70] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [71] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [72] F. Milletari, N. Navab, and S.-A. Ahmadi, “V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation,” in *3DV*, 2016.
- [73] H. Rezatofighi, N. Tsoi, J. Gwak, et al., “Generalized Intersection over Union,” in *CVPR*, 2019.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [76] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [77] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [78] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [79] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [80] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [81] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [82] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [83] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [84] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [85] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [86] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [87] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [88] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [89] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [90] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.

- [91] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [92] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [93] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [94] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [95] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [96] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [97] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [98] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [99] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [100] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [101] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.

Chia dữ liệu và giao thức đánh giá (Data Splitting and Evaluation Protocol)

PHỤ THUỘC B/05-thiet-ke-bai-toan (đơn vị phân tích (*unit of analysis*) quyết định cách chia); B/06-chat-luong-du-lieu (trần đồng thuận và trùng lặp gần đúng). Chương này chuẩn bị trực tiếp cho E/24-thi-nghiem-va-ablation.

TÓM TẮT

Chương này nói về cách để có một con số dự báo được hiệu năng tương lai, và về mọi cách con số đó nói dối. Đọc xong, nhìn một bảng kết quả là bạn biết cách chia của nó giả định gì và metric của nó đang giấu loại lỗi nào. Bạn tính được chênh lệch bao nhiêu điểm mới đáng gọi là thật. Bạn thiết kế được một giao thức đánh giá dám đem ra tranh luận. Nếu chỉ nhớ một điều: mọi lỗi trong chương này đều làm kết quả *đẹp lên* chứ không báo lỗi, nên chúng chỉ bị phát hiện ở chỗ đắt nhất — sau khi triển khai — trừ khi bạn đi tìm chúng một cách có hệ thống.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ba tập train / val / test, tham số và siêu tham số (hyperparameter)** — dữ liệu được cắt thành ba phần có ba việc khác nhau: *train* để thuật toán tự tìm ra các tham số của mô hình (trọng số); *val* để bạn chọn các siêu tham số — những số bạn đặt bằng tay trước khi huấn luyện như tốc độ học, số tầng, ngưỡng quyết định — và để biết khi nào nên dừng; *test* chỉ dùng đúng một lần ở cuối để đo. Nhầm vai trò của ba tập này là gốc của mọi vấn đề trong chương.
- **rò rỉ dữ liệu (data leakage)** — bất kỳ đường nào làm thông tin của tập kiểm tra lọt vào quá trình huấn luyện hoặc chọn mô hình — ảnh hưởng nằm hai bên, thống kê chuẩn hoá tính trên cả tập, hai khung hình liền nhau bị tách đôi, hoặc chính bạn xem kết quả test rồi đổi thiết kế. Đặc điểm nguy hiểm là nó không báo lỗi, chỉ làm con số đẹp lên.
- **metric và hàm mất mát (loss function)** — **không phải một** — hàm mất mát là đại lượng *khả vi* mà thuật toán tối ưu giảm xuống trong lúc huấn luyện, tính trên từng mẫu. Metric là đại lượng bạn báo cáo và dùng để ra quyết định, thường không khả vi nên không tối ưu trực tiếp được. Cả chương này nói về metric.
- **accuracy, precision, recall, TPR, FPR và tỉ lệ dương (prevalence)** — accuracy là tỉ lệ đúng trên toàn bộ mẫu. Precision là trong các lần báo “có” thì bao nhiêu phần đúng. Recall, còn gọi là TPR (tỉ lệ dương thật), là trong các ca “có” thật thì bắt được bao nhiêu phần. FPR là trong các ca “không” thật thì bao nhiêu phần bị báo nhầm thành “có”. Prevalence là tỉ lệ ca “có” thật trong toàn bộ dữ liệu — khi nó nhỏ, chẳng hạn 1%, bốn con số trên rời xa nhau tới mức đọc nhầm một cái là kết luận sai hoàn toàn.
- **đường cong ROC, đường cong PR, AUROC và AUPRC** — mọi bộ phân loại đều có một ngưỡng; kéo ngưỡng từ chặt sang lỏng thì được cả một dãy điểm làm việc thay vì một điểm. Vẽ dãy đó trên trục (FPR, TPR) ra đường cong ROC; trên trục (recall, precision) ra đường cong PR. Diện tích dưới mỗi đường là một con số tóm tắt, gọi lần lượt là AUROC và AUPRC.
- **metric riêng của từng bài toán: IoU, mAP, ATE, RPE** — IoU là tỉ số diện tích giao trên diện tích hợp của hai vùng, dùng để quyết định một hộp dự đoán có “khớp” hộp thật hay không; mAP là trung bình diện tích dưới đường PR lấy trên mọi lớp, và “mAP@0.5” nghĩa là coi khớp khi IoU ≥ 0.5 . Trong SLAM, ATE (*absolute trajectory error*) đo độ lệch của toàn bộ quỹ đạo so với quỹ đạo

thật, sau khi căn hai quỹ đạo về cùng hệ toạ độ. RPE (*relative pose error*) đo độ lệch của các đoạn dịch chuyển ngắn.

- **lát (slice), và mẫu (sample) — phân biệt hai nghĩa** — một lát là một tập con của tập kiểm tra được cắt theo một điều kiện có nghĩa vận hành: ảnh ban đêm, vật nhỏ, một camera cụ thể; đọc metric theo từng lát mới thấy được nhóm nào đang bị bỏ rơi. Từ mẫu thì có hai nghĩa: (a) một điểm dữ liệu đơn lẻ, ví dụ một ảnh; (b) một tập được rút ra từ tổng thể, như trong “lấy mẫu có hoàn lại”.
- **sai số chuẩn và khoảng tin cậy (confidence interval)** — sai số chuẩn ước lượng mức dao động của một con số nếu bạn lặp lại phép đo trên một tập khác cùng cỡ; khoảng tin cậy 95% là khoảng rộng khoảng hai lần sai số chuẩn về mỗi phía. Cần biết vì không có nó thì không cách nào phân biệt một cải tiến thật với nhiễu đo đạc.
- **mẫu số của một tỉ lệ** — con số nằm dưới gạch ngang. Nghe tầm thường, nhưng gần như mọi hiểu nhầm về metric là hiểu nhầm mẫu số: precision chia cho số lần hệ thống báo “có”, recall chia cho số ca thật sự dương, còn FPR chia cho số ca thật sự âm. Ba mẫu số khác nhau, nên ba con số kể ba câu chuyện khác nhau về cùng một lỗi.
- **ma trận nhầm lẫn (confusion matrix)** — bảng đếm bốn kết cục có thể xảy ra khi một hệ thống trả lời “có/không” cho một câu hỏi có đáp án “có/không”: báo có và đúng, báo có và sai, báo không và sai, báo không và đúng. Mọi metric phân loại trong chương đều là một phân số lấy từ bốn số này.
- **trung bình điều hoà (harmonic mean)** — nghịch đảo của trung bình cộng các nghịch đảo. Tính chất cần nhớ là nó bị con số nhỏ kéo xuống mạnh hơn trung bình cộng, nên nó thường cho sự cân bằng giữa hai đại lượng. F1 dùng nó chính vì lý do đó.
- **điểm vận hành (operating point)** — một ngưỡng cụ thể mà bạn chọn để biến điểm số liên tục của mô hình thành quyết định “có/không”. Đổi ngưỡng thì mọi metric đổi theo, nên báo cáo một metric mà không nói ngưỡng là báo cáo nửa thông tin.
- **bootstrap** — cách ước lượng độ dao động của một con số khi không có công thức giải tích: lấy mẫu có hoàn lại từ chính tập dữ liệu đã có, tính lại con số, lặp vài nghìn lần, rồi đọc phân vị. Cần nó vì mAP, HOTA hay AUPRC không phải trung bình của các mẫu độc lập nên không có công thức sai số chuẩn.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao một mô hình đạt 99% accuracy vẫn có thể vô dụng hoàn toàn, và bạn phát hiện điều đó bằng đúng hai con số nào?
- Q2. Bạn không hề chạm vào nhãn của tập test — chỉ tính mean và std trên toàn bộ dữ liệu trước khi chia. Vì sao con số cuối cùng đã hỏng?
- Q3. Vì sao thứ hạng trên leaderboard thường không tái lập được, dù không ai gian lận và không ai làm lộ tập test?
- Q4. Quỹ đạo SLAM của bạn có ATE rất đẹp nhưng chạy thật thì lệch dần theo thời gian. Bước căn chỉnh đã hấp thụ mất cái gì?
- Q5. Mô hình mới hơn mô hình cũ 1 điểm phần trăm trên 1000 ảnh. Phép tính nào cho biết bạn có quyền viết điều đó vào báo cáo hay không?
- Q6. Precision 0,95 nhưng recall 0,40 — hệ thống của bạn đang hành xử thế nào, và ba nguyên nhân nào giải thích được nó?
- Q7. Hai hệ thống có cùng $F_1 = 0,600$. Vì sao một trong hai vẫn có thể tốt hơn hẳn cái kia cho bài toán đóng vòng?
- Q8. Vì sao một mô hình có AUC hoàn hảo vẫn có thể sai hoàn toàn về xác suất, và bạn phát hiện điều đó bằng biểu đồ nào?

5.1 Đặt vấn đề

Bạn cần đúng một thứ từ giao thức đánh giá: *một con số dự báo được hiệu năng tương lai*. Không phải một con số đẹp, cũng không phải một con số so được với paper. Cái bạn cần là: khi bạn nói “mô hình này đạt 0,87”, hệ thống thật sự chạy quanh mức đó trên dữ liệu nó chưa từng thấy.

Điều làm chương này khó chịu là mọi lỗi ở đây đều làm kết quả đẹp lên. **Rò rỉ dữ liệu** (*data leakage*) không báo lỗi, không làm loss tăng, không làm gì cả ngoài việc tặng bạn thêm vài điểm phần trăm và một cảm giác tiến bộ. Không có bug nào có cơ chế tự vệ tốt hơn một bug làm bạn vui.

Người đã tự xây benchmark harness cho SLAM có sẵn phần lớn phản xạ cần thiết, chỉ là chưa gọi tên chúng: bạn biết một lần chạy không nói lên điều gì vì nhiều giữa các lần chạy có thể lớn hơn hiệu ứng cần đo; bạn biết chọn sequence nào để báo cáo là quyết định có sức nặng ngang việc sửa thuật toán; bạn biết một con số ATE trung bình có thể che một lần mất tracking. Chương này là ba trực giác đó viết lại bằng ngôn ngữ học máy, cộng thêm các dạng rò rỉ mà pipeline SLAM không có.

TRỰC GIÁC

Coi tập test như một *mẫu chuẩn niêm phong* trong phòng đo lường, không như một thước đo dùng bao nhiêu lần cũng được. Mẫu chuẩn giữ được giá trị chỉ khi nó ở trong tủ; mỗi lần bạn mở ra để chỉnh thiết bị, một chút đặc tính của thiết bị được in ngược vào cái mà bạn coi là chuẩn. Sau ba mươi lần “chỉnh theo mẫu chuẩn”, thiết bị khớp mẫu chuẩn hoàn hảo và sai với mọi thứ khác. Val là mẫu làm việc, dùng thoải mái và chấp nhận nó hao mòn; test là mẫu niêm phong, mở một lần ở cuối.

5.2 Ba vai trò và các đường rò rỉ (Splits and Leakage)

Việc tách train/val/test hay bị dạy như một thủ tục: cắt 70/15/15. Dạy như vậy thì người học nhớ tỉ lệ và quên mất phần quan trọng hơn — ba tập có *ba vai trò khác nhau về bản chất*.

- **Train** — ước lượng tham số của mô hình.
- **Val** — chọn siêu tham số và quyết định lúc nào dừng.
- **Test** — ước lượng hiệu năng, đúng một lần, ở cuối.

Nhằm vai trò thì mọi kết luận sau đó vô giá trị. Và nhằm vai trò không tạo ra thông báo lỗi nào.

A. Sáu đường rò rỉ, theo thứ tự khó phát hiện dần

Mạch 1 — rò rỉ và cách chặn từng đường

- Vấn đề** đo trên dữ liệu đã học là vô nghĩa.

Giải pháp tách ba tập với ba vai trò rõ ràng.

Giả định bạn thật sự tôn trọng vai trò đó, tức không bao giờ dùng test để quyết định bất cứ điều gì.

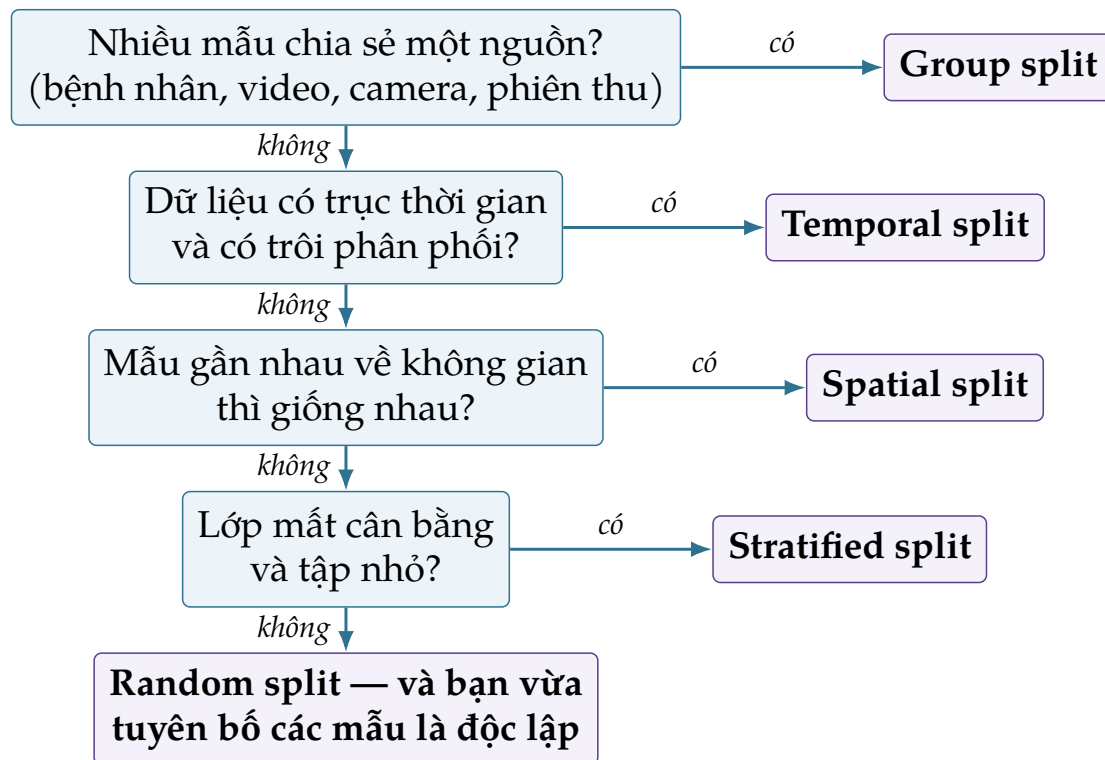
Hệ quả bước duy nhất ai cũng làm, và cũng là bước duy nhất không đủ.
- Vấn đề** rò rỉ qua khâu tiền xử lý.

Giải pháp tính mean/std, PCA, chọn đặc trưng, khớp bộ chuẩn hoá *sau* khi chia và *chỉ* trên train.

- Hệ quả thống kê của test đã đi vào mô hình — nhẹ, nhưng đủ để mọi chênh lệch nhỏ trở nên vô nghĩa.
- 3 Vấn đề** rò rỉ qua mẫu trùng.
- Giải pháp** khử **trùng lặp gần đúng** (*near-duplicate*) trước khi chia, bằng *embedding* chứ không bằng mã băm (*B/06*).
- Hệ quả với ảnh crawl web và frame video, đây là nguồn tạo kết quả đẹp mà giả phổ biến nhất.
- 4 Vấn đề** rò rỉ qua nhóm.
- Giải pháp** nếu nhiều mẫu chia sẻ một nguồn — cùng bệnh nhân, cùng video, cùng camera, cùng phiên thu — thì mọi mẫu của một nhóm phải nằm trọn một bên.
- Hệ quả bỏ qua điều này là cách rẻ nhất để tự tặng mình mười tới hai mươi điểm hiệu năng ảo; đây là kinh nghiệm được lặp lại rộng rãi trong y tế và video, không phải một con số đo chính xác.
- 5 Vấn đề** rò rỉ qua thời gian.
- Giải pháp** khi dữ liệu có trục thời gian, test phải nằm *sau* train.
- Hệ quả chia ngẫu nhiên trên dữ liệu có trôi cho phép mô hình nội suy giữa quá khứ và tương lai, một đặc quyền nó sẽ không có khi chạy thật.
- 6 Vấn đề** rò rỉ qua chính bạn.
- Vấn đề mới** ***adaptive overfitting***: mỗi lần bạn nhìn test rồi ra một quyết định, một chút thông tin từ test rò vào mô hình qua bộ não bạn.
- Hệ quả sau vài chục vòng như vậy, test đã trở thành val — lời giải thích tự nhiên cho việc thứ hạng leaderboard thường không tái lập được trên một tập test mới thu thập cùng quy trình [26].

B. Chọn cách chia là phát biểu một giả định, không phải chọn một hàm

Nhận thức then chốt của cả chương nằm ở đây. Khi bạn gọi `train_test_split` với `shuffle=True`, bạn không chọn một tiện ích; bạn đang tuyên bố rằng dữ liệu tương lai sẽ đến từ cùng nhóm, cùng thời điểm, cùng địa điểm với dữ liệu hiện tại. Đó là một phát biểu mạnh về thế giới, và nó thường sai. Hình 5.1 biến phát biểu đó thành một chuỗi câu hỏi trả lời được, còn Bảng 5.1 nói cái giá của mỗi lựa chọn.



Hình 5.1. Cây quyết định chọn cách chia. Thứ tự các câu hỏi là thứ tự mức thiệt hại: rò rỉ qua nhóm gây sai lệch lớn nhất nên phải loại trừ trước, còn *stratified* chỉ giảm phương sai chứ không chặn rò rỉ nào — nó đứng cuối vì nó là lựa chọn *cộng thêm*, không phải lựa chọn thay thế. Nhánh cuối cùng không phải mặc định an toàn mà là một tuyên bố cần được biện minh.

Bảng 5.1. Mỗi cách chia là một giả định về cách mô hình sẽ được dùng. Cột cuối nói điều gì xảy ra nếu giả định đó sai.

Cách chia	Giả định tương ứng	Dùng khi	Rủi ro nếu chọn sai
Random	Các mẫu độc lập với nhau	Ảnh rời rạc, không có cấu trúc nhóm	Rò rỉ qua nhóm và qua thời gian, không có dấu hiệu cảnh báo
Stratified	Cần giữ tỉ lệ lớp giữa các tập	Lớp mất cân bằng, tập nhỏ	Giảm phương sai ước lượng nhưng <i>không</i> chặn rò rỉ nhóm
Group	Sẽ gặp nhóm hoàn toàn mới	Y tế, video, đa camera, đa phiên thu	Nếu bỏ qua: hiệu năng ảo cao hàng chục điểm
Temporal	Sẽ dự đoán về tương lai	Có trôi phân phối theo thời gian	Đánh giá lạc quan giả tạo; mô hình được phép nội suy thời gian
Spatial	Sẽ gặp vùng địa lý mới	Ảnh vệ tinh, khảo sát hiện trường	Tự tương quan không gian làm hai bên split gần như trùng nhau

Một hệ quả ít được nói: cách chia phải khớp với đơn vị phân tích đã chọn ở B/05. Chọn frame làm đơn vị rồi chia ngẫu nhiên là công thức tạo rò rỉ. Hai frame cách nhau 33 ms không phải hai quan sát độc lập, đúng như hai keyframe liên kề không cho hai ràng buộc độc lập trong **hiệu chỉnh chùm tia** (*bundle adjustment*). Nếu đơn vị thật sự là “một video” hay “một phiên thu” thì test phải cắt ở cấp đó. Hệ quả là số mẫu độc lập thật nhỏ hơn nhiều so với con số bạn tưởng: một nghìn khung hình lấy từ mười video chỉ đáng giá mười mẫu. Điều đó nói rộng khoảng tin cậy ở cuối chương theo đúng căn bậc hai của tỉ lệ ấy.

↪ **hệ quả của B/05** — đơn vị phân tích chọn ở khâu thiết kế quyết định cách chia nào là hợp lệ; chọn frame rồi chia ngẫu nhiên là rõ ràng ngay từ định nghĩa.

↪ **tiền đề B/06** — khử trùng lặp gần đúng phải xong *trước* khi chia, nếu không thì mọi cách chia đều rõ ràng qua bản sao.

C. Khi dữ liệu quá ít để cắt một tập test tử tế

Kiểm định chéo (*cross-validation*) là câu trả lời cổ điển, và ***nested cross-validation*** là cách tune siêu tham số mà không làm ô nhiễm ước lượng cuối: vòng trong chọn siêu tham số, vòng ngoài chỉ để đo. Cái giá là chi phí tính toán nhân lên theo số fold. Với deep learning hiện đại, nhân năm lần một lượt huấn luyện kéo dài vài ngày là không khả thi. Đó là lý do *thực dụng*, không phải lý do lý thuyết, khiến kiểm định chéo hiếm gặp trong thị giác máy tính ngày nay. Biết điều này để không nhầm sự vắng mặt của nó với một chuẩn mực đúng.

I 5.3 Chọn metric khớp chi phí thật (Metric Selection)

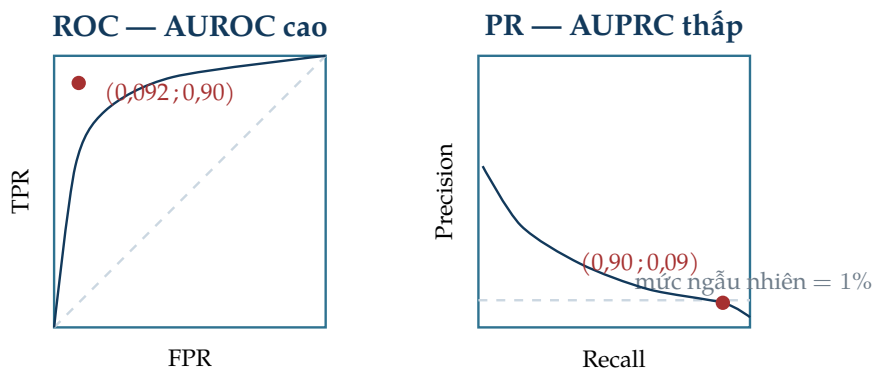
Giả sử đã chia đúng. Vẫn còn một cách nói dối tinh vi hơn: chọn sai thứ để đo. Metric là một hàm nén toàn bộ hành vi hệ thống xuống một số thực, và mọi phép nén đều mất thông tin — câu hỏi duy nhất đáng hỏi là *nó vứt đi loại lỗi nào*.

A. Ví dụ nhỏ nhất: accuracy 99% mà vô dụng

Tập test có 1000 ảnh, trong đó 10 ảnh thuộc lớp dương (1%).

- **Mô hình rỗng** — luôn trả lời “âm”. Accuracy $990/1000 = 99\%$, cao hơn phần lớn hệ thống thật, trong khi hoàn toàn vô dụng.
- **Mô hình có ích** — bắt được 9 trong 10 ca dương (*recall* 0,90) với tỉ lệ báo động giả 9,2% trên lớp âm, tức $0,092 \times 990 \approx 91$ báo động giả.
 - **Trên ROC** — điểm làm việc là $(FPR, TPR) = (0,092, 0,90)$, sát góc trên trái, trông xuất sắc.
 - **Người dùng thấy** — mô hình báo động $9 + 91 = 100$ lần, đúng 9 lần: ***precision* = 9%**.

Cơ chế rất rõ khi viết ra: FPR chia cho 990 mẫu âm nên một tỉ lệ nhỏ vẫn là rất nhiều mẫu, còn precision chia cho tổng số lần báo động nên nó cảm nhận đúng cái người dùng cảm nhận. Đó là toàn bộ lý do **AUPRC** phải thay **AUROC** khi lớp dương hiếm, và Hình 5.2 cho thấy cùng một điểm làm việc trông khác nhau đến mức nào trên hai không gian.



Hình 5.2. Cùng một mô hình, cùng một điểm làm việc, hai không gian đánh giá (sơ đồ minh họa, không phải số đo thật). Bên trái điểm đó nằm sát góc lý tưởng; bên phải chính nó nằm sát sàn. Khác biệt sinh ra vì trục tung bên trái chuẩn hoá theo 990 mẫu âm, còn bên phải chuẩn hoá theo 100 lần báo động — và người dùng sống ở trục bên phải.

B. Mỗi metric giấu loại lỗi nào

Bảng 5.2. Vài metric quen thuộc và loại lỗi mà mỗi cái làm cho vô hình. Cột giữa là lý do phải đọc ít nhất hai metric cùng lúc.

Metric	Giấu loại lỗi nào	Nên đọc kèm
Accuracy	Toàn bộ hành vi trên lớp hiếm	Precision/recall theo từng lớp
AUROC	Số lượng tuyệt đối báo động giả khi lớp dương hiếm	AUPRC ở đúng mức tỉ lệ dương thật
mAP@0.5	Sai lệch định vị: hộp lệch nhẹ vẫn được tính là đúng	mAP@0.5:0.95, hoặc phân rã lỗi kiểu TIDE [34]
mIoU	Lớp nhỏ và biên; một lớp chiếm phần lớn pixel có thể kéo con số lên	IoU từng lớp, và Panoptic Quality nếu cần tách thực thể [35]
MOTA	Việc liên tục đổi ID: MOTA cao vẫn tương thích với danh tính bị hoán loạn	IDF1 và HOTA [36]
PSNR, SSIM	Chất lượng cảm nhận; ảnh mờ trung bình hoá thường thắng	LPIPS [37], và với mô hình sinh thì FID [38]
ATE sau alignment	Trôi toàn cục, vì phép căn chỉnh hấp thụ đúng thành phần đó	RPE trên nhiều độ dài đoạn [39]

C. Metric hình học và cái bẫy của phép căn chỉnh

Dòng cuối Bảng 5.2 đáng nói kỹ vì người đọc sống ở đó. **sai số quỹ đạo tuyệt đối** (*absolute trajectory error*) được tính *sau* khi căn chỉnh quỹ đạo ước lượng vào quỹ đạo tham chiếu bằng một phép biến đổi cứng, hoặc một phép biến đổi có tỉ lệ trong trường hợp đơn mắt. Phép căn chỉnh này chính là thứ hấp thụ sai lệch toàn cục: một quỹ đạo trôi tỉ lệ đều đặn sẽ bị kéo về gần tham chiếu và cho ra ATE dễ chịu, trong khi lỗi cục bộ vẫn còn nguyên. **sai số tương đối** (*relative pose error*) thì ngược lại — nó đo tính nhất quán trên các đoạn ngắn và mù với sai lệch toàn cục.

Không cái nào sai; sai là đọc một cái rồi kết luận. Nguyên tắc tổng quát: *mỗi metric là một phép chiếu, và bạn cần ít nhất hai phép chiếu không song song để định vị được lỗi.*

↔ **cùng nguyên lý với** C/16 — trong thị giác 3D, cặp ATE-RPE và cặp Chamfer-độ phủ đều là hai phép chiếu bù nhau của cùng một sai số.

Một khía cạnh nữa thường bị đẩy ra ngoài giao thức rồi phải trả giá sau là **calibration**: mô hình có thể xếp hạng đúng mà vẫn tự tin sai lệch một cách hệ thống, và mạng hiện đại nổi tiếng là quá tự tin [40]. Nếu đầu ra đi vào một quyết định có ngưỡng, hoặc vào một bộ hợp nhất cảm biến coi nó như xác suất, thì calibration là một phần của tính đúng đắn chứ không phải chuyện phụ — khai triển đầy đủ ở E/25-do-tin-cay-va-van-hanh.

↔ **tiền đề** E/25 — một đầu ra xếp hạng đúng nhưng lệch hiệu chuẩn sẽ phá mọi ngưỡng quyết định đặt phía sau nó.

I 5.4 Bốn ô của ma trận nhầm lẫn (Reading the Confusion Matrix)

Từ đây tới hết chương, trọng tâm đổi. Phần trên dạy *chọn* metric; phần này dạy *đọc* metric. Đây là hai kỹ năng khác nhau. Chọn đúng metric mà đọc sai con số thì vẫn ra quyết định sai.

Mọi thứ bắt đầu từ một bảng bốn ô. Một bộ phân loại nhị phân nhận một mẫu và trả lời “có” hoặc “không”. Chân trị cũng chỉ có hai giá trị. Ghép hai bên lại được đúng bốn kết cục, không thừa không thiếu. Mọi metric phân loại mà bạn từng gặp đều là một phân số lắp từ bốn số này.

Người đọc đã sống với bảng này nhiều năm mà chưa gọi tên nó. Trong đòng vòng của SLAM, bước kiểm chứng hình học nhận một cặp khung hình ứng viên và quyết định “đây là cùng một chỗ” hay “không phải”. Đó chính là một bộ phân loại nhị phân. Ngưỡng số điểm inlier chính là **điểm vận hành** (*operating point*) của nó. Cả chương này vì thế nói về đúng thứ bạn đang chỉnh bằng tay.

A. Bốn ô, gọi bằng lời thường

- **TP — dương thật** (*true positive*). Hệ thống báo “có” và đúng là có. Đây là công việc hệ thống được thuê để làm.
- **FP — dương giả** (*false positive*). Hệ thống báo “có” nhưng thật ra không có. Đây là báo động giả. Người dùng nhìn thấy nó ngay.
- **FN — âm giả** (*false negative*). Hệ thống báo “không” nhưng thật ra có. Đây là ca bị bỏ sót. Người dùng *không* nhìn thấy nó, và đó mới là chỗ nguy hiểm.
- **TN — âm thật** (*true negative*). Hệ thống báo “không” và đúng là không. Ô này thường đông đảo tới mức vô nghĩa, và chính nó là nguồn của phần lớn ảo giác về hiệu năng.

Ghi nhớ theo một quy tắc: chữ thứ hai là *cái hệ thống nói*, chữ thứ nhất là *hệ thống có nói đúng không*. “False positive” nghĩa là hệ thống nói dương, và nó sai.

Hình 5.3 vẽ bốn ô đó cùng một bộ số cụ thể. Bộ số này đến từ một bộ phân loại tổng hợp mà chúng ta sẽ dùng lại suốt phần còn lại của chương: 20 000 mẫu, trong đó 200 mẫu dương, tức tỉ lệ nền 1%. Ở ngưỡng 0,50, nó cho TP 134, FP 1043, FN 66 và TN 18 757. Đây là số sinh từ mô phỏng, không phải số đo trên một hệ thống thật; giá trị của nó nằm ở chỗ mọi con số phía sau đều tính ra được từ đúng bốn số này.

		CHÂN TRỊ		
		thật sự có (200 ca)	thật sự không (19 800 ca)	
HỆ THỐNG BÁO	báo “có” (1177 lần)	TP = 134 báo có, đúng là có <i>việc hệ thống được thuê để làm</i>	FP = 1043 báo có, thật ra không <i>báo động giả — ai cũng thấy</i>	mẫu số của precision mọi lần hệ thống mở miệng
	báo “không” (18 823 lần)	FN = 66 báo không, thật ra có <i>bỏ sót — không ai thấy</i>	TN = 18 757 báo không, đúng là không <i>ô khổng lồ và gần như vô nghĩa</i>	
		mẫu số của recall mọi ca thật sự có	mẫu số của FPR mọi ca thật sự không	

Hình 5.3. Bốn ô của ma trận nhầm lẫn, với số từ bộ phân loại tổng hợp ở ngưỡng 0,50 (20 000 mẫu, 1% dương; số mô phỏng, không phải số đo). Ba dấu ngoặc là ba mẫu số khác nhau: precision chia theo hàng trên, recall chia theo cột trái, FPR chia theo cột phải. Cùng một ô FP nằm trong hai phân số có mẫu số lệch nhau gần hai mươi lần, và đó là toàn bộ lý do hai metric kể hai câu chuyện khác nhau.

Hai cái bẫy nhỏ nhưng tốn thời gian

Trước khi đi tiếp, hai chỗ vấp rất hay xảy ra khi đọc bảng của người khác.

Thứ nhất là *chiều của bảng*. Một số thư viện đặt hàng là lớp thật và cột là lớp dự đoán; một số tài liệu làm ngược lại. Nếu đọc nhầm chiều thì precision và recall hoán vị cho nhau, và bạn sẽ kết

luận ngược hoàn toàn. Cách kiểm tra trong ba giây: cộng theo hàng, xem tổng có bằng số mẫu thật của từng lớp không. Nếu có, hàng là chân trị.

Thứ hai là *ai được tính vào mẫu số*. Ở phát hiện vật thể, một dự đoán không khớp hộp nào là FP, nhưng một hộp thật không được khớp là FN — và không có ô TN nào cả, vì số “vùng ảnh không chứa vật” là vô hạn. Đây là lý do accuracy không tồn tại trong phát hiện vật thể, và cũng là lý do mọi metric ở đó đều xây trên precision và recall chứ không xây trên bốn ô đầy đủ.

B. Bốn ô là bốn chi phí, và chính chênh lệch chi phí mới chọn metric

Bốn ô không chỉ là bốn cách đếm. Chúng là bốn loại thiệt hại có giá khác nhau trong đời thực, và cái giá đó do bài toán quyết định chứ không do thuật toán.

- **Đóng vòng trong SLAM.** Một vòng sai (FP) khâu hai chỗ khác nhau lại thành một, làm cong toàn bộ bản đồ, và thường không hồi phục được. Một vòng bị bỏ lỡ (FN) chỉ có nghĩa là sai số tích lũy chưa được xoá lần này; lát nữa qua chỗ đó lần nữa vẫn còn cơ hội. Chi phí lệch hẳn về một phía, có thể tới vài bậc độ lớn — đây là đánh giá định tính từ cơ chế, không phải một con số đo được.
- **Sàng lọc y tế.** Ngược lại hoàn toàn. Một ca bỏ sót (FN) là một người bệnh về nhà. Một báo động giả (FP) chỉ tốn thêm một lần xét nghiệm.
- **Phát hiện vật cản cho robot.** FN là va chạm. FP là robot phanh vô cớ. Cả hai đều tổn, nhưng tổn theo hai đơn vị khác nhau, và người thiết kế phải quy đổi.

Từ đây rút ra nguyên tắc chi phối cả phần còn lại của chương. *Metric nào đáng nhìn phụ thuộc vào ô nào đấy*. Nếu FP đắt thì precision là con số quan trọng. Nếu FN đắt thì recall là con số quan trọng. Nếu cả hai cùng đắt và đắt ngang nhau, bạn cần một con số gộp — và sẽ thấy ở mục sau rằng gộp cũng có giá của nó.

Còn một điều nữa cần nói thẳng. Ô TN gần như không bao giờ đắt. Nó chỉ đông. Trong bảng trên, TN chiếm 93,8% tổng số mẫu. Bất kỳ metric nào có TN trong tử số hoặc trong mẫu số đều bị ô này chi phối, và accuracy là ví dụ tiêu biểu. Đó là lý do accuracy nói dối một cách hệ thống trên dữ liệu lệch lớp.

C. BẮC CẦU: kiểm chứng đóng vòng là một bộ phân loại nhị phân

Đặt bảng bốn ô lên đúng hệ thống mà người đọc đang bảo trì. Tầng kiểm chứng hình học của đóng vòng nhận một cặp khung hình ứng viên, chạy RANSAC trên các cặp điểm khớp, đếm số inlier, và so với một ngưỡng. Ánh xạ như sau.

- **Mẫu** là một cặp khung hình ứng viên, không phải một khung hình. Đơn vị phân tích ở đây là cặp, và điều đó ảnh hưởng tới cả cách chia dữ liệu lẫn cách bootstrap về sau.
- **Lớp dương** là “hai khung này thật sự nhìn cùng một chỗ”. Định nghĩa của nó do bạn đặt — thường bằng một ngưỡng khoảng cách trên chân trị quỹ đạo — nên nó là một quyết định thiết kế, không phải một sự thật khách quan.
- **Điểm số** là số inlier sau RANSAC, và **ngưỡng inlier** chính là điểm vận hành.
- **TP** là một vòng đúng được chấp nhận; **FP** là một vòng sai được chấp nhận; **FN** là một vòng đúng bị từ chối; **TN** là một cặp không liên quan bị từ chối, và ô này chiếm gần như toàn bộ dữ liệu.

Đặt xong ánh xạ này thì mọi thứ trong chương trở nên áp dụng được trực tiếp. Nâng ngưỡng inlier từ 20 lên 40 là dịch điểm vận hành sang phải trên đường cong. Việc đó tăng precision, giảm recall, và không đổi một chút nào bản thân bộ mô tả hay bộ ghép cặp. Nếu bạn muốn cải thiện

thật sự thì phải làm cả đường cong dịch lên, chứ không phải trượt dọc theo nó — và đó là hai loại công việc hoàn toàn khác nhau.

5.5 Các tỉ lệ dẫn xuất và mẫu số của từng cái (Derived Rates and Denominators)

Gần như mọi hiểu nhầm về metric phân loại là hiểu nhầm mẫu số. Người ta nhớ tử số vì nó là “số ca đúng”, rồi quên hỏi con số đó được chia cho cái gì. Mục này đi qua từng tỉ lệ theo đúng bốn bước: định nghĩa bằng lời, câu hỏi nó trả lời, cao và thấp nghĩa là gì, và điều nó *không* nói.

A. Precision — đọc theo hàng “hệ thống đã báo”

Precision là tỉ lệ đúng trong những lần hệ thống báo “có”. Mẫu số là $TP + FP$, tức toàn bộ số lần hệ thống mở miệng.

$$\text{precision} = \frac{TP}{TP + FP}.$$

Câu hỏi nó trả lời, ở ngôi thứ hai: *trong những cái tôi báo là dương, bao nhiêu phần là đúng?* Nói cách khác, nếu bây giờ hệ thống báo động, bạn có nên tin không.

Precision cao nghĩa là hệ thống ít khi làm phiền bạn vô cớ: mỗi lần nó nói, gần như chắc chắn có chuyện thật. Precision thấp nghĩa là bạn phải tự lọc lại phần lớn những gì nó đưa ra, và niềm tin vào hệ thống bốc hơi rất nhanh.

Điều precision *không* nói: nó không nói gì về số ca bị bỏ sót. Một hệ thống chỉ báo đúng một ca duy nhất trong cả năm, và báo đúng, có precision bằng 1,0. Mẫu số của precision không hề chứa FN, nên FN có thể lớn tùy ý mà precision không nhúc nhích.

B. Recall — đọc theo cột “thật sự có”

Recall, còn gọi là độ nhạy (*sensitivity*) hoặc TPR, là tỉ lệ bắt được trong số các ca thật sự dương. Mẫu số là $TP + FN$, tức toàn bộ số ca dương có trong dữ liệu.

$$\text{recall} = \frac{TP}{TP + FN}.$$

Câu hỏi nó trả lời: *trong những cái thật sự dương, tôi bắt được bao nhiêu phần?*

Recall cao nghĩa là ít thứ lọt lưới. Recall thấp nghĩa là hệ thống im lặng ở phần lớn các ca thật, và sự im lặng đó không để lại dấu vết nào trong log.

Điều recall *không* nói: nó không nói gì về số lần báo bừa. Một hệ thống luôn luôn báo “có” cho mọi mẫu có recall bằng 1,0. Mẫu số của recall không chứa FP.

Hai đoạn trên gộp lại thành một câu đáng thuộc: *precision và recall bỏ qua hai ô khác nhau, nên một mình cái nào cũng vô nghĩa*. Precision mù với FN. Recall mù với FP. Chỉ khi đọc cùng nhau chúng mới khoá được cả bốn ô.

C. Các tỉ lệ còn lại, xếp theo mẫu số

Bảng 5.3 liệt kê các tỉ lệ còn lại. Cột quan trọng nhất là cột mẫu số; cột cuối là cái bẫy đặc trưng của từng cái.

Bảng 5.3. Các tỉ lệ dẫn xuất từ bốn ô. Số ở cột giá trị lấy từ bộ phân loại tổng hợp ở ngưỡng 0,50 (TP 134, FP 1043, FN 66, TN 18 757). Đọc bảng theo cột mẫu số: hai tỉ lệ khác mẫu số thì không so sánh trực tiếp được với nhau.

Tỉ lệ	Mẫu số là gì	Giá trị	Nó không nói điều gì
Precision	mọi lần báo “có”	0,114	Số ca bỏ sót. Precision 1,0 vẫn tương thích với việc bỏ sót 99% ca thật.
Recall = TPR = độ nhạy	mọi ca thật sự dương	0,670	Số lần báo bừa. Luôn báo “có” cho recall 1,0.
Độ đặc hiệu (<i>specificity</i>) = TNR	mọi ca thật sự âm	0,947	Nó gần như luôn đẹp khi lớp âm đông; 0,947 ở đây vẫn ứng với 1043 báo động giả.
FPR = 1 – specificity	mọi ca thật sự âm	0,053	Số lượng báo động giả. 5,3% của 19 800 là 1043 lần.
FNR = 1 – recall	mọi ca thật sự dương	0,330	Cùng thông tin với recall, chỉ đổi hướng; không phải một metric mới.
NPV (giá trị dự đoán âm)	mọi lần báo “không”	0,9965	Nó gần như luôn sát 1 khi lớp dương hiếm, nên hầu như không mang thông tin.
Accuracy	toàn bộ mẫu	0,945	Bị ô TN chi phối. Mô hình rỗng luôn báo “không” đạt 0,990, cao hơn.
Balanced accuracy	trung bình của recall và specificity	0,808	Nó chứa được lệch lớp nhưng vẫn không phản ánh chi phí thật khi FP và FN lệch giá.
Prevalence (tỉ lệ nền)	toàn bộ mẫu	0,010	Không phải metric của mô hình. Nó là tính chất của <i>dữ liệu</i> , và nó quyết định cách đọc mọi dòng trên.

Ba nhận xét đáng rút ra từ bảng.

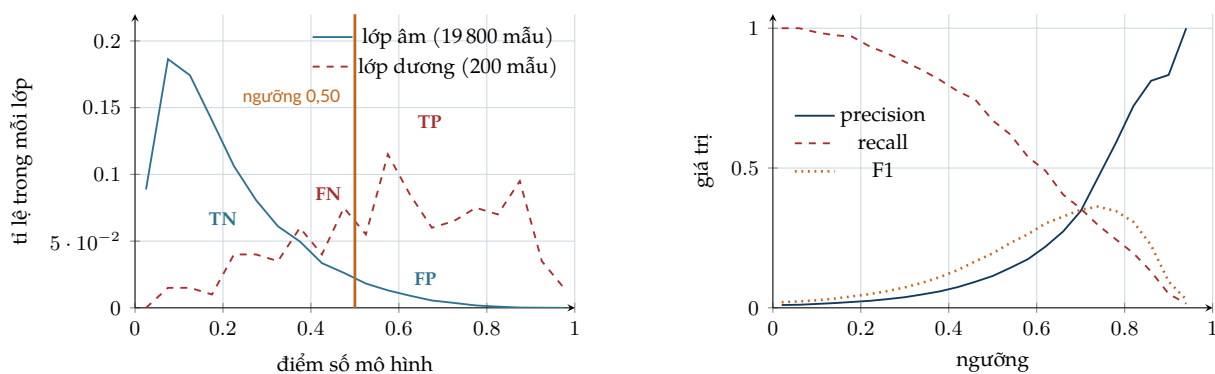
- **Specificity và precision hay bị lẫn.** Cả hai đều “liên quan tới báo động giả”, nhưng specificity chia FP cho số ca âm (19 800), còn precision chia FP cho số lần báo (1177). Mẫu số lệch gần hai mươi lần, nên specificity 0,947 nghe rất đẹp trong khi precision 0,114 thì thảm hại. Cùng một FP, hai câu chuyện.
- **NPV gần như vô dụng khi lớp dương hiếm.** Nếu chỉ 1% mẫu là dương thì đoán “không” đã đúng 99% số lần. NPV cao không chứng minh điều gì.
- **Prevalence không phải metric nhưng phải báo cáo cùng metric.** Precision phụ thuộc trực tiếp vào nó. Cùng một mô hình, đem sang tập có tỉ lệ nền thấp hơn mười lần thì precision tụt gần mười lần, dù mô hình không đổi một chút nào. Recall, specificity và FPR thì không đổi, vì mẫu số của chúng nằm trong một lớp duy nhất.

Nhận xét thứ ba là thứ hay cắn người nhất khi triển khai. Bạn đo precision trên tập test cân bằng nhân tạo, thấy 0,85, rồi đem ra hiện trường nơi tỉ lệ nền thấp hơn hai chục lần và precision rơi xuống 0,2. Không có gì hỏng cả; bạn chỉ đã đo một mẫu số khác.

D. Ngưỡng dịch chuyển bốn ô như thế nào

Bốn ô không phải một tính chất cố định của mô hình. Chúng là một tính chất của mô hình *cộng với* một ngưỡng. Đổi ngưỡng thì bốn số chày từ ô này sang ô kia theo một quy luật cứng: siết ngưỡng chặt lại thì TP chày sang FN và FP chày sang TN; nới lỏng thì ngược lại.

Hình 5.4 cho thấy điều đó trên chính bộ phân loại tổng hợp. Bên trái là phân bố điểm số của hai lớp, bên phải là precision, recall và F1 khi ngưỡng chạy từ lỏng sang chặt. Hai đường precision và recall đi ngược chiều nhau trên toàn dải, và không có ngưỡng nào làm cả hai cùng đẹp — đó là hình dạng của một sự đánh đổi, không phải một khuyết điểm cần sửa.

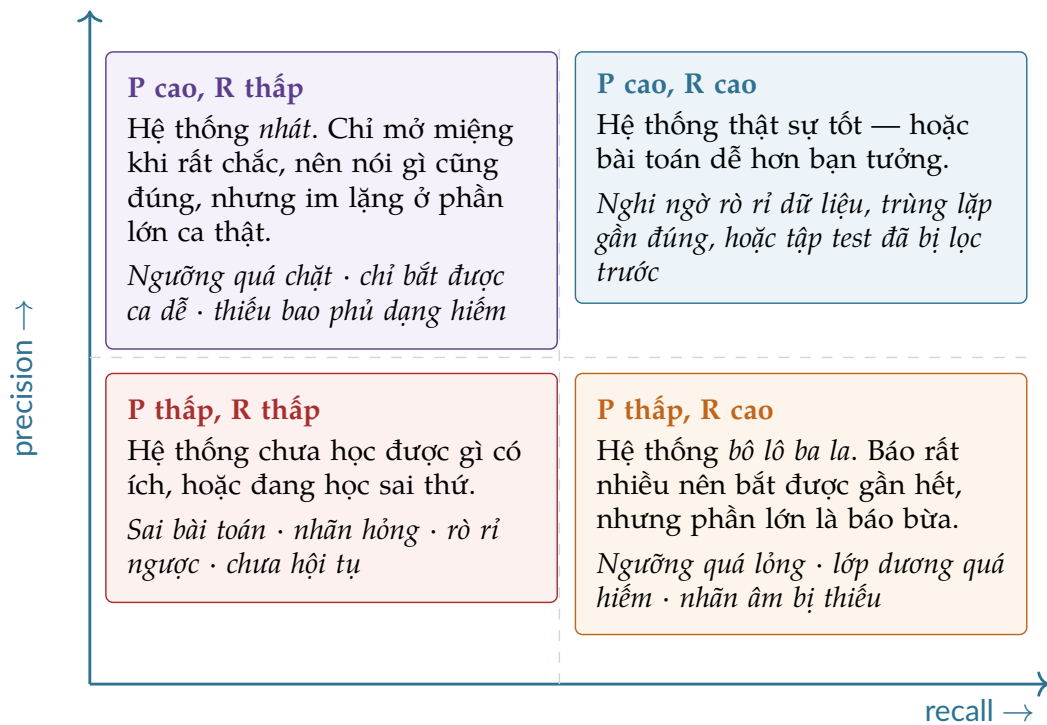


Hình 5.4. Ngưỡng dịch chuyển bốn ô (bộ phân loại tổng hợp, 20 000 mẫu, 1% dương). Bên trái: mỗi đường được chuẩn hoá trong lớp của nó, nên đừng đọc chiều cao như số lượng — đường đỏ đại diện 200 mẫu, đường xanh 19 800 mẫu. TN và FP nằm trên đường xanh (lớp âm); FN và TP nằm trên đường đỏ (lớp dương). Kéo ngưỡng sang phải thì FP giảm và FN tăng. Bên phải: precision và recall đi ngược chiều trên toàn dải, còn F1 đạt cực đại quanh ngưỡng 0,74 — một điểm không có lý do gì để trùng với điểm tối ưu theo chi phí thật của bạn.

Một hệ quả thực dụng: ngưỡng 0,5 không có gì thiêng liêng. Nó chỉ là giá trị mặc định của hàm `argmax` khi có hai lớp. Chọn ngưỡng là một quyết định thiết kế riêng, tách hẳn khỏi việc huấn luyện, và nó phải được ra dựa trên chi phí của FP và FN chứ không dựa trên thói quen.

5.6 Bốn góc precision–recall (The Four Corners)

Đây là phần trả lời trực tiếp câu hỏi “precision cao mà recall thấp thì nghĩa là gì”. Hai con số, mỗi con số cao hoặc thấp, cho bốn tổ hợp. Mỗi tổ hợp mô tả một *hành vi* của hệ thống, và mỗi hành vi lại có vài nguyên nhân khác nhau. Việc của bạn là đi từ tổ hợp về nguyên nhân, và điều đó cần một phép thử chứ không cần trực giác.



Hình 5.5. Bốn góc của mặt phẳng precision–recall và hành vi ứng với mỗi góc. Dòng in nghiêng trong mỗi ô là các nguyên nhân thường gặp — chúng khác nhau về cách sửa, nên phải phân biệt bằng phép thử chứ không đoán. Lưu ý góc trên phải cũng là một cảnh báo: kết quả quá đẹp trên bài toán khó là dấu hiệu của rò rỉ trước khi là dấu hiệu của tài năng.

A. Từ tổ hợp về nguyên nhân, kèm phép thử

Bảng 5.4 là phần dùng được nhất của mục này. Cột cuối là cột đáng giá: nó biến một suy đoán thành một việc làm được trong nửa giờ.

Bảng 5.4. Bốn tổ hợp precision–recall: hệ thống đang hành xử thế nào, vì sao, và làm gì để biết là vì sao. Mỗi ô ở cột giữa liệt kê các nguyên nhân *cạnh tranh nhau*; phép thử ở cột phải là cách tách chúng.

Tổ hợp	Nguyên nhân thường gặp	Phép thử để phân biệt
P cao, R cao	(a) mô hình thật sự tốt; (b) rò rỉ dữ liệu; (c) tập test đã bị lọc bỏ ca khó; (d) bài toán dễ hơn tưởng	So với một baseline ngu ngốc (đoán theo tỉ lệ nền, hoặc một quy tắc thủ công ba dòng). Nếu baseline cũng đẹp thì là (d). Chia lại theo nhóm và đo lại; nếu con số sập thì là (b). Đếm phân bố độ khó của tập test so với dữ liệu hiện trường để bắt (c).
P cao, R thấp	(a) ngưỡng đặt quá chặt; (b) mô hình chỉ học được dạng dễ, mù với dạng hiếm; (c) tập huấn luyện thiếu bao phủ; (d) định nghĩa nhãn dương hẹp hơn thực tế	Quét toàn dải ngưỡng và vẽ đường PR. Nếu nới ngưỡng làm recall tăng mạnh mà precision giảm ít thì là (a) — sửa bằng một dòng cấu hình. Nếu đường PR phẳng lì và tụt ngay thì không phải ngưỡng, mà là (b) hoặc (c). Phân biệt (b) với (c) bằng cách đọc recall theo từng lát.
P thấp, R cao	(a) ngưỡng quá lỏng; (b) lớp dương quá hiếm nên precision bị mẫu số chìm; (c) nhãn âm bị thiếu, cái mô hình bắt đúng lại bị tính là FP; (d) mô hình bám vào một đặc trưng tất	Siết ngưỡng và xem precision có leo lên không — nếu có thì là (a). Nếu precision vẫn thấp ở mọi ngưỡng, tính lại precision kỳ vọng của một bộ đoán ngẫu nhiên (bằng đúng tỉ lệ nền) để xem có phải (b). Với (c), bốc tay 50 mẫu FP và tự gán nhãn lại; tỉ lệ “FP thật ra đúng” nói ngay có phải nhãn thiếu hay không.
P thấp, R thấp	(a) mô hình chưa hội tụ hoặc có bug; (b) nhãn hỏng hoặc mâu thuẫn; (c) đặc trưng đầu vào không mang thông tin về nhãn; (d) rò rỉ ngược — tập huấn luyện chứa thứ mâu thuẫn với tập test	Kiểm tra mô hình có <i>học thuộc</i> nổi một batch nhỏ không; nếu không thì là (a). Đo mức đồng thuận giữa người gán nhãn trên một mẫu con để bắt (b), xem B/06-chat-luong-du-lieu . Nếu mô hình học thuộc được train mà chết trên test, và chia lại vẫn thế, thì nghi (c) hoặc (d).

B. Ba tình huống hai con số này đánh lừa bạn

Bốn góc ở trên giả định precision và recall được tính đúng. Có ba cách rất phổ biến để chúng được tính *sai* mà không ai nhận ra.

- **Accuracy 99% trên dữ liệu lệch 99:1.** Đây là bẫy kinh điển và nó vẫn bắt được người, vì con số 99 nghe như một lời khen. Kiểm tra chỉ mất một phút: tính accuracy của mô hình rỗng luôn trả lời lớp đa số. Nếu con số của bạn không cao hơn nó một cách rõ ràng thì bạn chưa có gì cả. Trong ví dụ đang chạy, mô hình rỗng đạt 0,990 còn mô hình thật đạt 0,945 — *thấp hơn*, dù mô hình thật rõ ràng có ích.
- **Precision đẹp vì tập test đã bị lọc.** Rất hay xảy ra khi tập đánh giá được xây từ đầu ra của một hệ thống cũ: bạn chỉ gán nhãn những ứng viên mà hệ thống cũ đề xuất. Khi đó mọi ca mà hệ thống cũ bỏ sót đều biến mất khỏi mẫu số của recall, và recall bạn đo là recall *có điều kiện đã lọt qua tầng trước*. Nó luôn đẹp hơn recall đầu-cuối, đôi khi đẹp hơn rất nhiều. Dấu hiệu nhận biết: bạn không thể trả lời câu hỏi “các mẫu âm trong tập test đến từ đâu”.
- **Recall đẹp vì chân trị thiếu.** Nếu chân trị bỏ sót một số ca dương thật, thì mẫu số của recall nhỏ hơn thực tế và recall bị thổi lên. Đồng thời, những ca mô hình bắt đúng nhưng chân trị không ghi lại bị tính thành FP, nên precision bị chìm xuống một cách oan uổng. Đây là lý do một mô hình mạnh đôi khi trông tệ hơn mô hình yếu: nó tìm ra những thứ mà người gán nhãn đã bỏ qua. Phép thử duy nhất đáng tin là bốc tay vài chục FP và gán nhãn lại chúng.

Trường hợp thứ ba có một biểu hiện quen thuộc trong SLAM. Khi bạn đánh giá tầng lấy ứng viên đóng vòng bằng nhãn “cùng chỗ” sinh tự động từ chân trị quỹ đạo, ngưỡng khoảng cách bạn chọn *chính là* định nghĩa của lớp dương. Nới ngưỡng từ 1 m lên 3 m thì recall của cùng một hệ thống thay đổi, dù hệ thống không đổi. Con số không mô tả hệ thống; nó mô tả cặp hệ thống–định nghĩa.

5.7 Gộp hai số làm một, và cái giá của việc gộp (F1, F-beta, MCC)

Hai con số khó xếp hạng. Khi phải chọn giữa mười cấu hình, người ta muốn một con số. Mọi cách gộp đều làm mất thông tin, nên câu hỏi đúng vẫn là câu hỏi cũ: nó vứt đi cái gì.

A. Vì sao F1 dùng trung bình điều hoà chứ không phải trung bình cộng

F1 là **trung bình điều hoà** (*harmonic mean*) của precision và recall:

$$F_1 = \frac{2PR}{P+R} = \left(\frac{P^{-1} + R^{-1}}{2} \right)^{-1}.$$

Lý do chọn trung bình điều hoà rất cụ thể: nó bị con số nhỏ kéo xuống. Trung bình cộng thì không. Lấy một hệ thống có precision 0,95 và recall 0,40. Trung bình cộng cho 0,675, nghe khá ổn. F1 cho 0,563, và con số đó phản ánh đúng hơn cái mà người dùng gặp: một hệ thống bỏ sót 60% ca thật.

Nói chặt hơn, trung bình điều hoà luôn nhỏ hơn hoặc bằng trung bình cộng, và khoảng cách giữa hai cái càng lớn khi hai số càng lệch nhau. Bằng nhau chỉ khi $P = R$. Vì vậy F1 thưởng cho sự cân bằng và phạt sự lệch — đúng thứ ta muốn khi *không* có lý do ưu tiên một phía. F1 xuất phát từ thước đo F trong truy hồi thông tin [41].

B. Cái F1 che mất, và F-beta chữa được tới đâu

Ba đoạn số sau nói hết vấn đề của F1. Xét hai hệ thống:

- Hệ A: $P = 0,60, R = 0,60$.
- Hệ B: $P = 0,90, R = 0,45$.

Cả hai đều có $F_1 = 0,600$. Nhưng đây là hai hệ thống hoàn toàn khác nhau. Hệ B đúng chín trên mười lần nó lên tiếng, đổi lại bỏ sót 55% ca thật. Hệ A sai bốn trên mười lần lên tiếng, nhưng bắt được nhiều hơn hẳn. Nếu bạn đang làm đóng vòng, hệ B tốt hơn hẳn; nếu bạn đang sàng lọc y tế, hệ A tốt hơn hẳn. F1 nói chúng bằng nhau. Đó là ý nghĩa của câu “F1 che mất bất đối xứng chi phí”.

F-beta cho phép nói ra sự bất đối xứng đó:

$$F_\beta = \frac{(1 + \beta^2)PR}{\beta^2 P + R}.$$

Cách nhớ ý nghĩa của β : β là số lần recall quan trọng hơn precision. $\beta = 2$ nghĩa là bạn coi một ca bỏ sót đắt gấp đôi một báo động giả; $\beta = 0,5$ nghĩa là ngược lại. Áp vào hai hệ trên: F_2 cho A là 0,600 và cho B là 0,500, nên nó chọn A. $F_{0,5}$ cho A là 0,600 và cho B là 0,750, nên nó chọn B. Cùng một cặp hệ thống, hai kết luận ngược nhau, và cái quyết định là β chứ không phải dữ liệu.

Bài học không phải “hãy dùng F-beta”. Bài học là: chọn β chính là phát biểu tỉ lệ chi phí giữa FP và FN, nên hãy phát biểu nó ra thành lời trước, rồi mới chọn số. Nếu bạn không nói được “một vòng sai đắt bằng bao nhiêu vòng bỏ lỡ” thì bạn cũng chưa đủ dữ kiện để chọn β .

C. Macro, micro, weighted — ba cách gộp cho ra ba kết luận

Với nhiều lớp, còn một tầng gộp nữa: gộp qua các lớp. Ba cách phổ biến, và chúng có thể xếp hạng ngược nhau.

Ví dụ nhỏ nhất, tính tay được. Ba lớp, 1000 mẫu: A có 900, B có 90, C có 10. Mô hình dự đoán như sau. Trong 900 mẫu A, đúng 891, còn 9 bị gọi thành B. Trong 90 mẫu B, đúng 54, còn 36 bị gọi thành A. Trong 10 mẫu C, đúng 1, còn 9 bị gọi thành A.

Viết ra thành bảng thì tính tay được trong hai phút. Bảng 5.5 là ma trận nhầm lẫn của ví dụ đó, kèm precision và recall từng lớp.

Bảng 5.5. Ma trận nhầm lẫn ba lớp của ví dụ tính tay. Hàng là lớp thật, cột là lớp dự đoán. Đọc theo hàng ra recall, đọc theo cột ra precision — và chính hai hướng đọc đó là gốc của chênh lệch giữa macro và micro.

thật	dự đoán			tổng	recall	precision	F1
	A	B	C				
A	891	9	0	900	0,990	0,952	0,971
B	36	54	0	90	0,600	0,857	0,706
C	9	0	1	10	0,100	1,000	0,182
tổng cột	936	63	1	1000			

Tính F1 từng lớp: lớp A được 0,971, lớp B được 0,706, lớp C được 0,182. Ba cách gộp cho ba con số:

- **Micro** — gộp bốn ô của mọi lớp lại rồi mới tính. Ở bài toán một nhãn, micro-F1 bằng đúng accuracy: $946/1000 = 0,946$.
- **Weighted** — trung bình F1 các lớp, trọng số theo số mẫu mỗi lớp: 0,939.
- **Macro** — trung bình cộng F1 các lớp, mỗi lớp một phiếu: 0,619.

Chênh lệch giữa 0,946 và 0,619 không phải sai số. Nó là hai câu hỏi khác nhau. Micro hỏi “một mẫu ngẫu nhiên có được xử lý đúng không”, nên lớp đông quyết định. Macro hỏi “một lớp ngẫu nhiên có được phục vụ tử tế không”, nên lớp C với 10 mẫu có quyền ngang lớp A với 900 mẫu. Weighted nằm giữa nhưng gần micro tới mức thường không thêm thông tin.

Quy tắc chọn rất đơn giản: *nếu bạn quan tâm tới lớp hiếm thì phải báo macro*. Và luôn báo cả hai. Một cặp micro cao, macro thấp là chữ ký của “con số tổng đang che một lớp bị bỏ rơi”, đúng chủ đề của mục kế tiếp trong chương.

D. MCC và kappa — vì sao chúng ổn định hơn accuracy

hệ số tương quan Matthews (*Matthews correlation coefficient*) dùng cả bốn ô trong một công thức đối xứng [42]:

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

Nó là hệ số tương quan giữa hai biến nhị phân “dự đoán” và “chân trị”, nên nó nằm trong $[-1, 1]$: 1 là hoàn hảo, 0 là không hơn đoán mò, âm là tệ hơn đoán mò. Tính chất quan trọng nhất: MCC chỉ cao khi cả bốn ô đều tốt. Không ô nào bị bỏ ra ngoài công thức, khác hẳn precision, recall và F1 — cả ba đều không đụng tới TN [43].

Điều đó cho MCC một tính chất mà accuracy không có. Mô hình rỗng luôn trả lời “không” đạt accuracy 0,990 nhưng MCC bằng đúng 0, vì nó không tương quan gì với chân trị. Mô hình thật ở ngưỡng 0,50 đạt accuracy 0,945 — thấp hơn — nhưng MCC 0,261. Hai con số accuracy xếp hạng ngược, MCC xếp hạng đúng.

hệ số kappa của Cohen (*Cohen's kappa*) [31] đi theo cùng ý tưởng nhưng phát biểu khác: nó đo mức đồng thuận vượt trên mức đồng thuận ngẫu nhiên tính từ tỉ lệ nền. Trên cùng ví dụ, kappa bằng 0,181. Cả hai đều là cách trả lời câu hỏi “con số này hơn đoán mò bao nhiêu”, và cả hai vì thế miễn nhiễm với trò 99% accuracy.

Cái giá phải trả cũng nên nói thẳng: MCC và kappa khó diễn giải hơn hẳn. Không ai nói được “MCC 0,26 nghĩa là người dùng gặp bao nhiêu báo động giả”. Chúng tốt để xếp hạng và để phát

hiện ảo giác, không tốt để *báo cáo cho người vận hành*. Trong thực tế, cặp precision–recall vẫn là thứ đi vào báo cáo, còn MCC là thứ bạn nhìn để tự kiểm.

5.8 Đường cong, và cái đường cong giấu đi (PR and ROC Curves)

Một điểm vận hành cho bốn ô. Nhưng ngưỡng là thứ bạn tự chọn, nên báo cáo một điểm duy nhất là báo cáo một lựa chọn chứ không phải một mô hình. Đường cong sinh ra để tách hai thứ đó: nó cho thấy *toàn bộ* các điểm vận hành mà mô hình có thể đạt tới.

A. Hai đường cong, trục là gì và một điểm nghĩa là gì

- **Đường ROC** — trục hoành là FPR, trục tung là TPR (tức recall). Một điểm trên đường là một ngưỡng cụ thể, và nó nói: “ở ngưỡng này, tôi bắt được TPR phần các ca dương, và làm phiền FPR phần các ca âm”. Cả hai trục chuẩn hoá *trong lớp*, nên đường ROC không phụ thuộc vào tỉ lệ nền.
- **Đường PR** — trục hoành là recall, trục tung là precision. Một điểm nói: “ở ngưỡng này, tôi bắt được recall phần các ca dương, và trong những gì tôi báo thì precision phần là đúng”. Trục tung chuẩn hoá theo *số lần báo*, nên đường PR phụ thuộc mạnh vào tỉ lệ nền.

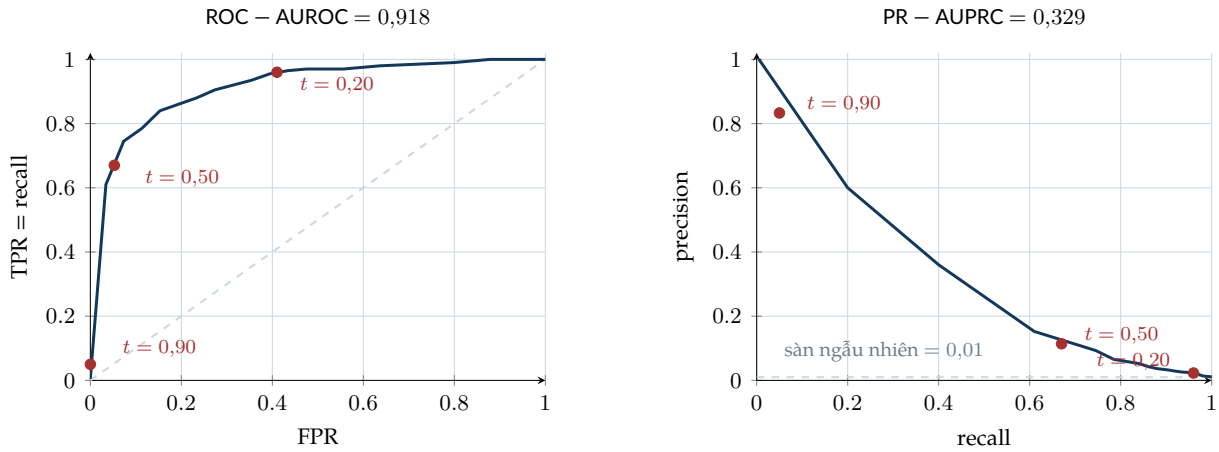
Đường của một bộ đoán ngẫu nhiên nằm ở hai chỗ khác nhau, và đây là chi tiết hay bị quên. Trên ROC, đoán ngẫu nhiên là đường chéo từ (0, 0) tới (1, 1), tức AUROC = 0,5, bất kể tỉ lệ nền. Trên PR, đoán ngẫu nhiên là một đường *nằm ngang ở đúng độ cao bằng tỉ lệ nền*. Với tỉ lệ nền 1%, sàn ngẫu nhiên của đường PR là 0,01. Vì thế AUPRC 0,33 không phải “kém hơn một nửa” như trục giác mách bảo — nó gấp ba mươi ba lần sàn.

B. Vì sao ROC trông đẹp giả tạo khi lớp dương hiếm

Đây là ý quan trọng nhất của cả mục, và nó nằm gọn trong một câu: *FPR có mẫu số rất lớn, nên FP tăng mạnh mà FPR gần như không nhúc nhích*.

Chạy con số cho cụ thể, vẫn trên bộ phân loại tổng hợp. Ở ngưỡng 0,50, FPR bằng 0,053. Nghe nhỏ. Nhưng 0,053 nhân với 19 800 mẫu âm ra 1043 lần báo động giả, trong khi chỉ có 134 lần báo đúng. Người vận hành nhận được 1177 cảnh báo mỗi ngày và 89% trong số đó là rác. Trục ROC không có cách nào hiển thị điều này, vì nó chia cho 19 800 chứ không chia cho 1177.

Hình 5.6 vẽ cả hai đường cho cùng một mô hình. AUROC bằng 0,918 — một con số bạn sẽ vui vẻ viết vào báo cáo. AUPRC bằng 0,329 trên cùng dữ liệu, cùng mô hình, cùng mọi thứ. Không cái nào sai. Chúng trả lời hai câu hỏi khác nhau, và chỉ một trong hai là câu hỏi mà người dùng đang hỏi.



Hình 5.6. Cùng một mô hình, cùng một dữ liệu, hai đường cong (bộ phân loại tổng hợp: 20 000 mẫu, 200 dương). Ba điểm đỏ là cùng ba ngưỡng t trên cả hai hình. Ngưỡng 0,50 nằm gần góc lý tưởng trên ROC nhưng nằm sát sàn trên PR, vì trục tung bên trái chia cho 19 800 mẫu âm còn bên phải chia cho 1177 lần báo động. Sàn ngẫu nhiên của ROC là đường chéo; sàn ngẫu nhiên của PR là đường ngang ở độ cao bằng tỉ lệ nền.

Quan hệ giữa hai không gian đã được chứng minh chặt: một đường cong chi phối trên ROC thì cũng chi phối trên PR và ngược lại, nhưng thứ hạng theo diện tích thì *không* bảo toàn [44]. Trên dữ liệu lệch lớp, đường PR phân biệt các mô hình rõ hơn hẳn [45]. Giới thiệu cổ điển về ROC và cách đọc nó vẫn là bài của Fawcett [46].

C. Chọn điểm vận hành bằng ma trận chi phí, không bằng thói quen

Nếu bạn gán được giá cho FP và FN thì việc chọn ngưỡng không còn là chuyện khẩu vị. Gọi c_{FP} và c_{FN} là chi phí một báo động giả và một ca bỏ sót. Chi phí kỳ vọng ở một ngưỡng là

$$\mathcal{C}(t) = c_{FP} \cdot FP(t) + c_{FN} \cdot FN(t).$$

Quét t qua toàn dải, tính $\mathcal{C}$, lấy điểm nhỏ nhất. Đó là toàn bộ thuật toán. Nó tầm thường về mặt tính toán và gần như không ai làm, vì phần khó là gán giá chứ không phải quét ngưỡng.

Với đóng vòng SLAM, phép tính này ra kết quả rất lệch. Đặt $c_{FN} = 1$ — một vòng bỏ lỡ mất một cơ hội xóa sai số. Đặt $c_{FP} = 100$ — một vòng sai lầm cong bản đồ và thường phải chạy lại. Tỉ lệ 100:1 là một ước lượng bậc thang từ cơ chế, không phải số đo; nhưng ngay cả khi bạn tin nó chỉ đúng trong vòng một bậc, kết luận vẫn không đổi: điểm vận hành phải nằm hẳn ở phần precision cao của đường cong, chấp nhận recall thấp. Đó chính xác là lý do ORB-SLAM3 đòi nhiều khung hình liên tiếp cùng đồng ý trước khi chấp nhận một vòng [1]. Cũng vì thế bộ từ vựng nhị phân luôn đi kèm bước kiểm chứng thời gian và hình học, không ai dùng nó trần [47].

D. Một đường cong đơn điệu vẫn ứng với hai hệ thống rất khác nhau

Đường cong tóm tắt tốt hơn một điểm, nhưng nó vẫn là một phép nén. Hai chỗ nó giấu thông tin:

- **Diện tích bằng nhau, hình dạng khác nhau.** Một mô hình có precision rất cao ở recall thấp rồi tụt dốc, và một mô hình precision trung bình đều trên toàn dải, có thể cho cùng một AUPRC. Nếu bạn chỉ vận hành ở vùng recall thấp thì mô hình thứ nhất tốt hơn nhiều. *Hãy đọc đường cong ở vùng bạn thật sự vận hành, đừng đọc diện tích.*
- **Cùng một đường, hai tập lỗi khác nhau.** Đường cong chỉ đếm số lượng, không nói *mẫu* nào sai. Hai mô hình cùng đường PR có thể sai trên hai nhóm mẫu hoàn toàn tách biệt. Cách phát hiện là đọc metric theo lát, hoặc đơn giản là đếm số mẫu mà hai mô hình bất đồng.

I 5.9 Xác suất có đáng tin không (Calibration)

Ba mục vừa rồi chỉ dùng *thứ hạng* của điểm số. Đường ROC, đường PR, AUC — tất cả đều không đổi nếu bạn ép điểm số qua một hàm tăng nghiêm ngặt bất kỳ. Điều đó có một hệ quả khó chịu: *một mô hình có AUC hoàn hảo vẫn có thể sai hoàn toàn về xác suất*.

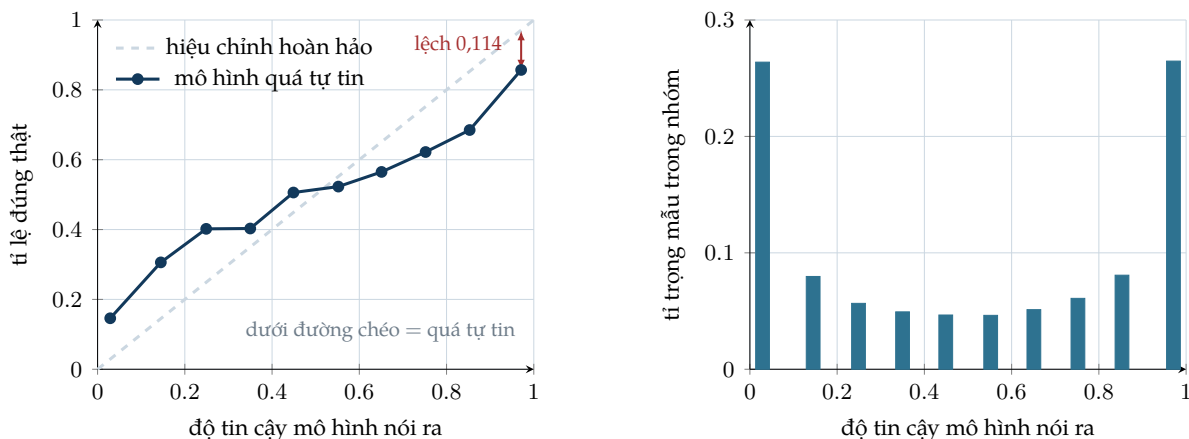
Chuyện này quan trọng khi đầu ra không dừng ở một quyết định nhị phân. Điểm số có thể đi vào một bộ hợp nhất cảm biến, vào một ma trận thông tin, hay vào một quy tắc “chỉ hành động khi độ tin cậy trên 0,9”. Trong cả ba trường hợp, giá trị số của nó là một phần của tính đúng đắn.

A. Biểu đồ độ tin cậy: đọc thế nào

biểu đồ độ tin cậy (*reliability diagram*) được vẽ theo ba bước. Chia các dự đoán thành các nhóm theo độ tin cậy mô hình nói ra — ví dụ mười nhóm, $[0; 0,1)$, $[0,1; 0,2)$, v.v. Trong mỗi nhóm, tính hai số: độ tin cậy trung bình mô hình *nói*, và tỉ lệ đúng *thật*. Vẽ cặp đó lên mặt phẳng. Mô hình hiệu chỉnh tốt nằm trên đường chéo.

Đọc lệch: điểm nằm *dưới* đường chéo nghĩa là mô hình nói mạnh hơn thực tế, tức quá tự tin; nằm trên là quá dè dặt. Mạng nơ-ron hiện đại lệch xuống một cách hệ thống ở vùng độ tin cậy cao [40].

Hình 5.7 vẽ một trường hợp như vậy. Ở nhóm mà mô hình nói 0,97, tỉ lệ đúng thật chỉ 0,86. Điều đáng chú ý nhất: mô hình này có AUROC *y* hết mô hình hiệu chỉnh đúng, vì nó chỉ bị làm sắc lại chứ không đổi thứ hạng. Mọi metric dựa trên thứ hạng đều không thấy gì.



Hình 5.7. Biểu đồ độ tin cậy của một mô hình quá tự tin (dữ liệu tổng hợp, 20 000 mẫu). Bên trái: đường nằm dưới đường chéo ở nửa phải, nghĩa là mô hình nói mạnh hơn thực tế; nhóm “0,97” chỉ đúng 0,86. Bên phải là tỉ trọng mẫu trong từng nhóm, cần thiết vì ECE lấy trung bình có trọng số theo cột này — một nhóm lệch nặng nhưng gần như rỗng thì đóng góp không đáng kể. ECE của mô hình này là 0,115.

B. ECE, Brier, và phân rã Brier

- **ECE** (*expected calibration error*) — lấy độ lệch tuyệt đối giữa hai số trong mỗi nhóm, rồi trung bình có trọng số theo số mẫu trong nhóm [48]. Trong hình trên, ECE bằng 0,115. Đọc là: “trung bình, độ tin cậy mô hình nói ra lệch khoảng 11 điểm phần trăm so với tần suất đúng thật”. Nhược điểm phải biết: ECE phụ thuộc vào cách chia nhóm, nên hai bài báo dùng số nhóm khác nhau không so được với nhau.
- **Điểm Brier** — trung bình bình phương sai lệch giữa xác suất dự đoán và nhãn 0/1 [49]. Nó là một hàm mất mát bình phương, nên nó phạt cả việc xếp hạng sai lẫn việc hiệu chỉnh sai. Ở ví dụ trên, Brier của mô hình là 0,188 còn của mô hình hiệu chỉnh đúng là 0,173.

- **Phân rã Brier** — và đây là chỗ nó trở nên hữu ích thật sự. Điểm Brier tách được thành ba phần: độ tin cậy (*reliability*), độ phân giải (*resolution*) và độ bất định nền (*uncertainty*) [50]. Viết gọn: $\text{Brier} = \text{reliability} - \text{resolution} + \text{uncertainty}$. Phần *reliability* đo đúng cái ECE đo, và càng nhỏ càng tốt. Phần *resolution* đo khả năng mô hình tách các mẫu thành các nhóm có tần suất khác nhau, và càng lớn càng tốt. Phần *uncertainty* chỉ phụ thuộc tỉ lệ nền, không phụ thuộc mô hình.

Phân rã này đáng nhớ vì nó cho một chẩn đoán rõ. Brier tệ do *reliability* lớn thì bạn chỉ cần hiệu chỉnh lại đầu ra, một việc rẻ và hậu kiểm được. Brier tệ do *resolution* nhỏ thì mô hình thật sự không phân biệt nổi các mẫu, và không phép hậu xử lý nào cứu được.

Cách sửa và cách vận hành phần này — *temperature scaling*, hiệu chỉnh theo lát, đo lại sau khi triển khai — thuộc về E/25-do-tin-cay-va-van-hanh. Ở đây chỉ cần nhớ một điều: *AUC cao không cho phép bạn tin con số xác suất*.

5.10 Nhiều lớp và nhiều nhãn (Multi-class and Multi-label)

Bốn ô là chuyện của hai lớp. Với nhiều lớp, ma trận nhầm lẫn thành một bảng $K \times K$, và cách đọc nó là một kỹ năng riêng.

A. Đọc theo hàng và đọc theo cột là hai câu hỏi khác nhau

Quy ước phổ biến: hàng là lớp thật, cột là lớp dự đoán. Khi đó ô (i, j) là số mẫu thuộc lớp i bị gọi thành lớp j . Từ đó:

- **Đọc theo hàng i** — “những mẫu thật sự thuộc lớp i đã đi đâu”. Chia ô đường chéo cho tổng hàng ra *recall của lớp i* . Hàng có ô lớn lệch khỏi đường chéo cho biết lớp i đang bị nuốt bởi lớp nào.
- **Đọc theo cột j** — “những mẫu bị gọi là lớp j thật ra là gì”. Chia ô đường chéo cho tổng cột ra *precision của lớp j* . Cột dày cho biết lớp j là một “hồ đen” mà mọi thứ rơi vào.
- **Cặp lớp bị lẫn** — nhìn cặp ô đối xứng (i, j) và (j, i) . Nếu cả hai đều lớn thì hai lớp đó thật sự khó phân biệt, và có thể định nghĩa nhãn đang chồng lẫn. Nếu chỉ một chiều lớn thì đó là thiên lệch một phía, thường do mất cân bằng số lượng.
- **Lớp hiếm biến mất** — một hàng có tổng bằng vài chục trong bảng vài chục nghìn mẫu thì mọi con số tổng đều mù với nó. Đây là lý do phải báo macro cùng micro, như ở mục trước.

B. Top-1 và top-5

Top-1 là accuracy thông thường: nhãn có điểm cao nhất có đúng không. Top-5 hỏi nhẹ hơn: nhãn đúng có nằm trong năm nhãn điểm cao nhất không. Chênh lệch giữa hai con số là một chẩn đoán trực tiếp. Top-5 cao mà top-1 thấp nghĩa là mô hình *biết* vùng đáp án nhưng không xếp hạng nổi trong vùng đó — thường là các lớp cực giống nhau, hoặc nhãn mơ hồ ở chính chân trị. Cả hai cùng thấp nghĩa là mô hình chưa khoanh được vùng.

Cần một cảnh báo: top-5 có ý nghĩa vì ImageNet có 1000 lớp [51]. Với 8 lớp, top-5 gần như luôn bằng 1 và không mang thông tin nào.

C. Nhiều nhãn: subset accuracy so với Hamming

Bài toán nhiều nhãn cho phép một mẫu mang nhiều nhãn cùng lúc. Hai cách đo phổ biến nằm ở hai cực:

- **Subset accuracy** — chỉ tính đúng khi *toàn bộ* tập nhãn khớp chính xác. Rất nghiêm. Với 20 nhãn, một mẫu sai một nhãn cũng bị tính là sai hoàn toàn. Con số này rơi rất nhanh khi số nhãn tăng, và nó phạt như nhau dù bạn sai một nhãn hay sai cả hai mươi.
- **Hamming loss** — tỉ lệ ô nhãn sai trên tổng số ô, tính trên toàn bộ ma trận mẫu $\times$ nhãn. Rất dễ dài. Nếu mỗi mẫu chỉ có 2 trong 20 nhãn là dương, thì đoán “tất cả đều âm” đã cho Hamming loss 0,10, nghe như một kết quả tốt.

Hai cái đó đóng khung khoảng đúng, nên cách hành xử đúng là báo cả hai, cộng với macro-F1 theo nhãn. Riêng Hamming loss cần luôn kèm tỉ lệ nền trung bình, vì nếu không thì nó lặp lại đúng bấy accuracy 99% ở quy mô lớn hơn.

5.11 Metric của từng bài toán thị giác, và điều mỗi cái che giấu (Task-specific Metrics)

Các mục trên là bộ khung chung. Mỗi bài toán thị giác lại đóng gói bộ khung đó theo một cách riêng, và mỗi lần đóng gói lại giấu thêm một thứ. Mục này đi qua năm họ metric mà người đọc sẽ gặp, theo cùng một câu hỏi: *con số này tăng thì thật ra cái gì đã tốt lên*.

A. Phát hiện vật thể: IoU, AP, mAP và các biến thể

Ở phát hiện vật thể, trước khi đếm được TP và FP, phải quyết định một hộp dự đoán có “khớp” một hộp thật hay không. Đó là việc của IoU, tỉ số diện tích giao trên diện tích hợp. Khớp hay không là một quyết định nhị phân với ngưỡng riêng, nên bài toán có *hai* ngưỡng chứ không phải một: ngưỡng độ tin cậy, và ngưỡng IoU.

- **AP** — diện tích dưới đường PR của một lớp, quét theo ngưỡng độ tin cậy với ngưỡng IoU cố định. **mAP** là trung bình AP trên các lớp, tức một phép trung bình macro và vì thế lớp hiếm có quyền ngang lớp đông.
- **AP50 so với AP75** — cùng một mô hình, hai ngưỡng IoU. AP50 dễ dài với sai lệch định vị; AP75 đòi hộp bám sát. Chênh lệch giữa hai con số là một chẩn đoán: AP50 cao mà AP75 thấp nghĩa là mô hình *tìm* được vật nhưng *khoanh* không chuẩn.
- **AP theo kích thước (APs, APm, APl)** — COCO chia vật thể theo diện tích và báo riêng [52]. Đây là cách chia lát đã được đóng sẵn vào metric, và nó là chỗ ẩn nhiều thông tin nhất.
- **AR@k** — recall trung bình khi mỗi ảnh chỉ được giữ k đề xuất. Nó đo chất lượng của tầng sinh ứng viên, tách khỏi tầng phân loại [53]. Cùng tinh thần với Recall@K trong truy hồi ở mục C dưới đây.

Điều mAP che giấu, nói thẳng: “*mAP tăng 1 điểm*” có thể là vật lớn tốt lên trong khi vật nhỏ tệ đi. mAP là trung bình trên các lớp và ngưỡng, nên hai thay đổi ngược chiều có thể triệt tiêu nhau và để lại một con số tổng nhích lên. Nếu ứng dụng của bạn sống bằng vật nhỏ ở xa — đúng trường hợp phát hiện vật cản cho robot — thì con số tổng đang nói dối bạn. Cách chặn rất rẻ: luôn báo APs, APm, APl cùng mAP, và dùng một công cụ phân rã lỗi như TIDE khi cần biết lỗi thuộc loại nào [34], [54].

B. Phân đoạn: mIoU, Dice, boundary F và PQ

- **mIoU** — IoU tính theo pixel cho từng lớp rồi lấy trung bình *theo lớp*. Vì lấy trung bình theo lớp chứ không theo pixel, nó công bằng hơn accuracy pixel; nhưng nó vẫn giấu lớp hiếm theo một cách khác. Một lớp chỉ xuất hiện trong 2% số ảnh vẫn có một phiếu đầy đủ, nên IoU của nó dao động dữ dội giữa các lần chạy và kéo mIoU nhảy theo mà không có nguyên nhân thật.

- **Dice, tức F1 theo pixel** — $\text{Dice} = 2|A \cap B| / (|A| + |B|)$. Nó và IoU là hàm đơn điệu của nhau nên xếp hạng giống nhau, nhưng Dice luôn cho số cao hơn. Đừng so Dice của bài này với IoU của bài kia.
- **Boundary F** — đo riêng độ khớp của *đường biên* thay vì của cả vùng [55]. Cần nó vì IoU bị chi phối bởi phần lõi của vùng: một vật lớn có biên sai vài pixel gần như không làm IoU nhúc nhích, trong khi với một vật nhỏ thì đúng sai lệch ấy là toàn bộ câu chuyện. Đây là điểm đã được nêu từ lâu trong bàn luận về thước đo phân đoạn [56].
- **PQ (*panoptic quality*)** — tách thành hai phần nhân nhau: chất lượng nhận dạng (RQ, chính là F1 ở mức thực thể) và chất lượng phân đoạn (SQ, IoU trung bình của các thực thể đã khớp) [35]. Cấu trúc tích này rất đáng học: nó cho phép nói “PQ tụt vì RQ chứ không vì SQ”, tức vì đếm sai số thực thể chứ không vì khoanh xấu.

C. Truy hồi và nhận dạng địa điểm — và đóng vòng SLAM

Ở truy hồi, hệ thống trả về một danh sách xếp hạng chứ không một câu trả lời. Metric vì thế phải nói rõ “xét bao nhiêu ứng viên”.

- **Recall@K** — tỉ lệ truy vấn mà có ít nhất một kết quả đúng nằm trong K ứng viên đầu. Đây là metric chuẩn của nhận dạng địa điểm. Chú ý mẫu số: nó chia cho *số truy vấn*, không chia cho số ứng viên.
- **Precision@K** — tỉ lệ đúng trong K ứng viên đầu. Nó và Recall@K trả lời hai câu khác nhau, và với đóng vòng thì cái thứ hai mới là cái quan trọng, vì tầng sau phải xử lý cả K ứng viên.
- **mAP truy hồi** — trung bình AP trên các truy vấn, dùng khi một truy vấn có nhiều kết quả đúng.
- **Đường recall-precision theo K** — tăng K thì recall tăng và precision giảm. Đây chính là đường PR của mục trước, chỉ tham số hoá bằng K thay vì bằng ngưỡng.

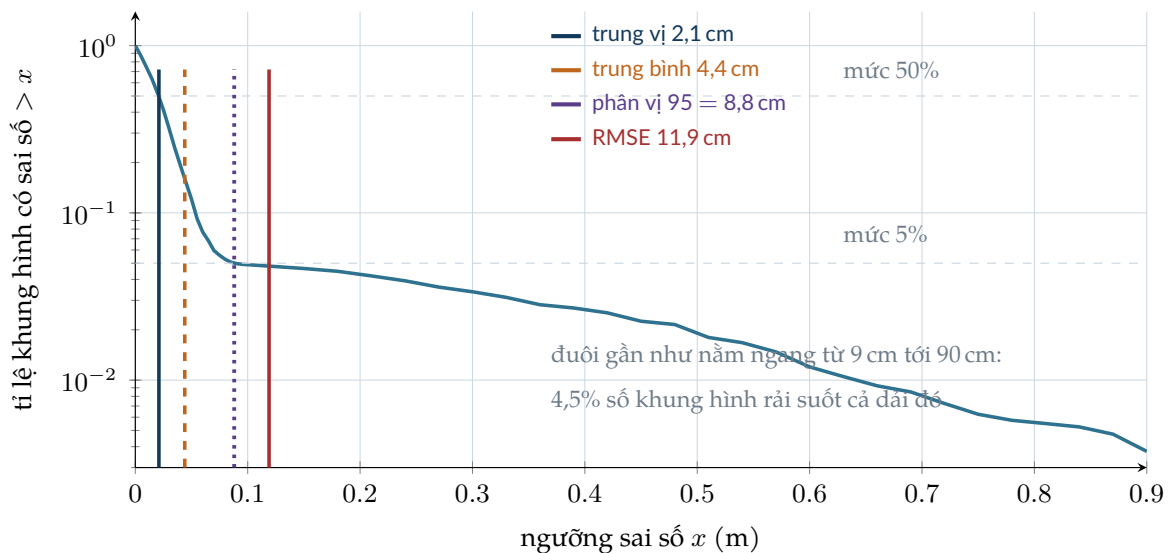
Nói thẳng sang đóng vòng. Tầng lấy ứng viên (DBoW2, NetVLAD, hay bất cứ thứ gì bạn dùng) được đánh giá bằng Recall@K, và điều đó *đúng*, vì việc của nó chỉ là không để lọt. Nhưng hệ thống đóng vòng đầy đủ thì phải được đánh giá bằng precision ở điểm vận hành cuối, sau kiểm chứng hình học. Hai tầng, hai metric, và lẫn lộn chúng là một lỗi thường gặp.

Lý do đã nói ở trên nhưng đáng lặp lại: *một vòng sai đắt hơn nhiều một vòng bỏ lỡ*. Vòng bỏ lỡ chỉ có nghĩa là sai số chưa được xoá; vòng sai khau hai nơi khác nhau thành một và làm hỏng bản đồ, thường không hồi phục. Cho nên điểm vận hành của hệ đóng vòng phải lệch hẳn về phía precision — gần góc trên trái của Hình 5.5 — và recall thấp là một lựa chọn có chủ ý, không phải một thất bại.

D. Hồi quy và quỹ đạo: MAE, RMSE, trung vị, phân vị

- **MAE** — trung bình trị tuyệt đối của sai số. Mỗi mẫu đóng góp tuyến tính.
- **RMSE** — căn của trung bình bình phương sai số. Bình phương làm mẫu lỗi lớn đóng góp gấp bội, nên RMSE là một thước đo *bị đuôi chi phối*.
- **Trung vị và phân vị** — trung vị nói “một mẫu điển hình sai bao nhiêu”; phân vị 95 và 99 nói “ngày tệ nhất tệ tới đâu”.

Hình 5.8 cho thấy vì sao ba nhóm này phải đi cùng nhau. Dữ liệu là sai số vị trí theo từng khung hình của một chuỗi tổng hợp: phần lớn thời gian bám tốt, và 4,5% số khung có một đợt bung. Trung vị là 2,1 cm, trung bình 4,4 cm, còn RMSE lên tới 11,9 cm. RMSE gấp gần sáu lần trung vị, và toàn bộ chênh lệch đó do 4,5% số khung tạo ra.



Hình 5.8. Cùng một chuỗi tổng hợp (4000 khung), vẽ theo lối “bao nhiêu phần khung hình tệ hơn mức x ” với trục tung log. Đường tụt rất nhanh tới mức 5% rồi *gần như nằm ngang* suốt từ 9 cm tới 90 cm — đó chính là cái đuôi. Bốn đường dọc là bốn cách tóm tắt cùng phân bố này. Trung vị cắt đường ở mức 50%, phân vị 95 cắt ở mức 5%, còn RMSE nằm hẳn bên phải phân vị 95, tức nó mô tả một khung hình bạn gặp ít hơn một lần trong hai mươi. Báo một mình RMSE là báo về cái đuôi và im lặng về hành vi thường ngày; báo một mình trung vị thì ngược lại.

Với quỹ đạo SLAM, hệ quả rất cụ thể và đáng thành thói quen.

- **Báo APE kèm trung vị và phân vị, không chỉ RMSE.** Công cụ [MichaelGrupp/evo](#) in sẵn cả bộ; vấn đề nằm ở việc người ta chỉ chép mỗi dòng RMSE vào bảng. Hai hệ có cùng RMSE có thể là một hệ luôn lệch đều và một hệ hoàn hảo trừ hai giây bung ra. Hướng dẫn đánh giá quỹ đạo của Zhang và Scaramuzza nói rõ điểm này [57]; thư mục metric của TUM RGB-D là nơi cặp APE–RPE được chuẩn hoá [39].
- **Luôn báo tỉ lệ hoàn thành chuỗi cùng APE.** Đây là chỗ dễ tự lừa nhất. Một hệ mất bám ở giây thứ 30 của chuỗi 120 giây, rồi chỉ nộp phần quỹ đạo nó còn giữ, có thể cho APE *rất đẹp* — vì nó chỉ được chấm trên đoạn dễ. So sánh một hệ hoàn thành 100% với một hệ hoàn thành 25% bằng APE là so sánh vô nghĩa. Quy tắc tối thiểu: mọi bảng APE phải có một cột tỉ lệ hoàn thành ngay bên cạnh.

E. Ghép cặp đặc trưng: matching score, MMA, tỉ lệ inlier

- **Matching score** — tỉ lệ điểm đặc trưng phát hiện được mà tìm đúng điểm tương ứng, chuẩn hoá theo số điểm phát hiện được [58].
- **MMA (mean matching accuracy)** — tỉ lệ cặp khớp có sai số chiều dưới một ngưỡng pixel, lấy trung bình trên các cặp ảnh; nó được báo theo từng ngưỡng chứ không một số [59].
- **Tỉ lệ inlier** — tỉ lệ cặp khớp sống sót sau RANSAC. Đây là con số bạn nhìn hằng ngày trong log.

Và đây là câu hỏi mà người đọc thật sự cần trả lời: *vì sao ba con số này tăng mà APE không đổi*. Có bốn lời giải thích, và chúng cần bốn phép thử khác nhau.

- **Bão hoà.** Bộ ước lượng đã có thừa ràng buộc. Thêm cặp khớp đúng thứ 200 không dịch chuyển nghiệm khi 80 cặp đã đủ. Phép thử: giảm số cặp giữ lại xuống một nửa rồi đo lại APE; nếu APE không đổi thì bạn đang ở vùng bão hoà.

- **Đúng nhưng không đa dạng.** Cặp khớp mới tập trung vào vùng đã có nhiều điểm, hoặc ở cùng một mặt phẳng độ sâu. Hình học của bài toán không được cải thiện. Phép thử: đo phân bố không gian của các điểm khớp, và đo số điều kiện của ma trận thông tin trước và sau.
- **Nút thắt nằm ở chỗ khác.** Nếu sai số bị chi phối bởi hiệu chỉnh, độ trễ thời gian, hay mô hình IMU, thì cải thiện tầng khớp không đụng tới nút thắt. Phép thử: thay tầng khớp bằng chân trị (dùng chính chân trị để sinh cặp khớp hoàn hảo) và xem APE tụt tới đâu. Con số đó là trần lý thuyết của mọi cải tiến ở tầng khớp.
- **Metric ghép cặp đo trên phân bố khác.** Matching score và MMA thường đo trên các cặp ảnh có góc nhìn cách xa nhau, còn tracking khung-tới-khung thì góc nhìn gần như nhau. Cải thiện ở chế độ khó không chuyển thành cải thiện ở chế độ dễ. Phép thử: đo lại MMA đúng trên phân bố cặp khung hình mà hệ thống thật sự dùng.

Bài học tổng quát vượt ra ngoài SLAM: *một metric trung gian chỉ có giá trị khi nó nằm trên đường tới hạn của metric cuối.* Trước khi tối ưu một metric trung gian, hãy đo xem nó có nằm trên đường tới hạn không — bằng đúng phép thử “thay bằng chân trị” ở trên.

F. Bảng tóm tắt: mỗi metric trả lời câu hỏi nào

Bảng 5.6 gom năm họ trên vào một chỗ, theo đúng bốn bước của cả khối: nó trả lời câu hỏi nào, mẫu số là gì, và nó im lặng về điều gì.

Bảng 5.6. Metric của từng bài toán, đọc theo cùng một khuôn. Cột “im lặng về” là cột đáng đọc trước, vì nó nói cho bạn biết phải đọc thêm metric nào nữa.

Metric	Câu hỏi nó trả lời	Mẫu số	Nó im lặng về
mAP	Trung bình chất lượng xếp hạng phát hiện trên các lớp	mỗi lớp một phiếu	Vật nhỏ so với vật lớn; chất lượng định vị nếu chỉ dùng ngưỡng IoU 0,5
AR@k	Tầng sinh ứng viên có để lọt vật nào không	số vật thật	Chất lượng phân loại và xếp hạng ở tầng sau
mIoU	Mỗi lớp được khoanh khớp tới đâu, tính theo pixel	mỗi lớp một phiếu	Đường biên; và lớp hiếm có IoU dao động mạnh kéo con số nhảy
Boundary F	Đường biên khớp tới đâu	số điểm biên	Phần lõi của vùng; một vùng lệch hẳn vị trí vẫn có thể có biên trơn
PQ	Vừa đếm đúng thực thể vừa khoanh đúng chưa	thực thể	Cần tách RQ và SQ mới biết hồng ở đếm hay ở khoanh
Recall@K	Ứng viên đúng có nằm trong K kết quả đầu không	số truy vấn	Chi phí xử lý K ứng viên đó ở tầng sau; và precision cuối cùng
APE (ATE)	Quỹ đạo lệch bao nhiêu sau khi căn chỉnh	số khung hình được chấm	Trôi toàn cục đã bị căn chỉnh hấp thụ; và phần chuỗi bị mất bám
RPE	Các đoạn dịch chuyển ngắn có nhất quán không	số đoạn	Sai lệch toàn cục
Tỉ lệ inlier	Bao nhiêu cặp khớp sống sót sau RANSAC	số cặp khớp đề xuất	Hình học có được cải thiện không; và nút thắt có nằm ở tầng khớp không

5.12 Bảng chẩn đoán ngược (Reading Numbers Backwards)

Đây là đích đến của cả khối. Mọi mục trên đi từ định nghĩa tới ý nghĩa. Trong công việc thật, bạn đi ngược: nhìn thấy một bộ số lạ, rồi phải tìm ra nó nghĩa là gì. Bảng 5.7 tổ chức việc đó theo ba cột — triệu chứng, các cách giải thích, và phép thử tách chúng ra.

Cách dùng bảng: đọc cột giữa như một danh sách nghi phạm, không phải một câu trả lời. Gần như mọi triệu chứng đều có từ hai tới bốn nguyên nhân khác nhau, và chúng đòi những cách sửa khác hẳn nhau. Cột phải là thứ biến nghi ngờ thành kết luận.

Bảng 5.7. Từ con số ngược về nguyên nhân. Cột phải luôn là một việc làm được trong một buổi, không phải một lời khuyên chung chung.

Bạn thấy	Các cách giải thích	Phép thử để biết là cách nào
Accuracy 0,99 mà recall lớp hiếm bằng 0	Mô hình đã học chiến lược “luôn đoán lớp đa số”; hoặc lớp hiếm quá ít mẫu để học; hoặc hàm mất mát không có trọng số lớp	Tính accuracy của mô hình rỗng. Nếu bằng nhau thì mô hình đúng là rỗng. Sau đó thử cân bằng lại hàm mất mát và xem recall lớp hiếm có nhúc nhích không; nếu vẫn 0 thì vẫn đề là số mẫu chứ không phải trọng số.
AUROC 0,95 mà AUPRC 0,25	Lớp dương hiếm — đây là hành vi <i>bình thường</i> , không phải lỗi; hoặc mô hình xếp hạng tốt ở vùng giữa mà tệ ở đỉnh danh sách	So AUPRC với tỉ lệ nền: nếu AUPRC gấp hàng chục lần tỉ lệ nền thì mô hình thật sự có ích và bạn chỉ đang đọc nhầm thang đo. Vẽ đường PR và nhìn riêng vùng recall thấp để bắt trường hợp thứ hai.
mAP tăng còn mIoU giảm	Cải thiện ở tầng phát hiện thực thể đi kèm suy giảm ở tầng phân loại pixel; hoặc hai metric được tính trên hai tập lớp khác nhau; hoặc thay đổi hậu xử lý làm hộp chặt hơn còn mặt nạ thô hơn	Đọc AP và IoU theo từng lớp rồi so cặp. Nếu chỉ vài lớp đảo chiều thì là thay đổi thật ở các lớp đó. Kiểm tra danh sách lớp của hai phép đánh giá có trùng nhau không — đây là nguyên nhân tầm thường nhưng rất hay gặp.
Val loss giảm mà metric không đổi	Mô hình đang cải thiện độ tin cậy chứ không đổi thứ hạng; hoặc metric có ngưỡng nên thay đổi nhỏ chưa vượt ngưỡng; hoặc metric bão hoà	Vẽ biểu đồ độ tin cậy trước và sau: nếu ECE giảm mà AUC không đổi thì đúng là chỉ hiệu chỉnh đổi. Quét lại toàn dải ngưỡng; nếu đường PR dịch lên mà điểm vận hành cố định thì là chuyện ngưỡng.
Precision khi triển khai thấp hơn hẳn trên tập test	Tỉ lệ nền ngoài đời thấp hơn trong tập test; hoặc phân phối đầu vào đã dịch chuyển; hoặc tập test đã bị lọc trước bởi một hệ thống cũ	Đo tỉ lệ nền thật ở hiện trường và tính lại precision kỳ vọng theo tỉ lệ đó. Nếu con số khớp thì nguyên nhân là mẫu số, không phải mô hình. Nếu vẫn lệch thì sang B/08-dịch-chuyen-phan-phoi .
F1 đứng yên trong khi cả precision lẫn recall đều đổi	Hai thay đổi bù trừ nhau — một hệ thống khác hẳn đang trộn sau một con số không đổi	Vẽ điểm vận hành cũ và mới trên cùng mặt phẳng PR. Rồi tính F_2 và $F_{0,5}$: nếu chúng xếp hạng ngược nhau thì hai hệ thật sự khác, và β đúng theo chi phí của bạn mới là trọng tài.
Recall@1 cao mà đóng vòng vẫn sai	Tầng lấy ứng viên tốt nhưng tầng kiểm chứng hình học yếu; hoặc nhãn “cùng chỗ” sinh tự động quá lỏng; hoặc lỗi nằm ở tối ưu sau khi chấp nhận vòng	Đếm riêng bốn tầng: ứng viên, ghép đặc trưng, kiểm chứng, tối ưu. Đo precision <i>sau</i> kiểm chứng chứ không trước. Siết ngưỡng khoảng cách của nhãn tự động xuống rồi đo lại Recall@1; nếu nó sập thì nhãn đang quá lỏng.
APE đẹp mà chạy thật thấy trôi	Phép căn chỉnh đã hấp thụ trôi toàn cục; hoặc chuỗi không hoàn thành và chỉ đoạn dễ được chấm	Đọc RPE trên nhiều độ dài đoạn. Kiểm tra tỉ lệ hoàn thành chuỗi trước mọi việc khác — đây là kiểm tra rẻ nhất và hay bị bỏ qua nhất.
APE trung vị tốt mà RMSE tệ	Có một hoặc vài đợt bùng ngắn; hoặc vài khung hình bị lệch mốc thời gian	Vẽ sai số theo thời gian. Nếu là vài đỉnh nhọn thì đọc log ở đúng các mốc đó. Nếu là bậc thang thì nghi mốc thời gian hoặc một lần khởi tạo lại.
Tỉ lệ inlier tăng mà APE không đổi	Bảo hoà ràng buộc; hoặc cặp khớp mới không đa dạng về hình học; hoặc nút thất nằm ở hiệu chỉnh, IMU hay độ trễ	Giảm nửa số cặp khớp và đo lại APE. Thay tầng khớp bằng cặp khớp sinh từ chân trị để lấy trần lý thuyết. Nếu trần đó cũng không tốt hơn hiện tại thì tầng khớp không phải nút thắt.
Macro-F1 thấp hơn micro-F1 rất nhiều	Các lớp hiếm đang bị bỏ rơi — đây gần như luôn là nguyên nhân	Xếp F1 theo lớp từ thấp lên và nhìn năm lớp đáy cùng số mẫu của chúng. Nếu chúng đều là lớp ít mẫu thì đã rõ; nếu có lớp nhiều mẫu nằm trong đáy thì đó là chuyện định nghĩa nhãn.
Metric nhảy mạnh giữa các lần chạy cùng cấu hình	Phương sai do huấn luyện (hạt giống, thứ tự dữ liệu, kernel không tắt định); hoặc tập đánh giá quá nhỏ; hoặc metric nhạy ngưỡng	Chạy lặp năm lần và đo độ lệch chuẩn. So nó với sai số chuẩn tính từ cỡ tập test. Nếu độ lệch giữa các lần chạy lớn hơn hẳn thì nguồn là huấn luyện, không phải cỡ mẫu.

Bạn thấy	Các cách giải thích	Phép thử để biết là cách nào (tiếp)
Mô hình mới thắng ở mAP nhưng thua ở APs	Cải thiện chỉ có ở vật lớn; hoặc thay đổi độ phân giải đầu vào; hoặc gán mẫu đối làm vật nhỏ mất mẫu dương	Báo tách APs, APm, API. Nếu chỉ APs giảm thì kiểm tra độ phân giải và quy tắc gán mẫu trước khi kiểm tra bất cứ thứ gì khác.
Top-5 cao mà top-1 thấp	Các lớp cực giống nhau; hoặc nhãn chân trị mơ hồ với ảnh nhiễu vật	Đọc ma trận nhầm lẫn và tìm các cặp ô đối xứng lớn. Bốc tay 30 ảnh sai và tự gán nhãn lại: nếu bạn cũng phân vân thì đó là trần đồng thuận, không phải lỗi mô hình.
Precision cao ở mọi ngưỡng nhưng recall trần thấp	Mô hình mù với một dạng con của lớp dương; hoặc tăng trước đã lọc mất dạng đó	Đọc recall theo lát. Tìm lát nào có recall gần 0. Rồi kiểm tra dạng đó có xuất hiện trong tập huấn luyện với đủ số lượng không.
ECE tốt mà mô hình vẫn ra quyết định tệ	Hiệu chỉnh tốt trên toàn cục nhưng lệch theo từng lát; hoặc vấn đề là độ phân giải chứ không phải độ tin cậy	Tính ECE riêng theo từng lát. Phân rã điểm Brier: nếu phần reliability nhỏ mà resolution cũng nhỏ thì mô hình không tách được mẫu, và hiệu chỉnh không cứu được.
Hai mô hình cùng AUPRC nhưng người dùng thích hẳn một cái	Hai đường cong cùng diện tích, khác hình dạng; hoặc chúng sai trên hai nhóm mẫu khác nhau	So hai đường cong tại đúng điểm vận hành đang dùng, không so diện tích. Đếm số mẫu hai mô hình bất đồng và xem chúng thuộc lát nào.

5.13 Mọi con số phải kèm một khoảng (Every Number Needs an Interval)

Cả khối trên đọc từng con số như thể nó chính xác. Nó không. Mỗi metric trong chương là một ước lượng tính từ một mẫu hữu hạn, nên nó có một khoảng dao động, và khoảng đó thường lớn hơn mức người ta tưởng.

Phần công cụ chung — sai số chuẩn, so sánh bất cặp, phương sai do huấn luyện — nằm ở mục sau của chính chương này, và kỷ luật thí nghiệm đầy đủ nằm ở E/24-thi-nghiem-va-ablation. Ở đây chỉ nói ba điều riêng của việc *đọc* metric.

- **Mẫu số nhỏ thì khoảng rất rộng, và mẫu số của metric theo lớp luôn nhỏ.** Recall của một lớp có 10 mẫu dương chỉ nhận được 11 giá trị rời rạc, cách nhau 0,1. Nói “recall lớp C tăng từ 0,1 lên 0,2” nghĩa là *một mẫu* đổi kết quả. Đó không phải một cải tiến, đó là một mẫu.
- **Metric không phải trung bình thì không có công thức sai số chuẩn.** mAP, HOTA, PQ, AUPRC đều là hàm phi tuyến của toàn bộ tập, nên công thức $\sqrt{p(1-p)/n}$ không áp dụng được. Cách đúng là *bootstrap* [60]: lấy mẫu có hoàn lại từ chính tập test, tính lại metric, lặp vài nghìn lần, rồi đọc phân vị 2,5 và 97,5. Nó tốn vài phút CPU và trả về đúng thứ bạn cần.
- **Bootstrap phải lấy mẫu theo đúng đơn vị phân tích.** Nếu tập test gồm nhiều khung hình từ ít video, thì lấy mẫu theo khung hình sẽ cho khoảng hẹp giả tạo, vì các khung trong cùng video không độc lập. Phải lấy mẫu theo *video*. Đây là cùng một nguyên tắc đã chi phối cách chia dữ liệu ở đầu chương, xuất hiện lại ở tầng đo độ bất định.

Quy tắc chốt lại: *đo sần nhiều trước, nói về cải tiến sau*. Không có sần nhiều thì mọi so sánh trong bảng chẩn đoán ở trên đều có thể đang chẩn đoán một dao động ngẫu nhiên.

5.14 Con số tổng che nhóm bị bỏ rơi (Slice-based and Behavioral Evaluation)

Ngay cả metric đúng, tính trên tập chia đúng, vẫn là một trung bình — và trung bình che các nhóm nhỏ một cách hoàn hảo. Hai công cụ bù lại chỗ đó, và cả hai đều rẻ:

- **slice-based evaluation** — cắt tập test theo các lát có nghĩa vận hành rồi đọc metric trên từng lát.
 - **Lát nào** — ban đêm, vật nhỏ, mức che khuất cao, từng camera, từng địa điểm, từng đợt thu.
 - **Điều kiện tiên quyết** — phải có metadata cho từng mẫu, nên hãy thu thập nó *từ lúc gán nhãn* (B/06); không có metadata thì không có lát nào cắt được.
 - **Giá trị** — theo kinh nghiệm thực hành, đây là bước có tỉ lệ “phát hiện vấn đề trên mỗi giờ công” cao nhất trong toàn bộ quy trình đánh giá. Khi nhóm tệ nhất mới là thứ quan trọng, có cả một dòng phương pháp tối ưu trực tiếp cho nó [61].
- **kiểm thử hành vi (behavioral testing)** — kiểm tra các tính chất mà bạn *biết* hệ thống phải có, theo kiểu unit test.
 - **Bất biến** — xoay ảnh vài độ thì nhãn không được đổi.
 - **Có hướng** — che vật thể đi thì độ tin cậy phải giảm, không được tăng.
 - **Vì sao cần** — loại test này không cho con số so với paper, nhưng nó bắt đúng loại lỗi mà tập test không bắt được, vì tập test chia sẻ mọi thiên lệch với tập train.

5.15 Chênh lệch 0,3% có thật không? (Uncertainty in Reported Numbers)

Đây là chỗ người đọc có lợi thế sẵn: bạn đã biết chạy cùng một cấu hình ba lần cho ba con số khác nhau, và phần lớn “cải tiến” được báo cáo trong một chiều tối nằm gọn trong phổ đó. Cần biến trực giác này thành phép tính, và có hai nguồn phương sai hoàn toàn khác nhau.

→ **tiền đề** E/24 — kỷ luật chạy lặp và báo cáo độ lệch ở đây chính là điều kiện tối thiểu để một bảng ablation có nghĩa.

- **Phương sai do tập test hữu hạn.** Với 1000 ảnh và accuracy 92%, sai số chuẩn là $\sqrt{0,92 \cdot 0,08/1000} \approx 0,0086$, nên **khoảng tin cậy (confidence interval)** 95% rộng khoảng $\pm 1,7$ điểm phần trăm. Công thức này nói: *nếu lấy một tập test khác cùng cỡ từ cùng phân phối, con số sẽ nhảy trong khoảng đó*. Một chênh lệch 0,3 điểm nằm sâu bên trong và không nói lên điều gì.
 - **Nhưng phải trung thực về một điểm tinh tế** — khi hai mô hình đo trên cùng một tập test, phép so sánh là bất cập, và chỉ những mẫu chúng bất đồng mới mang thông tin. Bất đồng ở 40 mẫu, chia 25 nghiêng về A và 15 về B: chênh lệch quan sát được là 1,0 điểm với sai số chuẩn cỡ $\sqrt{40}/1000 \approx 0,63$ điểm — chặt hơn khoảng tin cậy rời rạc ở trên, nhưng vẫn chưa đủ để tuyên bố A thắng.
 - **Bài học kép** — khoảng tin cậy rời rạc là ước lượng bảo thủ cho so sánh bất cập; và ngay cả phép so sánh chặt hơn cũng thường không cứu được một chênh lệch một điểm.
- **Phương sai do quá trình huấn luyện.** Hạt giống ngẫu nhiên, thứ tự dữ liệu, tính không tắt định của kernel GPU — nguồn này không giảm đi khi bạn thu thập thêm ảnh.
 - **Cách xử lý đúng** — chạy lặp, báo cáo trung bình kèm độ lệch, không chọn lần chạy tốt nhất; đúng kỷ luật bạn đã áp dụng cho benchmark harness của mình. Với metric phức tạp như mAP hay HOTA (không có công thức sai số chuẩn giải tích), dùng **bootstrap**: lấy mẫu có hoàn lại từ chính tập test vài nghìn lần rồi đọc phân vị.
 - **Bối cảnh** — việc báo cáo phương sai theo hạt giống từng bị chỉ ra là điểm yếu có hệ thống của cả lĩnh vực [9]. Trong một số nhánh, phần lớn “tiền bộ” biến mất khi **baseline** được tune ngang bằng [2], [3].

5.16 Giới hạn và trường hợp hỏng

Giao thức chặt tới đâu cũng chỉ đo được cái mà tập test đại diện. Nếu tập test lấy từ cùng một khung lấy mẫu với tập train, mọi thiên lệch chung giữa hai bên vẫn vô hình — chia đúng chỉ chặn rõ ràng, không chặn thiên lệch chung. Đây là ranh giới giữa chương này và chương sau: nội dung ở đây giả định phân phối không đổi, còn khi phân phối đổi thì mọi ước lượng ở đây đều lạc quan.

Nguyên tắc “chỉ chạm test một lần” cũng không sống sót nguyên vẹn: một dự án hai năm sẽ chạm test nhiều lần, không tránh được. Cách xử lý trung thực là ghi lại số lần chạm và thay tập test khi nó đã hao mòn — nghĩa là phải giữ sẵn ngân sách thu thập một tập test mới. Cuối cùng, chia theo nhóm và theo thời gian đều làm tập test nhỏ đi và phương sai lớn lên: một cái giá thật, và với dữ liệu ít bạn đôi khi buộc phải chọn giữa ước lượng thiên lệch có phương sai nhỏ và ước lượng không thiên lệch có phương sai lớn.

CẠM BẤY

Cạm bẫy tốn kém nhất không phải chia sai mà là *chọn tập test sau khi đã nhìn kết quả*. Nó xuất hiện dưới những dạng trông vô hại: bỏ vài sequence “bị lỗi thu”, thêm một tập test thứ hai vì tập đầu “không đại diện”, hay chọn checkpoint theo test vì val nhỏ quá. Cả ba đều là cùng một hành vi, và cả ba đều chuyển test thành val mà không ai ghi vào nhật ký.

CẦN NHỚ

Cách chia là một phát biểu về cách mô hình sẽ được dùng, và metric là một phép chiếu làm mất thông tin. Nên hai câu hỏi phải hỏi trước mọi bảng kết quả là: *cách chia này giả định gì*, và *metric này đang giấu loại lỗi nào*. Với một tập test 1000 mẫu, chênh lệch dưới khoảng một điểm phần trăm chưa phải là chênh lệch. [T2]

5.17 Thực hành

Repo và tài nguyên. [cocodataset/cocoapi](#) là định nghĩa tham chiếu của mAP — khi hai bài báo báo cáo mAP khác nhau, thủ phạm thường là cài đặt chứ không phải mô hình; [Lightning-AI/torchmetrics](#); [voxel51/fiftyone](#) (đánh giá theo lát); [dbolya/tide](#) (phân rã lỗi detection); [MichaelGrupp/evo](#) (ATE/RPE); [photosynthesis-team/piq](#) và [richzhang/PerceptualSimilarity](#) (LPIPS); [GaParmar/clean-fid](#). Danh sách chốt tại tháng 9/2026 và sẽ cũ trong 6–12 tháng; riêng nhóm hạ tầng metric (cocoapi, torchmetrics, evo) bền hơn hẳn tên các mô hình dẫn đầu bảng.

Câu hỏi kiểm tra hiểu. mAP@0.5 và mAP@0.5:0.95 khác nhau ở chỗ nào, và loại cải tiến nào chỉ hiện lên ở cái thứ hai? Vì sao AUROC gây hiểu lầm khi lớp dương chiếm 1%, trong khi AUPRC thì không? Bước căn chỉnh trước khi tính ATE hấp thụ mất loại lỗi nào, và metric nào bù lại chỗ đó?

Bài tập phụ. Tự cài mAP từ đầu — matching theo IoU, xây đường cong precision–recall, nội suy — rồi so với [pycocotools](#) trên cùng một file dự đoán. Chênh lệch sẽ xuất hiện, và việc truy ra nó đến từ quy tắc nội suy, từ cách xử lý mẫu iscrowd, hay từ giới hạn số phát hiện mỗi ảnh, dạy nhiều hơn bất kỳ bài đọc nào về metric.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Rò rỉ dữ liệu	Bất kỳ đường nào làm thông tin của tập test lọt vào huấn luyện hoặc chọn mô hình. Đặc điểm nguy hiểm là nó không báo lỗi, chỉ làm con số đẹp lên.
●●● Vì sao không được chọn checkpoint theo tập test?	Vì như vậy test đã đóng vai trò của val. Train ước lượng tham số, val chọn siêu tham số và quyết định lúc dừng, test chỉ được đo đúng một lần ở cuối.
●●● Vì sao chuẩn hoá trước khi chia đã là rò rỉ?	Vì mean và std tính trên cả tập test đã đi vào mô hình. Rò rỉ không cần tới nhãn — chỉ cần một thông tin nào đó của test tham gia vào huấn luyện.
●●● Chia theo nhóm	Mọi mẫu chung một nguồn — cùng bệnh nhân, cùng video, cùng camera — nằm trọn một bên. Hai khung hình cách nhau 33 ms không phải hai quan sát độc lập, nên bỏ qua nó là cách rẻ nhất để tự tặng mình hàng chục điểm hiệu năng ảo.
●● Chia theo thời gian	Test nằm sau train trên trục thời gian. Cần khi dữ liệu có trôi; chia ngẫu nhiên ở đây cho mô hình đặc quyền nội suy giữa quá khứ và tương lai, thứ nó không có khi chạy thật.
●●● Chia lại tập năm lần đều cho cùng con số, nhưng triển khai thì sập — nguyên nhân?	Chia đúng chỉ chặn rò rỉ, nó không chặn thiên lệch chung. Nếu train và test cùng lấy từ một khung lấy mẫu thì mọi cách chia đều cho cùng một con số lạc quan. Phân biệt bằng vài trăm mẫu thu ở điều kiện triển khai thật rồi đo lại.
●●● Adaptive overfitting	Test hao mòn dần vì mỗi lần bạn nhìn nó rồi ra quyết định là một lần thông tin rò qua bộ não bạn. Sau vài chục vòng, test đã trở thành val.
●● Thứ hạng leaderboard không tái lập trên tập test mới — nguyên nhân?	Adaptive overfitting ở quy mô cộng đồng: hàng nghìn cấu hình được thử trên một tập test cố định. Phân biệt với tiến bộ thật bằng cách thu một tập test mới theo đúng quy trình cũ rồi đo lại.
●● Prevalence	Tỉ lệ mẫu thuộc lớp dương trong dữ liệu. Khi nó nhỏ, accuracy, precision, recall và FPR rời xa nhau tới mức độc nhằm một con số là kết luận sai hoàn toàn.
●●● AUPRC	Diện tích dưới đường cong precision–recall. Phải dùng thay AUROC khi lớp dương hiếm, vì nó chuẩn hoá theo số lần báo động chứ không theo số mẫu âm.
●●● Vì sao AUROC gây hiểu lầm khi lớp dương chiếm 1%?	Vì FPR chia cho 990 mẫu âm, nên 9% báo động giả trông nhỏ mà thực ra là 91 lần báo nhầm. Precision chia cho tổng 100 lần báo động và cho ra 9%, đúng cái người dùng cảm nhận.
●● Accuracy 99% trên tập có 1% mẫu dương — đọc thêm gì?	Precision và recall ở đúng điểm làm việc. Một mô hình luôn trả lời “âm” cũng đạt 99%, nên chỉ hai con số đó mới tách được mô hình có ích khỏi mô hình rỗng.
●●● Bốn ô TP, FP, FN, TN	TP báo có và đúng; FP báo có nhưng sai, tức báo động giả ai cũng thấy; FN báo không nhưng thật ra có, tức ca bỏ sót không ai thấy; TN báo không và đúng. Chữ thứ hai là cái hệ thống nói, chữ thứ nhất là nó có nói đúng không.
●●● Mẫu số của precision, recall và FPR	Precision chia cho mọi lần báo “có”. Recall chia cho mọi ca thật sự dương. FPR chia cho mọi ca thật sự âm. Ba mẫu số khác nhau là lý do cùng một ô FP cho ba con số nghe khác hẳn nhau.
●●● Precision cao, recall thấp — hệ thống đang làm gì?	Nó nhất: chỉ báo khi rất chắc nên nói gì cũng đúng, nhưng bỏ sót phần lớn ca thật. Nguyên nhân cạnh tranh: ngưỡng quá chặt, chỉ bắt được dạng dễ, hoặc thiếu bao phủ. Quét toàn dải ngưỡng để tách nguyên nhân đầu khỏi hai cái sau.
●●● Precision thấp, recall cao — hệ thống đang làm gì?	Nó báo bừa: bắt được gần hết nhưng phần lớn cảnh báo là rác. Nguyên nhân: ngưỡng quá lỏng, tỉ lệ nền quá thấp, hoặc nhãn âm bị thiếu nên cái bắt đúng lại bị tính là FP. Bóc tay 50 mẫu FP và gắn nhãn lại để tách nguyên nhân cuối.
●● Vì sao F1 dùng trung bình điều hoà?	Vì trung bình điều hoà bị con số nhỏ kéo xuống, còn trung bình cộng thì không. Với $P = 0,95$ và $R = 0,40$, trung bình cộng cho 0,675 còn F1 cho 0,563 — con số sau phản ánh đúng hơn việc bỏ sót 60% ca thật.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Macro-F1 thấp hơn micro-F1 rất nhiều — nghĩa là gì?	Các lớp hiếm đang bị bỏ rơi. Micro cho mỗi mẫu một phiếu nên lớp đông quyết định; macro cho mỗi lớp một phiếu. Trong ví dụ ba lớp của chương, micro là 0,946 còn macro chỉ 0,619.
●●● Vì sao ROC trông đẹp giả tạo khi lớp dương hiếm?	Vì FPR có mẫu số rất lớn. FPR 0,053 nghe nhỏ, nhưng nhân với 19 800 mẫu âm ra 1043 báo động giả trong khi chỉ có 134 báo đúng. Đường PR chia cho số lần báo nên nó nhìn thấy điều đó.
●● Biểu đồ độ tin cậy	Chia dự đoán theo độ tin cậy thành nhóm, rồi vẽ độ tin cậy trung bình mô hình nói ra so với tỉ lệ đúng thật. Nằm dưới đường chéo là quá tự tin. Một mô hình quá tự tin có AUC y hết mô hình hiệu chỉnh đúng, vì thứ hạng không đổi.
●● Đọc ma trận nhầm lẫn nhiều lớp theo hàng và theo cột	Theo hàng: mẫu thật sự thuộc lớp này đã đi đâu, chia ra recall. Theo cột: mẫu bị gọi là lớp này thật ra là gì, chia ra precision. Cặp ô đối xứng cùng lớn nghĩa là hai lớp thật sự khó phân biệt.
●● mAP@0.5:0.95	mAP lấy trung bình trên nhiều ngưỡng IoU thay vì chỉ 0,5. Nó là chỗ duy nhất mà cải tiến về độ chính xác định vị hiện ra, vì ở ngưỡng 0,5 hộp lệch nhẹ vẫn được tính là đúng.
● HOTA	Metric theo dõi đối tượng cân bằng giữa phát hiện và giữ đúng danh tính. Cần nó vì MOTA cao vẫn hoàn toàn tương thích với việc ID bị hoán loạn liên tục.
●●● ATE	Sai số quỹ đạo tuyệt đối, tính sau khi căn quỹ đạo ước lượng vào quỹ đạo thật. Nó đo độ lệch toàn cục, nên phải đọc kèm RPE để thấy tính nhất quán trên các đoạn ngắn.
●●● Phép căn chỉnh trước khi tính ATE hấp thụ mất gì?	Trôi toàn cục. Một quỹ đạo trôi tỉ lệ đều đặn bị kéo về sát tham chiếu và cho ra ATE dễ chịu, trong khi lỗi cục bộ vẫn còn nguyên và chỉ RPE mới thấy.
●● Calibration	Mức khớp giữa độ tin cậy mô hình nói ra và tần suất nó đúng thật. Mạng hiện đại thường quá tự tin, nên mọi ngưỡng quyết định đặt sau đầu ra đều lệch theo.
●● Slice-based evaluation	Cắt tập test theo nhóm có nghĩa vận hành — ban đêm, vật nhỏ, từng camera — rồi đọc metric trên từng lát. Điều kiện là có metadata cho từng mẫu, nên phải thu thập từ lúc gán nhãn.
● Kiểm thử hành vi	Kiểm tra các tính chất bạn biết chắc phải đúng, theo kiểu unit test: xoay ảnh vài độ thì nhãn không đổi, che vật thì độ tin cậy phải giảm. Nó bắt loại lỗi mà tập test không bắt được.
●●● Vì sao đóng vòng phải lệch hẳn về precision?	Vì một vòng sai khâu hai nơi khác nhau thành một và làm cong bản đồ, thường không hồi phục; còn một vòng bỏ lỡ chỉ là một cơ hội xoá sai số chưa dùng tới. Recall thấp ở đây là lựa chọn có chủ ý.
●●● Vì sao RMSE của quỹ đạo phải đi kèm trung vị và phân vị?	Vì bình phương làm cái đuôi chi phối. Trong ví dụ của chương, trung vị 2,1 cm còn RMSE 11,9 cm, và toàn bộ chênh lệch đo 4,5% số khung tạo ra.
●● Bootstrap cho metric không phải trung bình	Lấy mẫu có hoàn lại từ chính tập test, tính lại mAP hoặc AUPRC, lặp vài nghìn lần, đọc phân vị. Phải lấy mẫu theo đúng đơn vị phân tích — theo video chứ không theo khung hình — nếu không khoảng sẽ hẹp giả tạo.
●●● Hai mô hình chênh 0,3 điểm trên 1000 ảnh — kết luận?	Chưa kết luận được gì. Sai số chuẩn của accuracy 92% trên 1000 mẫu khoảng 0,86 điểm, nên khoảng tin cậy 95% rộng chừng $\pm 1,7$ điểm. So sánh bất cập chặt hơn nhưng thường vẫn không cứu được một điểm.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
cocodataset/cocoapi	Là định nghĩa vận hành của mAP: quy tắc ghép theo IoU, nội suy đường PR, giới hạn số phát hiện mỗi ảnh	Hạ tầng; hai bài báo lệch mAP thường lệch vì cài đặt chứ không vì mô hình
WILDS (p-lambda/wilds)	Chia theo nhóm và theo miền thật — bệnh viện khác, năm khác, vùng khác — thay vì chia ngẫu nhiên	Bằng chứng rằng “chia theo nhóm” đã thành chuẩn thiết kế benchmark
TIDE (dbolya/tide)	Phân rã một con số mAP thành các loại lỗi tách biệt, đúng ý “metric là phép chiếu”	Cùng dòng với công trình phân tích lỗi detection sớm hơn [54]
ImageNet-v2	Thu một tập test mới theo đúng quy trình gốc để đo phần hiệu năng đã mất do tập test cũ hao mòn	Cách duy nhất đã biết để định lượng adaptive overfitting [26]
MichaelGrupp/evo	Chuẩn hoá việc báo cáo ATE cùng RPE và ghi rõ kiểu căn chỉnh đã dùng	Hạ tầng đánh giá SLAM; chính chỗ bẫy căn chỉnh trở nên hữu hình

MINI PROJECT SLAM · Định lượng rõ ràng do chia ngẫu nhiên theo khung hình · 1–2 ngày

Mục tiêu	biến câu “chia ngẫu nhiên trên video là rõ ràng” thành một con số điểm phần trăm đo được trên chính dữ liệu bạn dùng hằng ngày.
Dữ liệu	ba tới bốn chuỗi EuRoC. Nhãn “cùng chỗ / khác chỗ” lấy từ mini project chương B/06-chat-luong-du-lieu , mở rộng tự động bằng chân trị quỹ đạo: hai khung hình cách nhau dưới 1 m và lệch góc dưới 30° thì gán là cùng chỗ.
Các bước	dựng một bộ phân loại cặp thật đơn giản — một ngưỡng trên điểm tương đồng DBoW2, hoặc hồi quy logistic trên khoảng cách cosine giữa hai vectơ mô tả toàn ảnh. Huấn luyện và đo cùng bộ đó ba lần với ba cách chia: ngẫu nhiên theo khung hình, theo đoạn thời gian liền mạch trong cùng chuỗi, và theo chuỗi (giữ trọn một chuỗi ra ngoài). Mỗi cách chạy năm lần với năm hạt giống, ghi cùng một metric (AUPRC, hoặc precision tại recall 0,9).
Xong khi	bạn viết được một câu dạng “chia ngẫu nhiên theo khung hình cho AUPRC cao hơn chia theo chuỗi X điểm, khoảng tin cậy $\pm Y$ ”, trong đó Y tính từ năm lần chạy chứ không đoán.
Nối với	B/06-chat-luong-du-lieu là nguồn nhãn; B/08-dich-chuyen-phan-phoi vì chia theo chuỗi chính là mô phỏng dịch chuyển miền ở quy mô nhỏ; E/24-thi-nghiem-va-ablation cho cách đọc khoảng tin cậy đó.

MINI PROJECT SLAM · Đường PR của tầng lấy ứng viên đóng vòng, và điểm vận hành theo chi phí · 1–2 ngày

Mục tiêu	biến câu “một vòng sai đắt hơn một vòng bỏ lỡ” thành một ngưỡng cụ thể, chọn bằng phép tính chứ bằng cảm giác.
Dữ liệu	ba tới bốn chuỗi EuRoC. Nhãn “cùng chỗ” sinh từ chân trị quỹ đạo: hai khung hình cách nhau dưới 1 m và lệch góc dưới 30° . Ghi lại luôn kết quả với ngưỡng 2 m để đo xem con số nhạy thế nào với chính định nghĩa nhãn.
Các bước	lấy điểm tương đồng của tầng lấy ứng viên mà hệ bạn đang dùng (DBoW2 hoặc tương đương) cho mọi cặp khung hình đủ xa nhau về thời gian. Quét ngưỡng trên điểm đó, ghi TP, FP, FN, TN ở từng ngưỡng, vẽ cả đường PR lẫn đường ROC trên cùng một trang. Sau đó đặt $c_{FN} = 1$ và thử ba giá trị $c_{FP} \in \{10, 100, 1000\}$, rồi tìm ngưỡng cực tiểu hoá $c_{FP} \cdot FP + c_{FN} \cdot FN$ cho từng giá trị. Cuối cùng lập toàn bộ với nhãn ngưỡng 2 m.
Xong khi	bạn có một hình hai bảng cạnh nhau cho thấy ROC gần như không đổi trong khi PR đổi rõ; một bảng ba dòng “chi phí c_{FP} — ngưỡng tối ưu — precision và recall tại đó”; và một câu nói rõ ngưỡng tối ưu dịch bao nhiêu khi định nghĩa nhãn đổi từ 1 m sang 2 m.

Nối với [B/06-chat-luong-du-lieu](#) cho chất lượng nhãn tự động; mini project trước của chính chương này cho cách chia dữ liệu khi đo; E/25-do-tin-cay-va-van-hanh cho việc biến điểm tương đồng thành một xác suất đáng tin trước khi đặt ngưỡng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì với tỉ lệ dương 1%, một mô hình luôn trả lời “âm” đã đạt 99% . Hai con số cần thêm là precision và recall ở đúng điểm làm việc — hoặc tương đương, AUPRC thay cho AUROC.
- A2. Vì thống kê của tập test đã đi vào mô hình qua bộ chuẩn hoá. Rõ ràng không cần nhãn: chỉ cần một thông tin nào đó của test tham gia vào quá trình huấn luyện hay chọn mô hình là con số đã lạc quan.
- A3. Vì adaptive overfitting: cả cộng đồng thử hàng nghìn cấu hình trên một tập test cố định, nên phần thắng cuối cùng có một phần là khớp riêng tập đó. Thu một tập test mới cùng quy trình làm phần đó bốc hơi.
- A4. Hấp thụ trôi toàn cục: phép biến đổi cứng (hoặc có tỉ lệ, với đơn mắt) kéo quỹ đạo trôi đều đặn về sát tham chiếu. RPE bù lại chỗ đó vì nó đo tính nhất quán trên các đoạn ngắn và mù với sai lệch toàn cục.
- A5. Với 1000 ảnh, khoảng tin cậy 95% của một con số 92% rộng khoảng $\pm 1,7$ điểm. So sánh bất cặp chặt hơn: nếu hai mô hình bất đồng ở 40 mẫu chia 25/15 thì sai số chuẩn của chênh lệch cỡ 0,63 điểm — vẫn chưa đủ để tuyên bố thắng.
- A6. Nó đang *nhất*: chỉ mở miệng khi rất chắc, nên nói gì cũng đúng, nhưng im lặng ở 60% ca thật. Ba nghi phạm là ngưỡng quá chặt, mô hình chỉ bắt được dạng dễ, và tập huấn luyện thiếu bao phủ dạng hiếm. Quét toàn dải ngưỡng để tách nghi phạm thứ nhất khỏi hai cái còn lại.
- A7. Vì F1 gộp hai số bằng một phép trung bình đối xứng nên nó che mất bất đối xứng chi phí. Hệ $P = 0,90$, $R = 0,45$ và hệ $P = R = 0,60$ cùng cho $F_1 = 0,600$; với đóng vòng, nơi một vòng sai đắt hơn nhiều một vòng bỏ lỡ, hệ đầu tốt hơn hẳn. $F_{0,5}$ nói ra điều đó, F1 thì không.
- A8. Vì AUC chỉ dùng thứ hạng của điểm số, mà thứ hạng không đổi khi bạn ép điểm số qua một hàm tăng bất kỳ. Biểu đồ độ tin cậy phát hiện chỗ lệch: điểm nằm dưới đường chéo nghĩa là mô hình nói mạnh hơn thực tế. ECE và phân rã điểm Brier định lượng phần lệch đó.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ về một kỳ thi. Bạn có ba tập giấy: sách bài tập để học, một đề thi thử để biết mình yếu chỗ nào, và một đề thi thật nghiêm phong để chấm điểm cuối cùng. Điểm của đề thật chỉ có nghĩa khi thí sinh chưa từng nhìn thấy nó.

Bài toán gốc là mọi cách để đề bị lộ mà không ai cố ý làm lộ. Bạn photo đề thật lẫn vào sách bài tập rồi phát nhầm một trang. Bạn ra đề thử bằng đúng các câu của đề thật, chỉ đổi số. Bạn cho lớp này thi trước rồi mới thi lớp sau. Hoặc tình vi nhất: bạn cho thí sinh làm đề thật ba mươi lần, mỗi lần xem điểm rồi chỉnh cách ôn. Sau ba mươi lần, điểm cao không còn nói lên năng lực nữa, nó chỉ nói rằng họ đã quen với đúng tờ đề đó.

Mẹo có ba phần. Một, quyết định trước ai vào tập nào theo đúng cách bạn sẽ gặp người mới: nếu ngoài đời bạn sẽ gặp học sinh trường khác thì đề thật phải là học sinh trường khác. Hai, đừng đọc mỗi con số tổng — xem điểm theo từng nhóm, và luôn hỏi chênh lệch nửa điểm giữa hai thí sinh có lớn hơn sai số của chính kỳ thi hay không.

Ba, và đây là phần dễ quên nhất: mỗi lần thấy một tỉ lệ, hãy hỏi nó chia cho cái gì. “Chín mươi phần trăm” là chín mươi phần trăm của *cái gì*. Một người gác cổng chặn nhầm 5% số khách vào toà nhà nghe như rất giỏi; nhưng nếu mỗi ngày có hai vạn khách và chỉ hai trăm kẻ gian, thì 5% ấy là một nghìn người bị chặn oan, so với hơn một trăm kẻ gian bắt được. Cùng một sai lầm, chia cho hai thứ khác nhau, cho ra hai câu chuyện trái ngược. Con số nào cũng chỉ có nghĩa cùng với mẫu số của nó.

Cái giá là đề thi thật trở thành thứ tiêu hao: dùng ít lần thôi, và phải để dành ngân sách ra một đề mới.

Chỗ không rút gọn được: việc “chênh lệch này có thật không” thì không rút gọn được thành một quy tắc. Đo bằng khoảng dao động của từng con số thì quá dè dặt, mà chỉ đếm những chỗ hai bên trả lời khác nhau thì lại quá dễ dãi khi hai bên vốn rất giống nhau.

Một câu: Mọi kỹ thuật trong chương chỉ bảo vệ một thứ duy nhất: một con số đo trên phần dữ liệu mà bạn chưa từng chạm vào, kể cả bằng ánh mắt.

5.18 Kết nối

Chương này là hệ quả trực tiếp của [B/05-thiet-ke-bai-toan](#): đơn vị phân tích chọn ở đó quyết định cách chia hợp lệ ở đây. Nó dựa trên [B/06-chat-luong-du-lieu](#) ở hai chỗ — trùng lặp gần đúng là một đường rò rỉ, và trần đồng thuận quyết định chênh lệch nào đáng bàn. Nó là tiền đề của [B/08-dich-chuyen-phan-phoi](#), chương gỡ bỏ đúng giả định mà toàn bộ chương này đứng lên: rằng test và thực tế cùng một phân phối. Phần calibration nối sang [E/25-do-tin-cay-va-van-hanh](#), còn kỷ luật thí nghiệm và ablation được khai triển đầy đủ ở [E/24-thi-nghiem-va-ablation](#).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao chuẩn hoá dữ liệu trước khi chia là rò rỉ, dù bạn không hề dùng nhãn của test?
2. Trong [Hình 5.1](#), vì sao câu hỏi về nhóm đứng trước câu hỏi về mất cân bằng lớp?
3. Một mô hình đạt accuracy 99% trên tập có 1% mẫu dương. Cần thêm đúng hai con số nào để biết nó có ích hay không?
4. Vì sao ATE và RPE phải đọc cùng nhau, và phép căn chỉnh làm mất thông tin gì?
5. Adaptive overfitting không tạo ra bug, không tạo ra cảnh báo. Vậy dấu hiệu sớm nào cho thấy tập test của bạn đã hao mòn?
6. Hai mô hình chênh nhau 1 điểm phần trăm trên 1000 ảnh. Phép tính nào cho biết chênh lệch đó có ý nghĩa không, và vì sao khoảng tin cậy rời rạc lại là ước lượng bảo thủ ở đây?
7. Chạy lặp với nhiều hạt giống làm giảm nguồn phương sai nào, và *không* làm giảm nguồn nào?
8. Precision 0,95 và recall 0,40: hệ thống đang hành xử thế nào, và phép thử nào tách “ngưỡng quá chặt” khỏi “mô hình mù với dạng hiếm”?
9. Vì sao AUROC gần như không nhúc nhích khi số báo động giả tăng gấp mười lần trên dữ liệu có 1% mẫu dương, và metric nào thì có?

Nguồn

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *Rec-Sys*, 2019.
- [3] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [4] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [5] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [6] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [7] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [8] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [9] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [10] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [11] V. N. Vapnik, “An overview of statistical learning theory,” *IEEE Trans. Neural Networks*, vol. 10, no. 5, pp. 988–999, 1999.
- [12] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [13] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [15] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [16] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [17] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [18] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE Trans. Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.
- [19] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.

- [20] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [21] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [22] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [23] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [24] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [25] R. Geirhos, J.-H. Jacobsen, C. Michaelis, et al., “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.
- [26] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [27] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [28] T. Gebru, J. Morgenstern, et al., “Datasheets for Datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [29] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [30] M. Mitchell, S. Wu, et al., “Model Cards for Model Reporting,” in *FAT**, 2019.
- [31] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [32] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [33] C. G. Northcutt, L. Jiang, and I. L. Chuang, “Confident Learning: Estimating Uncertainty in Dataset Labels,” *JAIR*, vol. 70, pp. 1373–1411, 2021.
- [34] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [35] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [36] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [37] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [38] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [39] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [40] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [41] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London: Butterworths, 1979.
- [42] B. W. Matthews, “Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme,” *Biochimica et Biophysica Acta (BBA) — Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
- [43] D. Chicco and G. Jurman, “The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation,” *BMC Genomics*, vol. 21, p. 6, 2020.

- [44] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in *International Conference on Machine Learning (ICML)*, 2006, pp. 233–240.
- [45] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.
- [46] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [47] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [48] M. P. Naeni, G. F. Cooper, and M. Hauskrecht, "Obtaining Well Calibrated Probabilities Using Bayesian Binning," in *AAAI Conference on Artificial Intelligence*, 2015.
- [49] G. W. Brier, "Verification of Forecasts Expressed in Terms of Probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
- [50] A. H. Murphy, "A New Vector Partition of the Probability Score," *Journal of Applied Meteorology*, vol. 12, no. 4, pp. 595–600, 1973.
- [51] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [52] T.-Y. Lin, M. Maire, S. Belongie, et al., "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.
- [53] J. Hosang, R. Benenson, P. Dollár, et al., "What Makes for Effective Detection Proposals?" *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 4, pp. 814–830, 2016.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] F. Perazzi, J. Pont-Tuset, B. McWilliams, et al., "A Benchmark Dataset and Evaluation Methodology for Video Object Segmentation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [56] G. Csurka, D. Larlus, and F. Perronnin, "What Is a Good Evaluation Measure for Semantic Segmentation?" In *British Machine Vision Conference (BMVC)*, 2013.
- [57] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [58] K. Mikolajczyk and C. Schmid, "A Performance Evaluation of Local Descriptors," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 10, pp. 1615–1630, 2005.
- [59] M. Dusmanu, I. Rocco, T. Pajdla, et al., "D2-Net: A Trainable CNN for Joint Description and Detection of Local Features," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [60] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [61] S. Sagawa, P. W. Koh, T. B. Hashimoto, et al., "Distributionally Robust Neural Networks for Group Shifts," in *ICLR*, 2020.
- [62] M. Arjovsky, L. Bottou, I. Gulrajani, et al., "Invariant Risk Minimization," *arXiv:1907.02893*, 2019.
- [63] I. Gulrajani and D. Lopez-Paz, "In Search of Lost Domain Generalization," in *ICLR*, 2021.
- [64] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," in *ICLR*, 2019.

- [65] R. Taori, A. Dave, V. Shankar, et al., “Measuring Robustness to Natural Distribution Shifts in Image Classification,” in *NeurIPS*, 2020.
- [66] D. Hendrycks and K. Gimpel, “A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks,” in *ICLR*, 2017.
- [67] Y. Ovadia, E. Fertig, et al., “Can You Trust Your Model’s Uncertainty?” In *NeurIPS*, 2019.
- [68] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [69] A. Kendall, Y. Gal, and R. Cipolla, “Multi-Task Learning Using Uncertainty to Weigh Losses,” in *CVPR*, 2018.
- [70] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [71] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [72] F. Milletari, N. Navab, and S.-A. Ahmadi, “V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation,” in *3DV*, 2016.
- [73] H. Rezatofighi, N. Tsoi, J. Gwak, et al., “Generalized Intersection over Union,” in *CVPR*, 2019.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [76] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [77] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [78] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [79] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [80] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [81] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [82] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [83] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [84] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [85] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [86] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [87] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [88] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [89] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [90] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.

- [91] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [92] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [93] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [94] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [95] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [96] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [97] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [98] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [99] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [100] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [101] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.

Phân phối, dịch chuyển và ngoài phân phối (Distribution Shift and OOD)

PHỤ THUỘC B/04-nen-tang-hoc-tu-du-lieu (giả định IID và vì sao mọi bảo đảm học máy đứng trên nó); B/05-thiet-ke-bai-toan (thiên lệch chọn mẫu (*selection bias*), khung lấy mẫu); B/07-chia-du-lieu-va-danh-gia (chương này gỡ bỏ đúng giả định mà giao thức ở đó dựa vào).

TÓM TẮT

Chương này nói về chuyện xảy ra khi dữ liệu thật không còn cùng phân phối với dữ liệu huấn luyện — tức là gần như luôn luôn. Đọc xong bạn thay được câu “bị domain shift” bằng một chẩn đoán cụ thể chỉ ra nên gán nhãn lại, nên chỉnh đầu ra, hay nên thu thập thêm; bạn dựng được một quy trình giám sát trôi phân phối; và bạn biết vì sao mô hình lại mong manh đến vậy. Nếu chỉ nhớ một điều: ba loại dịch chuyển — $p(x)$ đổi, $p(y)$ đổi, $p(y | x)$ đổi — cho cùng một triệu chứng nhưng đòi ba cách chữa loại trừ nhau, và chỉ loại thứ ba mới bắt buộc phải gán nhãn lại.

KIẾN THỨC CẦN CÓ TRƯỚC

- **phân phối xác suất và ký hiệu** $p(\cdot)$ — $p(x)$ mô tả những ảnh nào hay xuất hiện và những ảnh nào hiếm; $p(y)$ mô tả tỉ lệ các lớp; $p(y | x)$ là “cho trước ảnh này, nhãn đúng của nó có khả năng là gì”; $p(x | y)$ là “ảnh của lớp này trông thế nào”. Toàn bộ chương xoay quanh đúng một câu hỏi: trong bốn thứ đó, thứ nào đã thay đổi.
- **giả định IID** — giả định rằng mỗi mẫu được rút ra một cách độc lập từ cùng một phân phối, cho cả dữ liệu huấn luyện lẫn dữ liệu tương lai. Mọi bảo đảm lý thuyết của học máy đứng trên giả định này, và chương này nói về chuyện xảy ra khi nó vỡ.
- **ba loại dịch chuyển phân phối** — *covariate shift* — $p(x)$ đổi (camera mới, ánh sáng khác) nhưng quan hệ ảnh–nhãn $p(y | x)$ vẫn đúng. *Label shift* — tỉ lệ các lớp $p(y)$ đổi trong khi ảnh của mỗi lớp vẫn y hệt. *Concept drift* — chính $p(y | x)$ đổi, tức cùng một ảnh nay phải gán nhãn khác. Ba loại cho cùng một triệu chứng nhưng đòi ba cách chữa loại trừ nhau.
- **mô hình, huấn luyện và tối thiểu hoá rủi ro thực nghiệm (empirical risk minimization, ERM)** — mô hình là hàm có tham số biến ảnh thành dự đoán; huấn luyện là tìm tham số làm trung bình mất mát trên tập huấn luyện nhỏ nhất — đúng cái nguyên tắc gọn ấy có tên là ERM. Nó là *baseline* mà mọi phương pháp phức tạp hơn trong chương này phải vượt qua.
- **tiên nghiệm (prior) và tỉ số cược (odds)** — tiên nghiệm là xác suất của một lớp *trước khi* nhìn ảnh, tức tỉ lệ lớp đó trong dữ liệu; tỉ số cược của một xác suất p là $p/(1 - p)$, một cách viết khác của cùng thông tin nhưng nhân chia được. Cần biết vì phép chữa rẻ nhất trong cả chương chỉ là nhân tỉ số cược với tỉ lệ giữa hai tiên nghiệm.
- **đuôi dài (long-tail)** — **phân biệt với mất cân bằng lớp** — mất cân bằng lớp là vài lớp có ít mẫu hơn hẳn các lớp khác. Đuôi dài là tình huống có *rất nhiều* lớp mà mỗi lớp chỉ có dăm bảy mẫu, nên vấn đề không còn là tỉ lệ mà là không đủ dữ liệu để học được lớp đó là gì. Phân biệt này quan trọng vì cách chữa của trường hợp đầu không dùng được cho trường hợp sau.
- **ngoài phân phối (OOD), softmax, độ tin cậy và hiệu chuẩn (calibration)** — softmax biến các điểm số thô ở đầu ra thành một bộ số dương cộng lại bằng 1, đọc như xác suất; giá trị lớn nhất trong đó gọi là độ tin cậy của mô hình. Hiệu chuẩn tốt nghĩa là khi mô hình nói 0,8 thì nó đúng khoảng 80% số lần. Một đầu vào *ngoài phân phối* là đầu vào khác hẳn mọi thứ mô hình từng thấy

lúc huấn luyện, và “phát hiện OOD” là cố nhận ra nó để hệ thống nói “tôi không chắc” thay vì trả lời bừa.

- **embedding, augmentation và corruption** — embedding là vector vài trăm chiều biểu diễn một ảnh, ảnh giống nhau cho vector gần nhau — dùng ở đây để so hai tập dữ liệu mà không cần nhãn. Augmentation là cố ý biến dạng ảnh *huấn luyện* (lật, đổi màu, thêm nhiễu, làm mờ) để mô hình bớt phụ thuộc chi tiết không quan trọng; corruption là cùng loại biến dạng đó nhưng áp lên tập *kiểm tra* để đo độ bền. Cùng một phép biến đổi, hai mục đích ngược nhau.

CÂU HỎI MỞ ĐẦU

- Q1. Mô hình đạt 95% trên tập test và 60% ở khách hàng đầu tiên. Ba câu hỏi nào tách được nguyên nhân, và câu nào cho phép chữa mà không cần huấn luyện lại một tham số nào?
- Q2. Vì sao một mạng có thể phân biệt bò với lạc đà rất giỏi mà thực ra chưa bao giờ nhìn con vật?
- Q3. Nếu hàng trăm phương pháp tổng quát hoá miền đều tốt hơn ERM trong bài báo của chính chúng, vì sao khi tune công bằng thì không cái nào thắng?
- Q4. Vì sao độ tin cậy của mô hình lại là tín hiệu cảnh báo tệ nhất, đúng vào lúc bạn cần nó nhất?

6.1 Đặt vấn đề

Mọi kết quả lý thuyết ở B/04 — từ chặn tổng quát hoá tới chính khái niệm “sai số kỳ vọng” — đứng trên một giả định duy nhất: dữ liệu huấn luyện và dữ liệu tương lai được rút độc lập từ cùng một phân phối. Trong thị giác máy tính, giả định đó gần như luôn sai. Camera được thay sau mười tám tháng, firmware ISP cập nhật, mùa đổi, khách hàng mới ở một thành phố khác, một loại xe mới xuất hiện trên đường.

Sai lầm đắt nhất của người mới ở đây không phải chọn sai phương pháp thích nghi, mà là chẩn đoán quá thô. Hệ thống tụt tám điểm sau khi triển khai, và kết luận duy nhất được đưa ra là “bị domain shift” — một câu vừa đúng vừa vô dụng, vì nó không dẫn tới hành động nào. Ba nguyên nhân hoàn toàn khác nhau đều cho ra cùng triệu chứng ấy, và chúng đòi ba cách chữa loại trừ lẫn nhau: một cái cần gán nhãn lại, một cái chỉ cần vài phép nhân ở đầu ra, một cái cần thu thập thêm dữ liệu miền mới.

Có một trục giác từ robotics giúp đóng khung cả chương. Một hệ định vị được điều chỉnh cho một mô hình nhiễu cụ thể chạy tuyệt vời trong đúng điều kiện nó được điều chỉnh; đổi cảm biến hay đổi bề mặt sàn thì nó không báo lỗi, nó chỉ *sai một cách tự tin*. Mạng nơ-ron còn tệ hơn ở chỗ nó không có **phần dư** (*residual*) để bạn nhìn vào, nên nó không có cách nào tự nói rằng mình đang ở ngoài vùng hợp lệ. Phần lớn chương này là các cách xây dựng nhân tạo cái phần dư ấy.

→ **hệ quả của B/04** — giả định IID là nền của mọi bảo đảm tổng quát hoá; gỡ nó ra thì các bảo đảm đó mất hiệu lực, chứ không phải yếu đi.

TRỰC GIÁC

Hãy hình dung một robot định vị bằng cách đếm số bước từ cửa phòng thay vì đọc bản đồ. Trong toà nhà nó được hiệu chỉnh, kết quả chính xác tuyệt đối và không có cách nào phân biệt nó với một hệ định vị thật. Sang toà nhà khác — cùng loại hành lang, cùng loại cửa, chỉ khác chiều dài — nó sai hoàn toàn ngay bước đầu tiên. Nó không hỏng; nó đã học đúng thứ tương quan mạnh nhất với đáp án trong dữ liệu nó thấy. Đây chính là cơ chế của **shortcut learning**, và nó giải thích vì sao độ chính xác cao trên tập test không hề bảo đảm hệ thống hiểu bài toán.

6.2 Ba loại dịch chuyển (Covariate, Label and Concept Shift)

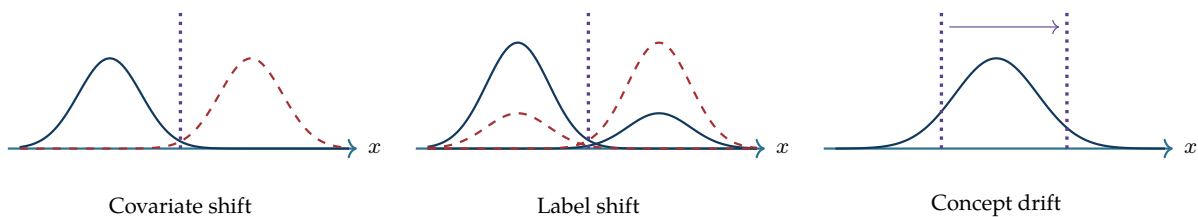
Muốn thay “bị shift” bằng một chẩn đoán dùng được, phải tách bài toán theo cấu trúc của phân phối chung $p(x, y)$. Nói gọn thì có ba cách phân rã nó, và mỗi cách cho một loại dịch chuyển với một cách chữa riêng.

A. Phân biệt bằng cấu trúc của $p(x, y)$

Bảng 6.1. Ba loại dịch chuyển. Cột cuối là lý do phải phân biệt: cách chữa của loại này vô dụng với loại kia.

Loại	Cái gì đổi	Dấu hiệu	Cách xử lý
dịch chuyển hiệp biến (<i>covariate shift</i>)	$p(x)$ đổi, $p(y x)$ giữ nguyên	Ảnh trông khác đi nhưng quan hệ ảnh–nhãn vẫn đúng	Reweighting theo mật độ, augmentation, domain adaptation
dịch chuyển nhãn (<i>label shift</i>)	$p(y)$ đổi, $p(x y)$ giữ nguyên	Tỉ lệ lớp đổi; ảnh của mỗi lớp vẫn y hệt	Hiệu chỉnh tiên nghiệm ngay ở đầu ra, không cần train lại
trôi khái niệm (<i>concept drift</i>)	$p(y x)$ đổi	Cùng một ảnh nay phải gán nhãn khác	Gán nhãn lại và huấn luyện lại; không có đường tắt

Hình 6.1 vẽ ba loại đó trên một trục đặc trưng duy nhất, và điểm đáng nhìn là chúng làm biến dạng những bộ phận khác nhau của cùng một bức tranh: hai loại đầu di chuyển *dữ liệu* quanh một biên quyết định đứng yên, còn loại thứ ba di chuyển *chính biên quyết định*. Đó là lý do hai loại đầu chữa được bằng số học trong khi loại thứ ba đòi nhãn mới.



Hình 6.1. Ba loại dịch chuyển trên một trục đặc trưng. Nét liền xanh là phân phối lúc huấn luyện, nét đứt đỏ là lúc triển khai, nét chấm tím là biên quyết định. Hai panel đầu: dữ liệu dịch chuyển, biên đứng yên. Panel cuối: dữ liệu y nguyên, biên dịch chuyển — và đó là lý do duy nhất trong ba trường hợp mà nhãn cũ trở thành nhãn sai.

B. Ví dụ tính tay: hiệu chỉnh tiên nghiệm khi có label shift

Label shift đáng có một ví dụ số vì nó là trường hợp duy nhất chữa được gần như miễn phí, và cũng là trường hợp hay bị bỏ qua nhất. Giả sử bạn huấn luyện một bộ phân loại nhị phân trên tập cân bằng (tỉ lệ dương 50%), còn khi triển khai tỉ lệ dương thật chỉ là 5%. Mô hình trả về $p = 0,60$ cho một mẫu.

- Đổi sang **tỉ số cược** (*odds*): $0,60/0,40 = 1,50$.
- Vì $p(x | y)$ không đổi, **tỉ số độ hợp lý** (*likelihood ratio*) giữ nguyên và chỉ phần tiên nghiệm cần sửa. Nhân với $\frac{0,05/0,95}{0,50/0,50} \approx 0,0526$, được 0,079.
- Đổi ngược về xác suất: $p \approx 0,079/1,079 \approx 0,073$.

Phép tính này nói điều gì? Rằng một dự đoán “khá chắc là dương” thật ra là “7% khả năng dương”, và toàn bộ sự chênh lệch đến từ việc mô hình mang theo tiên nghiệm của tập huấn

luyện chứ không phải của thế giới. Không train lại một tham số nào; chỉ cần biết tỉ lệ lớp mới. Ngược lại, nếu vấn đề thật là concept drift thì phép nhân này không những vô ích mà còn làm sai lệch thêm, vì giả định $p(x | y)$ bất biến đã vỡ ngay từ đầu.

↔ **tiền đề** E/25 — hiệu chỉnh tiên nghiệm là một thao tác vận hành: nó sửa đầu ra theo tỉ lệ lớp thật, và chỉ dùng được khi đầu ra đã hiệu chuẩn.

6.3 Biết là đã trôi, chưa biết có hại (Shift Detection)

Trước khi chữa phải phát hiện, và phát hiện rẻ hơn nhiều so với người ta tưởng. Hai công cụ thực dụng, cộng một giới hạn phải luôn nhớ:

- **Khoảng cách trên không gian *embedding*** — so phân phối đặc trưng của dữ liệu huấn luyện với dữ liệu đang chạy. Rẻ, chạy liên tục được, nhưng con số khoảng cách tự nó khó diễn giải.
- **Bộ phân loại phân biệt miền** — huấn luyện một bộ phân loại nhị phân để tách “mẫu train” khỏi “mẫu production”. Đây là dụng cụ đo tốt hơn vì kết quả đọc được ngay:
 - **AUC $\approx 0,5$** — hai tập không phân biệt được, chưa có dịch chuyển đáng kể.
 - **AUC $\approx 0,95$** — chỉ cần nhìn ảnh cũng biết nó đến từ đâu.
 - **Lợi ích kèm theo** — các đặc trưng mà bộ phân biệt dựa vào chính là mô tả cụ thể của dịch chuyển, nên nó vừa báo động vừa chẩn đoán.
- **Giới hạn chung của cả hai** — chúng trả lời “phân phối có đổi không”, không phải “việc đó có làm hỏng hệ thống không”. Hai điều đó tách rời theo cả hai chiều: đổi tông màu toàn cục là thay đổi lớn về thống kê nhưng vô hại nếu mô hình không dùng màu; ngược lại một thay đổi bé xíu đúng vào đặc trưng mô hình dựa vào lại đủ làm sập nó.

Hệ quả thực hành: tín hiệu dịch chuyển phải luôn được đối chiếu với một metric thật trên một tập nhỏ có nhãn thu ở miền đích. Giám sát dịch chuyển mà không có mẫu có nhãn nào ở miền đích là giám sát mù. Vì thế ngân sách nên đổ vào việc lấy vài trăm nhãn mỗi tháng từ production, trước khi đổ vào thuật toán thích nghi.

6.4 Thích nghi hay tổng quát hoá (Domain Adaptation vs Generalization)

Đến khi phải hành động, có một phân đôi quyết định toàn bộ chiến lược, và nó phụ thuộc vào đúng một câu hỏi: *bạn có mẫu từ miền đích hay không?*

Mạch 1 — từ thích nghi tới tổng quát hoá

1 Vấn đề

Giải pháp

Giả định

Giá phải trả

có mẫu miền đích (dù không nhãn), và chỉ cần chạy tốt ở đúng miền đó.

thích nghi miền (*domain adaptation*): căn chỉnh phân phối đặc trưng giữa hai miền, hoặc tự huấn luyện bằng nhãn giả trên miền đích.

hai miền chia sẻ cùng quan hệ $p(y | x)$, tức đây là covariate shift chứ không phải concept drift.

mô hình bị chuyên biệt hoá cho một phân phối cụ thể; miền thứ ba xuất hiện thì làm lại từ đầu.

2 Vấn đề

Giải pháp

chưa từng thấy miền đích và không biết nó sẽ là gì.

tổng quát hoá miền (*domain generalization*): học biểu diễn bất biến qua nhiều miền huấn luyện, với hy vọng cái bất biến đó cũng đúng ở miền chưa thấy [62].

- Giá phải trả** phải hy sinh hiệu năng trung bình để lấy độ bền — một biểu diễn chỉ giữ phần bất biến thì bỏ đi thông tin có ích ở miền hiện tại.
- Giả định** tồn tại một cơ chế nhân quả bất biến và bạn có đủ miền huấn luyện đa dạng để nhận ra nó.
- Vấn đề mới** giả định đó có được thoả trên dữ liệu thật không?
- Hệ quả** đây là chỗ cần trung thực: một khảo sát có kiểm soát cho thấy khi mọi phương pháp tổng quát hoá miền được tune công bằng với cùng ngân sách, không phương pháp nào vượt được ERM thuần một cách nhất quán [63]. Kết quả này không nói “đừng làm DG”; nó nói rằng phần lớn cải tiến báo cáo được đến từ quy trình chọn mô hình, không từ thuật toán. Đây đúng là *adaptive overfitting* ở B/07, lần này xuất hiện ở cấp độ cả một dòng nghiên cứu.
- 3 Vấn đề** cái ta thật sự quan tâm là nhóm tẻ nhất, không phải trung bình.
- Giải pháp** tối ưu trực tiếp cho nhóm tẻ nhất thay vì cho trung bình [61].
- Giả định** bạn biết trước các nhóm là gì — nghĩa là quay lại đúng vấn đề metadata ở B/07: không có nhãn nhóm thì không có phương pháp nhóm nào dùng được.

Đằng sau cả bốn bước là một nguyên nhân chung đã nêu ở hộp trực giác: *shortcut learning* [25]. Mô hình không cố hiểu bài toán; nó tìm đặc trưng dễ nhất tương quan đủ mạnh với nhãn trong tập huấn luyện. Nếu ảnh của lớp “bò” toàn chụp trên đồng cỏ thì cỏ là đặc trưng rẻ hơn hình dạng con bò rất nhiều; nếu ảnh bệnh lý đến từ một máy chụp và ảnh bình thường từ máy khác, thì dấu hiệu của máy là đặc trưng rẻ nhất trong toàn bộ ảnh. Đây là lời giải thích thống nhất cho hiện tượng “chính xác trên test, ngu ngốc ngoài đời”, và nó nổi ngược về B/05: shortcut hầu như luôn bắt nguồn từ một thiên lệch chọn mẫu ở khâu thu thập, nên nó không phải lỗi của mô hình mà là lỗi được cài vào từ trước khi mô hình tồn tại.

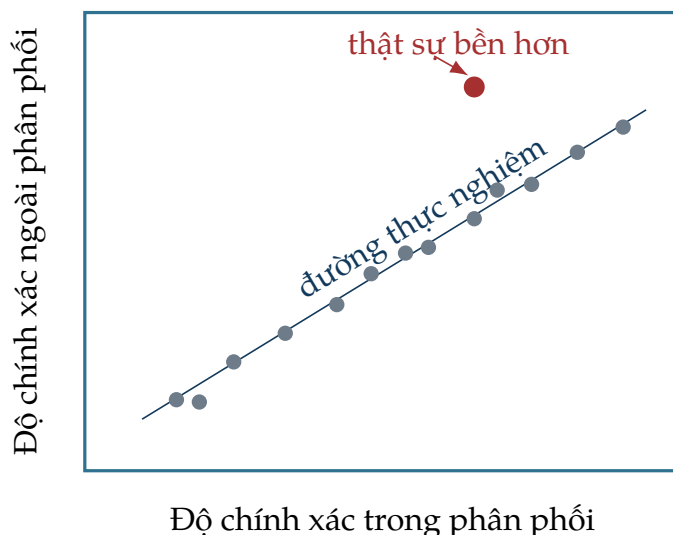
→ **hệ quả của B/05** — thiên lệch sinh ở khâu thu thập thì không hàm mất mát nào sửa được ở khâu cuối — shortcut chỉ là cách nó lộ ra.

6.5 Đo độ bền và quyền nói “tôi không biết” (Robustness and OOD Detection)

A. Benchmark corruption, và đường thẳng làm chuẩn nghiêm khắc

Nếu độ bền quan trọng thì nó phải được đo, không phải được hy vọng. Cách rẻ nhất là các bộ benchmark corruption tổng hợp: làm nhiễu, mờ, nén, đổi thời tiết ở nhiều mức nghiêm trọng rồi đọc metric theo từng loại và từng mức [64]. Giá trị thật của chúng nằm ở *cấu trúc* chứ không ở con số tổng — loại corruption làm sập mạnh nhất chính là loại phá huỷ đặc trưng mô hình đang dùng. Nếu mờ chuyển động làm sập trong khi đổi màu không ảnh hưởng, mô hình đang sống nhờ cấu trúc tần số cao, và đó là thông tin hành động được.

Giới hạn thì cũng rõ: corruption tổng hợp không thay thế được dịch chuyển thật. Vì thế các tập dữ liệu như WILDS được xây từ dịch chuyển xảy ra tự nhiên trong ứng dụng: bệnh viện khác, năm khác, vùng địa lý khác [27]. Và có một quan sát thực nghiệm hữu ích, hơi bất ngờ, được vẽ ở Hình 6.2: trên nhiều họ dịch chuyển tự nhiên, độ chính xác ngoài phân phối gần như là hàm tuyến tính của độ chính xác trong phân phối [26], [65].



Hình 6.2. “Accuracy on the line” (sơ đồ minh họa theo mô tả trong [65], không phải số đo lại). Hầu hết mô hình và hầu hết “phương pháp tăng độ bền” chỉ trượt dọc theo một đường thẳng: cải thiện ngoài phân phối đến hoàn toàn từ cải thiện trong phân phối. Chỉ điểm đỏ — nằm *trên* đường — mới xứng đáng gọi là tăng độ bền. Đường thẳng này vì thế là một *baseline* nghiêm khắc mà mọi tuyên bố về độ bền phải vượt qua.

B. Cho mô hình quyền nói “tôi không biết”

↔ **ràng buộc bởi** E/24 — đường thẳng đó là chuẩn so sánh: một ablation về độ bền chỉ có nghĩa khi nó tách được điểm ra khỏi đường.

Hướng bổ sung, không thay thế: **phát hiện ngoài phân phối** (*out-of-distribution detection*) nhằm nhận ra đầu vào nằm ngoài phân phối huấn luyện để hệ thống chuyển sang chế độ an toàn thay vì trả lời bừa. Ba điều cần biết, theo thứ tự quan trọng:

- **Đường cơ sở đơn giản đến bất ngờ** — dùng chính giá trị softmax cực đại làm điểm số [66]. Đây vừa là tin tốt (rẻ, không cần train lại) vừa là cảnh báo: nó cho thấy phần lớn công trình sau đó chỉ cải thiện tiệm tiến trên một tín hiệu vốn yếu.
- **Giới hạn thực nghiệm** — độ tin cậy của mạng sâu vốn đã kém hiệu chuẩn ngay trong phân phối, và nó xuống cấp thêm đúng dưới dịch chuyển, tức đúng lúc ta cần nó nhất [67].
- **Giới hạn nguyên tắc** — không thể có bộ phát hiện OOD phổ quát: “ngoài phân phối” chỉ có nghĩa khi đã cố định một phân phối tham chiếu, nên mọi bộ phát hiện đều là bộ phát hiện của *một loại* bất thường mà bạn đã ngầm chọn.

6.6 Đuôi dài và vòng phản hồi (Long-Tail and Feedback Loops)

Hai hiện tượng cuối không phải dịch chuyển theo nghĩa thời gian, nhưng chúng phá cùng một giả định.

- **Long-tail khác mất cân bằng lớp ở cơ chế, không chỉ ở mức độ.**
 - **Mất cân bằng lớp** — vài lớp có ít mẫu hơn hẳn; tăng trọng số hay lấy mẫu lại giúp được, vì vấn đề nằm ở hàm mất mát.
 - **Đuôi dài** — *rất nhiều* lớp, mỗi lớp cực hiếm; vấn đề không còn là trọng số mà là không đủ mẫu để học biểu diễn. Tăng trọng số cho một lớp có bốn mẫu chỉ làm mô hình thuộc lòng bốn mẫu đó nhanh hơn.

- **Hướng đi có cơ sở** — tách hai việc vốn bị gộp: học biểu diễn trên toàn bộ dữ liệu (phần đầu phân phối cung cấp tín hiệu), rồi học riêng bộ phân loại với dữ liệu cân bằng lại; hoặc đổi hẳn bài toán sang truy hồi theo độ tương đồng, nơi thêm một lớp mới chỉ là thêm một mẫu tham chiếu.
- **vòng phản hồi (*feedback loop*) là thứ nguy hiểm nhất chương, vì nó vô hình với mọi phép đo ngoại tuyến.**
 - **Cơ chế** — hệ thống quyết định cái gì được nhìn thấy: ảnh nào tới tay người kiểm duyệt, vùng nào được quét kỹ, mẫu nào được gửi đi gắn nhãn. Dữ liệu thu về sau đó đã bị chính mô hình lọc.
 - **Vì sao không đo được** — mô hình tiếp theo học trên dữ liệu ấy sẽ củng cố đúng thiên lệch của mô hình trước. Mọi metric ngoại tuyến vẫn xác nhận nó tốt lên, vì tập đánh giá cũng đi qua cùng cái phễu. Đây là một dạng cụ thể của khoản nợ kỹ thuật ẩn trong hệ thống học máy [4].
 - **Cách phá vòng** — cố tình giữ một luồng dữ liệu không bị mô hình lọc: một tỉ lệ nhỏ mẫu chọn ngẫu nhiên, thu thập bất kể mô hình nói gì.

Từ đó suy ra hành động chủ động cuối cùng, và cũng là hành động rẻ nhất trong cả chương: *tự xây một bộ test đối kháng*. Không phải đối kháng theo nghĩa nhiều tối ưu hoá. Ý ở đây là vài trăm ảnh được chọn có chủ ý để lộ đúng shortcut bạn nghi ngờ: cùng vật thể trên nền khác, cùng cảnh chụp bằng camera khác, đúng lớp hiếm ở điều kiện xấu. Vài trăm ảnh như vậy thường nói nhiều hơn cả một bộ benchmark corruption, vì chúng nhắm vào giả thuyết cụ thể của bạn thay vì rải đều.

↔ **đối lập với B/07** — tập test chuẩn được thiết kế để *đại diện*; bộ test đối kháng được thiết kế để *phá*, nên hai thứ trả lời hai câu hỏi khác nhau.

6.7 Giới hạn và trường hợp hỏng

Cách phân loại ba dịch chuyển ở đầu chương gọn gàng hơn thực tế: dịch chuyển thật hầu như luôn là hỗn hợp của cả ba, và không có phép kiểm định nào tách chúng ra sạch sẽ khi bạn không có nhãn ở miền đích. Thứ tự chẩn đoán thực dụng là bắt đầu bằng cái rẻ nhất — kiểm tra tiên nghiệm lớp có đổi không, vì nếu có thì hiệu chỉnh gần như miễn phí — rồi mới tới các giả thuyết đắt hơn.

Hiệu chỉnh theo tiên nghiệm giả định $p(x | y)$ bất biến, một giả định thường bị vi phạm âm thầm: khi tỉ lệ lớp đổi vì camera mới, thì ảnh của mỗi lớp cũng đã đổi cùng lúc, và bạn có cả label shift lẫn covariate shift chồng lên nhau. Reweighting theo tỉ số mật độ chỉ hoạt động khi vùng mà phân phối nguồn thật sự có mẫu của phân phối đích nằm trong giá đỡ của phân phối nguồn; với miền thật sự mới, tỉ số mật độ tiến ra vô cực ở đúng vùng bạn quan tâm và phương sai của ước lượng nổ tung.

CẠM BÃY

Cạm bẫy hay gặp nhất là dùng chính độ tin cậy của mô hình làm cảnh báo dịch chuyển. Nghe hợp lý — mô hình gặp thứ lạ thì phải kém tự tin — nhưng đó chính là điều mạng sâu *không* làm: dưới dịch chuyển, nó thường vẫn tự tin và tự tin sai [67]. Một hệ thống giám sát chỉ báo động khi confidence tụt sẽ im lặng đúng trong những kịch bản hỏng nguy hiểm nhất.

CẦN NHỚ

“Bị domain shift” không phải một chẩn đoán. Chẩn đoán là trả lời được $p(x)$ đổi, $p(y)$ đổi, hay $p(y | x)$ đổi. Ba câu trả lời đó dẫn tới ba hành động loại trừ nhau, và chỉ cái cuối cùng bắt buộc phải gán nhãn lại. Muốn chẩn đoán được thì phải có một dòng nhỏ dữ liệu *có nhãn* từ miền đích; không có nó thì mọi thuật toán thích nghi đều là đoán mò. [T2]

6.8 Thực hành

Repo và tài nguyên. [bethgelab/imagecorruptions](#) cùng ImageNet-C và ImageNet-R cho corruption tổng hợp; [RobustBench/robustbench](#) để so với các *baseline* đã được đo lại cẩn thận; [p-lambda/wilds](#) cho dịch chuyển thật thay vì nhân tạo; [Jingkang50/OpenOOD](#) và [kkirchheim/pytorch-ood](#) cho OOD detection; [evidentlyai/evidently](#) và [SeldonIO/alibi-detect](#) để giám sát trôi phân phối trong production. Danh sách chốt tại tháng 9/2026 và tên phương pháp dẫn đầu sẽ cũ trong 6–12 tháng; các tập dữ liệu (WILDS, ImageNet-C) và hạ tầng giám sát bền hơn nhiều so với bảng xếp hạng.

Câu hỏi kiểm tra hiểu. Accuracy trên production tụt 8%: ba phép đo nào phân biệt được covariate shift với concept drift, và phép nào rẻ nhất nên làm trước? Vì sao tăng trọng số cho lớp hiếm không giải quyết được đuôi dài? Vì sao một bộ phát hiện OOD “phổ quát” là khái niệm không nhất quán?

Bài tập phụ. Đo accuracy trên 15 loại corruption ở 5 mức nghiêm trọng, vẽ bản đồ nhiệt, tìm loại làm sập mạnh nhất, rồi giải thích nó bằng ngôn ngữ của chuỗi tạo ảnh ở A/02-vat-ly-tao-anh: corruption đó phá huỷ đặc trưng nào, và điều đó nói gì về thứ mô hình đang thật sự dùng?

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Giả định IID	Mỗi mẫu được rút độc lập từ cùng một phân phối, cho cả dữ liệu huấn luyện lẫn dữ liệu tương lai. Mọi bảo đảm lý thuyết của học máy đứng trên nó, nên khi nó vỡ thì các bảo đảm mất hiệu lực chứ không chỉ yếu đi.
••• Covariate shift	$p(x)$ đổi nhưng quan hệ ảnh–nhân giữ nguyên: ảnh trông khác, quy luật vẫn đúng. Chữa bằng thích nghi miền, augmentation, hoặc đánh trọng số lại theo mật độ.
••• Label shift	Tỉ lệ các lớp đổi trong khi ảnh của mỗi lớp vẫn y hệt. Đây là trường hợp duy nhất chữa được gần như miễn phí, bằng cách chỉnh lại tiên nghiệm ở đầu ra.
••• Concept drift	Chính $p(y x)$ đổi: cùng một ảnh nay phải gán nhãn khác. Không có đường tắt — phải gán nhãn lại rồi huấn luyện lại.
•• Vì sao label shift chữa được mà không cần huấn luyện lại?	Vì $p(x y)$ không đổi nên tỉ số độ hợp lý giữ nguyên, chỉ phân tiên nghiệm sai. Nhân tỉ số cực của đầu ra với tỉ lệ giữa hai tiên nghiệm là xong.
•• Mô hình cho 0,60 nhưng tỉ lệ dương thật chỉ 5% — xác suất đúng?	Khoảng 0,073. Tỉ số cực 1,5 nhân với 0,0526 ra 0,079, đổi ngược về xác suất là 7,3%. Một dự đoán “khá chắc” hoá ra chỉ là 7%.
••• Bộ phân biệt miền	Một bộ phân loại nhị phân tách “mẫu train” khỏi “mẫu production”. AUC quanh 0,5 nghĩa là chưa có dịch chuyển đáng kể; AUC 0,95 nghĩa là nhìn ảnh cũng biết nó đến từ đâu.
••• Vì sao phát hiện được dịch chuyển vẫn chưa đủ?	Vì nó trả lời “có đổi không”, không trả lời “có hại không”. Đổi tông màu toàn cục là thay đổi lớn mà vô hại; một thay đổi bé đúng vào đặc trưng mô hình dựa vào lại đủ làm sập nó.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Thích nghi miền	Căn chỉnh phân phối đặc trưng, hoặc tự huấn luyện bằng nhân giả, khi bạn đã có mẫu từ miền đích. Nó chuyên biệt hoá mô hình cho một phân phối, nên miền thứ ba xuất hiện thì phải làm lại.
●● Tổng quát hoá miền	Học biểu diễn bất biến qua nhiều miền huấn luyện, dùng khi chưa từng thấy miền đích. Giá phải trả là hy sinh hiệu năng trung bình để lấy độ bền.
●● Vì sao tổng quát hoá miền không thắng nổi ERM?	Vì phần lớn cải tiến báo cáo được đến từ quy trình chọn mô hình, không từ thuật toán. Cho mọi phương pháp cùng ngân sách tune và cùng thủ tục chọn thì lợi thế biến mất.
●●● Shortcut learning	Mô hình học đặc trưng dễ nhất mà tương quan đủ mạnh với nhãn: cổ thay cho hình dạng con bò, dấu hiệu máy chụp thay cho bệnh lý. Đây là lời giải thích thống nhất cho “chính xác trên test, ngu ngốc ngoài đời”.
●● Accuracy on the line	Quan sát rằng trên nhiều dịch chuyển tự nhiên, độ chính xác ngoài phân phối gần như tuyến tính theo độ chính xác trong phân phối. Dùng nó làm chuẩn: chỉ điểm nằm trên đường mới đáng gọi là tăng độ bền.
●● Benchmark corruption	Đo lại metric qua nhiều loại nhiễu, mờ, nén, thời tiết ở nhiều mức. Giá trị nằm ở bản đồ nhiệt theo từng loại, vì loại làm sập mạnh nhất chính là loại phá đặc trưng mô hình đang dùng.
●●● OOD detection	Nhận ra đầu vào nằm ngoài phân phối huấn luyện để hệ thống nói “tôi không chắc”. Không thể có bộ phát hiện phổ quát, vì “ngoài phân phối” chỉ có nghĩa khi đã cố định một phân phối tham chiếu.
●●● Vì sao độ tin cậy của mô hình là cảnh báo tệ?	Vì mạng sâu vốn đã quá tự tin ngay trong phân phối, và xuống cấp thêm đúng dưới dịch chuyển. Hệ giám sát chỉ báo động khi độ tin cậy tụt sẽ im lặng đúng lúc nguy hiểm nhất.
●●● Đuôi dài	Rất nhiều lớp mà mỗi lớp chỉ có dăm bảy mẫu. Khác mất cân bằng lớp ở chỗ nút thắt là thiếu dữ liệu để học biểu diễn, nên tăng trọng số chỉ làm mô hình thuộc lòng vài mẫu đó nhanh hơn.
●● Vòng phản hồi	Mô hình quyết định cái gì được nhìn thấy, nên dữ liệu thu về sau đó đã bị chính nó lọc. Phá vòng bằng cách giữ một tỉ lệ nhỏ mẫu chọn ngẫu nhiên, thu bắt kể mô hình nói gì.
●● Accuracy production tụt 8% — ba phép đo nào, làm cái nào trước?	Đo tỉ lệ lớp, đo AUC của bộ phân loại phân biệt miền, và gán tay vài chục mẫu để xem nhãn cũ còn đúng không. Làm phép đầu trước vì nếu là label shift thì chữa gần như miễn phí.
●● Mọi metric ngoại tuyến đều tốt lên mà người dùng vẫn phàn nàn — vì sao?	Vòng phản hồi: tập đánh giá đi qua đúng cái phễu mà mô hình đã lọc. Phân biệt bằng một tập giữ riêng thu ngẫu nhiên, không qua mô hình; nếu con số ở đó không nhích thì vòng lặp đã khoá.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
WILDS (p-lambda/wilds)	Đóng gói dịch chuyển tự nhiên theo bệnh viện, theo năm, theo vùng, và yêu cầu báo cáo hiệu năng nhóm tẻ nhất thay vì trung bình	Bằng chứng rằng phân biệt các loại dịch chuyển đã thành chuẩn thiết kế benchmark
ImageNet-C, ImageNet-R, RobustBench	Đo độ bền theo từng loại corruption và từng mức, đúng ý “cấu trúc quan trọng hơn con số tổng”	Hạ tầng đánh giá; RobustBench tồn tại vì các báo cáo tự đo không tái lập được

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenOOD	So các phương pháp phát hiện ngoài phân phối trên cùng giao thức, kể cả <i>baseline</i> softmax cực đại	Cho thấy khoảng cách giữa baseline đơn giản và phương pháp mới hẹp hơn ấn tượng từ từng bài báo
evidentlyai/evidently, SeldonIO/alibi-detect	Giám sát production bằng đúng hai công cụ của chương: khoảng cách phân phối trên embedding và bộ phân loại phân biệt miền	Hạ tầng vận hành, ít phụ thuộc mô hình cụ thể
Mô hình nền tảng huấn luyện trên dữ liệu web quy mô lớn (dòng CLIP, DINOv2/v3)	Được cho là nhắc hẳn đường “accuracy on the line” nhờ đa dạng dữ liệu, chứ không nhờ một thuật toán bền hơn	Đây là cách đọc phổ biến tính tới 9/2026, không phải một kết luận đã khép lại

MINI PROJECT SLAM · Chẩn đoán loại dịch chuyển khi đổi tập dữ liệu · 1–2 ngày

Mục tiêu	lấy đúng bộ phân loại “cùng chỗ” đã dựng ở chương trước, chạy nó trên một tập dữ liệu khác, rồi trả lời bằng bằng chứng số: mức sụt là covariate shift, label shift, hay concept drift.
Dữ liệu	EuRoC làm miền nguồn; TUM-VI (chuỗi room hoặc corridor) hoặc OpenLORIS làm miền đích. Sinh nhân cùng chỗ ở miền đích theo <i>đúng</i> quy tắc hình học đã dùng ở B/07-chia-du-lieu-va-danh-gia , không đổi ngưỡng.
Các bước	bốn phép đo, theo thứ tự từ rẻ tới đắt. (1) Đo metric ở cả hai miền, ghi mức sụt. (2) Huấn luyện một bộ phân loại phân biệt miền trên embedding của hai tập và ghi AUC của nó. (3) Đo tỉ lệ cặp dương ở hai miền, áp hiệu chỉnh tiên nghiệm, ghi phần hiệu năng lấy lại được. (4) Gán tay 50 cặp ở miền đích và đếm tỉ lệ bất đồng giữa nhãn hình học và nhãn tay.
Xong khi	bạn kết luận được một loại dịch chuyển kèm ba con số chống lưng: AUC của bộ phân biệt miền, phần trăm hiệu năng lấy lại nhờ hiệu chỉnh tiên nghiệm, và tỉ lệ bất đồng ở bước (4). Tỉ lệ bất đồng cao chính là bằng chứng của concept drift, vì định nghĩa “cùng chỗ” đã đổi theo cấu trúc cảnh.
Nối với	B/07-chia-du-lieu-va-danh-gia cho bộ đo và cách chia; E/25-do-tin-cay-va-van-hanh vì nếu kết luận là concept drift thì bước kế tiếp là thiết kế chế độ hồng an toàn chứ không phải tinh chỉnh thêm.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Ba câu hỏi: tỉ lệ lớp có đổi không; ảnh có trông khác không; cùng một ảnh nay có phải gán nhãn khác không. Câu đầu — label shift — chưa được gần như miễn phí bằng hiệu chỉnh tiên nghiệm ở đầu ra, nên phải kiểm tra trước tiên vì nó rẻ nhất.
- A2. Vì nó học nền chứ không học con vật: bò gần như luôn đứng trên cỏ, lạc đà trên cát. Đặc trưng nền rẻ hơn hình dạng rất nhiều và tương quan đủ mạnh với nhãn trong tập huấn luyện, nên ERM chọn đúng nó.
- A3. Vì phần lớn cải tiến báo cáo được đến từ quy trình chọn mô hình — mỗi bài chọn siêu tham số theo cách thuận lợi cho phương pháp của mình. Cho mọi phương pháp cùng ngân sách tune và cùng thủ tục chọn thì lợi thế đó biến mất [63].
- A4. Vì mạng sâu vốn đã quá tự tin ngay trong phân phối, và độ tin cậy xuống cấp thêm đúng dưới dịch chuyển. Một hệ giám sát chỉ báo động khi confidence tụt sẽ im lặng đúng trong những kịch bản hồng nguy hiểm nhất.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn bán hàng ở khu phố A suốt ba năm, quen tới mức nhìn khách bước vào là đoán được họ sẽ mua gì. Rồi bạn chuyển sang khu phố B và tỉ lệ đoán đúng tụt hẳn. Chỉ có đúng ba lý do, và ba cách chữa khác hẳn nhau.

Một, khách trông khác đi — ăn mặc khác, đến vào giờ khác — nhưng quy luật cũ vẫn đúng: ai vội thì mua nhanh. Cái này chỉ cần thời gian làm quen với kiểu khách mới. Hai, tỉ lệ khách đổi: khu cũ toàn dân văn phòng, khu mới toàn sinh viên. Quy luật không sai một chút nào, chỉ là bạn đang kỳ vọng sai tỉ lệ, nên chỉnh lại kỳ vọng là xong và gần như không tốn gì. Ba, nghĩa của dấu hiệu đã đổi: ở khu cũ, khách đứng lâu trước quầy nghĩa là sắp mua; ở khu mới, đứng lâu nghĩa là đang đợi bạn. Chỉ trường hợp thứ ba mới bắt bạn quan sát lại từ đầu.

Mẹo là đừng gộp ba thứ đó thành một câu than “khu này khó bán”. Hỏi lần lượt: khách có khác không, tỉ lệ có khác không, dấu hiệu có còn nghĩa cũ không — và hỏi cái rõ nhất trước, vì cái rõ nhất cũng là cái chữa nhanh nhất.

Cái giá là bạn cần vài chục đơn hàng thật ở khu mới mới trả lời được ba câu hỏi ấy. Không có chúng thì bạn chỉ đang đoán, và đoán một cách rất tự tin.

Chỗ không rút gọn được: vì sao máy lại bám vào một dấu hiệu ăn may thay vì dấu hiệu đúng thì không có cách nói gọn nào không gây hiểu lầm. Nói “máy lười” là sai: nó chọn đúng thứ gắn chặt nhất với đáp án trong đồng dữ liệu nó được xem, và nếu chỉ nhìn đồng đó thì không ai — kể cả người — phân biệt được đâu là dấu hiệu thật.

Một câu: Sự hiệu năng không phải một chẩn đoán; chẩn đoán là nói được cái gì đã đổi, vì mỗi thứ đổi có đúng một cách chữa.

6.9 Kết nối

Chương này đóng lại Phần B ở phía dữ liệu bằng cách gỡ bỏ giả định mà [B/04-nen-tang-hoc-tu-du-lieu](#) phải giả sử và [B/07-chia-du-lieu-va-danh-gia](#) phải dựa vào. Nó nối ngược về [B/05-thiet-ke-bai-toan](#) qua khái niệm shortcut, vốn chỉ là thiên lệch chọn mẫu nhìn từ phía mô hình, và về [B/06-chat-luong-du-lieu](#) theo quan hệ đối lập: ở đó nhấn sai vì con người, ở đây nhấn đúng nhưng thế giới đã đổi. Nó là tiền đề của [B/09-thiet-ke-ham-mat-mat](#), nơi một số dịch chuyển được xử lý ngay trong hàm mất mát, và của [E/25-do-tin-cay-va-van-hanh](#), nơi OOD detection và calibration trở thành một phần của thiết kế chế độ hồng an toàn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một hệ thống có thể có dịch chuyển phân phối rất lớn theo phép đo mà hiệu năng không giảm chút nào, và ngược lại?
2. Bạn đo được tỉ lệ lớp dương đã đổi từ 50% xuống 5%. Vì sao đây là tin tốt, và phép sửa cụ thể gồm mấy bước?
3. Tổng quát hoá miền phải hy sinh cái gì để lấy độ bền, và kết quả của [63] buộc bạn nghi ngờ điều gì trong chính thí nghiệm của mình?
4. Trong Hình 6.2, vì sao một điểm nằm *trên* đường thẳng mới đáng gọi là tăng độ bền, còn một điểm nằm xa về bên phải thì không?
5. Vì sao dùng confidence của mô hình làm cảnh báo dịch chuyển lại im lặng đúng lúc nguy hiểm nhất?
6. Đuôi dài và mất cân bằng lớp khác nhau ở cơ chế nào, và vì sao sự khác nhau đó khiến reweighting thất bại?
7. Một vòng phản hồi làm mọi metric ngoại tuyến tốt lên trong khi hệ thống tụt đi. Cơ chế nào cho phép điều đó, và biện pháp rõ nhất để phá vòng là gì?

Nguồn

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *Rec-Sys*, 2019.
- [3] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [4] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [5] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [6] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [7] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [8] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [9] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [10] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [11] V. N. Vapnik, “An overview of statistical learning theory,” *IEEE Trans. Neural Networks*, vol. 10, no. 5, pp. 988–999, 1999.
- [12] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [13] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [15] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [16] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [17] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [18] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE Trans. Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.
- [19] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.

- [20] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [21] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [22] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [23] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [24] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [25] R. Geirhos, J.-H. Jacobsen, C. Michaelis, et al., “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.
- [26] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [27] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [28] T. Gebru, J. Morgenstern, et al., “Datasheets for Datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [29] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [30] M. Mitchell, S. Wu, et al., “Model Cards for Model Reporting,” in *FAT**, 2019.
- [31] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [32] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [33] C. G. Northcutt, L. Jiang, and I. L. Chuang, “Confident Learning: Estimating Uncertainty in Dataset Labels,” *JAIR*, vol. 70, pp. 1373–1411, 2021.
- [34] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [35] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [36] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [37] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [38] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [39] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [40] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [41] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London: Butterworths, 1979.
- [42] B. W. Matthews, “Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme,” *Biochimica et Biophysica Acta (BBA) — Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
- [43] D. Chicco and G. Jurman, “The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation,” *BMC Genomics*, vol. 21, p. 6, 2020.

- [44] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in *International Conference on Machine Learning (ICML)*, 2006, pp. 233–240.
- [45] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.
- [46] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [47] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [48] M. P. Naeni, G. F. Cooper, and M. Hauskrecht, "Obtaining Well Calibrated Probabilities Using Bayesian Binning," in *AAAI Conference on Artificial Intelligence*, 2015.
- [49] G. W. Brier, "Verification of Forecasts Expressed in Terms of Probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
- [50] A. H. Murphy, "A New Vector Partition of the Probability Score," *Journal of Applied Meteorology*, vol. 12, no. 4, pp. 595–600, 1973.
- [51] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [52] T.-Y. Lin, M. Maire, S. Belongie, et al., "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.
- [53] J. Hosang, R. Benenson, P. Dollár, et al., "What Makes for Effective Detection Proposals?" *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 4, pp. 814–830, 2016.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] F. Perazzi, J. Pont-Tuset, B. McWilliams, et al., "A Benchmark Dataset and Evaluation Methodology for Video Object Segmentation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [56] G. Csurka, D. Larlus, and F. Perronnin, "What Is a Good Evaluation Measure for Semantic Segmentation?" In *British Machine Vision Conference (BMVC)*, 2013.
- [57] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [58] K. Mikolajczyk and C. Schmid, "A Performance Evaluation of Local Descriptors," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 10, pp. 1615–1630, 2005.
- [59] M. Dusmanu, I. Rocco, T. Pajdla, et al., "D2-Net: A Trainable CNN for Joint Description and Detection of Local Features," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [60] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [61] S. Sagawa, P. W. Koh, T. B. Hashimoto, et al., "Distributionally Robust Neural Networks for Group Shifts," in *ICLR*, 2020.
- [62] M. Arjovsky, L. Bottou, I. Gulrajani, et al., "Invariant Risk Minimization," *arXiv:1907.02893*, 2019.
- [63] I. Gulrajani and D. Lopez-Paz, "In Search of Lost Domain Generalization," in *ICLR*, 2021.
- [64] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," in *ICLR*, 2019.

- [65] R. Taori, A. Dave, V. Shankar, et al., "Measuring Robustness to Natural Distribution Shifts in Image Classification," in *NeurIPS*, 2020.
- [66] D. Hendrycks and K. Gimpel, "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks," in *ICLR*, 2017.
- [67] Y. Ovadia, E. Fertig, et al., "Can You Trust Your Model's Uncertainty?" In *NeurIPS*, 2019.
- [68] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [69] A. Kendall, Y. Gal, and R. Cipolla, "Multi-Task Learning Using Uncertainty to Weigh Losses," in *CVPR*, 2018.
- [70] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [71] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., "Rethinking the Inception Architecture for Computer Vision," in *CVPR*, 2016.
- [72] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation," in *3DV*, 2016.
- [73] H. Rezatofighi, N. Tsoi, J. Gwak, et al., "Generalized Intersection over Union," in *CVPR*, 2019.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in *CVPR*, 2015.
- [75] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [76] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [77] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [78] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [79] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [80] Y. LeCun, L. Bottou, Y. Bengio, et al., "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [81] F. Yu and V. Koltun, "Multi-Scale Context Aggregation by Dilated Convolutions," in *ICLR*, 2016.
- [82] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention Is All You Need," in *NeurIPS*, 2017.
- [83] Z. Liu, Y. Lin, Y. Cao, et al., "Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows," in *ICCV*, 2021.
- [84] K. He, X. Zhang, S. Ren, et al., "Deep Residual Learning for Image Recognition," in *CVPR*, 2016.
- [85] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *ICML*, 2015.
- [86] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," *arXiv:1607.06450*, 2016.
- [87] Y. Wu and K. He, "Group Normalization," in *ECCV*, 2018.
- [88] S. Santurkar, D. Tsipras, A. Ilyas, et al., "How Does Batch Normalization Help Optimization?" In *NeurIPS*, 2018.
- [89] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in *CVPR*, 2018.
- [90] J. Dai, H. Qi, et al., "Deformable Convolutional Networks," in *ICCV*, 2017.

- [91] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [92] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [93] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [94] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [95] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [96] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [97] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [98] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [99] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [100] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [101] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.

Thiết kế hàm mất mát (Loss Function Design)

PHỤ THUỘC B/04-nen-tang-hoc-tu-du-lieu (ERM và *inductive bias* — loss là một trong bốn chỗ cài giả định); B/05-thiet-ke-bai-toan (đầu ra và tín hiệu giám sát); A/01-toan-toi-thieu/03-xac-suat-va-thong-tin (KL, entropy, ước lượng độ hợp lý cực đại).

TÓM TẮT

Chương này nói về khoảng cách giữa ba thứ hay bị gọi chung là “mục tiêu”: điều bạn thật sự muốn, phép đo bạn báo cáo, và hàm khả vi bạn thực sự tối ưu. Nó cho một nguyên lý duy nhất sinh ra hầu hết công thức loss: mọi loss là một phát biểu về phân phối. Các họ loss còn lại ra đời vì phát biểu mặc định đó không khớp với một thực tế nào đó. Đọc xong, bạn chọn được loss bằng cách trả lời một câu hỏi về dữ liệu thay vì tra bảng công thức. Bạn chẩn đoán được “loss giảm mà kết quả không tốt lên”. Và bạn nhìn ra chỗ một loss bị *phẳng*, nơi mô hình mù hoàn toàn, trước khi nó kịp gây hại. Nếu chỉ nhớ một điều: loss không phải công thức mà là một phát biểu về nhiễu, về mẫu nào đáng được chú ý, và về loại sai lầm nào đắt hơn; nếu bạn không nói được phát biểu đó thành một câu tiếng Việt thì bạn mới sao chép loss chứ chưa chọn nó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình và tham số** — “mô hình” ở đây là một hàm số có hàng triệu con số điều chỉnh được gọi là tham số (*parameter*, hay trọng số); huấn luyện nghĩa là tìm bộ con số đó. Siêu tham số (*hyperparameter*) thì ngược lại: bạn đặt bằng tay trước khi huấn luyện và thuật toán không tự sửa.
- **gradient và hạ gradient** — gradient là vector đạo hàm của một hàm theo mọi tham số, tức hướng làm hàm đó tăng nhanh nhất. Huấn luyện là lặp đi lặp lại việc đi ngược hướng gradient một bước nhỏ. Hệ quả quan trọng cho cả chương: nơi nào hàm mất mát *phẳng*, gradient bằng 0 và mô hình không nhận được tín hiệu học nào từ chỗ đó.
- **logit và softmax** — logit là điểm số thô mà mạng xuất ra cho mỗi lớp, có thể âm hoặc lớn tùy ý. Softmax là phép biến đổi exp rồi chuẩn hoá, đưa các logit về một bộ số dương cộng lại bằng 1 để đọc được như xác suất.
- **nhãn one-hot** — cách mã hoá nhãn đúng thành một vector toàn số 0 trừ một số 1 ở vị trí lớp đúng. Nó phát biểu “lớp này đúng tuyệt đối, mọi lớp khác sai tuyệt đối” — một phát biểu mạnh hơn thực tế, và vài kỹ thuật trong chương ra đời chính vì điều đó.
- **độ hợp lý (likelihood)** — với một mô hình xác suất và một tập dữ liệu, độ hợp lý là xác suất mà mô hình gán cho chính dữ liệu đã quan sát. “Cực đại hoá độ hợp lý” nghĩa là chọn tham số làm dữ liệu thật trở nên khả dĩ nhất có thể.
- **metric so với hàm mất mát** — metric là con số bạn *báo cáo* (accuracy, mAP, IoU) — thường không khả vi nên không dùng để học được. Hàm mất mát là con số bạn *tối ưu*. Toàn bộ chương này nói về khoảng cách giữa hai thứ đó.
- **IoU và Dice** — IoU (*intersection over union*) là diện tích phần giao chia cho diện tích phần hợp của hai vùng — thước đo chuẩn cho hộp và mặt nạ. Dice là biến thể lấy hai lần phần giao chia cho tổng hai diện tích; cùng tinh thần, khác mẫu số.
- **hiệu chỉnh xác suất (calibration)** — mức độ mà xác suất mô hình xuất ra khớp với tần suất đúng thật sự: nếu mô hình nói “chắc 80%” trên một nghìn mẫu thì khoảng tám trăm mẫu phải đúng. Một mô hình phân loại chính xác vẫn có thể hiệu chỉnh rất tệ, và điều đó quan trọng khi có ngưỡng ra quyết định ở hạ nguồn.

CÂU HỎI MỞ ĐẦU

- Q1. Hai mô hình có cùng giá trị hàm mất mát trên cùng một tập, vì sao một cái dùng được để ra quyết định có ngưỡng còn cái kia thì không?
- Q2. Nếu bạn chưa bao giờ ngồi chọn giả định về phân phối của nhiễu, vì sao chỉ cần gỡ MSE Loss là bạn đã chọn xong rồi?
- Q3. Vì sao một mô hình phân vùng đạt độ chính xác pixel 99% lại có thể chưa từng đoán đúng một pixel tiền cảnh nào?
- Q4. Hai hộp dự đoán cùng trượt khỏi hộp thật, một cái trượt sát và một cái trượt rất xa. Vì sao thước đo chuẩn của bài toán không phân biệt được chúng, và mô hình học được gì từ chỗ đó?
- Q5. Vì sao cổ tình bôi bẩn nhãn đúng — nói rằng lớp đúng chỉ đúng 90% — lại làm mô hình tốt lên ở một khía cạnh và tệ đi ở khía cạnh khác?

7.1 Đặt vấn đề

Mô hình tối ưu chính xác cái bạn viết ra, không phải cái bạn nghĩ trong đầu. Câu này nghe hiển nhiên tới mức nhàm, nhưng nó là nguồn của phần lớn những buổi chiều mà loss giảm đều đặn còn hệ thống thì tệ đi. Gốc rễ nằm ở chỗ có *ba* thứ khác nhau thường bị gọi chung là “mục tiêu”, và chúng nằm trên một chuỗi xấp xỉ mà mỗi bước đều rò rỉ:

- **Mục tiêu thực tế** — giảm số vụ va chạm của con robot. Hầu như không bao giờ khả vi, thường còn không đo được trong lúc huấn luyện: nó là biến cố hiếm, quan sát được sau nhiều tháng, và phụ thuộc cả những khối không phải mạng nơ-ron.
- **Metric đánh giá** — mAP trên tập test. Đo được ngay, nhưng là hàm bậc thang theo tham số: thay trọng số một lượng vô cùng nhỏ thì nó hoặc đứng yên hoặc nhảy một bậc, nên *gradient* của nó bằng 0 gần như mọi nơi và vô dụng cho việc học.
- **Hàm mất mát khả vi** — một tổ hợp *focal* cộng GloU cộng vài số hạng nữa. Trơn, có gradient khác 0 ở mọi chỗ ta cần, và ta *hy vọng* đẩy nó xuống thì kéo metric lên.

Chữ “hy vọng” ở dòng cuối là một phát biểu toán học có tên: tính **nhất quán** (*consistency*) của **hàm thay thế** (*surrogate loss*) — điều kiện để nghiệm tối ưu của hàm thay thế trùng với nghiệm tối ưu của mục tiêu gốc. Khi điều kiện đó không thành lập, bạn đang tối ưu sai thứ một cách có hệ thống, và không có lượng dữ liệu nào cứu được. Đây là chỗ mà chương này nổi thẳng về chương 4: hàm mất mát là một trong bốn chỗ cài *inductive bias*, và cũng như ba chỗ kia, giả định cài vào có thể sai.

→ **tiền đề B/04** — bốn nguồn của inductive bias — kiến trúc, hàm mất mát, dữ liệu, thuật toán tối ưu — được liệt kê ở đó; chương này khai triển nguồn thứ hai.

Câu hỏi dẫn dắt do đó không phải “có những loss nào” mà là: *làm sao viết ra đúng thứ bạn muốn, khi thứ bạn muốn thường không khả vi?*

TRỰC GIÁC

Gradient chỉ nhìn thấy *độ dốc cục bộ* của loss, nên chỗ nào loss phẳng thì chỗ đó mô hình mù hoàn toàn. Hãy đọc mọi hàm mất mát như một bản đồ độ dốc: nó quyết định mẫu nào còn “phát ra tín hiệu” và mẫu nào đã tàng hình. **Hinge loss** cố tình làm phẳng vùng đã vượt lề, nên mẫu phân đúng biến mất khỏi tầm nhìn; **entropy chéo** (*cross-entropy*) không bao giờ phẳng, nên một mẫu đã đúng với xác suất 0,99 vẫn tiếp tục kéo mô hình. Focal loss là hành động ngược lại: *cố ý* bơm vùng phẳng vào chỗ dễ để tín hiệu của mẫu khó nổi lên.

Nhìn theo bản đồ độ dốc thì ba công thức trông rất khác nhau kia trở thành ba lựa chọn của cùng một câu hỏi: “tôi muốn ai còn nói được?”

7.2 Loss là một phát biểu xác suất (Loss as a Probabilistic Statement)

A. Một nguyên lý sinh ra hầu hết công thức

Có hàng chục công thức loss trong các thư viện, và học thuộc chúng là cách tệ nhất để dùng chúng. Có một nguyên lý sinh ra gần hết: **cực đại hoá độ hợp lý** (*maximum likelihood estimation*). Giả sử bạn mô hình hoá dữ liệu bằng một phân phối có điều kiện $p_\theta(y | x)$; khi đó tối thiểu hoá $-\sum_i \log p_\theta(y_i | x_i)$ chính là ước lượng độ hợp lý cực đại, và mọi thứ còn lại chỉ là chọn họ phân phối:

Gaussian $\Rightarrow \frac{1}{2}\|y - \hat{y}\|^2$ (MSE), Laplace $\Rightarrow |y - \hat{y}|$ (MAE), Categorical $\Rightarrow -\log \hat{p}_y$ (CE).

Ý nghĩa của ba dòng trên, nói gọn: *bạn không cần nhớ công thức nào cả, chỉ cần trả lời một câu — tôi giả định phần dư phân bố thế nào?* Nếu bạn tin sai số là Gaussian thì bạn đã chọn MSE dù không biết mình chọn; nếu dữ liệu có ngoại lai nặng đuôi thì giả định Gaussian sai, và MSE để một mẫu hỏng kéo cả mô hình vì nó phạt bậc hai [24].

B. Chính quy hoá là prior, không phải mẹo

Nguyên lý này mở rộng tự nhiên. Thay *MLE* bằng **cực đại hoá hậu nghiệm** (*maximum a posteriori*) thì thêm một số hạng $-\log p(\theta)$: prior Gaussian cho phạt L2, prior Laplace cho phạt L1 và tính thừa đi kèm [68]. Điều này biến câu “thêm *weight decay* cho đỡ *overfitting*” — một câu vô nghĩa về mặt nội dung — thành một phát biểu kiểm chứng được: *tôi tin rằng trước khi nhìn dữ liệu, trọng số nhỏ khả dĩ hơn trọng số lớn*. Và như mọi niềm tin, nó có thể sai: với mạng có **chuẩn hoá** (*normalization*), phạt L2 lên trọng số mất phần lớn nghĩa hình học vì thang của trọng số bị chuẩn hoá triệt tiêu ngay sau đó.

Một hệ quả ít được nói: nếu loss là log-likelihood thì đầu ra của mô hình là *tham số của một phân phối*, không phải một con số. Đi theo hướng đó ta được *heteroscedastic loss* — mạng xuất ra cả $\hat{y}$ lẫn $\log \sigma^2$ và tối thiểu hoá $\frac{(y-\hat{y})^2}{2\sigma^2} + \frac{1}{2} \log \sigma^2$. Số hạng thứ nhất cho phép mô hình giảm phạt ở mẫu nó tự nhận là khó; số hạng thứ hai ngăn nó tuyên bố mọi thứ đều khó. Kết quả là mô hình học luôn độ bất định của chính mình, và độ bất định đó dùng được ở hạ nguồn [69]. Đó đúng là thứ một bộ lọc hợp nhất cảm biến cần, và là chỗ nối tự nhiên nhất giữa chương này với nền tảng ước lượng trạng thái bạn đã có.

C. Từ “chọn công thức” tới “phát biểu giả định”

Mạch 1 — hồi quy, đọc theo giả định chứ không theo tên

1 Vấn đề

Giải pháp

Hệ quả

có quá nhiều công thức loss để nhớ, và không có tiêu chí chọn.

coi mọi loss là $-\log p_\theta(y | x)$ và chỉ chọn họ phân phối.

câu hỏi “dùng MSE hay MAE” trở thành “phần dư của tôi có đuôi nặng không”, một câu trả lời được bằng cách vẽ biểu đồ phần dư thay vì bằng cách thử cả hai rồi chọn con số đẹp hơn.

2 Vấn đề

giả định Gaussian sai khi có ngoại lai, nhưng MAE có gradient hằng nên hội tụ chậm quanh nghiệm.

- Giải pháp** **hàm Huber** (*Huber loss*) — bậc hai trong bán kính δ , tuyến tính ngoài bán kính đó.
- Giả định** ngoại lai là hiếm và nằm xa; δ tách được hai chế độ.
- Giá phải trả** thêm một siêu tham số, và nếu δ đặt sai thì bạn được cái tệ nhất của cả hai. Người làm SLAM nhận ra ngay đây chính là hàm robust quen thuộc trong **hiệu chỉnh chùm tia** (*bundle adjustment*), và trực giác từ đó chuyển sang nguyên vẹn.
- 3 Vấn đề** đôi khi ta không cần một điểm mà cần một khoảng.
- Giải pháp** **quantile loss** — phạt bất đối xứng để mô hình học phân vị thứ τ thay vì kỳ vọng; ba đầu ra ở $\tau = 0,1; 0,5; 0,9$ cho một khoảng dự đoán mà không cần giả định Gaussian nào.
- Vấn đề mới** nếu loss là giả định phân phối thì *prior cũng vậy* → chính quy hoá thôi là mẹo và trở thành một phát biểu có thể sai, có thể kiểm chứng, và có thể gỡ ra khi sai.

7.3 Phân loại: chọn loss là chọn hành vi (Classification: Choosing Behaviour)

A. Hai bản đồ độ dốc trên cùng một bài toán

Bảng 7.1. Hinge và cross-entropy: cùng một bài toán, hai hành vi hoàn toàn khác nhau.

Tiêu chí	Hinge (multiclass SVM)	Cross-entropy (softmax)
Cơ chế	Phạt cho tới khi vượt lề Δ , sau đó gradient tắt hẳn	Phạt – log xác suất đúng, không bao giờ tắt
Giả định ngầm	Chỉ cần phân đúng với một lề an toàn là đủ	Ta cần cả xác suất, không chỉ nhãn
Đầu ra	Điểm số, không diễn giải được như xác suất	Xác suất dùng được ở hạ nguồn
Ngoại lai đã phân đúng	Bị bỏ qua hoàn toàn	Vẫn kéo mô hình
Mạnh khi nào	Cần biên quyết định ổn định, ít quan tâm độ tin cậy	Cần ngưỡng hoá, hợp nhất, hoặc chưng cất về sau
Hổng khi nào	Khi quyết định hạ nguồn cần một xác suất có nghĩa	Nhãn nhiều: mô hình bị kéo vô hạn về phía nhãn sai

Bài học tổng quát vượt ra ngoài hai công thức: *chọn loss là chọn hành vi*. Đặc tính “không bao giờ tắt” của cross-entropy là ưu điểm khi ta cần xác suất được hiệu chỉnh, và là nhược điểm chí mạng khi trong tập có 5 % nhãn sai — mô hình sẽ dành phần lớn sức chứa cuối buổi huấn luyện để ghi nhớ đúng những nhãn sai đó [32].

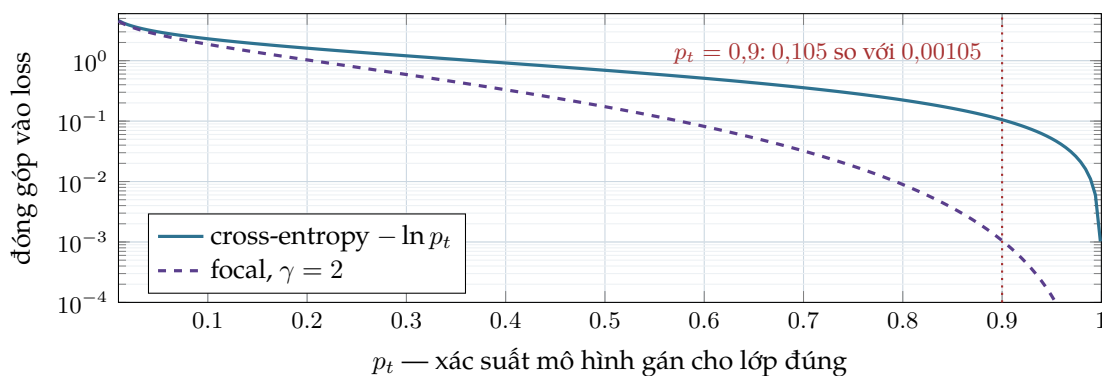
B. Ba cách định hình lại sự chú ý

Cross-entropy cộng đều đóng góp của mọi mẫu. Trong một bài phát hiện vật thể một giai đoạn, số *anchor* nên có thể gấp mười nghìn lần số anchor có vật; dù mỗi anchor nên chỉ đóng góp một lượng nhỏ, tổng của chúng nhấn chìm tín hiệu. Ba sửa chữa phổ biến tấn công ba chỗ khác nhau, và nhằm lẫn giữa chúng là lỗi rất hay gặp.

Mạch 2 — ba cơ chế khác nhau cho ba triệu chứng khác nhau

- 1 Vấn đề** mẫu dễ áp đảo về số lượng.
Giải pháp *focal loss*: nhân CE với $(1 - p_t)^\gamma$ [70].
Giả định đã dễ thì đã học xong, không cần học thêm.
Giá phải trả một siêu tham số γ , và nếu nhân nhiều thì focal *khuếch đại* nhiều, vì một mẫu có nhãn sai luôn trông y hệt một mẫu khó.
- 2 Vấn đề** lớp hiếm ít mẫu — đây là chuyện khác hẳn “mẫu dễ nhiều”.
Giải pháp *class-balanced loss* — cân theo số mẫu hiệu dụng của từng lớp.
Giả định nguyên nhân là tần suất, không phải độ khó hình ảnh.
Hệ quả trên một tập đuôi dài mà mọi lớp đều dễ về mặt thị giác, focal không giúp gì còn class-balanced thì giúp; trên một tập cân bằng nhưng có vài mẫu cực khó thì ngược lại.
- 3 Vấn đề** mô hình quá tự tin, xác suất không dùng được.
Giải pháp *làm mượt nhãn* (label smoothing): thay nhãn one-hot bằng $(1 - \varepsilon)$ cho lớp đúng và $\varepsilon/(C - 1)$ cho phần còn lại [71].
Hệ quả nó đặt trần hữu hạn cho logit đúng nên chấm dứt cuộc đua logit vô hạn.
Giá phải trả nó cũng làm co cụm các lớp trong không gian đặc trưng, và điều đó *làm hại* nếu bạn định dùng chính đặc trưng đó cho truy hồi hoặc chứng cất.

Hình 7.1 vẽ ra điều mà hộp trực giác vừa nói bằng lời: nó cho thấy chính xác vùng nào của trục xác suất bị focal loss “tắt tiếng”.



Hình 7.1. Vì sao focal loss “tắt tiếng” của mẫu dễ. Ở $p_t = 0,9$, hệ số $(1 - p_t)^2 = 0,01$ chia đóng góp cho một trăm lần; ở $p_t = 0,1$ hệ số là 0,81, gần như không đổi. Tỷ lệ chú ý giữa mẫu khó và mẫu dễ do đó bị nhân lên khoảng tám mươi lần. Đó cũng là lý do một nhãn sai được khuếch đại đúng chừng ấy: với mô hình, nhãn sai trông y hệt một mẫu cực khó.

C. Chiều của KL quyết định mô hình sinh ra thứ gì

Một trực thứ tư ít được nêu nhưng quyết định hành vi của cả họ mô hình sinh: *đo khoảng cách giữa hai phân phối theo chiều nào*. $KL(p \parallel q)$ (*forward*, tức cross-entropy quen thuộc) phạt rất nặng khi q gán xác suất gần 0 vào chỗ p có khối lượng, nên nghiệm tối ưu *phủ hết các mode* và bôi trơn qua mọi khả năng. $KL(q \parallel p)$ (*reverse*) chỉ phạt khi q đặt khối lượng vào chỗ p không có, nên nghiệm tối ưu *bám lấy một mode* và bỏ qua phần còn lại. Hệ quả rất cụ thể: hướng phủ mode sinh ra thứ trung bình mờ nhạt, hướng tìm mode sinh ra thứ sắc nét nhưng thiếu đa dạng. Chương 17 dùng lại đúng trục này để giải thích vì sao VAE mờ còn GAN thì sắc mà hay sập mode.

↔ **tiền đề** C/17 — bốn họ mô hình sinh khác nhau ở chỗ chúng chọn chiều nào của KL, hoặc thay KL bằng một định nghĩa “giống” khác hẳn.

7.4 Khi đầu ra có cấu trúc (Structured Outputs)

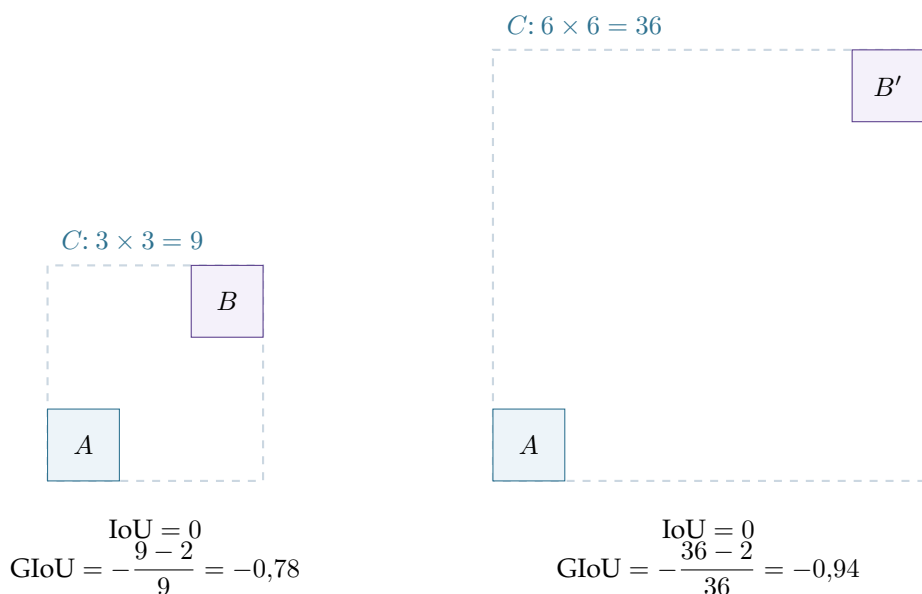
Nếu đầu ra là một mặt nạ, một hộp, một chuỗi hay một tập hợp thì trung bình theo từng phần tử là xấp xỉ tồi của thứ ta thật sự quan tâm. Nguyên tắc chung: *loss phải tôn trọng cấu trúc của đầu ra*, và cách làm điển hình là tìm một xấp xỉ khả vi cho chính metric của bài toán.

A. Phân vùng: vì sao Dice không bị đánh lừa

Với **phân vùng** (*segmentation*), CE theo từng pixel bị chi phối bởi lớp nền. Lấy ví dụ nhỏ nhất tính tay được: ảnh 100×100 với đúng 100 pixel tiền cảnh, tức 1 %. Một mô hình đoán “toàn bộ là nền” đạt độ chính xác pixel 99 % và CE thấp một cách đáng khen, trong khi hệ số **Dice** (*Dice coefficient*) $= 2|A \cap B|/(|A| + |B|) = 2 \cdot 0/(0 + 100) = 0$ — bằng không tuyệt đối [72]. Dice không bị đánh lừa vì nó chỉ đếm phần giao trên tổng diện tích hai mặt nạ, và pixel nền đúng không được cộng điểm. Cái giá là gradient của Dice phụ thuộc *toàn ảnh* chứ không tách được theo từng pixel, nên nó ồn hơn khi *batch* nhỏ và có mẫu số tiến về 0 khi ảnh không chứa tiền cảnh nào. Trong thực hành người ta hầu như luôn dùng tổ hợp CE cộng Dice, còn **Tversky** (*Tversky loss*) là bản tổng quát cho phép đặt giá khác nhau cho dương tính giả và âm tính giả — tức là chọn mặt nạ “béo” hay “gầy” ngay từ công thức, trước khi nhìn kết quả.

B. Hộp: lỗ hổng gradient của IoU và cách GIoU vá nó

Với **hộp giới hạn** (*bounding box*), L1 trên bốn tọa độ không phải thứ ta đo, vì metric là IoU. Nhưng IoU có một lỗ hổng chí mạng và nó tính tay được, như Hình 7.2 cho thấy.



Hình 7.2. Hai dự đoán rời hoàn toàn khỏi hộp thật A. IoU bằng 0 trong cả hai trường hợp, nên loss phẳng và mô hình không biết B tốt hơn B'. GIoU trừ đi phần thừa của hộp bao nhỏ nhất C và nhờ vậy phân biệt được hai tình huống, cho gradient chỉ đúng hướng.

Công thức là $\text{GIoU} = \text{IoU} - (|C| - |A \cup B|)/|C|$ [73], và ý nghĩa của số hạng thứ hai bằng lời: phạt theo mức độ “lãng phí” của hộp bao nhỏ nhất chứa cả hai hộp — càng xa nhau thì hộp bao càng rộng, càng bị phạt nặng. Đây là ví dụ mẫu mực cho cả chương: người ta không phát minh GIoU vì nó đẹp, mà vì họ đi tìm đúng chỗ loss bị phẳng.

C. Chuỗi và tập hợp

Cùng một nguyên tắc sinh ra hai họ nữa. CTC xử lý chuỗi không cần chỉnh bằng cách cộng xác suất trên mọi cách căn chỉnh hợp lệ, biến một bài toán không khả vi vì thiếu nhân từng khung thành một bài toán khả vi. Và **ghép cặp Hungary** (*Hungarian matching*) xử lý đầu ra dạng tập hợp: DETR bỏ hẳn NMS bằng cách ghép một-một tối ưu giữa dự đoán và nhãn *trước* khi tính loss, khiến loss trở nên bất biến với thứ tự [8]. Cả hai minh hoạ cùng một câu. Khi metric của bài toán không có xấp xỉ khả vi sẵn, bạn phải tự thiết kế một cái. Việc thiết kế nằm ở chỗ gỡ đúng điểm không khả vi, không phải ở chỗ làm công thức phức tạp hơn.

7.5 Ba nguồn giám sát ngoài nhãn lớp (Supervision Beyond Class Labels)

Ba họ loss còn lại chia nhau một ý tưởng: tín hiệu giám sát có thể đến từ quan hệ giữa các mẫu, từ vật lý, hoặc từ một mô hình khác.

- **Từ quan hệ giữa các mẫu — *metric learning*.** Thay vì hỏi “ảnh này thuộc lớp nào”, ta hỏi “hai ảnh này có giống nhau không”, và tối ưu để cặp dương gần hơn cặp âm một lẽ nào đó [74], [75]. Hai chi tiết quyết định thành bại, và cả hai đều là chỗ hay hỏng:
 - **Mẫu âm** — nếu toàn mẫu âm dễ thì loss về 0 mà biểu diễn chẳng học được gì; nhưng khai thác mẫu âm khó quá tay thì bạn chủ yếu đang huấn luyện trên các dương giả bị gán nhầm là âm.
 - **Nhiệt độ (*temperature*)** — nó điều khiển độ tập trung của phân phối tương tự; nhiệt độ thấp biến *InfoNCE* thành “chỉ quan tâm mẫu âm khó nhất”, tức là đưa cả loss về sát chế độ hỏng vừa nói.

Đây là nền của toàn bộ tự giám sát hiện đại.

↔ **tiền đề** C/18 — mọi phương pháp tự giám sát hiện đại đều là một lựa chọn cụ thể của cặp “mẫu dương là gì” và “mẫu âm lấy từ đâu”.

- **Từ vật lý — ràng buộc hình học nhúng vào mục tiêu.** Thị giác máy tính có sẵn một kho tri thức không cần học lại: **sai số tái chiếu** (*reprojection error*), **nhất quán trắc quang** (*photometric consistency*) giữa hai khung nhìn, prior trơn cho trường độ sâu. Đưa chúng vào loss là cách nói “đừng học lại hình học, tôi biết rồi”,

↔ **cùng nguyên lý với** C/16 — sai số tái chiếu trong hình học đa khung nhìn và mất mát tự giám sát cho độ sâu là cùng một hàm, chỉ khác chỗ ai giữ biến nào cố định.

và đó chính là cây cầu giữa SLAM cổ điển với học sâu — lý do các phương pháp ước lượng độ sâu tự giám sát huấn luyện được mà không cần một nhãn độ sâu nào. Giả định ngầm — bề mặt Lambert, không có vật chuyển động, phơi sáng không đổi — chính là danh sách các chế độ hỏng của chúng.

- **Từ một mô hình khác. Mất mát tri giác (*perceptual loss*)** so sánh đặc trưng của một mạng đã huấn luyện thay vì so sánh pixel [37], [76]; nó chứa một hiện tượng rất cụ thể. Khi nhiều đáp án đều đúng, loss trung bình theo pixel kéo đầu ra về *trung bình của các đáp án* — mà trung bình của nhiều ảnh sắc nét là một ảnh mờ. Cùng họ, **style loss** qua ma trận Gram cho thấy thống kê bậc hai của đặc trưng mang thông tin kết cấu còn bậc một mang thông tin bố cục [77]. Và **chưng cất tri thức** (*knowledge distillation*) [78] khai thác đúng chỗ nhãn one-hot lãng phí: phân phối mềm của *teacher* nói rằng con mèo này hơi giống con cáo, một thông tin về cấu trúc lớp mà nhãn cứng vứt bỏ hoàn toàn.

7.6 Cân bằng nhiều thành phần (Balancing Multi-Term Losses)

Khi một loss có bốn số hạng, siêu tham số khó nhất của cả hệ thống không nằm trong paper mà nằm trong file cấu hình. Ba triệu chứng cần nhận ra:

- **Thang đo lệch nhau** — một số hạng cỡ 10^{-3} và một số hạng cỡ 10^1 , nên số hạng nhỏ chỉ là trang trí. Kiểm tra bằng cách log riêng từng thành phần và chuẩn của gradient mà nó sinh ra, vì một loss nhỏ vẫn có thể có gradient lớn.
- **Xung đột gradient** — hai số hạng đẩy trọng số về hai hướng có tích vô hướng âm, nên tiến bộ ở nhánh này là thoái bộ ở nhánh kia. Đây là bản chất của học đa nhiệm chứ không phải lỗi cài đặt, nên đừng đi tìm bug.
- **Trọng số không chuyển được** — bộ trọng số tinh chỉnh trên một tập dữ liệu thường vô nghĩa trên tập khác, vì thang của từng thành phần phụ thuộc phân bố dữ liệu.

↔ **ràng buộc bởi** E/24 — hai bộ trọng số chỉ so sánh được khi mọi thứ khác trong giao thức thí nghiệm được giữ cố định — nếu không, bạn đang đo nhiễu.

Có cách đặt trọng số có nguyên lý thay vì dò tay: cho mô hình tự học một σ_k cho mỗi nhánh, rồi cân bằng theo $1/\sigma_k^2$ [69]. Nhưng hãy trung thực rằng đây là kinh nghiệm thực hành chứ không phải định lý. Trong nhiều hệ thống thật, một vòng tìm kiếm ngẫu nhiên trên hai ba trọng số vẫn thắng nó.

Hai khung liên quan đầy ý tưởng đi xa hơn:

- **DRO** — tối thiểu hoá loss của *nhóm tệ nhất* thay vì loss trung bình [61]. Đây là thứ bạn muốn khi trung bình đẹp che giấu một nhóm hỏng hoàn toàn.
- **Cost-sensitive learning** — thổi giá vờ rằng dương tính giả và âm tính giả đắt như nhau. Nếu bỏ sót một người đi bộ đất gấp nghìn lần một cảnh báo thừa, con số đó phải nằm trong loss chứ không phải trong một cuộc họp.

7.7 Thực hành

Repo và tài nguyên. [qubvel-org/segmentation_models.pytorch](https://github.com/qubvel-org/segmentation_models.pytorch) cho Dice, Tversky và focal đã cài sẵn; [KevinMusgrave/pytorch-metric-learning](https://github.com/kevinmusgrave/pytorch-metric-learning) cho toàn bộ họ *contrastive* và các chiến lược khai thác mẫu âm; [lightly-ai/lightly](https://github.com/lightly-ai/lightly) cho InfoNCE; [open-mmlab/mmdetection](https://github.com/open-mmlab/mmdetection) cho GIoU/DIoU/CIoU và Hungarian matching. Với loss nhiều thành phần, hãy đọc code chứ đừng đọc paper: trọng số thật luôn nằm trong config. Danh sách này chốt tại tháng 9/2026 và tên mô hình dẫn đầu sẽ cũ trong 6–12 tháng, trong khi các repo hạ tầng kể trên bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Vì sao một mô hình đạt hinge loss bằng 0 vẫn có thể hiệu chỉnh xác suất rất tệ? Focal loss và class-balanced loss chứa cùng một triệu chứng bằng hai cơ chế khác nhau — khác ở đâu, và tập dữ liệu nào làm lộ ra sự khác biệt? Trong một loss bốn thành phần, làm sao biết thành phần nào đang chi phối gradient?

Mini project. Trên CIFAR-10-LT, huấn luyện cùng một *backbone* với ba cấu hình: CE, CE cộng label smoothing, và focal. Báo cáo ba con số cho mỗi cấu hình — độ chính xác tổng, độ chính xác nhóm đuôi, và ECE. Ba con số kể ba câu chuyện khác nhau, và chỗ chúng mâu thuẫn nhau mới là chỗ đáng viết ra.

7.8 Giới hạn và trường hợp hỏng

Giới hạn lớn nhất là giới hạn khái niệm: loss chỉ là hàm thay thế, và không có gì bảo đảm nó nhất quán với metric, càng không bảo đảm metric nhất quán với mục tiêu thực tế. Khi khe hở đó đủ rộng, mô hình sẽ khai thác nó — đây là *Goodhart's law* phiên bản học máy: khi một phép đo trở thành mục tiêu, nó thôi là một phép đo tốt. Điều đáng sợ không phải mô hình thất bại, mà là nó *thành công* theo cách bạn không tưởng tượng nổi: một mô hình phân vùng đạt Dice cao nhờ phình mặt nạ ra vì bạn đặt Tversky phạt âm tính giả nặng gấp bốn; một mô hình phát hiện đạt mAP cao nhờ đổ ra hàng trăm hộp điểm thấp mà metric không phạt.

Ba trường hợp hỏng cụ thể đáng thuộc:

- **Loss giảm mà metric không lên** — hoặc hàm thay thế không nhất quán, hoặc bạn đang đo trên một phân bố khác với phân bố huấn luyện (chương 8). Hai nguyên nhân này cần hai hành động ngược nhau, nên phải phân biệt trước khi sửa.
- **Loss vượt rồi đột ngột thành NaN** — hầu như luôn là một thành phần có mẫu số tiến về 0: Dice trên ảnh không có tiền cảnh, chia cho σ trong loss heteroscedastic, log của xác suất bằng 0. Phải truy nguyên từ bước *đầu tiên* xuất hiện chứ không phải từ chỗ nó nổ.
- **Loss đúng công thức nhưng sai giả định** — focal trên tập nhãn nhiều, MSE trên dữ liệu đuôi nặng, contrastive với mẫu âm toàn là dương giả. Đây là loại lỗi không bao giờ lộ ra trong đường cong huấn luyện.

CẠM BẮY

Trước khi tin bất kỳ đường cong loss nào, hãy kiểm tra giá trị tại lúc khởi tạo. Với C lớp và mô hình chưa học gì, cross-entropy kỳ vọng phải bằng $\ln C$: $\ln 10 \approx 2,30$ cho CIFAR-10, $\ln 1000 \approx 6,91$ cho ImageNet. Nếu epoch 0 in ra 0,4 thì bạn đang rò rỉ nhãn hoặc đánh giá nhầm tập; nếu in ra 15 thì logit đang nổ hoặc chỉ số nhãn lệch. Phép kiểm tra này mất ba mươi giây và bắt được nhiều lỗi hơn bất kỳ thứ gì khác cùng chi phí. Kèm theo nó là hai thói quen: tắt hết chính quy hoá để nhìn riêng *data loss*, và log từng thành phần riêng thay vì chỉ log tổng.

CẦN NHỚ

Loss không phải là công thức mà là một *phát biểu*: về phân phối của nhiễu, về việc mẫu nào đáng được chú ý, và về việc loại sai lầm nào đắt hơn. Nếu bạn không nói được phát biểu đó thành một câu tiếng Việt thì bạn chưa chọn loss — bạn mới sao chép nó. Và ở chỗ nào loss phẳng, ở chỗ đó mô hình mù. [T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Hàm thay thế	Tiếng Anh là <i>surrogate loss</i> : hàm khả vi ta thực sự tối ưu, đứng thay cho metric vốn không khả vi. Nó chỉ hợp lệ khi nghiệm tối ưu của nó trùng nghiệm tối ưu của mục tiêu gốc; điều đó gần như không bao giờ được chứng minh trong thị giác.
••• Vì sao không tối ưu thẳng metric như mAP?	Vì metric là hàm bậc thang theo tham số: đổi trọng số một lượng vô cùng nhỏ thì nó hoặc đứng yên hoặc nhảy một bậc. Gradient của nó bằng 0 gần như mọi nơi nên vô dụng cho việc học.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
••• Cực đại hoá độ hợp lý	Chọn tham số làm dữ liệu đã quan sát trở nên khả dĩ nhất. Nó sinh ra hầu hết công thức loss: nhiều Gaussian cho MSE, Laplace cho MAE, Categorical cho entropy chéo.
•• Hàm Huber	Bậc hai trong bán kính δ , tuyến tính ngoài bán kính đó, nên một ngoại lai không kéo được cả mô hình. Giả định là ngoại lai hiếm và nằm xa; đặt δ sai thì bạn được cái tệ nhất của cả MSE lẫn MAE.
•• Heteroscedastic loss	Mạng xuất ra cả giá trị dự đoán lẫn phương sai của chính nó, và phương sai đó vào thẳng mẫu số của số hạng phạt. Kết quả là mô hình học luôn độ bất định của mình, dùng được cho khâu hợp nhất cảm biến ở hạ nguồn.
•• Vì sao hinge và cross-entropy hành xử khác nhau?	Hinge tắt gradient ngay khi mẫu vượt lề, nên mẫu đã đúng biến mất khỏi tầm nhìn. Cross-entropy không bao giờ tắt, nên nó cho xác suất hiệu chỉnh tốt hơn nhưng bị nhãn sai kéo vô hạn.
••• Focal loss	Nhân entropy chéo với $(1 - p_t)^\gamma$ để dim đóng góp của mẫu dễ. Ở $p_t = 0,9$ với $\gamma = 2$, hệ số là 0,01, tức chia đóng góp cho một trăm lần. Nó hỏng khi nhãn nhiều, vì nhãn sai trông y hệt mẫu khó.
•• Vì sao focal loss không chứa được tập đuôi dài?	Vì focal cân theo <i>độ khó của từng mẫu</i> , còn đuôi dài là vấn đề <i>tần suất của từng lớp</i> . Nếu mọi lớp đều dễ về thị giác thì focal không đổi gì; phải dùng loss cân theo số mẫu hiệu dụng.
•• Label smoothing	Thay nhãn one-hot bằng $(1 - \varepsilon)$ cho lớp đúng và chia đều ε cho phần còn lại. Nó đặt trần hữu hạn cho logit đúng nên hiệu chỉnh xác suất tốt lên, nhưng làm các lớp co cụm lại nên hại cho truy hồi và chưng cất.
• Reverse KL	Chỉ phạt khi mô hình đặt khối lượng vào chỗ dữ liệu không có, nên nghiệm tối ưu bám lấy một mode. Hệ quả: đầu ra sắc nét mà thiếu đa dạng, ngược hẳn với forward KL vốn phủ hết mode và cho ảnh mờ.
••• Hệ số Dice	Hai lần phần giao chia cho tổng diện tích hai mặt nạ. Nó không cộng điểm cho pixel nền đúng, nên đoán “toàn nền” cho 99% độ chính xác pixel nhưng Dice bằng 0. Mẫu số tiến về 0 khi ảnh không có tiền cảnh.
••• GIoU	IoU trừ đi tỉ lệ phần rỗng của hộp bao nhỏ nhất chứa cả hai hộp. Nó vá đúng chỗ IoU bị phẳng: với hai hộp rời nhau, IoU đều bằng 0 còn GIoU cho $-0,78$ và $-0,94$, nên gradient lại chỉ đúng hướng.
•• Ghép cặp Hungary	Ghép một-một tối ưu giữa dự đoán và nhãn <i>trước</i> khi tính loss, khiến loss bất biến với thứ tự. Đây là thứ cho phép DETR bỏ hẳn bước triết phi cực đại.
•• InfoNCE	Loss đối chiếu một mẫu dương với nhiều mẫu âm trong một không gian nhúng. Hai thứ quyết định thành bại là nguồn mẫu âm và nhiệt độ; mẫu âm quá dễ thì loss về 0 mà không học được gì.
•• Vì sao loss theo pixel cho ảnh mờ?	Khi nhiều đáp án đều đúng, trung bình bình phương kéo đầu ra về <i>trung bình của các đáp án</i> , mà trung bình của nhiều ảnh sắc nét là một ảnh mờ. Mất mát tri giác và loss đối kháng ra đời để thoát khỏi chính hiện tượng này.
• Chưng cất tri thức	Huấn luyện theo phân phối mềm của một mô hình lớn thay vì theo nhãn cứng. Phân phối mềm chứa thông tin về sự tương tự giữa các lớp mà nhãn one-hot vứt bỏ hoàn toàn.
•• Vì sao trọng số của loss nhiều thành phần khó chỉnh?	Vì thang của từng số hạng phụ thuộc phân bố dữ liệu, nên bộ trọng số tinh chỉnh trên tập này thường vô nghĩa trên tập khác. Ngoài ra hai số hạng có thể xung đột gradient, và đó là bản chất của học đa nhiệm chứ không phải lỗi cài đặt.
••• Epoch 0 in ra loss 6,5 trên CIFAR-10 — nghĩa là gì?	Sai, vì giá trị kỳ vọng lúc khởi tạo phải là $\ln 10 \approx 2,30$. Cao hơn nhiều nghĩa là logit đang nổ hoặc chỉ số nhãn lệch; thấp bất thường thì ngược lại, nghi ngờ rò rỉ nhãn.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Loss giảm mượt rồi đột ngột thành NaN — nguyên nhân?	Gần như luôn là một thành phần có mẫu số tiến về 0: Dice trên ảnh không có tiền cảnh, chia cho σ trong loss heteroscedastic, hoặc log của xác suất bằng 0. Truy nguyên từ bước <i>đầu tiên</i> xuất hiện, không phải từ chỗ nó nổ.
●● Loss giảm đều mà mAP đứng yên — nguyên nhân?	Hoặc hàm thay thế không nhất quán với metric, hoặc bạn đang đo trên một phân bố khác với phân bố huấn luyện. Phân biệt bằng cách đo lại metric ngay trên chính tập huấn luyện: nếu ở đó metric cũng đứng yên thì lỗi nằm ở loss.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
<code>mmdetection</code> , <code>detectron2</code>	Focal loss cho phân loại anchor và họ GIoU/DIoU/CIoU cho hồi quy hộp là mặc định của gần như mọi detector một giai đoạn	Hạ tầng, ít hết hạn; ý tưởng loss bền hơn tên kiến trúc
Dòng DETR (Deformable DETR, DINO-DETR)	Ghép cặp Hungary biến phát hiện vật thể thành bài toán dự đoán tập hợp và bỏ được triết phi cực đại	Phần được kế thừa rộng rãi là loss, không phải kiến trúc
<code>nnU-Net</code>	Tổ hợp entropy chéo cộng Dice là cấu hình mặc định, chính vì lý do mất cân bằng nền đã phân tích ở chương	Vẫn là đối chứng mạnh cho ảnh y sinh tính tới 9/2026
CLIP, DINOv2/DINOv3	Toàn bộ tín hiệu huấn luyện là contrastive: InfoNCE với nhiệt độ học được, không có nhân phân lớp nào	Nền của phần lớn mô hình nền tảng thị giác hiện nay
Bộ mã hoá tự động của các mô hình khuếch tán tiềm ẩn	Mất mát tri giác cộng một số hạng đối kháng là lý do ảnh giải mã không bị mờ như khi chỉ dùng MSE	Đây đúng là hiện tượng “trung bình của nhiều đáp án” đã nêu trong chương

MINI PROJECT SLAM · Hai hàm mất mát cho cùng một bài ghép cặp đặc trưng ORB · 1–2 ngày

Mục tiêu	kiểm chứng bằng số luận điểm trung tâm của chương — chọn loss là chọn hành vi — trên đúng bài toán bạn đang làm: quyết định hai mảnh ảnh quanh hai đặc trưng ORB có ứng với cùng một điểm 3D hay không.
Dữ liệu	một chuỗi EuRoC; sinh cặp dương và cặp âm từ chân trị pose cộng độ sâu stereo, hoặc đơn giản hơn là lấy các ghép cặp đã sống sót qua kiểm định hình học của chính ORB-SLAM3 làm cặp dương. Cắt mảnh 32×32 quanh mỗi đặc trưng; vài chục nghìn cặp là đủ.
Các bước	huấn luyện một CNN nhỏ theo hai cách, giữ nguyên backbone và mọi siêu tham số, chỉ đổi đầu ra và hàm mất mát: (1) đầu phân loại nhị phân với entropy chéo trên nhãn “cùng điểm / khác điểm”; (2) đầu nhúng với loss có biên — triplet hoặc InfoNCE. Sau đó tính khoảng cách trong không gian nhúng cho tập kiểm tra và vẽ hai biểu đồ phân bố, một cho cặp đúng và một cho cặp sai. Lặp lại phiên bản (2) với khai thác mẫu âm khó và vẽ lại.
Xong khi	bạn có ba con số cho mỗi cấu hình — tỉ lệ dương giả tại ngưỡng cho recall 95%, diện tích chồng lấn của hai phân bố, và khoảng cách giữa phân vị 5% của cặp sai với phân vị 95% của cặp đúng — và nói được loss nào tách tốt hơn ở <i>đuôi</i> , tức đúng vùng mà quyết định ghép cặp thật sự xảy ra.
Nối với	C/18 cho vai trò của mẫu âm và nhiệt độ; B/10 vì kết quả phụ thuộc cả backbone chứ không riêng loss; B/11 nếu bạn định chạy bộ mô tả này thời gian thực.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai mô hình có thể cùng giá trị loss mà khác nhau ở độ hiệu chỉnh xác suất: hinge tất gradient ngay khi vượt lề nên điểm số của nó không có nghĩa xác suất, còn cross-entropy là proper scoring rule nên xác suất của nó đọc được. Quyết định có ngưỡng cần thứ hai, không cần thứ nhất.
- A2. Vì MSE là log-likelihood của một mô hình nhiễu Gaussian. Bạn không tránh được việc chọn giả định; bạn chỉ chọn nó một cách vô thức, và giả định đó sai ngay khi phần dư có đuôi nặng.
- A3. Vì entropy chéo theo pixel cộng điểm cho mọi pixel nên đoán đúng, mà nên chiếm 99%. Dice không cộng điểm cho nên nên nó bằng 0 tuyệt đối trong đúng tình huống đó — hai hàm đo hai thứ khác nhau trên cùng một dự đoán.
- A4. Vì IoU bằng 0 cho mọi cặp hộp rời nhau, bất kể xa gần: loss phẳng nên gradient bằng 0 và mô hình không nhận được tín hiệu nào về hướng phải đi. GIoU trừ thêm phần rỗng của hộp bao nhỏ nhất, cho $-0,78$ so với $-0,94$ trong ví dụ ở chương, nên hướng lại xuất hiện.
- A5. Label smoothing đặt một trần hữu hạn cho logit đúng nên chấm dứt cuộc đua logit vô hạn, và độ hiệu chỉnh xác suất tốt lên. Cái giá là các lớp bị co cụm trong không gian đặc trưng, nên chính đặc trưng đó dùng cho truy hồi hay chứng cất thì tệ đi.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn muốn một đứa trẻ dọn phòng cho sạch, nhưng “sạch” thì không đo được, nên bạn treo thưởng theo số món đồ được cất vào tủ. Đứa trẻ rất chăm và tuyệt đối theo chữ nghĩa: nó nhét cả rác vào tủ, còn những góc bạn quên tính điểm thì nó không đụng tới.

Máy học đúng như vậy. Thứ bạn thật sự muốn — robot không đâm vào ai — không đo được trong lúc dạy, còn máy thì chỉ hiểu một thứ duy nhất là một con số càng nhỏ càng tốt.

Mẹo thì đơn giản: đừng đi nhớ công thức, hãy trả lời một câu hỏi về dữ liệu của bạn — “sai số của tôi thường trông thế nào?”. Nếu sai số thường nhỏ và cân đối thì ra một công thức; nếu thỉnh thoảng có một mẫu hỏng nặng thì ra công thức khác. Mỗi câu trả lời là một công thức.

Cái giá gồm hai phần. Thứ nhất, máy tối ưu đúng con số bạn viết chứ không phải ý bạn định, nên chỗ nào bạn viết cầu thả thì nó thắng còn bạn thua. Thứ hai, chỗ nào con số không thay đổi thì máy hoàn toàn mù: hai cái hộp cùng đoán trượt, một cái sát và một cái rất xa, mà cùng bị chấm không điểm, thì máy không có cách nào biết cái nào tốt hơn.

Chỗ không rút gọn được: không ai chứng minh được rằng đẩy con số thay thế xuống thì thứ bạn thật sự muốn cũng tốt lên. Với hầu hết công thức đang dùng trong thị giác máy tính, đây là niềm tin dựa trên kinh nghiệm chứ không phải một định lý. Tôi không biết cách nói đơn giản nào che được chỗ hờ này mà không nói dối.

Một câu: Máy làm đúng thứ bạn viết ra, nên phần khó không nằm ở chỗ dạy máy mà ở chỗ viết cho đúng.

7.9 Kết nối

Chương này là hệ quả trực tiếp của chương 4: loss là một trong bốn chỗ cài inductive bias, và mọi thứ vừa bàn chỉ là những cách khác nhau để phát biểu “tôi biết gì về lời giải”. Nó là *đối tác* của chương 10: kiến trúc cài giả định về *dạng* của hàm, loss cài giả định về *mục tiêu*; một giả định cài sai ở chỗ này thường phải trả giá ở chỗ kia. Và nó là *tiền đề ngược* của chương 7 và 8: nếu giao thức đánh giá sai thì việc loss có nhất quán với metric hay không cũng thành vô nghĩa.

Đọc tiếp theo bốn hướng. C/12-co-che-huan-luyen cho biết gradient của những loss này thực sự chảy thế nào và vì sao vài loss khó tối ưu dù đúng về nguyên lý. C/14-bai-toan-cv-loi là nơi focal, GIoU, Dice và Hungarian matching xuất hiện trong ngữ cảnh thật, kèm lý do lịch sử vì sao từng cái ra đời. C/17-mo-hinh-sinh phát triển đoạn về chiều của KL thành bốn triết lý khác nhau của chữ “giống dữ liệu thật” [79]. Cuối cùng, E/25-do-tin-cay-va-van-hanh nhặt lại sợi chỉ hiệu chỉnh xác suất mà label smoothing và *proper scoring rule* vừa thả ra [40].

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao “loss giảm nhưng mAP không tăng” là triệu chứng có *nhiều* nguyên nhân, và ba phép đo nào phân biệt được chúng?
2. Focal loss giả định gì về quan hệ giữa “mẫu khó” và “mẫu có nhãn sai”? Giả định đó bị vi phạm khi nào và hậu quả định lượng là bao nhiêu?
3. Cho hai hộp dự đoán cùng có IoU bằng 0 với hộp thật, vì sao GIoU phân biệt được chúng còn IoU thì không, và hai con số cụ thể là bao nhiêu?
4. Nếu bạn thay MSE bằng MAE mà kết quả tệ đi, điều đó nói gì về phân bố phần dư của bài toán?
5. Trong loss bốn thành phần, vì sao log giá trị từng thành phần là chưa đủ, và bạn cần log thêm cái gì?
6. Label smoothing cải thiện hiệu chỉnh xác suất nhưng làm hại điều gì, và vì sao điều đó quan trọng nếu bạn định dùng đặc trưng cho truy hồi?
7. Nêu một trường hợp trong hệ thống của bạn mà mục tiêu thực tế, metric và loss lệch nhau — và cái giá của khe hở đó là gì?

Nguồn

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *Rec-Sys*, 2019.
- [3] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [4] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [5] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [6] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [7] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [8] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [9] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [10] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [11] V. N. Vapnik, “An overview of statistical learning theory,” *IEEE Trans. Neural Networks*, vol. 10, no. 5, pp. 988–999, 1999.
- [12] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [13] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [15] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [16] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [17] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [18] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE Trans. Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.
- [19] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.

- [20] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [21] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [22] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [23] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [24] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [25] R. Geirhos, J.-H. Jacobsen, C. Michaelis, et al., “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.
- [26] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [27] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [28] T. Gebru, J. Morgenstern, et al., “Datasheets for Datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [29] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [30] M. Mitchell, S. Wu, et al., “Model Cards for Model Reporting,” in *FAT**, 2019.
- [31] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [32] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [33] C. G. Northcutt, L. Jiang, and I. L. Chuang, “Confident Learning: Estimating Uncertainty in Dataset Labels,” *JAIR*, vol. 70, pp. 1373–1411, 2021.
- [34] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [35] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [36] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [37] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [38] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [39] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [40] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [41] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London: Butterworths, 1979.
- [42] B. W. Matthews, “Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme,” *Biochimica et Biophysica Acta (BBA) — Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
- [43] D. Chicco and G. Jurman, “The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation,” *BMC Genomics*, vol. 21, p. 6, 2020.

- [44] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in *International Conference on Machine Learning (ICML)*, 2006, pp. 233–240.
- [45] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.
- [46] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [47] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [48] M. P. Naeni, G. F. Cooper, and M. Hauskrecht, "Obtaining Well Calibrated Probabilities Using Bayesian Binning," in *AAAI Conference on Artificial Intelligence*, 2015.
- [49] G. W. Brier, "Verification of Forecasts Expressed in Terms of Probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
- [50] A. H. Murphy, "A New Vector Partition of the Probability Score," *Journal of Applied Meteorology*, vol. 12, no. 4, pp. 595–600, 1973.
- [51] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [52] T.-Y. Lin, M. Maire, S. Belongie, et al., "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.
- [53] J. Hosang, R. Benenson, P. Dollár, et al., "What Makes for Effective Detection Proposals?" *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 4, pp. 814–830, 2016.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] F. Perazzi, J. Pont-Tuset, B. McWilliams, et al., "A Benchmark Dataset and Evaluation Methodology for Video Object Segmentation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [56] G. Csurka, D. Larlus, and F. Perronnin, "What Is a Good Evaluation Measure for Semantic Segmentation?" In *British Machine Vision Conference (BMVC)*, 2013.
- [57] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [58] K. Mikolajczyk and C. Schmid, "A Performance Evaluation of Local Descriptors," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 10, pp. 1615–1630, 2005.
- [59] M. Dusmanu, I. Rocco, T. Pajdla, et al., "D2-Net: A Trainable CNN for Joint Description and Detection of Local Features," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [60] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [61] S. Sagawa, P. W. Koh, T. B. Hashimoto, et al., "Distributionally Robust Neural Networks for Group Shifts," in *ICLR*, 2020.
- [62] M. Arjovsky, L. Bottou, I. Gulrajani, et al., "Invariant Risk Minimization," *arXiv:1907.02893*, 2019.
- [63] I. Gulrajani and D. Lopez-Paz, "In Search of Lost Domain Generalization," in *ICLR*, 2021.
- [64] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," in *ICLR*, 2019.

- [65] R. Taori, A. Dave, V. Shankar, et al., "Measuring Robustness to Natural Distribution Shifts in Image Classification," in *NeurIPS*, 2020.
- [66] D. Hendrycks and K. Gimpel, "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks," in *ICLR*, 2017.
- [67] Y. Ovadia, E. Fertig, et al., "Can You Trust Your Model's Uncertainty?" In *NeurIPS*, 2019.
- [68] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [69] A. Kendall, Y. Gal, and R. Cipolla, "Multi-Task Learning Using Uncertainty to Weigh Losses," in *CVPR*, 2018.
- [70] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [71] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., "Rethinking the Inception Architecture for Computer Vision," in *CVPR*, 2016.
- [72] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation," in *3DV*, 2016.
- [73] H. Rezatofighi, N. Tsoi, J. Gwak, et al., "Generalized Intersection over Union," in *CVPR*, 2019.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in *CVPR*, 2015.
- [75] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [76] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [77] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [78] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [79] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [80] Y. LeCun, L. Bottou, Y. Bengio, et al., "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [81] F. Yu and V. Koltun, "Multi-Scale Context Aggregation by Dilated Convolutions," in *ICLR*, 2016.
- [82] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention Is All You Need," in *NeurIPS*, 2017.
- [83] Z. Liu, Y. Lin, Y. Cao, et al., "Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows," in *ICCV*, 2021.
- [84] K. He, X. Zhang, S. Ren, et al., "Deep Residual Learning for Image Recognition," in *CVPR*, 2016.
- [85] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *ICML*, 2015.
- [86] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," *arXiv:1607.06450*, 2016.
- [87] Y. Wu and K. He, "Group Normalization," in *ECCV*, 2018.
- [88] S. Santurkar, D. Tsipras, A. Ilyas, et al., "How Does Batch Normalization Help Optimization?" In *NeurIPS*, 2018.
- [89] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in *CVPR*, 2018.
- [90] J. Dai, H. Qi, et al., "Deformable Convolutional Networks," in *ICCV*, 2017.

- [91] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [92] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [93] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [94] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [95] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [96] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [97] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [98] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [99] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [100] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [101] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.

Nguyên lý thiết kế kiến trúc (Architecture Design Principles)

PHỤ THUỘC B/04-nen-tang-hoc-tu-du-lieu (inductive bias và trục “giả định mạnh + ít dữ liệu ↔ giả định yếu + nhiều dữ liệu” — chương này chính là trục đó áp cho kiến trúc); B/09-thiet-ke-ham-mat-mat (chỗ cài giả định thứ hai); A/01-toan-toi-thieu/04-hinh-hoc-va-tin-hieu (convolution, lấy mẫu).

TÓM TẮT

Chương này giải thích vì sao không thể dùng một MLP khổng lồ cho mọi thứ, và vì sao mỗi kiến trúc thực chất là một tập giả định về cấu trúc dữ liệu được đông cứng thành mã nguồn. Nó xây năm câu hỏi để mổ xẻ một kiến trúc lạ, kèm phép tính tay ra con số cho từng câu: giả định nào được cài vào, mô hình nhìn được bao xa, thông tin định tuyến cố định hay theo nội dung, gradient chảy thế nào, và bốn thước đo chi phí rơi vào đâu. Đọc xong, bạn dự đoán được một lớp lỗi trước khi huấn luyện, tức trường tiếp nhận không phủ hết vật thể. Bạn giải thích được vì sao thêm độ sâu và thêm độ rộng có giá hoàn toàn khác nhau. Và bạn nhận ra khi nào một bảng so sánh kiến trúc thực ra đang so sánh công thức huấn luyện. Nếu chỉ nhớ một điều: kiến trúc không làm mô hình “thông minh hơn”, nó quyết định *ai chạm được tới ai, sau bao nhiêu bước, và với chi phí nào*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tensor và feature map** — tensor là mảng số nhiều chiều — ảnh đầu vào là tensor $C \times H \times W$ (kênh, cao, rộng). Feature map là tensor ở giữa mạng: mỗi “kênh” của nó là một bản đồ cho biết một loại đặc trưng xuất hiện mạnh ở vị trí nào.
- **tầng nối đầy đủ và convolution** — tầng nối đầy đủ (*fully connected*) nhân toàn bộ đầu vào với một ma trận — mọi đầu ra nhìn thấy mọi đầu vào. Convolution trượt một cửa sổ nhỏ (kernel) khắp ảnh và dùng *cùng* bộ trọng số ở mọi vị trí, nên mỗi đầu ra chỉ nhìn thấy một vùng nhỏ.
- **kernel, stride, padding** — kernel là kích thước cửa sổ trượt (thường 3×3); stride là bước nhảy giữa hai vị trí liên tiếp (stride 2 làm ảnh nhỏ đi một nửa); padding là số pixel đệm thêm ở viền để kiểm soát kích thước đầu ra.
- **hàm kích hoạt (activation function)** — hàm áp lên từng phần tử giữa hai tầng tuyến tính, ví dụ ReLU $\max(0, x)$. Không có nó thì xếp chồng bao nhiêu tầng tuyến tính cũng chỉ tương đương một tầng tuyến tính — đó là lý do nó bắt buộc phải có.
- **lan truyền ngược và luồng gradient** — để cập nhật trọng số ở tầng đầu, đạo hàm phải nhân dồn ngược qua tất cả các tầng phía sau. Chuỗi phép nhân dài đó có thể tiêu biến hoặc bùng nổ, và đó là lý do “đường về của gradient” là một tiêu chí thiết kế thật sự chứ không phải ẩn dụ.
- **tham số với FLOPs** — tham số là số trọng số phải lưu; FLOPs là số phép nhân-cộng phải làm cho một lần chạy. Hai đại lượng độc lập với nhau: convolution có ít tham số nhưng nhiều FLOPs vì cùng bộ trọng số được dùng lại ở mọi vị trí.
- **inductive bias** — tập giả định mà bạn cài sẵn vào hệ thống về hình dạng của lời giải, trước khi nhìn dữ liệu. Nó đến từ bốn chỗ: kiến trúc, hàm mất mát, augmentation, và thuật toán tối ưu. Bạn không thể “không có giả định” — chỉ có thể chọn giả định nào.
- **batch** — số mẫu được xử lý cùng lúc trong một bước cập nhật. Nó ảnh hưởng tới bộ nhớ, tới độ ổn định của gradient, và — như phần cuối chương cho thấy — tới cả độ trễ đo được.

CÂU HỎI MỞ ĐẦU

- Q1. Một MLP đủ rộng xấp xỉ được mọi hàm liên tục — vậy vì sao không một hệ thống thị giác nghiêm túc nào dùng MLP cho ảnh?
- Q2. Trước ResNet, mạng 56 tầng có sai số *huấn luyện* cao hơn mạng 20 tầng. Vì sao đó chắc chắn không phải *overfitting*, và cái gì thực sự hỏng?
- Q3. Nếu *attention* mạnh hơn convolution ở mọi mặt lý thuyết, vì sao thể hệ ViT tiếp theo lại phải cắt nó về thành cục bộ trong từng cửa sổ?
- Q4. Hai mạng cùng số tham số, một sâu gấp đôi và một rộng gấp đôi — vì sao cái rẻ hơn về tham số lại là cái đắt hơn về thời gian chạy?
- Q5. Bạn đọc một bảng nói kiến trúc mới hơn kiến trúc cũ 3 điểm mAP. Vì sao con số đó có thể không nói gì về kiến trúc cả?

8.1 Đặt vấn đề

Một mạng nơ-ron đủ rộng xấp xỉ được mọi hàm liên tục; định lý xấp xỉ phổ quát nói vậy từ những năm 1980. Nếu thế thì vì sao không dùng một **MLP** khổng lồ cho mọi thứ và để dữ liệu lo phần còn lại? Câu trả lời không nằm ở khả năng biểu diễn mà nằm ở *chi phí học*: định lý nói hàm đúng *tồn tại* trong không gian giả thuyết, nó không nói bạn tìm được hàm đó với lượng dữ liệu và lượng tính toán bạn có. Kiến trúc là công cụ thu hẹp không gian giả thuyết lại quanh những hàm khả dĩ, và mọi lần thu hẹp đều là một phát biểu về cấu trúc của dữ liệu.

Ba ví dụ để thấy phát biểu đó cụ thể tới mức nào:

- **Convolution** phát biểu rằng đặc trưng hữu ích thì *cục bộ*, và một đặc trưng có nghĩa ở góc trên trái thì cũng có nghĩa y như vậy ở góc dưới phải.
- **Pooling** phát biểu rằng vị trí chính xác ít quan trọng hơn sự hiện diện.
- **Attention** phát biểu rằng quan hệ giữa hai vị trí nên do *nội dung* quyết định chứ không do khoảng cách.

Nếu phát biểu khớp với dữ liệu, bạn được tổng quát hoá tốt với ít mẫu; nếu không khớp, bạn đã cài sẵn một sai lệch mà không lượng dữ liệu nào gỡ được. Đây chính xác là trục ở chương 4: giả định mạnh đối lấy ít dữ liệu. Toàn bộ lịch sử kiến trúc thị giác đọc được như một chuỗi các lần *nói lòng* giả định, mỗi lần dữ liệu và tính toán rẽ đi một bậc.

↪ **tiền đề B/04** — trục đánh đổi giả định–dữ liệu là thứ giải thích vì sao ViT cần tiền huấn luyện quy mô lớn còn CNN thì không.

Giá trị của chương: sau khi đọc, bạn không cần nhớ tên kiến trúc nào cũng đọc được một bài báo kiến trúc lạ, vì bạn có năm câu hỏi cố định và ba phép tính tay để tự trả lời chúng.

TRỰC GIÁC

Hãy nghĩ về kiến trúc như hệ thống ống dẫn chứ không như một cái hộp đen: nó quyết định *thông tin nào có thể chạm tới thông tin nào, sau bao nhiêu bước*. Một MLP là bể chứa — mọi thứ chạm được mọi thứ ngay lập tức, nên nó không có cấu trúc để dựa vào và phải trả bằng tham số. Convolution là mạng ống ngăn nối các điểm lân cận, nên hai pixel ở hai góc ảnh chỉ “nói chuyện” được sau nhiều tầng. Nếu số tầng không đủ, chúng *không bao giờ* nói chuyện được, dù bạn huấn luyện bao lâu. **Kết nối tắt** (*residual connection*) là đường ống phụ chạy song song, đặt ở đó không phải để mang thêm thông tin xuôi mà để **gradient** có lối về ngấn. Đọc kiến trúc theo hình dạng đường ống thì receptive field, attention và skip

connection thôi là ba khái niệm rời rạc và trở thành ba câu trả lời cho cùng một câu hỏi: ai chạm được tới ai.

8.2 Từ template matching tới convolution (From Template Matching to Convolution)

A. Điểm xuất phát nhìn được bằng mắt

Điểm xuất phát đáng ngạc nhiên là bộ phân loại tuyến tính $f = Wx + b$. Cách đọc bề bợt không phải đại số mà là hình học: mỗi hàng của W là một **mẫu chuẩn** (*template*) có cùng số chiều với ảnh, và điểm số của một lớp là tích vô hướng giữa ảnh và mẫu chuẩn đó. Ý nghĩa bằng lời: *mô hình chấm điểm bằng cách hỏi ảnh này giống bản mẫu của lớp đến đâu*. Vẽ các hàng của W ra thành ảnh sau khi huấn luyện trên CIFAR-10 là bài tập rất đáng làm. Bạn sẽ thấy mẫu “ngựa” có hai đầu. Lý do là mỗi lớp chỉ có *một* mẫu chuẩn, nên nó buộc phải trung bình mọi tư thế lại. Giới hạn của mô hình lộ ra từ hình ảnh đó, trước cả khi bạn đọc con số accuracy.

B. Ba lần thu hẹp không gian giả thuyết

Mạch 1 — vì sao convolution ra đời, và nó lấy đi cái gì

- 1 Vấn đề** một mẫu chuẩn cho mỗi lớp là quá ít, và biên quyết định chỉ là siêu phẳng.

Giải pháp xếp chồng nhiều tầng tuyến tính xen phi tuyến, tức MLP.

Giả định hàm cần học liên tục và mạng đủ rộng.

Giá phải trả số tham số nổ tung — ảnh $224 \times 224 \times 3 = 150\,528$ chiều nối đầy đủ vào 1000 nơ-ron là ≈ 150 triệu tham số cho *một* tầng.

Vấn đề mới tệ hơn cả chi phí, mô hình không hề biết hai pixel cạnh nhau thì liên quan, nên phải học lại sự thật đó từ dữ liệu, và học lại nó riêng cho từng vị trí.
- 2 Vấn đề** quá nhiều tham số và không có chút tri thức nào về ảnh.

Giải pháp **chia sẻ trọng số** (*weight sharing*) cộng tính cục bộ, tức convolution [80].

Hệ quả ta được **đẳng biến tịnh tiến** (*translation equivariance*) gần như miễn phí, và số tham số giảm hàng nghìn lần: một tầng conv 3×3 từ 3 kênh sang 64 kênh chỉ có $3 \cdot 3 \cdot 3 \cdot 64 = 1728$ trọng số, *bất kể ảnh to bao nhiêu*.

Giá phải trả bất biến với xoay và với tỉ lệ thì *không* được cài vào, nên mạng vẫn phải học chúng từ dữ liệu hoặc từ **augmentation** — tức chỗ cài giả định thứ ba của chương 4; và quan hệ giữa hai vùng xa nhau trở nên đắt đỏ.

Vấn đề mới nếu thông tin chỉ lan ra từng chút qua mỗi tầng thì *mô hình nhìn được bao xa?*

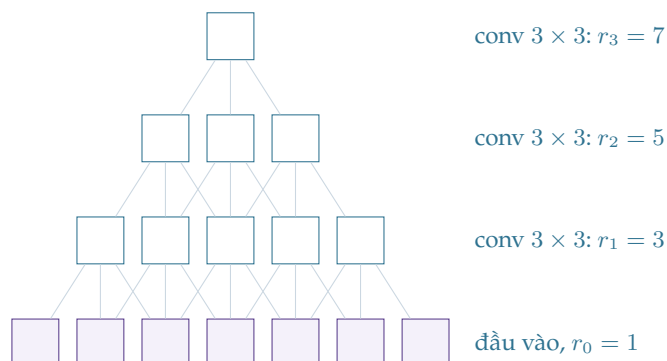
Giải pháp tính receptive field bằng tay *trước* khi huấn luyện, thay vì phát hiện vấn đề sau ba ngày.

C. Receptive field: con số tính được trước khi train

Kích thước đầu ra của một tầng conv là $(W - F + 2P)/S + 1$, và **trường tiếp nhận** (*receptive field*) tích lũy theo hai công thức truy hồi rất đơn giản:

$$j_l = j_{l-1} \cdot s_l, \quad r_l = r_{l-1} + (k_l - 1) j_{l-1}, \quad j_0 = 1, r_0 = 1.$$

Diễn giải bằng lời: mỗi tầng mở rộng tầm nhìn thêm đúng $(k - 1)$ lần bước nhảy hiện tại, và mỗi lần lấy mẫu xuống thì bước nhảy nhân lên. Hệ quả là stride mở rộng tầm nhìn theo cấp số nhân, còn độ sâu chỉ mở rộng theo cấp số cộng. Hình 8.1 minh hoạ trường hợp nhỏ nhất tính được trong đầu.



Hình 8.1. Ba tầng 3×3 stride 1 (vẽ theo một chiều cho gọn): một nơ-ron ở tầng trên cùng nhìn thấy đúng 7 pixel đầu vào, tô đậm. Cùng tầm nhìn với một tầng 7×7 nhưng chỉ tốn $3 \cdot 9C^2 = 27C^2$ tham số thay vì $49C^2$, và có ba phi tuyến thay vì một — đây chính là lập luận của VGG.

Vì sao con số này quan trọng đến thế? Vì *receptive field* nhỏ hơn vật thể là nguyên nhân của một lớp lỗi rất khó chẩn đoán. Mô hình sẽ sai có hệ thống trên vật lớn, và trên những gì cần ngữ cảnh rộng: phân biệt một con thuyền với một cái xe đời mới nhìn thấy nước hay đường. Triệu chứng thì trông giống hệt “thiếu dữ liệu” hoặc “mô hình nhỏ quá”, và cả hai chẩn đoán đó đều dẫn tới hành động sai.

↔ **đối lập với B/08** — cùng triệu chứng “sai có hệ thống trên một nhóm”, nhưng ở đó nguyên nhân là dịch chuyển phân phối chứ không phải giới hạn kiến trúc; phép đo phân biệt hai thứ là khác nhau.

Điều dễ chịu là receptive field tính được trong hai phút bằng giấy bút.

Nhưng ở đây phải phức tạp hoá một chút, vì cắt gọn sẽ thành sai. Với ResNet-18 ở tầng conv cuối, con số lý thuyết rơi vào cỡ vài trăm pixel, lớn hơn ảnh 224×224 , nên thoát nhìn có vẻ không có vấn đề gì. Hiệu chỉnh quan trọng là **trường tiếp nhận hiệu dụng** (*effective receptive field*) nhỏ hơn nhiều: đóng góp của các pixel giảm dần theo dạng gần Gaussian tính từ tâm, và bán kính hiệu dụng chỉ tăng cỡ căn bậc hai của số tầng. Đây là kết quả đã được công bố (dòng công trình về *effective receptive field*), và hệ quả thực dụng là hãy coi con số lý thuyết như một chặn trên rất lỏng chứ đừng coi là mô tả hành vi thật.

8.3 Bớt cách nhìn xa hơn, bớt cái giá (Widening the Receptive Field)

Khi receptive field không đủ, có bốn đòn bẩy và không cái nào miễn phí.

Bảng 8.1. Bốn cách mở rộng receptive field: cơ chế, cái được, và chế độ hồng riêng của từng cách.

Cách	Cơ chế	Được	Hồng khi nào
Tăng độ sâu	Cộng dồn $(k - 1)j$ qua nhiều tầng	Cả tầm nhìn lẫn độ trừu tượng	Khó tối ưu nếu không có residual; độ trễ tăng <i>tuần tự</i> , không song song hoá được
Lấy mẫu xuống	Nhân bước nhảy j lên	Rẻ nhất, mở rộng theo cấp số nhân	Mất chi tiết không gian vĩnh viễn; vật nhỏ biến mất trước khi tới tầng sâu
<i>Dilated conv</i> [81]	Giãn kernel mà giữ nguyên độ phân giải	Vừa xa vừa chi tiết — đúng thứ phân vùng cần	<i>Gridding artifact</i> : điểm lấy mẫu không phủ liên tục nên có tần số bị bỏ sót
Attention toàn cục [82]	Mọi vị trí chạm mọi vị trí trong một bước	Quan hệ xa tùy ý, do nội dung quyết định	Chi phí bậc hai theo số <i>token</i> ; bỏ bớt inductive bias nên đối dữ liệu

Cột chi phí của attention đáng làm thành con số vì nó giải thích gần hết thiết kế của các ViT hiện đại. Ở 224×224 với patch 16×16 ta có $14 \times 14 = 196$ token và ma trận tương tự $196^2 \approx 3,8 \cdot 10^4$ phần tử — hoàn toàn chấp nhận được. Nhưng phân vùng cần đặc trưng ở 56×56 , tức 3136 token, và $3136^2 \approx 9,8 \cdot 10^6$: gấp 256 lần. Đó là lý do *vật lý*, không phải thẩm mỹ, khiến Swin chia attention thành cửa sổ cục bộ rồi dịch cửa sổ giữa các tầng [83]. Nói cách khác, Swin *cài lại* tính cục bộ mà ViT vừa gỡ bỏ, sau khi cả ngành đã học được rằng gỡ hết thì quá đắt.

↔ **ràng buộc bởi B/11** — con số $256 \times$ này vẫn chưa phải toàn bộ câu chuyện: thời gian chạy phụ thuộc lưu lượng bộ nhớ chứ không chỉ số phép tính.

8.4 Residual và luồng gradient (Residual Connections and Gradient Flow)

A. Nghịch lý mà residual thực sự giải quyết

Đây là điểm hay bị hiểu sai nhất trong toàn chương. Trước ResNet, người ta quan sát một nghịch lý: mạng 56 tầng có sai số *huấn luyện* cao hơn mạng 20 tầng [84]. Đây không thể là *overfitting* — overfitting làm sai số test cao chứ không làm sai số train cao. Mạng sâu hơn về nguyên tắc biểu diễn được mọi thứ mạng nông biểu diễn được (chỉ cần các tầng thừa học hàm đồng nhất), nên vấn đề nằm ở chỗ SGD *không tìm được* nghiệm đó. Kết nối tắt $y = x + F(x)$ sửa đúng chỗ đó: hàm đồng nhất trở thành mặc định (chỉ cần $F \rightarrow 0$), và gradient có một đường về thẳng không bị nhân qua chuỗi Jacobian. Nói gọn thì *residual giải quyết bài toán tối ưu trước*, còn *độ sâu chỉ là hệ quả của việc bài toán tối ưu đã dễ đi*.

B. Chuẩn hoá đặt ở đâu, và nó giả định gì

Hệ quả kéo dài tới hôm nay: mọi kiến trúc sâu đều có dạng “một luồng chính cộng dồn, các khối chỉ ghi thêm một lượng nhỏ vào luồng đó”. Chỗ đặt *normalization* là chi tiết đi kèm và có hệ quả rất thực. Đặt sau phép cộng (*post-norm*) thì luồng chính bị chuẩn hoá lại ở mỗi tầng, đường về của gradient không còn sạch, và *transformer* sâu cần khởi động học suất rất cẩn thận mới không phân kỳ; đặt trước khối (*pre-norm*) thì luồng chính không bị đung vào, huấn luyện ổn định hơn

nhiều nhưng thường mất một chút chất lượng cuối. Bản thân họ chuẩn hoá cũng là những giả định khác nhau.

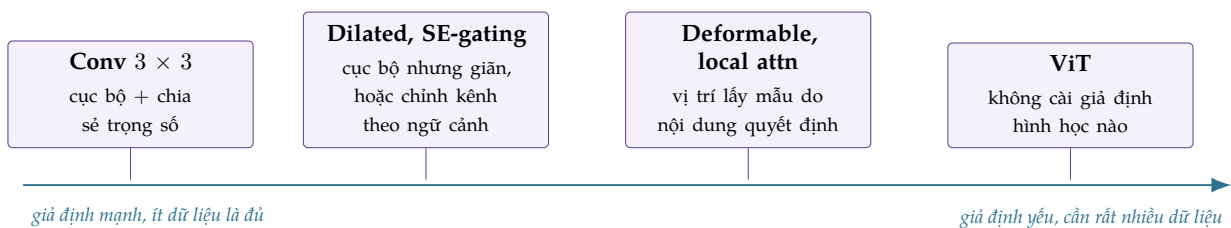
Bảng 8.2. Ba họ chuẩn hoá: mỗi họ là một giả định khác nhau về chỗ nào thống kê đáng tin.

Họ	Chuẩn hoá theo chiều nào	Giả định	Hỏng khi nào
BatchNorm [85]	Trên cả batch, từng kênh	Batch đủ lớn và đồng nhất; thống kê lúc suy luận giống lúc huấn luyện	Batch bằng 2 (cấu hình huấn luyện detection); dữ liệu suy luận lệch phân bố; suy luận trực tuyến từng mẫu
LayerNorm [86]	Trong từng mẫu, trên toàn bộ đặc trưng	Các kênh trong một mẫu có thang so sánh được	Ảnh có kênh với ý nghĩa vật lý rất khác nhau
GroupNorm [87]	Trong từng mẫu, theo nhóm kênh	Có cấu trúc nhóm tự nhiên giữa các kênh	Số nhóm chọn tùy tiện; mất một phần lợi ích chính quy hoá của BN

Ở đây cần trung thực về mức độ chắc chắn: lý do *vì sao* chuẩn hoá giúp vẫn còn tranh luận. Lời giải thích “giảm internal covariate shift” của bài gốc đã bị phản bác bằng thực nghiệm [88], và cách hiểu được ủng hộ hơn hiện nay là nó làm mất loss trơn hơn. Điều *đã được đo* là hành vi ở cột cuối bảng; điều còn là giả thuyết là cơ chế.

8.5 Định tuyến cố định hay theo nội dung (Fixed vs Content-Based Routing)

Convolution trộn thông tin theo một mẫu hình *cố định*: cùng một kernel áp lên mọi vị trí, bất kể nội dung. Đây vừa là sức mạnh (ít tham số, tổng quát hoá tốt) vừa là giới hạn cứng. Ba thể hệ giải pháp nối lỏng dần sự cố định đó, và xếp chúng lên một quang phổ như Hình 8.2 là cách gọn nhất để nhớ cả họ.



Hình 8.2. Quang phổ giả định của kiến trúc thị giác: đi từ trái sang phải, mỗi bậc gỡ bớt một ràng buộc hình học và đổi lấy nhu cầu dữ liệu cao hơn. Đây chính là trục của chương 4, lần này áp cho kiến trúc thay vì cho toàn hệ thống.

Ba bậc giữa và bậc cuối, đọc theo cái được và cái mất:

- **Channel gating** kiểu Squeeze-and-Excitation chỉ học một trọng số cho mỗi kênh dựa trên ngữ cảnh toàn cục [89]. Rất rẻ, và nó chứng minh rằng chỉ một chút định tuyến động đã đủ tạo khác biệt.

- **Deformable convolution** để mạng học luôn *vị trí lấy mẫu* thay vì cố định lưới [90]. Mạnh khi vật thể biến dạng nhiều; khó tối ưu hơn và khó tăng tốc trên phần cứng vì truy cập bộ nhớ không đều.
- **Self-attention** tính lại toàn bộ mẫu hình tròn cho từng đầu vào một. Cái mất không chỉ là FLOPs mà còn là *lượng dữ liệu tối thiểu*: ViT huấn luyện từ đầu trên ImageNet-1k thua CNN, và chỉ vượt lên khi có tiền huấn luyện quy mô lớn hoặc khi được thêm lại một ít inductive bias qua chưng cất và augmentation mạnh [91], [92].

Có một hướng ngược lại đáng nhắc: thay vì nói lỏng giả định, *cài thêm toán học vào kiến trúc* — tăng tối ưu khả vi, vòng lặp tối ưu được trải phẳng, ràng buộc hình học cứng đặt thẳng vào đồ thị tính toán. Tất cả đều nói “đừng học lại thứ tôi đã biết”. Chúng mạnh khi dữ liệu ít và mô hình toán đúng; chúng cứng nhắc và sai một cách khó gỡ khi giả định toán không khớp. Người làm hình học sẽ thấy trực này quen thuộc: đây đúng là cuộc tranh luận giữa pipeline SLAM cổ điển và các mô hình **tiền thẳng** (*feed-forward*), và chương 16 sẽ mở nó ra đầy đủ.

↪ **đối lập với** C/16 — hình học cổ điển cài toán vào hệ thống, mô hình tiến thẳng để mạng học lại toán đó — cùng một bài toán, hai vị trí đặt giả định.

8.6 Bốn thước đo chi phí (Four Cost Metrics)

A. Ba trục phân tích được bằng giấy bút

Có bốn thước đo và chúng *không* tỉ lệ với nhau: số tham số, FLOPs, bộ nhớ, và độ trễ thực đo. Trước khi chạy bất cứ thứ gì, ba trục kiến trúc cơ bản đã phân tích được.

Bảng 8.3. Nhân đôi theo ba trục: chi phí không đi cùng nhau, nên “cùng ngân sách FLOPs” không có nghĩa là “cùng giá”.

Nhân đôi	Tham số	FLOPs	Bộ nhớ kích hoạt	Độ trễ (batch 1)
Độ sâu	×2	×2	×2	×2 và <i>tuần tự</i> — không giấu được bằng song song hoá
Độ rộng	×4	×4	×2	< ×4: kernel to hơn dùng GPU hiệu quả hơn
Độ phân giải	×1	×4	×4	≈ ×4; là trục dễ tràn bộ nhớ nhất

Bảng 8.3 giải thích vài điều thoạt nhìn nghịch lý. Tăng độ rộng làm tham số tăng *bậc hai* — vì một tầng conv có $k^2 C_{in} C_{out}$ trọng số và cả hai chỉ số cùng nhân đôi — trong khi tăng độ sâu chỉ tăng tuyến tính; xét theo tham số thì sâu rẻ hơn rộng. Nhưng xét theo độ trễ ở batch 1 thì ngược lại: các tầng phải chạy nối tiếp, còn độ rộng thì trải ra được trên hàng nghìn lõi. Đây là lý do các mạng thiết kế cho thiết bị biên thường nông và rộng hơn ta tưởng. Và vì ba trục không tương đương, dồn hết ngân sách vào một trục là lựa chọn tồi. Mở rộng cả ba cùng lúc theo một tỉ lệ cố định — **compound scaling** — cho kết quả tốt hơn [93].

Điểm nối trực tiếp sang chương 11 nằm ở cột cuối: *FLOPs thấp không có nghĩa là nhanh*. MobileNet dùng convolution tách theo chiều sâu để giảm FLOPs hơn một bậc so với ResNet-50 [94]. Nhưng ở batch 1 trên GPU nó thường không nhanh hơn tương ứng, đôi khi còn chậm hơn. Lý do là mỗi FLOP của nó vẫn phải kéo theo gần như cùng lượng dữ liệu như trước, nên phép tính chuyển từ chế độ nghẽn tính toán sang chế độ nghẽn băng thông. Chương 11 biến câu đó thành hai con số byte cụ thể.

→ **hệ quả của** $D/20$ — mọi kỹ thuật nén mô hình ở đó đều nhằm vào một trong bốn thước đo này, và chọn nhầm thước đo là lý do phổ biến nhất khiến nén xong không nhanh hơn.

B. Các mô típ lặp lại: cách đọc nhanh nhất một kiến trúc lạ

- **Tách ba tầng chức năng** — *backbone* trích đặc trưng đa tỉ lệ, *neck* trộn các tỉ lệ (ví dụ FPN [95]), *head* biến đặc trưng thành đầu ra của bài toán. Nhận ra ba tầng này là bước đầu tiên khi mở một repo lạ, vì nó cho biết chỗ nào thay được.
- **Encoder-decoder với skip connection** [96] — dùng khi đầu ra cần độ phân giải cao: nhánh xuống lấy ngữ nghĩa, skip mang chi tiết không gian đã bị mất khi lấy mẫu xuống.
- **Bottleneck** — giảm số kênh ở giữa để cắt chi phí bậc hai của tầng 3×3 , rồi phục hồi.
- **Đa tỉ lệ và coarse-to-fine** — xử lý thô trước rồi tinh dần, để chi phí tập trung vào chỗ cần.

Chín mươi phần trăm kiến trúc thị giác là các mô típ này lặp lại theo thứ tự khác nhau, và khi bạn thấy chúng thì phần còn lại của bài báo đọc rất nhanh.

8.7 Thực hành

Repo và tài nguyên. [huggingface/pytorch-image-models](https://github.com/huggingface/pytorch-image-models) (timm) là nơi đối chiếu hàng trăm backbone trên cùng một khung; [TylerYep/torchinfo](https://github.com/TylerYep/torchinfo) để in bảng hình dạng và tham số từng tầng; [facebookresearch/fvcore](https://github.com/facebookresearch/fvcore) để đếm FLOPs; [lutzroeder/netron](https://github.com/lutzroeder/netron) để nhìn đồ thị tính toán thật thay vì tin hình vẽ trong paper; [Fangyh09/pytorch-receptive-field](https://github.com/Fangyh09/pytorch-receptive-field) để đối chiếu với phép tính tay của bạn. Danh sách này chốt tại tháng 9/2026 và tên các mô hình dẫn đầu sẽ cũ trong 6–12 tháng, còn các repo hạ tầng kể trên bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Tính tay receptive field của ResNet-18 ở tầng cuối — nó có phủ hết ảnh 224×224 không, và nếu có thì vì sao mô hình vẫn hỏng trên vật rất lớn? Vì sao tăng độ sâu rẻ hơn tăng độ rộng xét theo tham số nhưng đắt hơn xét theo độ trễ ở batch 1? Với batch 32 ảnh 512×512 , tầng nào chiếm nhiều bộ nhớ kích hoạt nhất, và vì sao câu trả lời không phải là tầng có nhiều tham số nhất?

Mini project. Chọn ba backbone trong timm có cùng mức accuracy ImageNet nhưng khác họ (một CNN cổ điển, một mạng *depthwise*, một ViT nhỏ). Tính tay tham số và FLOPs, *viết ra dự đoán thứ tự độ trễ trước khi đo*, rồi đo thật trên GPU và trên CPU. Mọi chỗ dự đoán sai là một bài học của chương 11; ghi lại cụ thể sai bao nhiêu lần và vì sao.

8.8 Giới hạn và trường hợp hỏng

Khung “kiến trúc là tập giả định” rất mạnh nhưng có ba chỗ nó nói dối.

- **Nó giả vờ rằng ta biết mình đang cài giả định gì.** Trong thực tế nhiều lựa chọn kiến trúc là kết quả của tìm kiếm thực nghiệm rồi được kể lại như một lập luận có nguyên lý. Bằng chứng rõ nhất là ConvNeXt [97]: khi lấy một ResNet và áp lần lượt các chi tiết huấn luyện lẫn thiết kế của ViT, phần lớn khoảng cách vốn được quy cho “attention” biến mất.
- **Mọi phép đếm ở đây là chặn, không phải hành vi thật.** Receptive field hiệu dụng nhỏ hơn lý thuyết rất nhiều, và FLOPs không dự đoán được độ trễ. Ai tối ưu theo con số lý thuyết mà không đo sẽ tối ưu nhầm hướng — chương 11 tồn tại đúng vì lý do này.
- **“Giả định mạnh thì cần ít dữ liệu” chỉ đúng khi giả định khớp.** Ảnh y sinh, ảnh vệ tinh và camera gắn trên người đều có xoay tự do. Với chúng, đẳng biến tịnh tiến của conv không

giúp gì cả, nên mạng phải tiêu sức chứa để học lại phép xoay — thứ mà một biểu diễn phù hợp hơn cho không mất gì. Cùng logic, nếu ảnh đã được cắt căn giữa và luôn cùng tỉ lệ thì phần lớn lợi thế của conv so với MLP biến mất.

CẠM BẤY

Đừng so sánh hai kiến trúc khi công thức huấn luyện của chúng khác nhau — và chúng gần như luôn khác nhau. Số epoch, augmentation, optimizer, độ phân giải, EMA của trọng số, độ dài warm-up: mỗi thứ có thể tạo chênh lệch lớn hơn chênh lệch giữa hai backbone. Khi đọc một bảng kết quả, câu hỏi đầu tiên không phải “kiến trúc nào cao hơn” mà là “hai dòng này có được huấn luyện như nhau không”; câu trả lời nằm ở phụ lục, không nằm ở phần method.

CẦN NHỚ

Kiến trúc không làm mô hình “thông minh hơn”, nó quyết định *ai chạm được tới ai, sau bao nhiêu bước, và với chi phí nào*. Ba câu hỏi tương ứng — receptive field, cách định tuyến, bốn thước đo chi phí — đủ để đọc gần như mọi kiến trúc thị giác, kể cả kiến trúc chưa ra đời.

[T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Inductive bias	Tập giả định về lời giải được cài sẵn trước khi nhìn dữ liệu. Nó đến từ bốn chỗ: kiến trúc, hàm mất mát, augmentation, và thuật toán tối ưu. Bạn không thể “không có giả định”, chỉ có thể chọn giả định nào.
●● Template matching	Cách đọc bộ phân loại tuyến tính: mỗi hàng của ma trận trọng số là một ảnh mẫu, và điểm số là tích vô hướng với ảnh đầu vào. Vẽ các hàng đó ra sẽ thấy mẫu “ngựa” có hai đầu, vì một lớp chỉ có một mẫu chuẩn.
●●● Vì sao không dùng một MLP khổng lồ cho ảnh?	Định lý xấp xỉ phổ quát chỉ nói hàm đúng <i>tồn tại</i> , không nói bạn tìm được nó. Một tầng nổi đầy đủ từ ảnh $224 \times 224 \times 3$ sang 1000 nơ-ron đã là 150 triệu tham số, và mô hình vẫn không biết hai pixel cạnh nhau thì liên quan.
●●● Đăng biến tịnh tiến	Dời vật trong ảnh thì bản đồ đặc trưng dời theo đúng chừng ấy. Convolution có tính chất này gần như miễn phí nhờ chia sẻ trọng số, nhưng <i>không</i> có tính chất tương ứng cho xoay hay đổi tỉ lệ.
●●● Trường tiếp nhận	Vùng ảnh đầu vào ảnh hưởng tới một vị trí đầu ra. Tính bằng hai công thức truy hồi: bước nhảy nhân với stride, bán kính cộng thêm $(k - 1)$ lần bước nhảy hiện tại. Nhỏ hơn vật thể là một lớp lồi tính được trước khi huấn luyện.
●● Trường tiếp nhận hiệu dụng	Vùng thực sự đóng góp, nhỏ hơn con số lý thuyết rất nhiều vì đóng góp giảm dần theo dạng gần Gaussian từ tâm, và chỉ tăng cỡ căn bậc hai của số tầng. Con số lý thuyết chỉ là chặn trên rất lỏng.
●● Dilated convolution	Giãn khoảng cách giữa các điểm lấy mẫu của kernel để mở rộng tầm nhìn mà không giảm độ phân giải. Đúng thứ phân vùng cần, nhưng các điểm lấy mẫu không phủ liên tục nên sinh gridding artifact.
●● Vì sao attention đắt ở bài toán dày đặc?	Chi phí bậc hai theo số token. Ở 224×224 với patch 16×16 chỉ có 196 token, nhưng đặc trưng 56×56 cần 3136 token, đắt hơn 256 lần. Đó là lý do vật lý khiến Swin cắt attention về từng cửa sổ.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Kết nối tắt	Cộng thẳng đầu vào vào đầu ra của một khối, khiến hàm đồng nhất thành mặc định và mở một đường về sạch cho gradient. Nó giải quyết bài toán <i>tối ưu</i> trước; độ sâu chỉ là hệ quả.
●●● Vì sao mạng sâu hơn lại có sai số huấn luyện cao hơn?	Quan sát này có thật trước ResNet: mạng 56 tầng thua mạng 20 tầng ngay trên tập huấn luyện. Vì SGD không tìm được nghiệm, chứ không phải vì mạng không biểu diễn nổi. Đây chắc chắn không phải overfitting: overfitting làm sai số <i>kiểm thử</i> cao, không làm sai số huấn luyện cao.
●● Ba họ chuẩn hoá	BatchNorm chuẩn hoá theo chiều batch nên giả định batch đủ lớn và đồng nhất. LayerNorm và GroupNorm chuẩn hoá trong từng mẫu nên bỏ được giả định đó. Vì sao chuẩn hoá giúp thì tới nay vẫn còn tranh luận.
● Squeeze-and-Excitation	Học một trọng số cho mỗi kênh dựa trên ngữ cảnh toàn ảnh, rồi nhân lại vào kênh đó. Rất rẻ, và nó cho thấy chỉ một chút định tuyến theo nội dung đã đủ tạo khác biệt.
● Deformable convolution	Để mạng học luôn <i>vị trí lấy mẫu</i> thay vì cố định trên lưới. Mạnh khi vật thể biến dạng nhiều, nhưng truy cập bộ nhớ không đều nên khó tăng tốc trên phần cứng.
●●● Thước đo chi phí	Có bốn cái: số tham số, FLOPs, bộ nhớ, và độ trễ thực đo. Chúng không tỉ lệ với nhau, nên “cùng ngân sách FLOPs” không có nghĩa là “cùng giá”.
●● Vì sao nên mở rộng cả ba trục cùng lúc?	Vì ba trục có giá khác nhau: độ rộng làm tham số tăng bậc hai, độ sâu tăng tuyến tính nhưng cộng thẳng vào độ trễ tuần tự, độ phân giải nhân bộ nhớ lên bốn khi tăng gấp đôi. Dồn hết vào một trục thì chậm trễ của trục đó trước.
●● Backbone–neck–head	Ba tầng chức năng của gần như mọi hệ thống thị giác: trích đặc trưng đa tỉ lệ, trộn các tỉ lệ, rồi biến đặc trưng thành đầu ra của bài toán. Nhận ra ba tầng này là bước đầu tiên khi mở một repo lạ.
●● Mô hình sai có hệ thống trên vật rất lớn — nguyên nhân?	Hai khả năng: trường tiếp nhận hiệu dụng không phủ hết vật, hoặc tập huấn luyện thiếu vật cỡ đó. Phân biệt bằng cách cắt vật lớn thành mảnh nhỏ rồi đo lại: nếu điểm số hồi phục thì lỗi nằm ở tầm nhìn, không ở dữ liệu.
●● Huấn luyện detection với batch 2, kết quả tụt bất thường — nguyên nhân?	Nhiều khả năng là BatchNorm: thống kê tính trên hai mẫu thì quá nhiều, và thống kê chạy dừng lúc suy luận lệch hẳn. Đổi sang GroupNorm hoặc đóng băng BatchNorm rồi đo lại.
● Bảng nói kiến trúc mới hơn 3 điểm mAP — kiểm tra gì trước?	Kiểm tra phụ lục xem hai dòng có cùng công thức huấn luyện không: số epoch, augmentation, optimizer, độ phân giải, độ dài warm-up. ConvNeXt cho thấy phần lớn khoảng cách CNN–ViT biến mất khi giữ công thức cố định.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
timm	Nó tồn tại chính vì bài học “phải giữ công thức huấn luyện cố định mới so được kiến trúc”	Hạ tầng bền; là nơi duy nhất so backbone công bằng
ConvNeXt	Bằng chứng thực nghiệm trực tiếp rằng phần lớn khoảng cách CNN–ViT đến từ công thức huấn luyện chứ không từ attention	Kết quả này thay đổi cách đọc mọi bảng so sánh kiến trúc
Dòng Swin và các ViT phân cấp	Cài lại tính cục bộ và cấu trúc đa tỉ lệ vào ViT, đúng vì lý do bậc hai theo số token đã tính trong chương	Là kiến trúc mặc định cho bài toán dày đặc tính tới 9/2026

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Segment Anything (SAM)	Phân bổ chi phí cực lệch: bộ mã hoá ảnh rất nặng chạy một lần, còn phần giải mã mặt nạ rất nhẹ chạy nhiều lần	Minh hoạ sạch nhất cho việc tách backbone-head theo ràng buộc độ trễ
Họ EfficientNet và các biến thể sau nó	Mở rộng hợp nhất theo cả ba trục thay vì dồn ngân sách vào một trục	Ý tưởng còn sống dù tên mô hình đã cũ

MINI PROJECT SLAM · Đo lường tiếp nhận thực nghiệm của backbone frontend · vài giờ tới một ngày

Mục tiêu	kiểm chứng bằng số cái chặn trên lòng đã nêu trong chương: đặt con số trường tiếp nhận lý thuyết tính bằng giấy bút cạnh con số đo được bằng cách che ảnh, và xem chúng lệch nhau bao nhiêu lần.
Dữ liệu	vài trăm ảnh xám 752×480 từ một chuỗi EuRoC, cùng đúng backbone bạn định dùng cho frontend.
Các bước	(1) tính tay trường tiếp nhận lý thuyết ở ba tầng có độ sâu khác nhau bằng hai công thức truy hồi trong chương; (2) chọn một vị trí đầu ra cố định; (3) trượt một ô che 16×16 khắp ảnh và mỗi lần ghi lại mức thay đổi của vectơ đặc trưng tại vị trí đó; (4) vẽ bản đồ nhạy cảm và lấy bán kính chứa 90% tổng thay đổi; (5) lặp cho cả ba tầng.
Xong khi	bạn có một bảng ba dòng — tầng, trường tiếp nhận lý thuyết, bán kính 90% đo được — cùng tỉ số giữa hai cột, và trả lời được bằng số rằng backbone này có nhìn đủ xa cho vật thể lớn nhất xuất hiện trong chuỗi của bạn hay không.
Nối với	B/11 vì bản đồ nhạy cảm tốn hàng nghìn lượt xuôi và là bài tập tốt để đo thông lượng; C/13 nếu bạn muốn lặp lại phép đo trên một ViT để so hai họ trên cùng thước đo; E/25 vì đây chính là một dạng bản đồ nổi bật.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì định lý chỉ nói hàm đúng *tồn tại* trong không gian giả thuyết, không nói bạn tìm được nó với lượng dữ liệu và tính toán bạn có. Một tầng nổi đầy đủ từ ảnh $224 \times 224 \times 3$ sang 1000 nơ-ron đã là 150 triệu tham số, và mô hình vẫn không biết hai pixel cạnh nhau thì liên quan.
- A2. Vì overfitting làm sai số *test* cao chứ không làm sai số *train* cao. Mạng sâu về nguyên tắc biểu diễn được mọi thứ mạng nông biểu diễn được, nên cái hỏng là bài toán tối ưu: SGD không tìm được nghiệm đó. Kết nối tắt biến hàm đồng nhất thành mặc định và mở một đường về sạch cho gradient.
- A3. Vì chi phí bậc hai theo số token. Ở 224×224 với patch 16×16 chỉ có 196 token, nhưng bài toán dày đặc cần đặc trưng ở 56×56 , tức 3136 token và đắt hơn 256 lần. Swin cài lại tính cục bộ mà ViT vừa gỡ bỏ, sau khi đã biết rằng gỡ hết thì quá đắt.
- A4. Vì tham số của một tầng conv là $k^2 C_{in} C_{out}$, nên nhân đôi độ rộng nhân tham số lên bốn còn nhân đôi độ sâu chỉ nhân lên hai. Nhưng các tầng phải chạy nối tiếp, còn các kênh trải ra được trên hàng nghìn lõi — nên ở batch 1, độ sâu là thứ không giấu được.
- A5. Vì hai dòng trong bảng gần như luôn khác nhau ở số epoch, augmentation, optimizer, độ phân giải và độ dài warm-up — những thứ có thể tạo chênh lệch lớn hơn chênh lệch kiến trúc. ConvNeXt là bằng chứng: giữ công thức cố định thì phần lớn khoảng cách biến mất.

NÓI LẠI THẬT ĐƠN GIẢN

Có hai cách xây nhà. Cách thứ nhất là đổ bê tông tự do: tạo được bất kỳ hình dạng nào, nhưng tốn rất nhiều vật liệu và thời gian. Cách thứ hai là lắp bằng một bộ gạch có sẵn, mỗi viên chỉ khớp với viên bên cạnh theo vài kiểu nhất định. Bộ gạch giới hạn hình dạng, nhưng bạn xây nhanh hơn với ít vật liệu hơn hẳn.

Mạng nơ-ron cũng vậy. Nối mọi điểm ảnh với mọi thứ là bê tông tự do: chỉ một tầng đã tốn hàng trăm triệu con số cần chỉnh. Hoặc bạn cài sẵn một hiểu biết rất tầm thường — hai điểm ảnh cạnh

nhau thì liên quan tới nhau, và một hoa văn có nghĩa ở góc này thì cũng có nghĩa y hệt ở góc kia. Cài xong, số thứ phải học giảm hàng nghìn lần.

Cái giá là gì? Vì mỗi viên gạch chỉ nhìn thấy vùng nhỏ quanh nó, muốn hai đầu bức ảnh “nói chuyện” được với nhau thì phải xếp rất nhiều lớp gạch chồng lên. Xếp không đủ lớp thì hai đầu đó không bao giờ gặp nhau, dù bạn dạy máy bao lâu — và đây là con số tính được bằng giấy bút trước khi bắt đầu. Còn nếu ảnh của bạn bị xoay lung tung thì bộ gạch không giúp gì, bạn tốn công hơn cả khi dùng bê tông.

Chỗ không rút gọn được: vì sao việc chuẩn hoá các con số giữa các lớp lại làm việc học dễ đi thì tới nay vẫn chưa có lời giải thích vừa gọn vừa đúng; cách giải thích trong bài báo gốc đã bị chính thực nghiệm bác bỏ.

Một câu: Kiến trúc là bộ gạch bạn chọn: chọn khớp với dữ liệu thì xây nhanh với rất ít vật liệu, chọn lệch thì tốn hơn cả xây tự do.

8.9 Kết nối

Chương này là *trục của chương 4 áp cho kiến trúc*: quang phổ ở Hình 8.2 chính là trục “giả định mạnh + ít dữ liệu ↔ giả định yếu + nhiều dữ liệu”. Nó *đối tác* với chương 9: kiến trúc cài giả định về dạng của hàm, loss cài giả định về mục tiêu. Nó là *tiền đề trực tiếp* của chương 11, vì cột “độ trễ” trong mọi bảng ở đây chỉ có nghĩa khi bạn hiểu vì sao FLOPs không dự đoán được thời gian.

Sang Phần C, C/13-kien-truc-backbone đọc lịch sử CNN, attention và ViT như một chuỗi các lần gỡ nút thắt — đó là bản kể chi tiết của quang phổ trong chương này. C/14-bai-toan-cv-loi cho thấy tách backbone/neck/head hoạt động ra sao trên bài toán thật và vì sao đa tỉ lệ là bắt buộc chứ không phải trang trí. C/16-thi-giac-3d mở ra cuộc tranh luận “nhúng toán vào kiến trúc hay để mạng học”. Và D/20-toi-uu-va-nen-mo-hinh nhật lại chỗ này khi bàn về việc thay đổi kiến trúc *sau* huấn luyện.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao định lý xấp xỉ phổ quát *không* phải lý do để dùng MLP cho ảnh — giới hạn thật nằm ở khả năng biểu diễn hay ở chi phí học?
2. Residual connection giải quyết vấn đề gì *trước tiên*, và bằng chứng thực nghiệm nào cho thấy đó không phải vấn đề overfitting?
3. Một mô hình phân vùng hồng có hệ thống trên vật rất lớn: hai chẩn đoán nào giải thích được triệu chứng đó, và phép đo nào phân biệt chúng?
4. Vì sao Swin phải chia attention thành cửa sổ, và phép tính nào về số token cho thấy đó là bắt buộc chứ không phải một lựa chọn thẩm mỹ?
5. Nếu dữ liệu của bạn có xoay tự do, đẳng biến tịnh tiến của convolution còn giá trị gì, và bạn sẽ cài giả định thiếu vào chỗ nào trong bốn chỗ của chương 4?
6. Hai mạng cùng số tham số, một sâu gấp đôi và một rộng gấp đôi: chênh lệch độ trễ ở batch 1 khác chênh lệch ở batch 64 ra sao, và vì sao?
7. BatchNorm hồng ở ba tình huống nào, và mỗi tình huống vi phạm giả định nào của nó?

Nguồn

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *Rec-Sys*, 2019.
- [3] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [4] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [5] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [6] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [7] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [8] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [9] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [10] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [11] V. N. Vapnik, “An overview of statistical learning theory,” *IEEE Trans. Neural Networks*, vol. 10, no. 5, pp. 988–999, 1999.
- [12] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [13] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [15] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [16] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [17] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [18] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE Trans. Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.
- [19] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.

- [20] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [21] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [22] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [23] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [24] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [25] R. Geirhos, J.-H. Jacobsen, C. Michaelis, et al., “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.
- [26] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [27] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [28] T. Gebru, J. Morgenstern, et al., “Datasheets for Datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [29] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [30] M. Mitchell, S. Wu, et al., “Model Cards for Model Reporting,” in *FAT**, 2019.
- [31] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [32] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [33] C. G. Northcutt, L. Jiang, and I. L. Chuang, “Confident Learning: Estimating Uncertainty in Dataset Labels,” *JAIR*, vol. 70, pp. 1373–1411, 2021.
- [34] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [35] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [36] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [37] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [38] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [39] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [40] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [41] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London: Butterworths, 1979.
- [42] B. W. Matthews, “Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme,” *Biochimica et Biophysica Acta (BBA) — Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
- [43] D. Chicco and G. Jurman, “The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation,” *BMC Genomics*, vol. 21, p. 6, 2020.

- [44] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in *International Conference on Machine Learning (ICML)*, 2006, pp. 233–240.
- [45] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.
- [46] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [47] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [48] M. P. Naeni, G. F. Cooper, and M. Hauskrecht, "Obtaining Well Calibrated Probabilities Using Bayesian Binning," in *AAAI Conference on Artificial Intelligence*, 2015.
- [49] G. W. Brier, "Verification of Forecasts Expressed in Terms of Probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
- [50] A. H. Murphy, "A New Vector Partition of the Probability Score," *Journal of Applied Meteorology*, vol. 12, no. 4, pp. 595–600, 1973.
- [51] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [52] T.-Y. Lin, M. Maire, S. Belongie, et al., "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.
- [53] J. Hosang, R. Benenson, P. Dollár, et al., "What Makes for Effective Detection Proposals?" *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 4, pp. 814–830, 2016.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] F. Perazzi, J. Pont-Tuset, B. McWilliams, et al., "A Benchmark Dataset and Evaluation Methodology for Video Object Segmentation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [56] G. Csurka, D. Larlus, and F. Perronnin, "What Is a Good Evaluation Measure for Semantic Segmentation?" In *British Machine Vision Conference (BMVC)*, 2013.
- [57] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [58] K. Mikolajczyk and C. Schmid, "A Performance Evaluation of Local Descriptors," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 10, pp. 1615–1630, 2005.
- [59] M. Dusmanu, I. Rocco, T. Pajdla, et al., "D2-Net: A Trainable CNN for Joint Description and Detection of Local Features," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [60] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [61] S. Sagawa, P. W. Koh, T. B. Hashimoto, et al., "Distributionally Robust Neural Networks for Group Shifts," in *ICLR*, 2020.
- [62] M. Arjovsky, L. Bottou, I. Gulrajani, et al., "Invariant Risk Minimization," *arXiv:1907.02893*, 2019.
- [63] I. Gulrajani and D. Lopez-Paz, "In Search of Lost Domain Generalization," in *ICLR*, 2021.
- [64] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," in *ICLR*, 2019.

- [65] R. Taori, A. Dave, V. Shankar, et al., “Measuring Robustness to Natural Distribution Shifts in Image Classification,” in *NeurIPS*, 2020.
- [66] D. Hendrycks and K. Gimpel, “A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks,” in *ICLR*, 2017.
- [67] Y. Ovadia, E. Fertig, et al., “Can You Trust Your Model’s Uncertainty?” In *NeurIPS*, 2019.
- [68] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [69] A. Kendall, Y. Gal, and R. Cipolla, “Multi-Task Learning Using Uncertainty to Weigh Losses,” in *CVPR*, 2018.
- [70] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [71] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [72] F. Milletari, N. Navab, and S.-A. Ahmadi, “V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation,” in *3DV*, 2016.
- [73] H. Rezatofighi, N. Tsoi, J. Gwak, et al., “Generalized Intersection over Union,” in *CVPR*, 2019.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [75] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [76] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [77] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [78] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [79] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [80] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [81] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [82] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [83] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [84] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [85] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [86] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [87] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [88] S. Santurkar, D. Tsipras, A. Ilyas, et al., “How Does Batch Normalization Help Optimization?” In *NeurIPS*, 2018.
- [89] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [90] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.

- [91] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [92] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [93] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [94] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [95] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [96] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [97] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [98] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [99] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [100] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [101] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.

Giới hạn tính toán và hệ thống (Compute and Systems Limits)

PHỤ THUỘC B/10-thiet-ke-kien-truc (bốn thước đo chi phí — chương này biến cột “độ trễ” ở đó thành con số); B/03-giai-phau-he-thong (ràng buộc cứng); hữu ích nhưng không bắt buộc: D/19-numpy-toi-pytorch nếu bạn muốn tự đo.

TÓM TẮT

Chương này trả lời một câu hỏi rất cụ thể: vì sao bạn giảm FLOPs một nửa mà mô hình không nhanh hơn. Hầu hết phép toán trong một mạng thị giác không chờ đơn vị tính toán; chúng chờ bộ nhớ. Đại lượng phân biệt hai chế độ đó là cường độ số học, tức số FLOP làm được trên mỗi byte phải chuyển. Nó tính được bằng giấy bút trong hai phút. Đọc xong bạn ước lượng được bộ nhớ và độ trễ trước khi viết code. Bạn cũng đọc được một bảng “FLOPs thấp hơn ba lần” mà không bị nó dẫn sai hướng, và chọn đúng biện pháp cho đúng chế độ nghẽn thay vì thử tất cả. Nếu chỉ nhớ một điều: FLOPs là thuộc tính của mô hình, còn thời gian là thuộc tính của cặp mô hình–máy. Cây cầu giữa chúng là cường độ số học đem so với điểm gãy roofline của chính cỗ máy bạn chạy: khoảng 150 FLOP/byte trên A100, khoảng 330 trên Jetson Orin.

KIẾN THỨC CẦN CÓ TRƯỚC

- **FLOPs** — số phép nhân–cộng dấu phẩy động cần cho một lần chạy mô hình. Nó đo *khối lượng công việc tính toán*, không đo thời gian. Phân biệt hai thứ này là toàn bộ nội dung của chương. Có một nhập nhằng phổ biến: nhiều bài báo ghi “FLOPs” nhưng thực ra đếm số phép nhân–cộng, tức một nửa con số theo định nghĩa chặt.
- **băng thông bộ nhớ (memory bandwidth)** — số byte mỗi giây chuyển được giữa bộ nhớ chính của thiết bị và chip tính toán, ví dụ 2000 GB/s trên một GPU máy chủ và khoảng 200 GB/s trên một mô-đun nhúng. Đây là một hằng số vật lý của máy, không phải thứ tối ưu phần mềm thay đổi được.
- **cường độ số học (arithmetic intensity)** — tỉ số giữa số FLOP thực hiện và số byte phải chuyển để thực hiện chúng. Nó trả lời câu “mỗi byte kéo về được dùng để làm bao nhiêu việc”, và so nó với một hằng số của máy là ra ngay bạn đang bị giới hạn bởi tính toán hay bởi bộ nhớ.
- **độ trễ (latency)** — thời gian từ lúc một đầu vào đi vào hệ thống tới lúc có kết quả cho *chính* đầu vào đó. Đây là đại lượng mà hệ thống thời gian thực quan tâm.
- **thông lượng (throughput)** — số mẫu xử lý xong trên một đơn vị thời gian khi được phép gom nhiều mẫu lại cùng lúc. Hai đại lượng này xung đột nhau: gom mẫu làm thông lượng tăng và độ trễ xấu đi.
- **kích hoạt (activation)** — các tensor trung gian sinh ra ở giữa mạng trong *lượt xuôi* (một lần chạy mô hình từ đầu vào tới đầu ra). *Lượt ngược* là lần chạy gradient theo chiều ngược lại, do thuật toán lan truyền ngược điều khiển. Nó cần đúng các tensor đó, nên khi huấn luyện ta phải giữ lại tất cả. Vì vậy kích hoạt là phần bộ nhớ tỉ lệ với số mẫu trong một batch và với độ phân giải ảnh, khác hẳn tham số vốn cố định.
- **kernel (trong ngữ cảnh GPU)** — một chương trình nhỏ chạy trên GPU thực hiện một phép toán, ví dụ “một tầng convolution” hoặc “một phép cộng tensor”. CPU phải *phóng* từng kernel và mỗi lần phóng tốn một khoản thời gian cố định. Đừng nhầm với “kernel” nghĩa cửa sổ convolution ở chương 10 — hai nghĩa hoàn toàn khác nhau.

- **GPU utilization và MFU** — “utilization” chỉ cho biết GPU có đang bận hay không, kể cả khi nó bận chờ bộ nhớ; MFU (*model FLOPs utilization*) là tỉ lệ giữa FLOPs thật sự làm được và đỉnh lý thuyết của chip. Một GPU có thể util 95% mà MFU chỉ 10%, và chính khoảng cách giữa hai con số đó là manh mối chẩn đoán mạnh nhất bạn có.

CÂU HỎI MỞ ĐẦU

- Q1. Hai mô hình cùng số FLOPs, vì sao một cái chạy nhanh gấp năm lần cái kia trên cùng một GPU?
- Q2. Vì sao mô hình vừa khít bộ nhớ khi suy luận lại tràn khi huấn luyện, dù trọng số không đổi một byte nào?
- Q3. Điểm gây roofline của một mô-đun nhúng *cao gấp đôi* của một GPU máy chủ — điều đó nói gì về danh sách tối ưu bạn mang từ máy chủ xuống thiết bị?
- Q4. Bạn tối ưu mô hình suốt một tuần và GPU vẫn nhàn rỗi bốn mươi phần trăm thời gian. Vì sao lỗi có thể hoàn toàn không nằm trong mô hình?
- Q5. Bạn đo được lượt xuôi của một mạng lớn mất 0,2 ms. Vì sao con số đó gần như chắc chắn là đối trá, và đối trá theo cơ chế nào?

9.1 Đặt vấn đề

Bạn thay một khối trong mô hình bằng một khối rẻ hơn một bậc về **FLOPs**, đo lại, và độ trễ không nhúc nhích — đôi khi còn tệ hơn. Đây không phải chuyện hiếm và cũng không phải lỗi cài đặt: nó là hệ quả trực tiếp của một sự thật vật lý mà bảng so sánh trong paper không bao giờ nhắc tới. Bộ xử lý hiện đại *tính* nhanh hơn khả năng *nạp dữ liệu* của nó hàng trăm lần, nên phần lớn các phép toán trong một mạng nơ-ron không chờ đơn vị tính toán mà chờ bộ nhớ. Khi bạn đang bị giới hạn bởi băng thông mà lại đi tối ưu số phép nhân, bạn đang tối ưu đúng cách nhưng nhầm chỗ.

Đọc nhanh, nếu bạn chỉ có năm phút. Có một đại lượng duy nhất quyết định bạn đang bị giới hạn bởi cái gì: **cường độ số học** (*arithmetic intensity*) — số phép tính thực hiện được trên mỗi byte phải chuyển qua bộ nhớ. Đem so nó với một hằng số của phần cứng (**điểm gây** (*ridge point*) = đỉnh tính toán chia cho băng thông) là ra ngay chế độ nghẽn, và mỗi chế độ có một tập biện pháp riêng mà các tập kia hoàn toàn vô ích. Trên Jetson AGX Orin, điểm gây nằm quanh 330 FLOP/byte, nghĩa là gần như mọi thứ trừ *convolution* dày và *GEMM* lớn đều nghẽn bộ nhớ.

Giá trị của chương này rất cụ thể. Bạn dự đoán được *trước khi viết một dòng code* rằng một tối ưu sẽ giúp hay không. Bạn ước lượng được bộ nhớ và độ trễ bằng giấy bút với sai số một bậc. Và bạn đọc được một bảng “FLOPs thấp hơn ba lần” mà không bị nó dẫn đi sai hướng. Nếu bạn đã từng đo latency của một pipeline SLAM trên thiết bị nhúng, chương này chủ yếu là đặt tên và công thức hoá những thứ bạn đã cảm nhận bằng tay.

TRỰC GIÁC

Hãy hình dung GPU như một nhà máy khổng lồ với một cửa nhập hàng rất hẹp. Bên trong có hàng nghìn thợ (**CUDA core** và **lõi tensor** (*tensor core*)) làm việc nhanh kinh khủng; nhưng nguyên liệu chỉ vào được qua một cửa duy nhất với lưu lượng cố định. **Cường độ số học** chính là số thao tác nhà máy làm được trên mỗi kilogram nguyên liệu nhập vào. Nếu công đoạn của bạn làm nhiều việc trên mỗi kilogram — như nhân hai ma trận lớn — thì thợ bận rộn và cửa nhập hàng thành thơi. Nếu nó chỉ làm một hai phép trên mỗi kilogram — như **BatchNorm** hay hàm kích hoạt — thì thợ đứng chơi và cả nhà máy chạy đúng bằng

tốc độ cửa. Giảm số việc phải làm trong trường hợp thứ hai không rút ngắn được gì cả, vì nút thất nằm ở cửa. Đây là toàn bộ nội dung của mô hình *roofline*, và cũng là lý do **hợp nhất kernel** (*kernel fusion*) — gộp nhiều công đoạn để nguyên liệu chỉ phải vào cửa một lần — lại là biện pháp mạnh đến vậy.

9.2 Roofline và cường độ số học (Roofline and Arithmetic Intensity)

A. Một công thức và một hằng số

Với một phép toán bất kỳ, định nghĩa **cường độ số học** (*arithmetic intensity*) là

$$I = \frac{\text{số FLOP thực hiện}}{\text{số byte phải chuyển giữa bộ nhớ và chip}}.$$

Nói gọn thì I đo mức độ “đáng công vận chuyển” của một phép toán. Hiệu năng đạt được bị chặn bởi hai thứ cùng lúc — đỉnh tính toán của chip và băng thông nhân với cường độ — nên [10]

$$P_{\text{đạt được}} = \min(P_{\text{đỉnh}}, B \cdot I).$$

Câu diễn giải: nếu mỗi byte không mang theo đủ việc để làm, chip sẽ chạy theo nhịp của bộ nhớ chứ không theo nhịp của chính nó. Chỗ hai chặn cắt nhau, $I^* = P_{\text{đỉnh}}/B$, là **điểm gãy** (*ridge point*): bên trái nó bạn nghẽn bộ nhớ, bên phải bạn nghẽn tính toán. Điểm gãy là một hằng số của máy, không phải của mô hình, và nó đáng thuộc lòng cho từng thiết bị bạn dùng.

- **NVIDIA A100 80 GB** — đỉnh khoảng 312 TFLOP/s fp16 (lỗi tensor, dày), băng thông HBM2e 2039 GB/s. Điểm gãy ≈ 153 FLOP/byte.
- **Jetson AGX Orin 64 GB** — đỉnh khoảng 68 TFLOP/s fp16 (suy ra từ con số công bố 275 TOPS INT8 thừa; tôi chưa tự đo), băng thông LPDDR5 204,8 GB/s. Điểm gãy ≈ 332 FLOP/byte.

Con số thứ hai là điều đáng ngạc nhiên nhất của cả chương: *điểm gãy của thiết bị nhúng cao gấp đôi máy chủ*, vì bộ nhớ LPDDR chậm hơn nhiều so với đà tăng của đơn vị tính toán. Hệ quả thực hành rất nặng: một tối ưu chứng minh được là hữu ích trên máy chủ có thể vô ích trên Orin, và ngược lại, kernel fusion trên Orin thường đáng giá hơn nhiều so với việc cắt FLOPs.

B. Ví dụ nhỏ nhất tính tay được

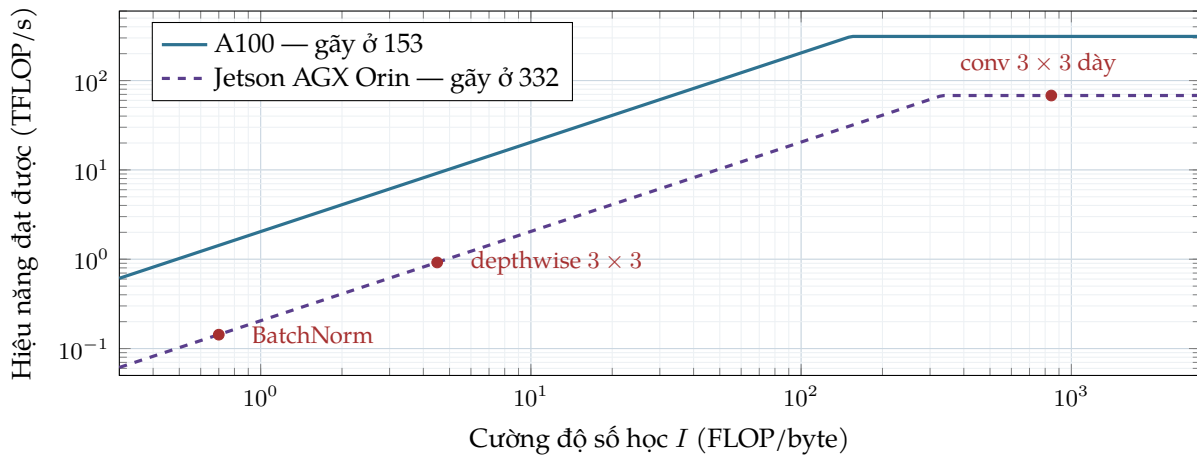
Lấy một *feature map* $256 \times 56 \times 56$ ở fp16 (2 byte mỗi phần tử), tức 802 816 phần tử và 1,61 MB mỗi lần đọc hoặc ghi. So sánh ba phép toán chạy trên chính nó, batch bằng 1.

- **Convolution 3×3 dày, 256 $\rightarrow$ 256 kênh.**
 - FLOPs = $2 \cdot 9 \cdot 256 \cdot 256 \cdot 56 \cdot 56 \approx 3,70 \cdot 10^9$.
 - Byte = vào 1,61 MB + ra 1,61 MB + trọng số $9 \cdot 256 \cdot 256 \cdot 2 = 1,18 \text{ MB} \approx 4,4 \text{ MB}$.
 - $I \approx 843$ FLOP/byte — nằm bên phải điểm gãy của cả hai máy, tức compute-bound thật sự.
- **BatchNorm trên cùng tensor đó.**
 - FLOPs ≈ 4 phép trên mỗi phần tử $\approx 3,2 \cdot 10^6$; byte ≈ 3 lượt qua tensor $\approx 4,8 \text{ MB}$.
 - $I \approx 0,7$ FLOP/byte — thấp hơn convolution hơn một nghìn lần. Đây là câu trả lời cho câu hỏi “vì sao BatchNorm memory-bound còn conv thì compute-bound”, và cũng là lý do mọi trình biên dịch đều cố gộp BatchNorm vào conv liền trước nó.
- **Convolution 3×3 tách theo chiều sâu (depthwise), 256 kênh.**

- FLOPs = $2 \cdot 9 \cdot 256 \cdot 56 \cdot 56 \approx 1,45 \cdot 10^7$ — ít hơn bản dày đúng 256 lần.
- Byte $\approx 3,2$ MB — chỉ ít hơn bản dày 1,4 lần.
- $I \approx 4,5$ FLOP/byte. Nằm sâu bên trái điểm gãy, nên thời gian chạy quyết định bởi 3,2 MB kia chứ không bởi 14 triệu phép tính.

Ba dòng trên đủ để trả lời câu hỏi mở đầu chương, và đáng làm cho tới cùng. Hãy thay một convolution dày 3×3 bằng cặp *depthwise* 3×3 cộng *pointwise* 1×1 , đúng công thức của MobileNet [94]. Tổng phép tính là $1,45 \cdot 10^7 + 4,11 \cdot 10^8 \approx 4,3 \cdot 10^8$ FLOP, tức ít hơn 8,7 lần. Nhưng tổng lưu lượng bộ nhớ lại là $3,2 + 3,3 \approx 6,5$ MB, nhiều hơn 1,5 lần so với 4,4 MB của bản dày. Lý do là bây giờ có hai kernel, nên tensor trung gian phải ghi ra rồi đọc lại. Trên một máy nghèo bộ nhớ, thời gian tỉ lệ với byte chứ không với FLOP, nên phép “tối ưu” này làm mô hình chậm đi khoảng một rưỡi lần trong khi bảng số liệu khoe giảm FLOPs gần chín lần. Không có gì mâu thuẫn ở đây cả; chỉ là hai đại lượng khác nhau bị gọi chung là “chi phí”.

Hình 9.1 đặt cả ba phép toán lên mái nhà roofline của hai máy để thấy vị trí tương đối của chúng.



Hình 9.1. Roofline của hai máy và vị trí ba phép toán (chấm đặt trên mái của Orin). Phần dốc bên trái là vùng nghèo băng thông, phần phẳng bên phải là vùng nghèo tính toán. Điểm gãy của Orin nằm xa bên phải hơn A100, nên trên thiết bị những có nhiều phép toán rơi vào vùng nghèo bộ nhớ hơn. Các đỉnh lấy từ thông số công bố, không phải đo thực tế.

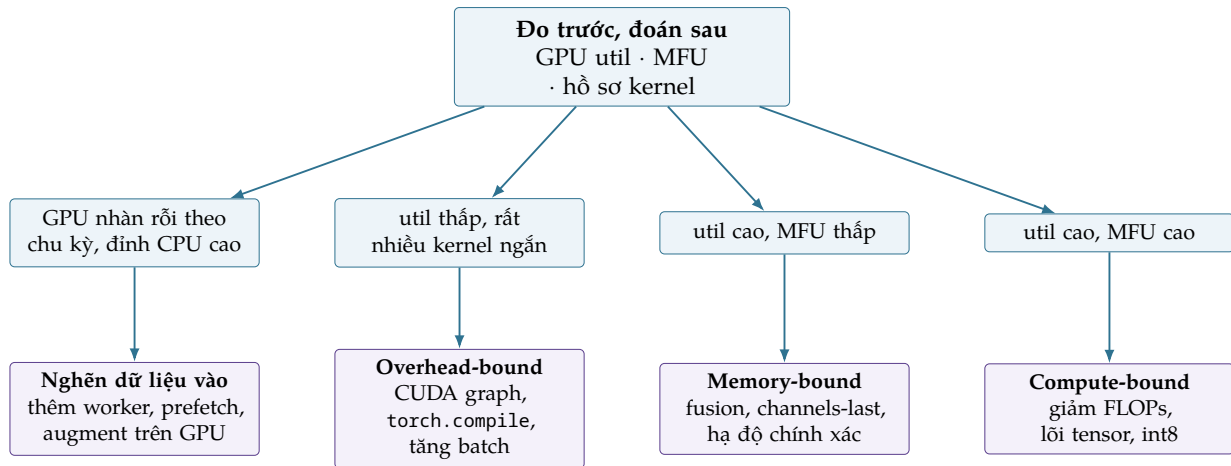
C. Ba chế độ nghèo và biện pháp tương ứng

Roofline mới chỉ phân biệt hai chế độ. Thực tế có chế độ thứ ba, và nó là chế độ phổ biến nhất ở suy luận batch nhỏ trên thiết bị nhúng: GPU không chờ tính toán cũng không chờ bộ nhớ mà chờ CPU gửi việc cho nó. Bảng 9.1 là bảng đáng dán lên tường.

Bảng 9.1. Ba chế độ nghèo: triệu chứng, biện pháp đúng, và biện pháp hoàn toàn vô ích.

Chế độ	Triệu chứng đo được	Biện pháp đúng	Vô ích
Compute-bound	GPU util cao, MFU cao ($> 50\%$)	Giảm FLOPs thật, dùng lõi tensor, lượng tử hoá xuống int8	Kernel fusion
Memory-bound	GPU util cao nhưng MFU thấp; thời gian tỉ lệ với kích thước tensor	Kernel fusion, bố cục <i>channels-last</i> , giảm số lượt đọc/ghi, hạ độ chính xác để giảm byte	Giảm FLOPs
Overhead-bound	GPU util thấp, hồ sơ đầy kernel ngắn dưới $10 \mu s$, khoảng trống giữa các kernel	<i>CUDA graph</i> , <i>torch.compile</i> , gộp kernel, tăng batch	Đổi kiến trúc, giảm FLOPs

Hình 9.2 biến bảng đó thành một quy trình chẩn đoán theo thứ tự, kèm một nhánh thứ tư mà người làm thị giác gặp thường xuyên hơn cả ba nhánh kia cộng lại.



Hình 9.2. Cây chẩn đoán: bốn nhánh, và điều quan trọng nhất là biện pháp của nhánh này gần như vô dụng ở nhánh kia. Nhánh trái ngoài cùng — nghẽn ở đường dữ liệu vào — là nhánh đặc thù của thị giác máy tính và hay bị bỏ sót nhất.

9.3 Cùng FLOPs, khác nhiều lần độ trễ (Equal FLOPs, Unequal Latency)

Phần trước đã cho một nguyên nhân. Còn ba nguyên nhân nữa, và cộng lại chúng giải thích được gần hết các trường hợp dự đoán sai.

1. **Cường độ số học khác nhau** — đã tính ở trên: hai mạng cùng FLOPs có thể chênh nhau hai bậc về lưu lượng bộ nhớ. Đây là nguyên nhân lớn nhất và cũng dễ ước lượng nhất bằng giấy bút.
2. **Số lượng kernel và chi phí phóng kernel** — mỗi lần phóng một kernel tốn cỡ 5–10 μs trên máy chủ và nhiều hơn trên nền CPU yếu của thiết bị nhúng (đây là bậc độ lớn thường được đo, không phải một con số chuẩn). Một mạng nhẹ kiểu MobileNet có thể có hai trăm kernel ở batch 1, tức một sàn cỡ 1–2 ms mà không lượng FLOPs nào ảnh hưởng tới. Với ngân sách 33 ms cho cả pipeline, cái sàn đó không hề nhỏ.
3. **Bố cục bộ nhớ và điều kiện dùng lỗi tensor** — lỗi tensor chỉ được kích hoạt khi tensor ở đúng bố cục (*NHWC*, tức *channels-last*) và khi số kênh chia hết cho một bội số nhất định (8 với fp16). Sai một trong hai điều kiện thì kernel rơi xuống đường chậm và mất nhiều lần hiệu năng *trong khi FLOPs không đổi một chút nào*. Một mạng có số kênh 100 thay vì 96 hoặc 104 có thể chậm hơn đáng kể mà biểu đồ FLOPs không hé lộ gì.
4. **Mức độ song song có sẵn** — đây là chỗ nổi thẳng về Bảng ba trực ở chương 10

↪ **hệ quả của $B/10$** — bảng ba trực ở chương kiến trúc giờ đọc được bằng đơn vị thời gian thay vì bằng đơn vị FLOPs.

: hai mạng cùng FLOPs, một sâu và hẹp, một nông và rộng, có độ trễ rất khác nhau ở batch 1 vì các tầng phải chạy nối tiếp còn các kênh thì trải ra được trên hàng nghìn lõi. Ở batch 64 thì khoảng cách này thu hẹp lại, vì batch cung cấp nguồn song song thay thế.

Điểm chung của cả bốn: *FLOPs là một đại lượng của mô hình, độ trễ là một đại lượng của cặp mô hình–máy*. Không có gì bảo đảm hai đại lượng đó tỉ lệ với nhau, và trong họ mạng nhẹ chúng thường xuyên đi ngược nhau.

9.4 Bộ nhớ: kích hoạt so với tham số (Memory: Activations vs Parameters)

A. Vì sao mô hình vừa lúc suy luận lại tràn lúc huấn luyện

Đây là câu hỏi ai cũng gặp trong tuần đầu tiên. Bộ nhớ lúc huấn luyện gồm bốn phần rất khác nhau về cách chúng lớn lên:

- **Tham số** — cố định, không phụ thuộc batch. N tham số ở fp32 tốn $4N$ byte.
- **Gradient** — cùng kích thước với tham số, cũng cố định.
- **Trạng thái optimizer** — với Adam có hai moment nên thêm $8N$ byte. Cộng cả ba: **16 byte cho mỗi tham số**, một con số đáng thuộc.
- **Kích hoạt (activation)** — các tensor trung gian phải giữ lại để dùng trong lượt ngược. Đây là phần *tỉ lệ với batch và với bình phương độ phân giải*, và nó là phần gây tràn.

Ba phần đầu là lý do một mô hình 25 triệu tham số cần khoảng 410 MB chỉ để tồn tại trong vòng huấn luyện. Phần thứ tư là lý do cùng mô hình đó tràn 24 GB khi bạn nâng batch hoặc độ phân giải.

B. Tính tay bộ nhớ kích hoạt của ResNet-50

Hãy làm thật với ResNet-50 ở 224×224 , fp16. Đếm số phần tử của các đầu ra convolution cho một ảnh, theo từng tầng ($56^2 = 3136$, $28^2 = 784$, $14^2 = 196$, $7^2 = 49$):

- **Stem**: $64 \cdot 112^2 = 0,80$ M, sau max-pool $64 \cdot 3136 = 0,20$ M.
- **Stage 1** (3 khối ở 56×56 , kênh $64/64/256$): mỗi khối $(64+64+256) \cdot 3136 = 1,20$ M $\Rightarrow$ 3,61 M, cộng nhánh chiều 0,80 M.
- **Stage 2** (4 khối ở 28×28 , $128/128/512$): mỗi khối $768 \cdot 784 = 0,60$ M $\Rightarrow$ 2,41 M.
- **Stage 3** (6 khối ở 14×14 , $256/256/1024$): mỗi khối $1536 \cdot 196 = 0,30$ M $\Rightarrow$ 1,81 M.
- **Stage 4** (3 khối ở 7×7 , $512/512/2048$): mỗi khối $3072 \cdot 49 = 0,15$ M $\Rightarrow$ 0,45 M.

Tổng $\approx 10,1$ triệu phần tử mỗi ảnh, tức 20,2 MB ở fp16. Với batch 32: ≈ 650 MB — và đó mới chỉ là đầu ra của các tầng convolution. Trong thực tế còn phải giữ đầu ra của BatchNorm, mặt nạ của ReLU, các tensor của nhánh cộng; hệ số nhân thực nghiệm thường rơi vào khoảng hai tới ba lần, cho $\approx 1,3$ –2 GB. Đối chiếu với 410 MB của phía tham số: ở batch 32, kích hoạt chiếm gấp ba tới năm lần, và mọi biện pháp tiết kiệm bộ nhớ phải nhắm vào nó. (Hệ số hai–ba lần là kinh nghiệm thực hành, không phải con số tôi đo được cho đúng mô hình này.)

Hai quan sát rơi ra ngay từ phép tính này. Thứ nhất, *tầng chiếm nhiều kích hoạt nhất là tầng đầu chứ không phải tầng có nhiều tham số nhất*: stem một mình đã bằng gần một nửa cả stage 4 cộng lại, trong khi tham số thì tập trung gần hết ở stage 3 và 4. Thứ hai, kích hoạt tỉ lệ với $H \cdot W$. Nâng độ phân giải từ 224 lên 448 do đó nhân bộ nhớ kích hoạt lên bốn lần, trong khi tham số không đổi một byte. Đó chính là cột thứ ba của bảng ba trục ở chương 10, giờ đã thành con số.

C. Đổi tính toán lấy bộ nhớ

Khi kích hoạt là nút thắt, có một cách đổi rất sạch: *gradient checkpointing* — chỉ lưu một số điểm mốc trên đồ thị và tính lại phần ở giữa trong lượt ngược [98]. Đặt các mốc cách nhau $\sqrt{N}$ tầng thì bộ nhớ giảm từ $O(N)$ xuống $O(\sqrt{N})$. Cái giá là một lượt xuôi phụ, cỡ +30% thời gian huấn luyện. Ba biện pháp cùng họ nhắm ba phần bộ nhớ khác nhau, nên phải biết mình đang tràn vì phần nào rồi mới chọn:

- **Gradient checkpointing** — cắt phần kích hoạt, trả bằng thời gian tính lại [98].

- **Độ chính xác hỗn hợp** — cắt một nửa số byte của mỗi kích hoạt [99].
- **Phân mảnh trạng thái optimizer** — cắt phần $16N$ byte của tham số, gradient và hai moment, bằng cách chia chúng qua nhiều thiết bị [100].

→ **ràng buộc bởi** $C/12$ — bộ nhớ kích hoạt chặn batch tối đa, mà batch lại thay đổi động học của tối ưu — nên một ràng buộc phần cứng đi thẳng vào chất lượng mô hình.

I 9.5 Độ trễ, thông lượng và ngân sách thời gian thực (Latency, Throughput, Real-Time Budgets)

Hai mục tiêu này xung đột nhau và hầu hết tranh cãi về “mô hình nào tốt hơn” thực ra là tranh cãi về việc đang tối ưu cái nào. **Thông lượng** (*throughput*) là số mẫu xử lý được mỗi giây khi bạn được phép gom batch lớn; **độ trễ** (*latency*) là thời gian từ lúc một khung hình tới lúc có kết quả cho *chính* khung đó. Tăng batch gần như luôn cải thiện thông lượng — vì nó nâng cường độ số học và pha loãng chi phí phóng kernel — và gần như luôn làm xấu độ trễ, vì khung đầu tiên phải chờ khung cuối cùng của batch. Cấu hình tối ưu cho huấn luyện do đó thường là cấu hình tệ nhất cho robot.

Với hệ thống thời gian thực, cách làm đúng là *chia ngân sách mili-giây trước khi thiết kế*, giống hệt cách bạn phân bổ ngân sách cho các khối của một pipeline SLAM.

→ **trường hợp riêng của** $B/03$ — ngân sách mili-giây là một ràng buộc cứng, và ràng buộc cứng loại bỏ phương án trước khi độ chính xác kịp lên tiếng.

Bảng 9.2 là một cách chia ví dụ cho 30 Hz; con số cụ thể tùy hệ thống, nhưng kỷ luật thì không đổi: mỗi khối có một trần, và khối nào vượt trần phải bị cắt chứ không được “mượn” của khối khác.

Bảng 9.2. Một cách chia ngân sách 33 ms cho pipeline nhận thức 30 Hz (ví dụ minh họa, không phải số đo).

Khối	Trần (ms)	Điều xảy ra khi vượt trần
Thu ảnh + ISP	4	Không cắt được; là ràng buộc cứng của cảm biến
Tiền xử lý, chuyển bố cục	2	Gộp vào kernel đầu của mạng, hoặc làm trên GPU
Backbone	12	Trục đầu tiên bị cắt: hạ độ phân giải hoặc đổi backbone
Head + hậu xử lý	7	NMS trên CPU là thủ phạm quen thuộc
Hợp nhất, quyết định	3	Thường bị quên trong lúc đo mô hình
Dự phòng (jitter, throttling)	5	Không có nó thì hệ thống đúng 90% thời gian

Dòng cuối cùng đáng nói riêng. Trên thiết bị nhúng, năng lượng là ràng buộc thứ nhất chứ không phải thứ hai, và **giảm xung do nhiệt** (*thermal throttling*) khiến mọi phép đo ngắn nói dối: mô hình đo được 25 ms trong ba mươi giây đầu có thể thành 40 ms sau mười phút chạy liên tục trong vỏ kín. Đây là kinh nghiệm thực hành phổ biến trên Jetson và là lý do benchmark phải chạy đủ dài để nhiệt bão hòa.

→ **ràng buộc bởi** $E/24$ — một phép đo tốc độ là một thí nghiệm, nên nó chịu đúng các yêu cầu về làm nóng, lặp lại và báo cáo phân vị.

Nó cũng là một phần lời giải thích vì sao nhiều phương pháp dẫn đầu bảng xếp hạng không dùng được ngoài đời: bảng xếp hạng đo trên máy chủ có tản nhiệt và điện không giới hạn.

9.6 Nút cổ chai không nằm ở GPU (Bottlenecks Outside the GPU)

Đặc thù của thị giác máy tính so với xử lý ngôn ngữ: đầu vào rất nặng và phải được giải mã. Giải một ảnh JPEG 1080p trên một lõi CPU tốn cỡ 5–15 ms, và augmentation còn tốn thêm. Với tám worker, bạn cấp được cỡ vài trăm tới hơn một nghìn ảnh mỗi giây — trong khi một GPU máy chủ nuốt ResNet-50 ở tốc độ cao hơn thế. Kết quả là GPU nhàn rỗi theo chu kỳ, và mọi giờ bạn bỏ ra tối ưu mô hình đều không thay đổi gì. Các lỗi thoát theo thứ tự chi phí tăng dần: tăng số worker và bật prefetch; chuyển augmentation lên GPU; chuyển sang định dạng dữ liệu tuần tự đọc theo khối thay vì hàng triệu file nhỏ; giải mã JPEG bằng phần cứng.

Một nút cổ chai họ hàng là đường truyền giữa host và thiết bị. Một batch 32 ảnh $3 \times 224 \times 224$ ở fp32 là 19 MB. Qua PCIe 4.0 x16 với thông lượng thực tế cỡ 25 GB/s, phép sao chép tốn khoảng 0,8 ms. Con số đó không lớn, nhưng nếu bạn quên dùng bộ nhớ ghim và sao chép bất đồng bộ thì nó nằm trọn trên đường tới hạn. Trên Jetson thì câu chuyện khác hẳn và theo hướng có lợi: CPU và GPU dùng chung bộ nhớ vật lý, nên bản sao đó có thể biến mất hoàn toàn nếu bạn cấp phát đúng loại bộ nhớ — một tối ưu không tồn tại trên máy chủ.

↔ **hệ quả của D/21** — đây đúng là loại tối ưu mà runtime triển khai khai thác, và là lý do cùng một mô hình phải được đo lại trên từng nền tảng đích.

9.7 Các con số cần thuộc lòng (Numbers Worth Memorizing)

Đây là phần duy nhất của chương thực sự là một danh sách. Nó đáng thuộc vì nó cho phép ước lượng bậc độ lớn ngay trong đầu, giữa một cuộc họp.

- **Kích thước dữ liệu.** fp32 = 4 byte, fp16/bf16 = 2, int8 = 1. Một triệu tham số fp32 = 4 MB; huấn luyện với Adam = 16 MB.
- **Công thức chi phí.** Tích chập: $FLOPs = 2k^2 C_{in} C_{out} H_{out} W_{out}$ cho mỗi ảnh. Tầng nối đầy đủ: $2 C_{in} C_{out}$. Activation tỉ lệ với $batch \times H \times W$ — nhân đôi độ phân giải là nhân bốn bộ nhớ.
- **Hằng số của máy.** Điểm gãy roofline của phần cứng hiện đại nằm trong khoảng 100–350 FLOP/byte; đọc 1 GB từ bộ nhớ tốn 0,5 ms ở 2 TB/s và 5 ms ở 200 GB/s; phóng một kernel tốn cỡ 5 μ s; PCIe 4.0 x16 cho cỡ 25 GB/s thực tế.
- **Ngưỡng chẩn đoán.** MFU dưới 30% thì gần như chắc chắn bạn *không* compute-bound, nên đừng đụng vào FLOPs. Kernel ngắn hơn 10 μ s là dấu hiệu overhead-bound.
- **Ngân sách thời gian.** 30 Hz = 33 ms mỗi khung; 60 Hz = 16,7 ms; một ảnh JPEG 1080p mất 5–15 ms để giải mã trên một lõi.

9.8 Thực hành

Repo và tài nguyên. torch.profiler kèm trace viewer của TensorBoard là điểm khởi đầu; NVIDIA Nsight Systems cho bức tranh toàn hệ thống và Nsight Compute cho từng kernel; [XuehaiPan/nvtop](#) để nhìn nhanh; [facebookresearch/fvcore](#) để đếm FLOPs; [triton-lang/triton](#) khi bạn cần tự viết kernel hợp nhất. Bài đọc ngắn gọn nhất về ba chế độ nghề là “Making Deep Learning Go Brrrr From First Principles” của Horace He. Danh sách này chốt tại tháng 9/2026; khác với các chương về mô hình, phần lớn nội dung ở đây dựa trên vật lý phần cứng nên bán rã dài hơn nhiều — chỉ các con số tuyệt đối của từng đời chip là sẽ cũ đi.

Câu hỏi kiểm tra hiểu. Mô hình của bạn đạt 30 % MFU: nghẽn ở đâu, và ba phép đo nào phân biệt được ba khả năng? Vì sao BatchNorm memory-bound trong khi convolution 3×3 compute-bound, và điều đó gợi ý gì về việc nên hợp nhất cái gì với cái gì? Nếu chuyển từ A100 sang Orin, những tối ưu nào bạn đang có sẽ mất tác dụng và những tối ưu nào trở nên đáng giá hơn?

Mini project. Hồ sơ hoá một vòng huấn luyện thật và lập bảng thời gian ở năm chỗ: dataloader, lượt xuôi, lượt ngược, bước optimizer, và đồng bộ. Yêu cầu bắt buộc: *viết dự đoán ra giấy trước khi mở profiler*, rồi ghi lại mức sai của từng dự đoán. Bản ghi mức sai đó có giá trị hơn bảng số liệu.

9.9 Giới hạn và trường hợp hỏng

Roofline là một mô hình *thượng giới hạn* và nó nói dối theo những cách có thể liệt kê được. Nó giả định phép toán chồng lấn hoàn hảo giữa tính toán và truyền dữ liệu, giả định cache không tồn tại hoặc luôn trúng, và giả định chỉ có một nút thất tại một thời điểm. Trong thực tế một kernel có thể vừa nghẽn băng thông vừa nghẽn số thanh ghi, và một mô hình có thể compute-bound ở tầng này nhưng overhead-bound ở tầng khác trong cùng một lượt. Roofline dùng để *chọn hướng điều tra*, không dùng để dự đoán con số.

Ba trường hợp hỏng cụ thể đáng cảnh giác. Thứ nhất, *đỉnh công bố không phải đỉnh đạt được*: các con số marketing thường tính ở chế độ thừa hoặc ở độ chính xác thấp nhất, và khoảng cách với thực tế có thể là hai lần — vì vậy hãy đo đỉnh của chính máy mình một lần rồi dùng con số đó mãi. Thứ hai, *benchmark ngắn nói dối* vì nhiệt chưa bão hoà, vì xung chưa ổn định, và vì lần chạy đầu tiên còn gánh chi phí biên dịch kernel; phép đo đúng phải có giai đoạn làm nóng và phải lấy phân vị chứ không chỉ lấy trung bình. Thứ ba, *đo sai chỗ*: lệnh gọi GPU là bất đồng bộ, nên đo bằng đồng hồ của CPU mà quên đồng bộ hoá sẽ cho ra những con số nhanh đến phi lý — đây là lỗi phổ biến nhất trong toàn bộ lĩnh vực đo hiệu năng học sâu.

CẠM BÃY

Trực giác về hiệu năng gần như luôn sai cho tới khi đo, kể cả trực giác của người có kinh nghiệm. Nhưng “cứ đo đi” là lời khuyên chưa đủ, vì đo mà không dự đoán trước thì bạn chỉ thu được một con số chứ không cải thiện được trực giác. Quy trình đúng có ba bước và bước đầu là bước hay bị bỏ: *viết dự đoán ra trước*, đo, rồi ghi lại mức sai. Sau mười lần như vậy bạn sẽ biết mình có thiên kiến hệ thống ở đâu — hầu hết mọi người đánh giá thấp chi phí bộ nhớ và đánh giá cao ảnh hưởng của FLOPs.

CẦN NHỚ

Một câu để mang theo: *FLOPs là thuộc tính của mô hình, thời gian là thuộc tính của cặp mô hình–máy, và cây cầu giữa hai thứ đó là cường độ số học*. Trước khi tối ưu bất cứ điều gì, hãy tính I cho khối bạn định sửa. Rồi đem nó so với điểm gãy của máy bạn đang chạy. Nếu I nằm bên trái điểm gãy, mọi nỗ lực giảm FLOPs sẽ không đem lại gì. [T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• FLOPs	Số phép nhân–cộng dấu phẩy động cho một lần chạy. Nó đo khối lượng công việc tính toán, không đo thời gian, và đó là toàn bộ nội dung của chương.
••• Cường độ số học	Số FLOP làm được trên mỗi byte phải chuyển giữa bộ nhớ và chip. Nó trả lời câu “mỗi byte kéo về được dùng để làm bao nhiêu việc”, và là đại lượng duy nhất phân biệt nghẽn tính toán với nghẽn bộ nhớ.
••• Điểm gãy roofline	Đỉnh tính toán chia cho băng thông, một hằng số của <i>máy</i> chứ không của mô hình. Khoảng 150 FLOP/byte trên A100 và khoảng 330 trên Jetson Orin; bên trái nó bạn nghẽn bộ nhớ, bên phải nghẽn tính toán.
•• Vì sao định fp16 công bố không dùng thẳng được?	Vì con số marketing thường tính ở chế độ thừa hoặc ở độ chính xác thấp nhất, và khoảng cách với thực tế có thể tới hai lần. Hãy đo định của chính máy mình một lần bằng một GEMM lớn, rồi dùng con số đó mãi.
••• Vì sao BatchNorm memory-bound còn convolution compute-bound?	Vì cường độ số học chênh hơn một nghìn lần: convolution 3×3 dày ở 256 kênh cho khoảng 843 FLOP/byte, còn BatchNorm trên cùng tensor chỉ khoảng 0,7. Đó là lý do trình biên dịch luôn cố gộp BatchNorm vào convolution liền trước.
••• Memory-bound	Chế độ mà thời gian chạy do lưu lượng bộ nhớ quyết định chứ không do số phép tính. Triệu chứng là GPU util cao nhưng MFU thấp; chứa bằng hợp nhất kernel và bố cục channels-last, còn giảm FLOPs thì vô ích.
•• Overhead-bound	Chế độ mà GPU chờ CPU gửi việc. Triệu chứng là util thấp với rất nhiều kernel ngắn dưới $10 \mu s$; chứa bằng CUDA graph, torch.compile hoặc tăng batch, còn đổi kiến trúc thì vô ích.
••• Vì sao depthwise cộng pointwise giảm FLOPs mà không nhanh hơn?	Vì nó giảm FLOPs 8,7 lần nhưng <i>tăng</i> lưu lượng bộ nhớ từ 4,4 MB lên 6,5 MB: hai kernel nên tensor trung gian phải ghi ra rồi đọc lại. Trên máy nghẽn bộ nhớ, thời gian đi theo byte.
•• Channels-last (NHWC)	Bố cục xếp kênh liền nhau trong bộ nhớ. Lỗi tensor chỉ được kích hoạt khi tensor ở đúng bố cục này và số kênh chia hết cho 8 ở fp16; sai một điều kiện thì kernel rơi xuống đường chậm dù FLOPs không đổi.
•• Chi phí phóng kernel	Mỗi lần CPU gửi một kernel cho GPU tốn cỡ $5\text{--}10 \mu s$, và nhiều hơn trên CPU yếu của thiết bị nhúng. Một mạng nhẹ hai trăm kernel do đó có sần thời gian 1–2 ms mà không lượng FLOPs nào ảnh hưởng tới.
••• Bộ nhớ kích hoạt	Các tensor trung gian của lượt xuôi, phải giữ lại vì lượt ngược cần tới. Nó tỉ lệ với batch và với bình phương độ phân giải, nên nó mới là thứ chặn batch chứ không phải số trọng số.
•• Trạng thái optimizer	Với Adam là hai moment, cộng với trọng số và gradient thành 16 byte cho mỗi tham số ở fp32. Phần này cố định, không phụ thuộc batch, nên nó chỉ phổ biến khi mô hình rất lớn mà rất nông.
•• Gradient checkpointing	Chỉ lưu một số điểm mốc trên đồ thị rồi tính lại phần ở giữa trong lượt ngược. Đặt mốc cách nhau $\sqrt{N}$ tầng thì bộ nhớ giảm từ $O(N)$ xuống $O(\sqrt{N})$, trả bằng cỡ +30% thời gian huấn luyện.
•• Tăng batch xong thông lượng tăng nhưng độ trễ đuôi xấu hẳn — vì sao?	Vì hai đại lượng này xung đột. Gộp batch nâng cường độ số học và pha loãng chi phí phóng kernel, nên thông lượng tăng; nhưng khung hình đầu phải chờ khung cuối của batch, nên độ trễ xấu đi. Cấu hình tốt cho huấn luyện thường là cấu hình tệ nhất cho robot.
•• Ngân sách mili-giây	Chia trần thời gian cho từng khối <i>trước</i> khi thiết kế: 33 ms cho 30 Hz, 16,7 ms cho 60 Hz. Khối nào vượt trần phải bị cắt chứ không được mượn của khối khác, và phải chừa phần dự phòng cho jitter và giảm xung.
• Giảm xung do nhiệt	Thiết bị tự hạ tần số khi nóng, nên một mô hình đo 25 ms trong ba mươi giây đầu có thể thành 40 ms sau mười phút chạy trong vỏ kín. Vì vậy benchmark phải chạy đủ dài để nhiệt bão hoà.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Vì sao dataloader có thể là nút cổ chai của cả hệ?	Giải mã một ảnh JPEG 1080p tốn 5–15 ms trên một lõi CPU, nên tám worker chỉ cấp được vài trăm tới hơn một nghìn ảnh mỗi giây. Con số đó thấp hơn tốc độ nuốt của một GPU máy chủ, và mọi tối ưu mô hình khi đó đều vô nghĩa.
●●● GPU util 95 % nhưng MFU chỉ 12 % — nghĩa là gì?	Bạn memory-bound: GPU bận, nhưng nó chờ bộ nhớ chứ không bận tính. Phân biệt với compute-bound bằng chính MFU, và phân biệt với overhead-bound bằng util — overhead-bound thì util thấp chứ không cao.
●● GPU nhàn rỗi theo chu kỳ, đỉnh CPU cao — nguyên nhân?	Nghẽn ở đường dữ liệu vào, không ở mô hình. Thêm worker và bật prefetch rồi đo lại; nếu thời gian một bước giảm theo số worker thì đúng là nhánh này.
●●● Lượt xuôi của một mạng lớn đo được 0,2 ms — lỗi ở đâu?	Bạn quên đồng bộ hoá trước khi bấm giờ. Lệnh gọi GPU là bất đồng bộ nên đồng hồ CPU trả về ngay khi lệnh được xếp hàng, chứ không phải khi phép tính xong.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
FlashAttention	Hệ quả trực tiếp của phân tích roofline: nó không giảm FLOPs mà giảm số lượt đọc/ghi bộ nhớ — đúng biện pháp của chế độ memory-bound	Ví dụ sạch nhất trong ngành về việc đo đúng đại lượng [101]
torch.compile, TensorRT, ONNX Runtime	Hợp nhất kernel và ghi lại đồ thị thành CUDA graph — biện pháp của chế độ memory-bound và overhead-bound	Hạ tầng bền; là nơi phần lớn tốc độ “miễn phí” đến từ
NVIDIA DALI và các định dạng dữ liệu tuần tự	Tồn tại vì nút cổ chai giải mã ảnh trên CPU, không vì bất kỳ lý do nào liên quan tới mô hình	Đúng nhánh trái ngoài cùng của cây chẩn đoán trong chương
Gradient checkpointing trong PyTorch và Hugging Face	Đổi một lượt xuôi phụ lấy bộ nhớ kích hoạt $O(\sqrt{N})$	Bật bằng một cờ; là lý do nhiều mô hình lớn huấn luyện được trên phần cứng nhỏ [98]
DeepSpeed ZeRO và FSDP	Cắt đúng phần 16 byte mỗi tham số của trạng thái optimizer bằng cách phân mảnh qua nhiều thiết bị	Nhắm một phần bộ nhớ khác hẳn với checkpointing — biết mình tràn vì phần nào mới chọn đúng [100]

MINI PROJECT SLAM · Roofline đo thật cho Jetson Orin của bạn, với ba khối trong pipeline SLAM · 1–2 ngày

- Mục tiêu** thay mọi con số công bố trong chương này bằng con số đo được của chính máy bạn. Đáng ngờ nhất là đỉnh fp16: ở đây nó chỉ được *suy ra* từ thông số 275 TOPS INT8 thừa, chưa hề được đo. Có mái nhà đo thật rồi thì dùng nó để phân loại ba khối trong pipeline của bạn.
- Dữ liệu** một chuỗi EuRoC chạy qua pipeline hiện tại; cố định xung bằng jetson_clocks và chạy đủ lâu cho nhiệt bão hoà *trước khi* bắt đầu ghi số.
- Các bước** (1) đo đỉnh tính toán bằng một GEMM fp16 đủ lớn lặp nhiều lần; (2) đo bằng thông thực bằng một phép sao chép tensor lớn hơn hẳn cache; (3) vẽ mái nhà và tính điểm gãy đo được, đặt cạnh con số 332 FLOP/byte suy ra từ thông số; (4) chọn ba khối trong pipeline — ví dụ trích đặc trưng ORB trên GPU, một tầng convolution của backbone, và một bước chuẩn hoá hoặc đổi kích thước theo phần tử — rồi đếm bằng tay số FLOP và số byte của từng khối; (5) đặt ba điểm lên đồ thị và đối chiếu thời gian dự đoán với thời gian đo bằng Nsight Systems.

- Xong khi** bạn có một đồ thị roofline với đỉnh và bằng thông *đo được*, và ba khối của bạn nằm trên đó với toạ độ tính bằng tay. Mỗi khối kèm một câu kết luận dạng “khối này memory-bound ở I FLOP/byte, nên hợp nhất kernel đáng giá còn cắt FLOPs thì không”. Cuối cùng, bạn ghi được sai số giữa thời gian roofline dự đoán và thời gian đo thật.
- Nổi với** D/20 và D/21 để biến kết luận thành hành động; E/24 cho cách báo cáo đúng (làm nóng, lặp lại, phân vị thay vì trung bình); B/10 nếu kết luận buộc bạn đổi backbone thay vì đổi kernel.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì FLOPs không đo thứ chi phối thời gian. Hai mô hình cùng FLOPs có thể chênh nhau hai bậc về cường độ số học. Chúng cũng có thể chênh hàng trăm lần về số kernel phải phóng, và khác nhau ở chỗ có kích hoạt được lõi tensor hay không. Ba nguyên nhân này độc lập với nhau, và không cái nào xuất hiện trong con số FLOPs.
- A2. Vì trọng số chỉ là một trong bốn phần bộ nhớ. Huấn luyện thêm gradient, thêm trạng thái optimizer (Adam giữ hai moment, tổng cộng 16 byte mỗi tham số), và quan trọng nhất là kích hoạt phải giữ lại cho lượt ngược — phần duy nhất tỉ lệ với batch và với bình phương độ phân giải.
- A3. Rằng phần lớn danh sách đó sẽ đối thứ tự. Điểm gây cao hơn nghĩa là nhiều phép toán rơi vào vùng nghẽn bộ nhớ hơn, nên hợp nhất kernel và giảm số lượt đọc/ghi trở nên đáng giá hơn, còn cắt FLOPs kém hiệu quả hơn hẳn so với trên máy chủ.
- A4. Vì có một nhánh chẩn đoán thứ tư nằm ngoài GPU: giải mã ảnh và augmentation trên CPU. Một ảnh JPEG 1080p tốn 5–15 ms trên một lõi, nên tám worker chỉ cấp được vài trăm tới hơn một nghìn ảnh mỗi giây — thấp hơn tốc độ nuốt của một GPU máy chủ.
- A5. Vì lệnh gọi GPU là bất đồng bộ: đồng hồ CPU trả về ngay khi lệnh được xếp hàng chứ không phải khi phép tính xong. Không đồng bộ hoá trước khi bấm giờ là lỗi đo phổ biến nhất trong toàn bộ lĩnh vực này.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn gọi thợ sửa ống nước. Việc thật sự chỉ mất năm phút vặn cờ lê, nhưng anh ta phải lái xe bốn mươi phút mới tới nhà bạn. Đoán hoá đơn bằng cách đếm số phút vặn cờ lê thì bạn sai gần mười lần. Thuê thợ khéo gấp đôi thì hoá đơn gần như không đổi, vì phần lớn thời gian nằm ở đường đi chứ không ở tay nghề.

Máy tính chạy mạng nơ-ron y hệt vậy. Con số mọi người hay khoe, tức số phép tính phải làm, chính là số phút vặn cờ lê. Thứ tốn thời gian là chuyển dữ liệu từ bộ nhớ vào chỗ tính, tức quãng đường lái xe. Nên phép đo đáng giá không phải “bao nhiêu việc” mà là “mỗi chuyến đi làm được bao nhiêu việc”.

Nếu mỗi chuyến chở về đủ việc làm cả buổi thì cắt bớt việc mới giúp. Nếu mỗi chuyến chỉ làm được một hai thao tác thì cắt việc chẳng đổi gì, phải gộp nhiều thao tác vào chung một chuyến. Còn nếu xe cứ đứng chờ vì bạn gọi đặt lịch từng việc một, thì phải gộp các cuộc gọi.

Cái giá là bạn buộc phải đo. Trực giác về tốc độ gần như luôn sai, và tỉ lệ giữa thời gian đi với thời gian làm khác nhau ở mỗi máy: kết luận đúng trên máy chủ có thể sai hẳn trên thiết bị nhúng.

Chỗ không rút gọn được: một phép tính có thể vừa kẹt ở đường đi vừa kẹt ở chỗ khác cùng lúc, nên bức tranh “chỉ có đúng một nút thắt” là một xấp xỉ. Nó đủ tốt để chọn hướng điều tra, nhưng không đủ để dự đoán ra con số.

Một câu: Thời gian chạy không được quyết định bởi bạn làm bao nhiêu việc, mà bởi bạn phải đi lấy dữ liệu bao nhiêu lần — nên trước khi cắt việc, hãy đếm số chuyến đi.

9.10 Kết nối

Chương này là *mặt sau* của chương 10: mọi đánh đổi kiến trúc ở đó đều có một cái giá vật lý được định lượng ở đây, và cột “độ trễ” trong bảng ba trục chỉ đọc được sau khi có khái niệm cường độ số học. Nó cũng là *trường hợp cụ thể* của nguyên lý ràng buộc cứng ở chương 3: ngân sách mili-giây và ngân sách năng lượng loại bỏ phương án trước khi độ chính xác kịp lên tiếng, và đó là lý do phần lớn thiết kế trong hệ thống nhúng trông “bảo thủ” hơn trong paper.

Đọc tiếp: D/20-toi-uu-va-nen-mo-hinh-bien-cac-quan-sat-o-dây-thành-hành-động — lượng tử hoá, tia, chưng cất — và mỗi kỹ thuật ở đó nhắm vào một chế độ nghẽn khác nhau, nên chương này là điều kiện để chọn đúng. D/21-inference-va-trien-khai bàn về trình biên dịch và runtime, tức là nơi kernel fusion và CUDA graph thực sự xảy ra. C/12-co-che-huan-luyen giải thích vì sao batch lớn không chỉ là chuyện bộ nhớ mà còn thay đổi động học của tối ưu, nên bạn không được tự do tăng batch để lấy thông lượng. Và E/24-thi-nghiem-va-ablation dạy cách đo cho tử tế — phân vị, lặp lại, làm nóng — những thứ mà phần “giới hạn” ở trên vừa chỉ ra là bắt buộc.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao thay convolution dày bằng depthwise cộng pointwise giảm FLOPs gần chín lần mà vẫn có thể làm mô hình chậm đi, và hai con số byte nào chứng minh điều đó?
2. Điểm gãy roofline của Jetson Orin cao hơn của A100. Điều đó có nghĩa Orin “tốt hơn” ở khía cạnh nào và “tệ hơn” ở khía cạnh nào, và nó đổi hướng tối ưu của bạn ra sao?
3. Mô hình vừa khít GPU khi suy luận nhưng tràn khi huấn luyện với cùng batch: bốn thành phần bộ nhớ nào đang tồn tại, và thành phần nào chịu trách nhiệm?
4. Với ResNet-50 batch 32, vì sao tầng chiếm nhiều kích hoạt nhất lại không phải tầng có nhiều tham số nhất?
5. GPU util của bạn là 95 % nhưng MFU chỉ 12 %. Đó là chế độ nào, và vì sao “giảm FLOPs” là biện pháp sai?
6. Vì sao một benchmark 30 giây trên thiết bị nhúng có thể cho con số tốt hơn thực tế vận hành tới gần hai lần?
7. Bạn đo được lượt xuôi mất 0,2 ms trên một mạng lớn — lỗi đo nào gần như chắc chắn đã xảy ra?

Nguồn

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *Rec-Sys*, 2019.
- [3] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [4] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [5] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [6] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [7] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [8] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [9] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [10] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [11] V. N. Vapnik, “An overview of statistical learning theory,” *IEEE Trans. Neural Networks*, vol. 10, no. 5, pp. 988–999, 1999.
- [12] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [13] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [15] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [16] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [17] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [18] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE Trans. Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.
- [19] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.

- [20] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [21] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [22] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [23] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [24] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [25] R. Geirhos, J.-H. Jacobsen, C. Michaelis, et al., “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.
- [26] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [27] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [28] T. Gebru, J. Morgenstern, et al., “Datasheets for Datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [29] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [30] M. Mitchell, S. Wu, et al., “Model Cards for Model Reporting,” in *FAT**, 2019.
- [31] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [32] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [33] C. G. Northcutt, L. Jiang, and I. L. Chuang, “Confident Learning: Estimating Uncertainty in Dataset Labels,” *JAIR*, vol. 70, pp. 1373–1411, 2021.
- [34] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [35] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [36] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [37] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [38] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [39] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [40] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [41] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London: Butterworths, 1979.
- [42] B. W. Matthews, “Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme,” *Biochimica et Biophysica Acta (BBA) — Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
- [43] D. Chicco and G. Jurman, “The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation,” *BMC Genomics*, vol. 21, p. 6, 2020.

- [44] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in *International Conference on Machine Learning (ICML)*, 2006, pp. 233–240.
- [45] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.
- [46] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [47] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [48] M. P. Naeni, G. F. Cooper, and M. Hauskrecht, "Obtaining Well Calibrated Probabilities Using Bayesian Binning," in *AAAI Conference on Artificial Intelligence*, 2015.
- [49] G. W. Brier, "Verification of Forecasts Expressed in Terms of Probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
- [50] A. H. Murphy, "A New Vector Partition of the Probability Score," *Journal of Applied Meteorology*, vol. 12, no. 4, pp. 595–600, 1973.
- [51] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [52] T.-Y. Lin, M. Maire, S. Belongie, et al., "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.
- [53] J. Hosang, R. Benenson, P. Dollár, et al., "What Makes for Effective Detection Proposals?" *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 4, pp. 814–830, 2016.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] F. Perazzi, J. Pont-Tuset, B. McWilliams, et al., "A Benchmark Dataset and Evaluation Methodology for Video Object Segmentation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [56] G. Csurka, D. Larlus, and F. Perronnin, "What Is a Good Evaluation Measure for Semantic Segmentation?" In *British Machine Vision Conference (BMVC)*, 2013.
- [57] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [58] K. Mikolajczyk and C. Schmid, "A Performance Evaluation of Local Descriptors," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 10, pp. 1615–1630, 2005.
- [59] M. Dusmanu, I. Rocco, T. Pajdla, et al., "D2-Net: A Trainable CNN for Joint Description and Detection of Local Features," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [60] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [61] S. Sagawa, P. W. Koh, T. B. Hashimoto, et al., "Distributionally Robust Neural Networks for Group Shifts," in *ICLR*, 2020.
- [62] M. Arjovsky, L. Bottou, I. Gulrajani, et al., "Invariant Risk Minimization," *arXiv:1907.02893*, 2019.
- [63] I. Gulrajani and D. Lopez-Paz, "In Search of Lost Domain Generalization," in *ICLR*, 2021.
- [64] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," in *ICLR*, 2019.

- [65] R. Taori, A. Dave, V. Shankar, et al., "Measuring Robustness to Natural Distribution Shifts in Image Classification," in *NeurIPS*, 2020.
- [66] D. Hendrycks and K. Gimpel, "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks," in *ICLR*, 2017.
- [67] Y. Ovadia, E. Fertig, et al., "Can You Trust Your Model's Uncertainty?" In *NeurIPS*, 2019.
- [68] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [69] A. Kendall, Y. Gal, and R. Cipolla, "Multi-Task Learning Using Uncertainty to Weigh Losses," in *CVPR*, 2018.
- [70] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [71] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., "Rethinking the Inception Architecture for Computer Vision," in *CVPR*, 2016.
- [72] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation," in *3DV*, 2016.
- [73] H. Rezatofighi, N. Tsoi, J. Gwak, et al., "Generalized Intersection over Union," in *CVPR*, 2019.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in *CVPR*, 2015.
- [75] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [76] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [77] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [78] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [79] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [80] Y. LeCun, L. Bottou, Y. Bengio, et al., "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [81] F. Yu and V. Koltun, "Multi-Scale Context Aggregation by Dilated Convolutions," in *ICLR*, 2016.
- [82] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention Is All You Need," in *NeurIPS*, 2017.
- [83] Z. Liu, Y. Lin, Y. Cao, et al., "Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows," in *ICCV*, 2021.
- [84] K. He, X. Zhang, S. Ren, et al., "Deep Residual Learning for Image Recognition," in *CVPR*, 2016.
- [85] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *ICML*, 2015.
- [86] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," *arXiv:1607.06450*, 2016.
- [87] Y. Wu and K. He, "Group Normalization," in *ECCV*, 2018.
- [88] S. Santurkar, D. Tsipras, A. Ilyas, et al., "How Does Batch Normalization Help Optimization?" In *NeurIPS*, 2018.
- [89] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in *CVPR*, 2018.
- [90] J. Dai, H. Qi, et al., "Deformable Convolutional Networks," in *ICCV*, 2017.

- [91] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [92] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [93] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [94] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [95] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [96] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [97] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [98] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [99] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [100] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [101] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.

Nguồn

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *Rec-Sys*, 2019.
- [3] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [4] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [5] S. Keshav, *How to Read a Paper*, ACM SIGCOMM Computer Communication Review, 2007.
- [6] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [7] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [8] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [9] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.
- [10] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [11] V. N. Vapnik, “An overview of statistical learning theory,” *IEEE Trans. Neural Networks*, vol. 10, no. 5, pp. 988–999, 1999.
- [12] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [13] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [15] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [16] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [17] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [18] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE Trans. Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.
- [19] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.

- [20] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [21] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [22] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [23] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [24] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [25] R. Geirhos, J.-H. Jacobsen, C. Michaelis, et al., “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.
- [26] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [27] P. W. Koh, S. Sagawa, et al., “WILDS: A Benchmark of in-the-Wild Distribution Shifts,” in *ICML*, 2021.
- [28] T. Gebru, J. Morgenstern, et al., “Datasheets for Datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [29] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [30] M. Mitchell, S. Wu, et al., “Model Cards for Model Reporting,” in *FAT**, 2019.
- [31] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [32] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [33] C. G. Northcutt, L. Jiang, and I. L. Chuang, “Confident Learning: Estimating Uncertainty in Dataset Labels,” *JAIR*, vol. 70, pp. 1373–1411, 2021.
- [34] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [35] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [36] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [37] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [38] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [39] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [40] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [41] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London: Butterworths, 1979.
- [42] B. W. Matthews, “Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme,” *Biochimica et Biophysica Acta (BBA) — Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
- [43] D. Chicco and G. Jurman, “The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation,” *BMC Genomics*, vol. 21, p. 6, 2020.

- [44] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in *International Conference on Machine Learning (ICML)*, 2006, pp. 233–240.
- [45] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.
- [46] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [47] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [48] M. P. Naeni, G. F. Cooper, and M. Hauskrecht, "Obtaining Well Calibrated Probabilities Using Bayesian Binning," in *AAAI Conference on Artificial Intelligence*, 2015.
- [49] G. W. Brier, "Verification of Forecasts Expressed in Terms of Probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
- [50] A. H. Murphy, "A New Vector Partition of the Probability Score," *Journal of Applied Meteorology*, vol. 12, no. 4, pp. 595–600, 1973.
- [51] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [52] T.-Y. Lin, M. Maire, S. Belongie, et al., "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.
- [53] J. Hosang, R. Benenson, P. Dollár, et al., "What Makes for Effective Detection Proposals?" *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 4, pp. 814–830, 2016.
- [54] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [55] F. Perazzi, J. Pont-Tuset, B. McWilliams, et al., "A Benchmark Dataset and Evaluation Methodology for Video Object Segmentation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [56] G. Csurka, D. Larlus, and F. Perronnin, "What Is a Good Evaluation Measure for Semantic Segmentation?" In *British Machine Vision Conference (BMVC)*, 2013.
- [57] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [58] K. Mikolajczyk and C. Schmid, "A Performance Evaluation of Local Descriptors," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 10, pp. 1615–1630, 2005.
- [59] M. Dusmanu, I. Rocco, T. Pajdla, et al., "D2-Net: A Trainable CNN for Joint Description and Detection of Local Features," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [60] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [61] S. Sagawa, P. W. Koh, T. B. Hashimoto, et al., "Distributionally Robust Neural Networks for Group Shifts," in *ICLR*, 2020.
- [62] M. Arjovsky, L. Bottou, I. Gulrajani, et al., "Invariant Risk Minimization," *arXiv:1907.02893*, 2019.
- [63] I. Gulrajani and D. Lopez-Paz, "In Search of Lost Domain Generalization," in *ICLR*, 2021.
- [64] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," in *ICLR*, 2019.

- [65] R. Taori, A. Dave, V. Shankar, et al., "Measuring Robustness to Natural Distribution Shifts in Image Classification," in *NeurIPS*, 2020.
- [66] D. Hendrycks and K. Gimpel, "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks," in *ICLR*, 2017.
- [67] Y. Ovadia, E. Fertig, et al., "Can You Trust Your Model's Uncertainty?" In *NeurIPS*, 2019.
- [68] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [69] A. Kendall, Y. Gal, and R. Cipolla, "Multi-Task Learning Using Uncertainty to Weigh Losses," in *CVPR*, 2018.
- [70] T.-Y. Lin, P. Goyal, R. Girshick, et al., "Focal Loss for Dense Object Detection," in *ICCV*, 2017.
- [71] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., "Rethinking the Inception Architecture for Computer Vision," in *CVPR*, 2016.
- [72] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation," in *3DV*, 2016.
- [73] H. Rezatofighi, N. Tsoi, J. Gwak, et al., "Generalized Intersection over Union," in *CVPR*, 2019.
- [74] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in *CVPR*, 2015.
- [75] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," in *arXiv:1807.03748*, 2018.
- [76] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *ECCV*, 2016.
- [77] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image Style Transfer Using Convolutional Neural Networks," in *CVPR*, 2016.
- [78] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS Deep Learning Workshop*, 2015.
- [79] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *CVPR*, 2018.
- [80] Y. LeCun, L. Bottou, Y. Bengio, et al., "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [81] F. Yu and V. Koltun, "Multi-Scale Context Aggregation by Dilated Convolutions," in *ICLR*, 2016.
- [82] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention Is All You Need," in *NeurIPS*, 2017.
- [83] Z. Liu, Y. Lin, Y. Cao, et al., "Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows," in *ICCV*, 2021.
- [84] K. He, X. Zhang, S. Ren, et al., "Deep Residual Learning for Image Recognition," in *CVPR*, 2016.
- [85] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *ICML*, 2015.
- [86] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," *arXiv:1607.06450*, 2016.
- [87] Y. Wu and K. He, "Group Normalization," in *ECCV*, 2018.
- [88] S. Santurkar, D. Tsipras, A. Ilyas, et al., "How Does Batch Normalization Help Optimization?" In *NeurIPS*, 2018.
- [89] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in *CVPR*, 2018.
- [90] J. Dai, H. Qi, et al., "Deformable Convolutional Networks," in *ICCV*, 2017.

- [91] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [92] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [93] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [94] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [95] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [96] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [97] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [98] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [99] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [100] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [101] T. Dao, D. Y. Fu, S. Ermon, et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022.